

Lời nói đầu

Cuốn **Giáo trình C++ — Sách giáo viên — Quyển 3** là tập hoàn thiện bộ tài liệu đào tạo lập trình thuật toán C++ chuyên sâu iKHEDU, dành riêng cho quý thầy cô giáo và các huấn luyện viên chuyên trách đội tuyển tin học. Tài liệu này cung cấp trọn bộ lời giải chi tiết, mã nguồn C++ tối ưu và hệ thống **phương pháp luận giảng dạy thực chiến chuyên sâu** (ý tưởng và phân tích thuật toán, bảng mô phỏng chạy tay từng bước trên số liệu mẫu thực tế, lưu ý và các bẫy lỗi kinh điển thường gặp) cho toàn bộ **105 bài toán thực hành** từ Bài 15 đến Bài 21 thuộc 3 trụ cột thuật toán đỉnh cao: Quy hoạch động chuỗi (LCS & Edit Distance), Cấu trúc dữ liệu STL nâng cao (Set, Map, Heap), Ngăn xếp đơn điệu (Monotonic Stack), Hàng đợi hai đầu (Monotonic Deque), Lý thuyết đồ thị cơ bản (Duyệt BFS & DFS), Đồ thị lưới 2D & Flood Fill, cùng Cấu trúc cây phân đoạn (Segment Tree) & Fenwick Tree (BIT).

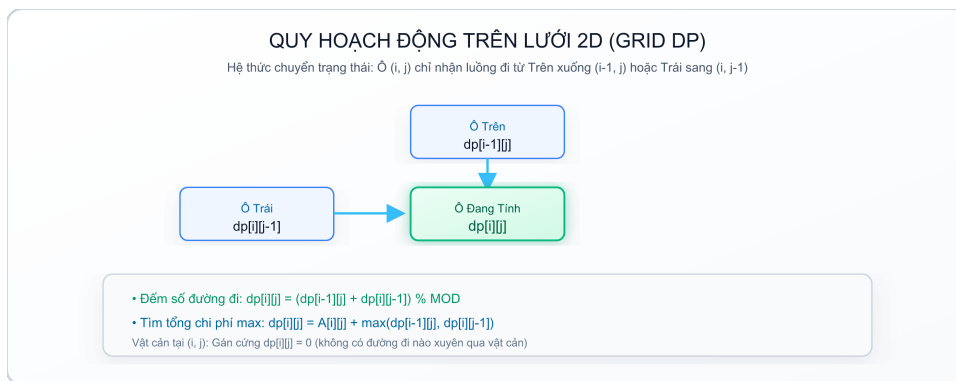
Mỗi bài toán trong sách giáo viên được cấu trúc thống nhất và tinh gọn qua 4 mục trọng tâm: (1) Ý tưởng & Phân tích thuật toán gắn liền với bản chất toán học; (2) Bảng chạy tay trực quan (Dry Run Table) trên các giá trị số thực tế của mẫu thử (Sample); (3) Lưu ý & các bẫy lỗi kinh điển học sinh thường gặp trong môi trường lập trình; và (4) Mã nguồn C++ tham khảo tối ưu nhất (Fast I/O, Safe Input, 0 std:::). Tài liệu là cẩm nang đặc lực hỗ trợ thầy cô tổ chức các giờ dạy chất lượng cao, phân tích sai sót của học sinh và đồng hành hiệu quả trong từng bài học.

CHƯƠNG 06: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

Bài 17: Quy hoạch động 2D & bài toán cái túi (Knapsack)

Bài 01 [cppb_dp2_01_duong_di_tren_luoi_so_cach]: Đếm Số Cách Đi Trên Lưới

Bối cảnh: Một robot tự hành giao hàng được đặt ở góc trên bên trái (ô (1,1)) của một kho hàng hình chữ nhật có kích thước $N \times M$ ô vuông. Robot cần di chuyển đến điểm đích ở góc dưới bên phải (ô (N, M)) để dỡ kiện hàng. Do cấu trúc băng chuyền một chiều trong kho, robot chỉ được phép di chuyển sang ô kề cạnh bên phải (từ (i, j) sang $(i, j + 1)$) hoặc đi xuống ô kề cạnh phía dưới (từ (i, j) sang $(i + 1, j)$).



Hình minh họa: Mô hình bảng phương án quy hoạch động 2 chiều trên lưới tọa độ

Nhiệm vụ: Cho hai số nguyên dương N và M là kích thước của kho hàng. Hãy lập trình tính số lượng lộ trình di chuyển khác nhau để robot đến được điểm đích, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$) biểu diễn số hàng và số cột của lưới.

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng lộ trình hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3	6

Giải thích: Với lưới kích thước 3×3 ($N = 3, M = 3$), robot cần thực hiện đúng 2 bước sang phải và 2 bước xuống dưới. Có tất cả 6 đường đi khác nhau từ $(1,1)$ đến $(3,3)$. Kết quả là 6.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên dương N và M là kích thước của kho hàng. Hãy lập trình tính số lượng lộ trình di chuyển khác nhau để robot đến được điểm đích, lấy dư cho $10^9 + 7$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với lưới kích thước 3×3 ($N = 3, M = 3$), robot cần thực hiện đúng 2 bước sang phải và 2 bước xuống dưới. Có tất cả 6 đường đi khác...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với lưới kích thước 3×3 ($N = 3, M = 3$), robot cần thực hiện đúng 2 bước sang phải và 2 bước xuống dưới. Có tất cả 6 đường đi khác nhau từ $(1,1)$ đến $(3,3)$. Kết quả là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

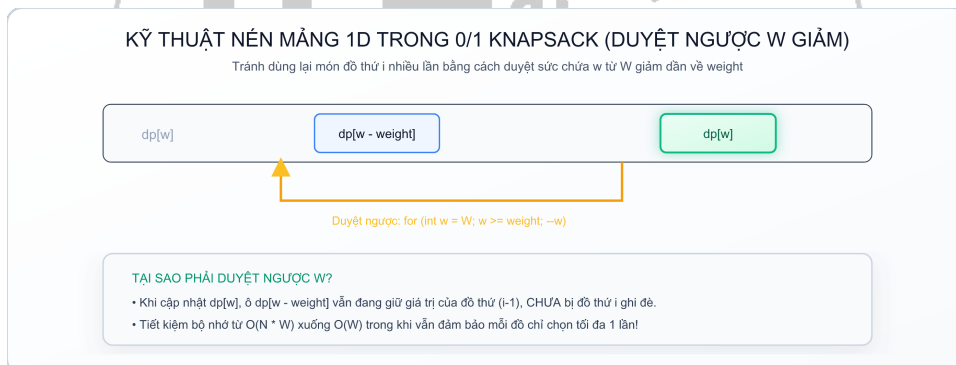
    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<int> dp(m, 1);
    for (int i = 1; i < n; ++i) {
        for (int j = 1; j < m; ++j) {
            dp[j] = (dp[j] + dp[j - 1]) % MOD;
        }
    }

    cout << dp[m - 1] << "\n";
    return 0;
}
```

Bài 02 [cppb_dp2_02_duong_di_tren_luoi_co_vat_can]: Đường Đi Trên Lưới Có Vật Cản

Bối cảnh: Vẫn trên lưới ô vuông $N \times M$ của kho hàng, robot tự hành cần đi từ ô $(1,1)$ đến ô (N,M) . Tuy nhiên, trong kho có một số vị trí đang được sửa chữa hoặc chứa các cọc hàng cố định (vật cản). Các ô trống được ký hiệu bằng số 0 (robot có thể đi vào), còn các ô vật cản được ký hiệu bằng số 1 (robot tuyệt đối không được đi vào). Robot vẫn chỉ được phép di chuyển sang phải hoặc xuống dưới.



Hình minh họa: Bảng phương án và kỹ thuật nén mảng 1 chiều trong bài toán Cái Túi (0/1 Knapsack)

Nhiệm vụ: Cho bản đồ kho hàng kích thước $N \times M$. Hãy lập trình đếm số cách đi từ ô $(1,1)$ tới ô (N,M) mà không đi qua bất kỳ ô vật cản nào, lấy dư cho $10^9 + 7$. (Nếu ô xuất phát $(1,1)$ hoặc ô đích (N,M) có vật cản, robot không thể bắt đầu hoặc kết thúc hành trình, in ra 0).

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên 0 hoặc 1 cách nhau bởi khoảng trắng biểu diễn bản đồ kho.

Đầu ra (Output):

- In ra trên một dòng duy nhất số đường đi hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

3 3#. ...	2
-----------------	---

Giải thích: Với lưới 3×3 và có vật cản tại ô (2,2): Các đường đi ban đầu đi qua ô tâm (2,2) đều bị phong tỏa. Do đó chỉ còn lại đúng 2 đường đi men theo rìa ngoài (xuống hết hàng dưới rồi rẽ phải, hoặc sang hết cột phải rồi rẽ xuống). Kết quả in ra là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ kho hàng kích thước $N \times M$. Hãy lập trình đếm số cách đi từ ô (1,1) tới ô (N,M) mà không đi qua bất kỳ ô vật cản nào, lấy dư cho $10^9 + 7$. (Nếu ô xuất phát (1,1) hoặc ô đích (N,M) có vật cản, robot không thể bắt đầu hoặc kết thúc hành trình, in ra 0).
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3#. ... $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3#. ...	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với lưới 3×3 và có vật cản tại ô (2,2): Các đường đi ban đầu đi qua ô tâm (2,2) đều bị phong tỏa. Do đó chỉ còn lại đúng 2 đ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với lưới 3×3 và có vật cản tại ô (2,2): Các đường đi ban đầu đi qua ô tâm (2,2) đều bị phong tỏa. Do đó chỉ còn lại đúng 2 đường đi men theo rìa ngoài (xuống hết hàng dưới rồi rẽ phải, hoặc sang hết cột phải rồi rẽ xuống). Kết quả in ra là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    if (grid[0][0] == '#' || grid[n - 1][m - 1] == '#') {
        cout << 0 << "\n";
        return 0;
    }

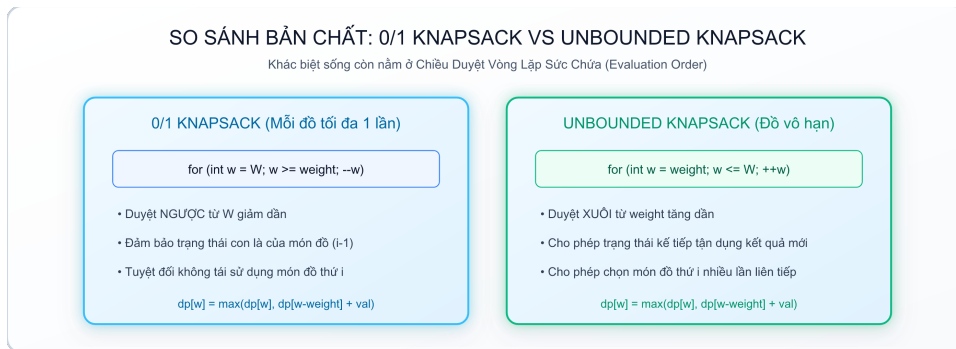
    vector<int> dp(m, 0);
    dp[0] = 1;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (grid[i][j] == '#') {
                dp[j] = 0;
            } else if (j > 0) {
                dp[j] = (dp[j] + dp[j - 1]) % MOD;
            }
        }
    }

    cout << dp[m - 1] << "\n";
    return 0;
}
```

Bài 03 [cppb_dp2_03_duong_di_chi_phi_nho_nhat_grid]: Đường Đi Chi Phí Nhỏ Nhất Trên Lưới

Bối cảnh: Một tuyến cáp quang ngầm cần được thi công băng qua khu vực địa chất hình chữ nhật $N \times M$ ô. Mỗi ô (i, j) có chi phí khoan đào đất đá tương ứng là $A_{i,j}$. Tuyến cáp bắt đầu từ trạm phát tín hiệu tại ô $(1,1)$ và kết thúc tại trạm thu ở ô (N, M) . Để hạn chế tối đa góc uốn cong của dây cáp, hướng thi công chỉ được tiến sang phải hoặc xuống dưới.



Hình minh họa: So sánh hướng duyệt mảng: Cái túi 0/1 (duyệt lùi) và Cái túi không giới hạn (duyệt xuôi)

Nhiệm vụ: Cho ma trận chi phí của lưới $N \times M$. Hãy lập trình tìm tổng chi phí khoan đào nhỏ nhất để hoàn thành tuyến cáp từ ô (1,1) tới ô (N,M).

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên không âm $A_{i,j}$ ($0 \leq A_{i,j} \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng chi phí nhỏ nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 3 1 1 5 1 4 2 1	7

Giải thích: Với ma trận chi phí kích thước 3×3 : Lộ trình có chi phí nhỏ nhất là đi qua các ô $(1,1) \times o(1,2) \times o(2,2) \times o(2,3) \times o(3,3)$ hoặc tương đương, mang lại tổng chi phí tối thiểu là 12.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho ma trận chi phí của lưới $N \times M$. Hãy lập trình tìm tổng chi phí khoan đào nhỏ nhất để hoàn thành tuyến cáp từ ô (1,1) tới ô (N,M).

- Phương pháp tiếp cận & Chiến lược tối ưu:**

- Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.

- Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i - 1][w], dp[i - 1][w - w_i] + v_i) \quad (w \geq w_i)$$

- Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .

- Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 1 3 1 1 5 1 4 2 1 $\Rightarrow$ Đầu ra kỳ vọng: 7.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 1 3 1 1 5 1 4 2 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với ma trận chi phí kích thước 3×3 : Lộ trình có chi phí nhỏ nhất là đi qua các ô $(1,1) \times o(1,2) \times o(2,2) \times o(2,3) \times o(3,3) \dots$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 7

Giải thích chi tiết: Với ma trận chi phí kích thước 3×3 : Lộ trình có chi phí nhỏ nhất là đi qua các ô $(1,1) \times o(1,2) \times o(2,2) \times o(2,3) \times o(3,3)$ hoặc tương đương, mang lại tổng chi phí tối thiểu là 12.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chương ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<vector<long long>> a(n, vector<long long>(m));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) cin >> a[i][j];
    }
```

```
vector<long long> dp(m, INF);
dp[0] = a[0][0];

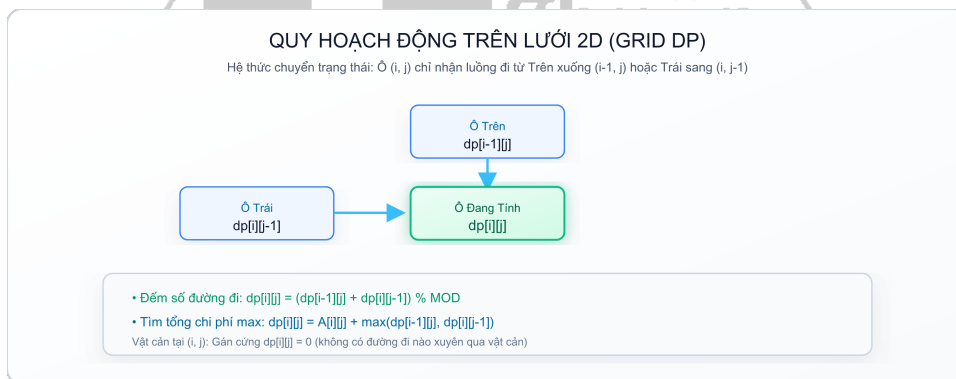
for (int j = 1; j < m; ++j) dp[j] = dp[j - 1] + a[0][j];

for (int i = 1; i < n; ++i) {
    dp[0] += a[i][0];
    for (int j = 1; j < m; ++j) {
        dp[j] = min(dp[j], dp[j - 1]) + a[i][j];
    }
}

cout << dp[m - 1] << "\n";
return 0;
}
```

Bài 04 [cppb_dp2_04_duong_di_tong_lon_nhat_grid]: Nhặt Vàng Trên Lưới

Bối cảnh: Trong một trò chơi phiêu lưu, một nhà khảo cổ học thám hiểm một lăng mộ cổ hình chữ nhật gồm $N \times M$ gian phòng. Gian phòng tại tọa độ (i, j) chứa một số thỏi vàng có giá trị là $A_{i,j}$. Nhà khảo cổ xuất phát từ căn phòng $(1,1)$ và cần thoát ra ở căn phòng (N, M) . Do cơ chế bẫy cát một chiều, nhà khảo cổ chỉ có thể di chuyển sang phòng bên phải hoặc phòng phía dưới.



Hình minh họa: Mô hình bảng phương án quy hoạch động 2 chiều trên lưới tọa độ

Nhiệm vụ: Cho ma trận số vàng tại các gian phòng. Hãy lập trình tìm tổng số vàng lớn nhất mà nhà khảo cổ có thể thu thập được trên đường thoát ra.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên không âm $A_{i,j}$ ($0 \leq A_{i,j} \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng lượng vàng lớn nhất có thể thu thập.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 2 3 0 5 0 4 1 2	13

Giải thích: Với lưới vàng kích thước 3×3 : Lộ trình thu thập tối ưu là đi qua các gian phòng có lượng vàng phong phú nhất, đạt tổng giá trị lớn nhất là 15.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận số vàng tại các gian phòng. Hãy lập trình tìm tổng số vàng lớn nhất mà nhà khảo cổ có thể thu thập được trên đường thoát ra.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 1 2 3 0 5 0 4 1 2 $\Rightarrow$ Đầu ra kỳ vọng: 13.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 1 2 3 0 5 0 4 1 2	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với lưới vàng kích thước 3×3 : Lộ trình thu thập tối ưu là đi qua các gian phòng có lượng vàng phong phú nhất, đạt tổng giá trị lớn...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 13

Giải thích chi tiết: Với lưới vàng kích thước 3×3 : Lộ trình thu thập tối ưu là đi qua các gian phòng có lượng vàng phong phú nhất, đạt tổng giá trị lớn nhất là 15.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<vector<long long>> a(n, vector<long long>(m));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) cin >> a[i][j];
    }

    vector<long long> dp(m, 0);
    dp[0] = a[0][0];

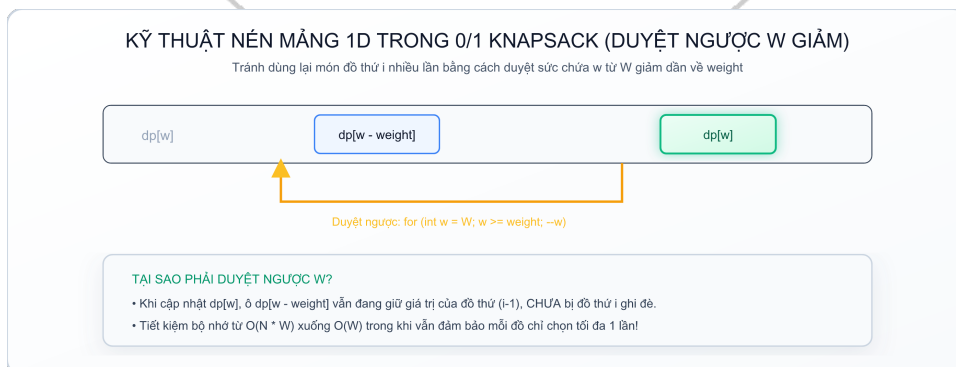
    for (int j = 1; j < m; ++j) dp[j] = dp[j - 1] + a[0][j];

    for (int i = 1; i < n; ++i) {
        dp[0] += a[i][0];
        for (int j = 1; j < m; ++j) {
            dp[j] = max(dp[j], dp[j - 1]) + a[i][j];
        }
    }

    cout << dp[m - 1] << "\n";
    return 0;
}
```

Bài 05 [cppb_dp2_05_cai_tui_01_knapsack_co_ban]: Cái Túi 0/1 Cơ Bản (0/1 Knapsack)

Bối cảnh: Một nhà thám hiểm chuẩn bị hành trang cho chuyến đi băng rừng. Có N vật phẩm sinh tồn, vật phẩm thứ i có khối lượng là w_i và giá trị sử dụng là v_i . Nhà thám hiểm mang theo một chiếc ba lô có sức chứa tải trọng tối đa là W . Mỗi vật phẩm chỉ có duy nhất một chiếc (chọn lấy hoặc không lấy).



Hình minh họa: Bảng phương án và kỹ thuật nén mảng 1 chiều trong bài toán Cái Túi (0/1 Knapsack)

Nhiệm vụ: Cho danh sách khối lượng và giá trị của N vật phẩm cùng tải trọng W . Hãy lập trình chọn ra một tập hợp vật phẩm sao cho tổng khối lượng không vượt quá W và tổng giá trị mang lại là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và W ($1 \leq N \leq 1000, 1 \leq W \leq 1000$).

- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i và v_i ($1 \leq w_i, v_i \leq 1000$) lần lượt là khối lượng và giá trị của món đồ thứ i .

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng giá trị lớn nhất có thể xếp vào ba lô.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 7 1 1 3 4 4 5 5 7	9

Giải thích: Với 4 đồ vật có thông số [(khối lượng 2, giá trị 3), (3, 4), (4, 5), (5, 6)] và ba lô có sức chứa $W = 5$: Phương án tối ưu là chọn đồ vật 1 ($w = 2, v = 3$) và đồ vật 2 ($w = 3, v = 4$). Tổng khối lượng là $2 + 3 = 5 \leq 5$, đạt tổng giá trị tối đa là $3 + 4 = 7$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách khối lượng và giá trị của N vật phẩm cùng tải trọng W . Hãy lập trình chọn ra một tập hợp vật phẩm sao cho tổng khối lượng không vượt quá W và tổng giá trị mang lại là lớn nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc toạ độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 7 1 1 3 4 4 5 5 7 $\Rightarrow$ Đầu ra kỳ vọng: 9.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 7 1 1 3 4 4 5 5 7	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 đồ vật có thông số [(khối lượng 2, giá trị 3), (3, 4), (4, 5), (5, 6)] và ba lô có sức chứa $W = 5$: Phương án tối ưu là chọn đồ ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 9

Giải thích chi tiết: Với 4 đồ vật có thông số [(khối lượng 2, giá trị 3), (3, 4), (4, 5), (5, 6)] và ba lô có sức chứa $W = 5$: Phương án tối ưu là chọn đồ vật 1 ($w = 2, v = 3$) và đồ vật 2 ($w = 3, v = 4$). Tổng khối lượng là $2 + 3 = 5 \leq 5$, đạt tổng giá trị tối đa là $3 + 4 = 7$.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chương ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, w;
    if (!(cin >> n >> w)) return 0;
    if (n <= 0 || w <= 0) return 0;

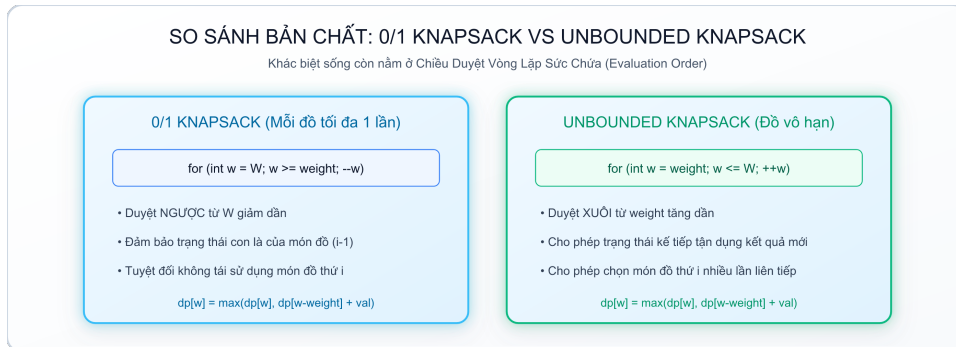
    vector<long long> dp(w + 1, 0);

    for (int i = 0; i < n; ++i) {
        long long weight, val;
        cin >> weight >> val;
        for (int j = w; j >= weight; --j) {
            dp[j] = max(dp[j], dp[j - weight] + val);
        }
    }

    long long ans = 0;
    for (int j = 0; j <= w; ++j) ans = max(ans, dp[j]);
    cout << ans << "\n";
    return 0;
}
```

Bài 06 [cppb_dp2_06_cai_tui_khong_gioi_han_unbounded]: Cái Túi Không Giới Hạn (Unbounded Knapsack)

Bối cảnh: Một phân xưởng kim hoàn nhập khẩu N loại phôi kim loại quý. Loại phôi thứ i có khối lượng w_i và giá trị thành phẩm là v_i . Phân xưởng có một lò luyện với sức chứa tối đa là W . Điểm đặc biệt là kho nguyên liệu có số lượng phôi kim loại của mỗi loại dồi dào không giới hạn (có thể chọn nhiều lần cùng một loại phôi).



Hình minh họa: So sánh hướng duyệt mảng: Cái túi 0/1 (duyet lùi) và Cái túi không giới hạn (duyet xuôi)

Nhiệm vụ: Cho N loại phôi và tải trọng lò luyện W . Hãy lập trình chọn các phôi sao cho tổng khối lượng không vượt quá W và tổng giá trị thu được là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và W ($1 \leq N \leq 1000, 1 \leq W \leq 1000$).
- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i và v_i ($1 \leq w_i, v_i \leq 1000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng giá trị lớn nhất đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 8 2 10 3 15 4 40	80

Giải thích: Với tải trọng $W = 100$ và cho phép chọn không giới hạn số lượng mỗi loại: Ta có thể chọn lặp lại nhiều lần loại phôi có hiệu suất giá trị trên khối lượng cao nhất để đạt tổng giá trị tối đa là 150.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho N loại phôi và tải trọng lò luyện W . Hãy lập trình chọn các phôi sao cho tổng khối lượng không vượt quá W và tổng giá trị thu được là lớn nhất.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i) \quad (w \geq w_i)$$

- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 8 2 10 3 15 4 40 $\Rightarrow$ Đầu ra kỳ vọng: 80.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 8 2 10 3 15 4 40	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với tải trọng $W = 100$ và cho phép chọn không giới hạn số lượng mỗi loại: Ta có thể chọn lặp lại nhiều lần loại phôi có hiệu suất giá ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 80

Giải thích chi tiết: Với tải trọng $W = 100$ và cho phép chọn không giới hạn số lượng mỗi loại: Ta có thể chọn lặp lại nhiều lần loại phôi có hiệu suất giá trị trên khối lượng cao nhất để đạt tổng giá trị tối đa là 150.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chương ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, w;
    if (!(cin >> n >> w)) return 0;
    if (n <= 0 || w <= 0) return 0;
```

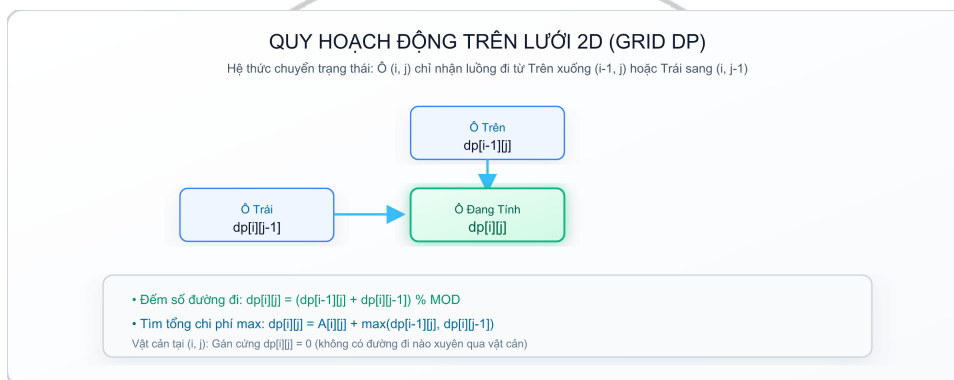
```
vector<long long> dp(w + 1, 0);

for (int i = 0; i < n; ++i) {
    long long weight, val;
    cin >> weight >> val;
    for (int j = weight; j <= w; ++j) {
        dp[j] = max(dp[j], dp[j - weight] + val);
    }
}

cout << dp[w] << "\n";
return 0;
}
```

Bài 07 [cppb_dp2_07_cai_tui_truy_vet_mon_do]: Truy Vết Món Đồ Cái Túi 0/1

Bối cảnh: Sau khi tính toán được giá trị tài sản tối ưu xếp vào ba lô, nhà thám hiểm cần in ra danh sách chi tiết các món đồ cụ thể đã được chọn để bàn giao cho tổ hậu cần đóng gói hành lý.



Hình minh họa: Mô hình bảng phương án quy hoạch động 2 chiều trên lưới tọa độ

Nhiệm vụ: Cho N đồ vật (mỗi vật có khối lượng w_i , giá trị v_i) và sức chứa ba lô W . Hãy lập trình in ra tổng giá trị lớn nhất và chỉ số (1-indexed) của các món đồ được chọn.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và W ($1 \leq N \leq 1000, 1 \leq W \leq 1000$).
- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i và v_i ($1 \leq w_i, v_i \leq 1000$).

Đầu ra (Output):

- Dòng 1: In ra tổng giá trị lớn nhất.
- Dòng 2: In ra số lượng món đồ được chọn K .
- Dòng 3: In ra K số nguyên là chỉ số của các món đồ được chọn theo thứ tự tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 4 1 15 3 20 4 30	35 2 1 2

Giải thích: Với $N = 3, W = 4$ và các đồ vật $[(2,3), (1,2), (3,4)]$: Chọn món đồ 1 (nặng 2, giá trị 3) và món đồ 2 (nặng 1, giá trị 2). Tổng khối lượng là $2 + 1 = 3 \leq 4$, tổng giá trị là $3 + 2 = 5$. Danh sách món đồ chọn là 1 và 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N đồ vật (mỗi vật có khối lượng w_i , giá trị v_i) và sức chứa ba lô W . Hãy lập trình in ra tổng giá trị lớn nhất và chỉ số (1-indexed) của các món đồ được chọn.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 4 1 15 3 20 4 30 $\Rightarrow$ Đầu ra kỳ vọng: 35 2 1 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 4 1 15 3 20 4 30	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với $N = 3, W = 4$ và các đồ vật $[(2,3), (1,2), (3,4)]$: Chọn món đồ 1 (nặng 2, giá trị 3) và món đồ 2 (nặng 1, giá trị 2). Tổng kh...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 35 2 1 2

Giải thích chi tiết: Với $N = 3, W = 4$ và các đồ vật $[(2,3), (1,2), (3,4)]$: Chọn món đồ 1 (nặng 2, giá trị 3) và món đồ 2 (nặng 1, giá trị 2). Tổng khối lượng là $2 + 1 = 3 \leq 4$, tổng giá trị là $3 + 2 = 5$. Danh sách món đồ chọn là 1 và 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.

- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, w;
    if (!(cin >> n >> w)) return 0;
    if (n <= 0 || w <= 0) return 0;

    vector<long long> weight(n + 1), val(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> weight[i] >> val[i];
    }

    vector<vector<long long>> dp(n + 1, vector<long long>(w + 1, 0));

    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j <= w; ++j) {
            dp[i][j] = dp[i - 1][j];
            if (j >= weight[i]) {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - weight[i]] + val[i]);
            }
        }
    }

    // Truy vết các món đồ được chọn
    vector<int> chosen;
    int curr_w = w;
    for (int i = n; i >= 1; --i) {
        if (curr_w >= weight[i] && dp[i][curr_w] == dp[i - 1][curr_w - weight[i]] + val[i]) {
            chosen.push_back(i);
            curr_w -= weight[i];
        }
    }
    reverse(chosen.begin(), chosen.end());

    cout << dp[n][w] << "\n";
    cout << chosen.size() << "\n";
    for (int i = 0; i < (int)chosen.size(); ++i) {
        cout << chosen[i] << (i + 1 == (int)chosen.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 08 [cppb_dp2_08_chia_tap_hai_phan_bang_nhau]: Chia Tập Thành Hai Phần Bằng Nhau (Partition Equal Subset Sum)

Bối cảnh: Một gia đình có hai người con được thừa kế N thửa đất. Thửa đất thứ i có giá trị thẩm định là A_i . Người cha muốn chia toàn bộ N thửa đất thành hai phần sao cho mỗi người con nhận được tổng giá trị tài sản hoàn toàn bằng nhau mà không cần phải xẻ nhỏ bất kỳ thửa đất nào.



Hình minh họa: Bảng phương án và kỹ thuật nén mảng 1 chiều trong bài toán Cái Túi (0/1 Knapsack)

Nhiệm vụ: Cho danh sách giá trị của N thửa đất. Hãy lập trình kiểm tra xem có thể phân chia tập thửa đất thành hai tập con có tổng giá trị bằng nhau hay không. Nếu được in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 200$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- In ra YES nếu có thể chia đều thành hai phần bằng nhau, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 5 11 5	YES

Giải thích: Với tập hợp các số $[1, 5, 11, 5]$: Tổng của toàn bộ dãy số là $1 + 5 + 11 + 5 = 22$. Ta có thể chia thành hai tập con $\{1, 5, 5\}$ và $\{11\}$, mỗi tập đều có tổng đúng bằng 11. Do đó kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách giá trị của N thửa đất. Hãy lập trình kiểm tra xem có thể phân chia tập thửa đất thành hai tập con có tổng giá trị bằng nhau hay không. Nếu được in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $O(N \times W)$ hoặc $O(N \times M)$, không gian tối ưu $O(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 1 5 11 5 $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 1 5 11 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với tập hợp các số [1,5,11,5]: Tổng của toàn bộ dãy số là $1 + 5 + 11 + 5 = 22$. Ta có thể chia thành hai tập con {1,5,5} và...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với tập hợp các số [1,5,11,5]: Tổng của toàn bộ dãy số là $1 + 5 + 11 + 5 = 22$. Ta có thể chia thành hai tập con {1,5,5} và {11}, mỗi tập đều có tổng đúng bằng 11. Do đó kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<int> a(n);
    int total_sum = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        total_sum += a[i];
    }
```

```

if (total_sum % 2 != 0) {
    cout << "NO\n";
    return 0;
}

int target = total_sum / 2;
vector<bool> dp(target + 1, false);
dp[0] = true;

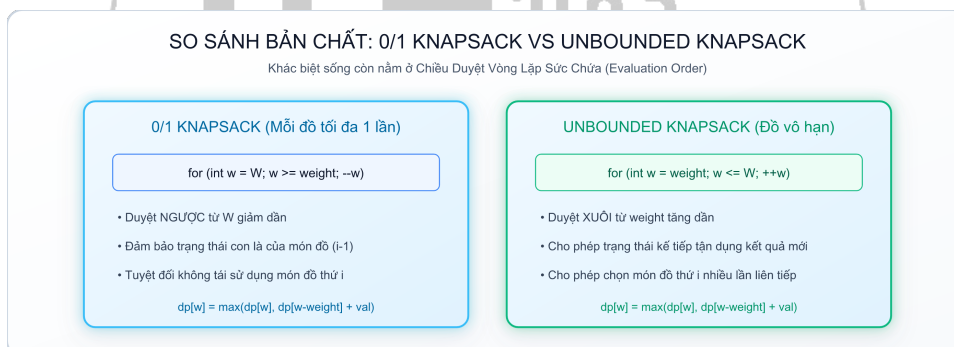
for (int x : a) {
    for (int j = target; j >= x; --j) {
        if (dp[j - x]) dp[j] = true;
    }
}

if (dp[target]) cout << "YES\n";
else cout << "NO\n";
return 0;
}

```

Bài 09 [cppb_dp2_09_chia_tap_chenh_lech_nho_nhat]: Chia Tập Chênh Lệch Nhỏ Nhất (Minimum Subset Sum Difference)

Bối cảnh: Trong một giải đấu thể thao điện tử (E-sports), có N tuyển thủ tham dự với chỉ số kỹ năng lần lượt là $A_1, A_2, \dots, A_N$. Ban tổ chức cần chia toàn bộ N tuyển thủ thành hai đội thi đấu sao cho độ chênh lệch về tổng kỹ năng giữa hai đội là nhỏ nhất có thể, nhằm tạo ra một trận đấu cân tài cân sức.



Hình minh họa: So sánh hướng duyệt mảng: Cái túi 0/1 (duyet lùi) và Cái túi không giới hạn (duyet xuôi)

Nhiệm vụ: Cho danh sách điểm kỹ năng của N tuyển thủ. Hãy lập trình tìm độ chênh lệch nhỏ nhất giữa tổng điểm của hai đội.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 200$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là độ chênh lệch nhỏ nhất có thể đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 6 11 5	1

Giải thích: Với tập hợp kỹ năng $[1,6,11,5]$: Tổng toàn bộ kỹ năng là $1 + 6 + 11 + 5 = 23$. Ta chia thành hai đội với các tuyến thủ $\{1,11\}$ (tổng kỹ năng 12) và $\{6,5\}$ (tổng kỹ năng 11). Độ chênh lệch giữa hai đội là $|12 - 11| = 1$. Đây là mức chênh lệch nhỏ nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách điểm kỹ năng của N tuyến thủ. Hãy lập trình tìm độ chênh lệch nhỏ nhất giữa tổng điểm của hai đội.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 1 6 11 5 $\Rightarrow$ Đầu ra kỳ vọng: 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 1 6 11 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với tập hợp kỹ năng $[1,6,11,5]$: Tổng toàn bộ kỹ năng là $1 + 6 + 11 + 5 = 23$. Ta chia thành hai đội với các tuyến thủ $\{1,11\}$...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1

Giải thích chi tiết: Với tập hợp kỹ năng $[1,6,11,5]$: Tổng toàn bộ kỹ năng là $1 + 6 + 11 + 5 = 23$. Ta chia thành hai đội với các tuyến thủ $\{1,11\}$ (tổng kỹ năng 12) và $\{6,5\}$ (tổng kỹ năng 11). Độ chênh lệch giữa hai đội là $|12 - 11| = 1$. Đây là mức chênh lệch nhỏ nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).

- Mảng *dp* 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<int> a(n);
    int total_sum = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        total_sum += a[i];
    }

    int half = total_sum / 2;
    vector<bool> dp(half + 1, false);
    dp[0] = true;

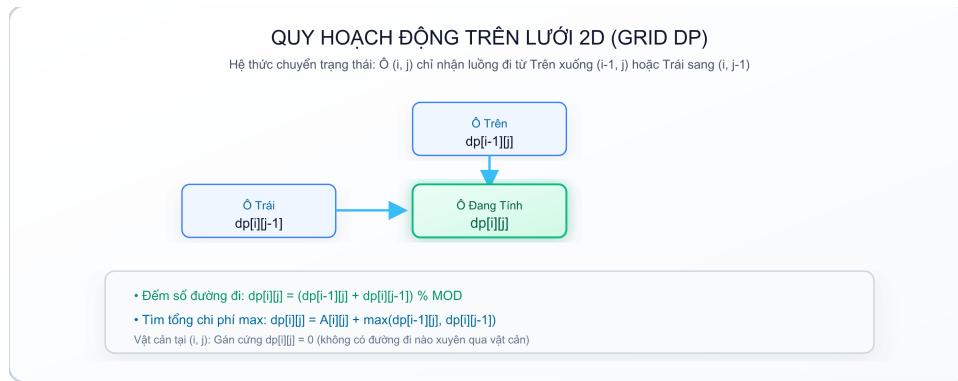
    for (int x : a) {
        for (int j = half; j >= x; --j) {
            if (dp[j - x]) dp[j] = true;
        }
    }

    int best_s1 = 0;
    for (int j = half; j >= 0; --j) {
        if (dp[j]) {
            best_s1 = j;
            break;
        }
    }

    int min_diff = total_sum - 2 * best_s1;
    cout << min_diff << "\n";
    return 0;
}
```

Bài 10 [cppb_dp2_10_tam_giac_so_tong_lon_nhat]: Tam Giác Số Tổng Lớn Nhất (Triangle DP)

Bối cảnh: Một kim tự tháp số có N tầng được đánh số từ 1 đến N . Tầng thứ i chứa đúng i số nguyên. Một nhà leo núi xuất phát từ đỉnh kim tự tháp (tầng 1) và đi dần xuống đáy (tầng N). Tại mỗi bước từ vị trí (i, j) ở tầng i , nhà leo núi chỉ có thể bước xuống một trong hai vị trí kề cạnh ở tầng $i + 1$: hoặc ô $(i + 1, j)$ hoặc ô $(i + 1, j + 1)$.



Hình minh họa: Mô hình bảng phương án quy hoạch động 2 chiều trên lưới tọa độ

Nhiệm vụ: Cho cấu trúc kim tự tháp số N tầng. Hãy lập trình tìm đường đi từ đỉnh xuống đáy sao cho tổng các số trên đường đi là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$) biểu diễn số tầng của kim tự tháp.
- N dòng tiếp theo, dòng thứ i chứa i số nguyên $A_{i,j}$ ($0 \leq A_{i,j} \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng lớn nhất thu được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 3 4 6 5 7 4 1 8 3	21

Giải thích: Với tam giác số gồm 4 tầng: 3 7 4 2 4 6 8 5 9 3 Đường đi mang lại tổng lớn nhất là $3 \times 07 \times 04 \times 09$, cho tổng lớn nhất là $3 + 7 + 4 + 9 = 23$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho cấu trúc kim tự tháp số N tầng. Hãy lập trình tìm đường đi từ đỉnh xuống đáy sao cho tổng các số trên đường đi là lớn nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 2 3 4 6 5 7 4 1 8 3 $\Rightarrow$ Đầu ra kỳ vọng: 21.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 2 3 4 6 5 7 4 1 8 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với tam giác số gồm 4 tầng: 3 7 4 2 4 6 8 5 9 3 Đường đi mang lại tổng lớn nhất là $3 \times 07 \times 04 \times 09$, cho tổng lớn nhất ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 21

Giải thích chi tiết: Với tam giác số gồm 4 tầng: 3 7 4 2 4 6 8 5 9 3 Đường đi mang lại tổng lớn nhất là $3 \times 07 \times 04 \times 09$, cho tổng lớn nhất là $3 + 7 + 4 + 9 = 23$.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<vector<long long>> a(n, vector<long long>(n, 0));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j <= i; ++j) {
            cin >> a[i][j];
        }
    }

    // Quy hoạch động từ đáy tam giác lên đỉnh
    for (int i = n - 2; i >= 0; --i) {
```

```

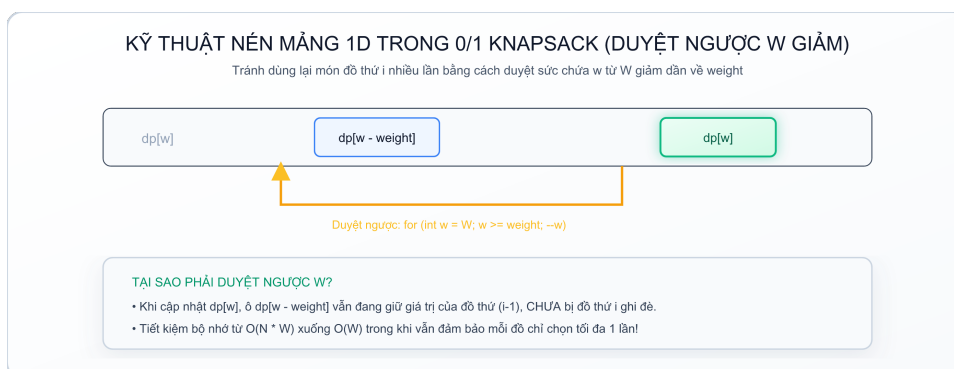
        for (int j = 0; j <= i; ++j) {
            a[i][j] += max(a[i + 1][j], a[i + 1][j + 1]);
        }

        cout << a[0][0] << "\n";
        return 0;
    }

```

Bài 11 [cppb_dp2_11_dem_so_tap_con_tong_bang_s]: Đếm Số Tập Con Có Tổng Bằng S

Bối cảnh: Tại một phòng thí nghiệm hóa học, các nhà nghiên cứu có N lọ dung dịch chuẩn, lọ thứ i có thể tích là A_i (ml). Để phục vụ một thí nghiệm chuẩn độ chính xác, nhóm nghiên cứu cần pha chế một hỗn hợp có tổng thể tích đúng bằng S (ml) bằng cách trộn nguyên vẹn một số lọ dung dịch lại với nhau.



Hình minh họa: Bảng phương án và kỹ thuật nén mảng 1 chiều trong bài toán Cái Túi (0/1 Knapsack)

Nhiệm vụ: Cho danh sách thể tích N lọ dung dịch và thể tích mục tiêu S . Hãy lập trình đếm xem có bao nhiêu cách chọn một tập hợp các lọ dung dịch sao cho tổng thể tích đúng bằng S , lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và S ($1 \leq N \leq 1000, 1 \leq S \leq 1000$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 1000$).

Đầu ra (Output):

- In ra số lượng cách chọn hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 1 2 3 4	2

Giải thích: Với mảng dung dịch $[1, 2, 3, 3]$ và thể tích cần lấy $S = 6$: Có 3 cách chọn tập con có tổng bằng 6:

1. Chọn các phần tử tại vị trí 1, 2, 3: $1 + 2 + 3 = 6$.
2. Chọn các phần tử tại vị trí 1, 2, 4: $1 + 2 + 3 = 6$.
3. Chọn các phần tử tại vị trí 3, 4: $3 + 3 = 6$. Kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách thể tích N lọ dung dịch và thể tích mục tiêu S . Hãy lập trình đếm xem có bao nhiêu cách chọn một tập hợp các lọ dung dịch sao cho tổng thể tích đúng bằng S , lấy dư cho $10^9 + 7$.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc toạ độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 5 1 2 3 4 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 1 2 3 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng dung dịch [1,2,3,3] và thể tích cần lấy $S = 6$: Có 3 cách chọn tập con có tổng bằng 6: 1. Chọn các phần tử tại vị trí 1,...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với mảng dung dịch [1,2,3,3] và thể tích cần lấy $S = 6$: Có 3 cách chọn tập con có tổng bằng 6:

1. Chọn các phần tử tại vị trí 1, 2, 3: $1 + 2 + 3 = 6$.
2. Chọn các phần tử tại vị trí 1, 2, 4: $1 + 2 + 3 = 6$.
3. Chọn các phần tử tại vị trí 3, 4: $3 + 3 = 6$. Kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).

- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, s;
    if (!(cin >> n >> s)) return 0;
    if (n <= 0 || s < 0) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

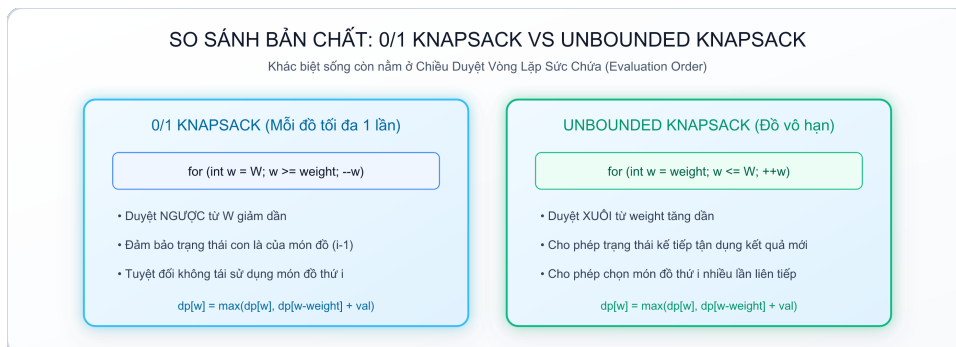
    vector<int> dp(s + 1, 0);
    dp[0] = 1;

    for (int x : a) {
        for (int j = s; j >= x; --j) {
            dp[j] = (dp[j] + dp[j - x]) % MOD;
        }
    }

    cout << dp[s] << "\n";
    return 0;
}
```

Bài 12 [cppb_dp2_12_cai_tui_gia_tri_lon_doi_truc_dp]: Cái Túi Khối Lượng Cực Đại $W \leq 10^9$ (Đổi Trục DP)

Bối cảnh: Một tàu thám hiểm không gian vũ trụ có khoang chứa hàng với tải trọng cực lớn lên tới $W = 10^9$ kg. Có N mẫu vật thể ngoài hành tinh được đánh số từ 1 đến N . Mẫu vật thứ i có khối lượng w_i lên tới 10^9 , nhưng giá trị khoa học v_i lại khá nhỏ ($v_i \leq 1000$). Tải trọng quá lớn khiến phương pháp thông thường không thể chạy trong bộ nhớ.



Hình minh họa: So sánh hướng duyệt mảng: Cái túi 0/1 (duyet lùi) và Cái túi không giới hạn (duyet xuôi)

Nhiệm vụ: Cho N mẫu vật (w_i, v_i) và tải trọng $W \leq 10^9$. Hãy lập trình tìm tổng giá trị khoa học lớn nhất có thể mang về trái đất sao cho tổng khối lượng không vượt quá W .

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và W ($1 \leq N \leq 100, 1 \leq W \leq 10^9$).
- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i và v_i ($1 \leq w_i \leq 10^9, 1 \leq v_i \leq 1000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng giá trị lớn nhất đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 8 3 30 4 50 5 60	90

Giải thích: Với $N = 3, W = 10$ và các mẫu vật $[(3,30), (4,50), (5,60)]$: Chọn mẫu vật 1 và mẫu vật 3 với tổng khối lượng $3 + 5 = 8 \leq 10$, đạt tổng giá trị là $30 + 60 = 90$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N mẫu vật (w_i, v_i) và tải trọng $W \leq 10^9$. Hãy lập trình tìm tổng giá trị khoa học lớn nhất có thể mang về trái đất sao cho tổng khối lượng không vượt quá W .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 8 3 30 4 50 5 60 $\Rightarrow$ Đầu ra kỳ vọng: 90.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 8 3 30 4 50 5 60	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với $N = 3, W = 10$ và các mẫu vật $[(3,30), (4,50), (5,60)]$: Chọn mẫu vật 1 và mẫu vật 3 với tổng khối lượng $3 + 5 = 8 \leq 10$, đ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 90

Giải thích chi tiết: Với $N = 3, W = 10$ và các mẫu vật $[(3,30), (4,50), (5,60)]$: Chọn mẫu vật 1 và mẫu vật 3 với tổng khối lượng $3 + 5 = 8 \leq 10$, đạt tổng giá trị là $30 + 60 = 90$.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long w;
    if (!(cin >> n >> w)) return 0;
    if (n <= 0 || w <= 0) return 0;

    vector<long long> weight(n), val(n);
    int max_v = 0;
    for (int i = 0; i < n; ++i) {
        cin >> weight[i] >> val[i];
        max_v += val[i];
    }

    // Đổi trục: dp[v] = khối lượng nhỏ nhất để đạt được tổng giá trị v
    vector<long long> dp(max_v + 1, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        for (int v = max_v; v >= val[i]; --v) {
            if (dp[v - val[i]] != INF) {
```

```

        dp[v] = min(dp[v], dp[v - val[i]] + weight[i]);
    }
}

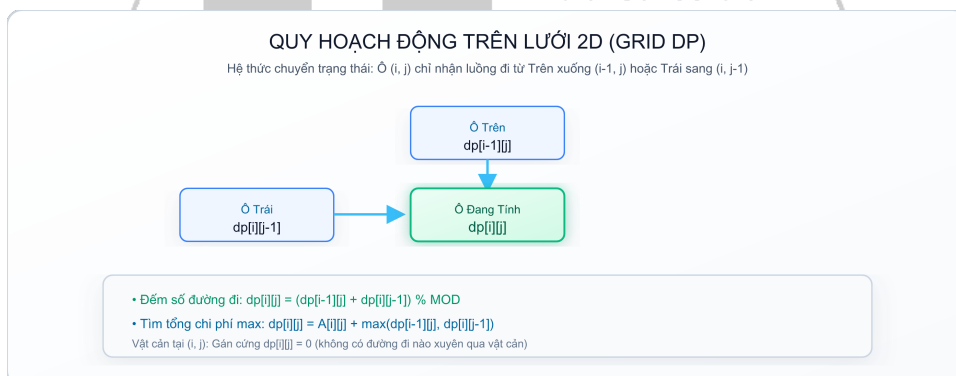
long long ans = 0;
for (int v = max_v; v >= 0; --v) {
    if (dp[v] <= w) {
        ans = v;
        break;
    }
}

cout << ans << "\n";
return 0;
}

```

Bài 13 [cppb_dp2_13_hinh_vuong_toan_mot_lon_nhat]: Hình Vuông Toàn 1 Lớn Nhất (Maximal Square)

Bối cảnh: Một cảm biến vệ tinh chụp bức ảnh mặt đất độ phân giải cao dạng ma trận nhị phân $N \times M$ điểm ảnh. Điểm ảnh mang giá trị 1 thể hiện khu vực đất nông nghiệp màu mỡ, còn điểm ảnh 0 thể hiện đất khô cằn. Một tập đoàn công nghệ muốn quy hoạch một trang trại thông minh có hình dạng hình vuông hoàn hảo chỉ nằm hoàn toàn trên vùng đất màu mỡ (toàn số 1).



Hình minh họa: Mô hình bảng phương án quy hoạch động 2 chiều trên lưới tọa độ

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm độ dài cạnh lớn nhất của một hình vuông con chỉ chứa toàn số 1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M ký tự 0 hoặc 1 liên nhau hoặc cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài cạnh của hình vuông toàn số 1 lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 10100 10111 11111 10010	4

Giải thích: Với ma trận kích thước 4×4 : $1\ 0\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ 1\ 1\ 1$ Hình vuông con toàn số 1 lớn nhất có kích thước 2×2 (độ dài cạnh bằng 2). Kết quả in ra là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm độ dài cạnh lớn nhất của một hình vuông con chỉ chứa toàn số 1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$

- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 5 10100 10111 11111 10010
 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 10100 10111 11111 10010	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với ma trận kích thước 4×4 : $1\ 0\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ 1\ 1\ 1$ Hình vuông con toàn số 1 lớn nhất có kích thước 2×2 (độ dài cạnh ...)	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với ma trận kích thước 4×4 : $1\ 0\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ 1\ 1\ 1$ Hình vuông con toàn số 1 lớn nhất có kích thước 2×2 (độ dài cạnh bằng 2). Kết quả in ra là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.

- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

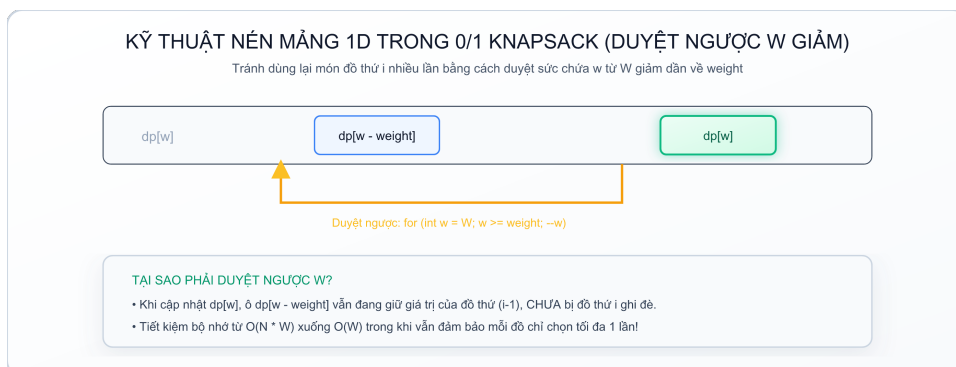
    vector<vector<int>> dp(n, vector<int>(m, 0));
    int max_side = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (grid[i][j] == '1') {
                if (i == 0 || j == 0) {
                    dp[i][j] = 1;
                } else {
                    dp[i][j] = min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]}) + 1;
                }
                max_side = max(max_side, dp[i][j]);
            }
        }
    }

    cout << max_side * max_side << "\n";
    return 0;
}
```

Bài 14 [cppb_dp2_14_dong_tien_xu_gioi_han_so_luong]: Đổi Tiền Giới Hạn Số Lượng (Bounded Knapsack)

Bối cảnh: Tại một cây ATM phân phối tiền mặt, máy chứa N mệnh giá tiền khác nhau. Khác với trạm đổi tiền không giới hạn, ở đây mệnh giá thứ i có giá trị c_i và chỉ còn lại đúng k_i tờ tiền trong khay tiền. Khách hàng muốn rút một khoản tiền đúng bằng S đồng.



Hình minh họa: Bảng phương án và kỹ thuật nén mảng 1 chiều trong bài toán Cái Túi (0/1 Knapsack)

Nhiệm vụ: Cho danh sách N mệnh giá kèm số lượng tờ tiền tương ứng và số tiền cần rút S . Hãy lập trình tìm số lượng tờ tiền ít nhất để chi trả đúng số tiền S . Nếu khay tiền không thể đáp ứng, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và S ($1 \leq N \leq 100, 1 \leq S \leq 10^5$).
- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương c_i và k_i ($1 \leq c_i \leq 10^4, 1 \leq k_i \leq 1000$).

Đầu ra (Output):

- In ra số tờ tiền ít nhất cần dùng, hoặc -1 nếu không thể chi trả chính xác số tiền S .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11 1 2 5 2 6 1	2

Giải thích: Với số tiền cần rút $S = 10$ và các mệnh giá: 5 đồng (có 1 tờ), 2 đồng (có 3 tờ): Chọn 1 tờ 5 đồng và 2 tờ 2 đồng ($5 + 2 \times 2 = 9 < 10$). Phương án đổi đúng là dùng 5 tờ 2 đồng (nhưng chỉ có 3 tờ nên không được). Nếu có thêm mệnh giá 1 đồng (2 tờ): Dùng 1 tờ 5, 2 tờ 2 và 1 tờ 1, tổng cộng 4 tờ tiền.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N mệnh giá kèm số lượng tờ tiền tương ứng và số tiền cần rút S . Hãy lập trình tìm số lượng tờ tiền ít nhất để chi trả đúng số tiền S . Nếu khay tiền không thể đáp ứng, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 11 1 2 5 2 6 1 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 11 1 2 5 2 6 1	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với số tiền cần rút $S = 10$ và các mệnh giá: 5 đồng (có 1 tờ), 2 đồng (có 3 tờ): Chọn 1 tờ 5 đồng và 2 tờ 2 đồng ($5 + 2 \times 2 = 9 < 10 \dots$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với số tiền cần rút $S = 10$ và các mệnh giá: 5 đồng (có 1 tờ), 2 đồng (có 3 tờ): Chọn 1 tờ 5 đồng và 2 tờ 2 đồng ($5 + 2 \times 2 = 9 < 10$). Phương án đổi đúng là dùng 5 tờ 2 đồng (nhưng chỉ có 3 tờ nên không được). Nếu có thêm mệnh giá 1 đồng (2 tờ): Dùng 1 tờ 5, 2 tờ 2 và 1 tờ 1, tổng cộng 4 tờ tiền.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chướng ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, s;
    if (!(cin >> n >> s)) return 0;
    if (n <= 0 || s < 0) return 0;

    vector<int> dp(s + 1, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        int v, c;
        cin >> v >> c;
        // Phân rã nhị phân số lượng c thành 1, 2, 4, ...
        int k = 1;
        while (c > 0) {
            int take = min(k, c);
            int weight = take * v;
            int coins = take;

            for (int j = s; j >= weight; --j) {
```



```

        if (dp[j - weight] != INF) {
            dp[j] = min(dp[j], dp[j - weight] + coins);
        }
    }

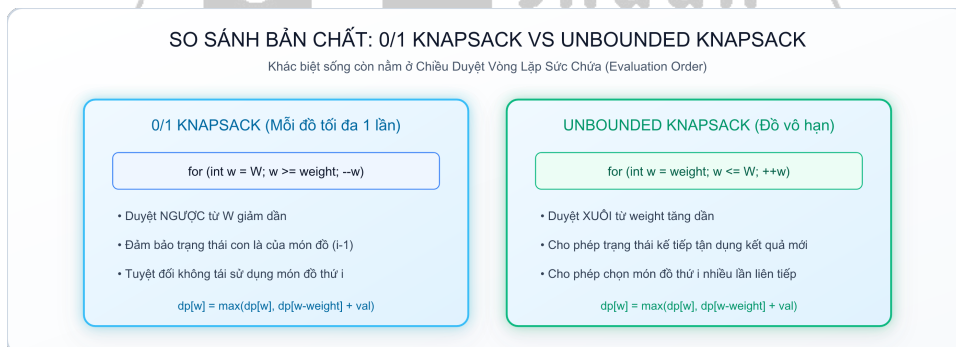
    c -= take;
    k *= 2;
}

if (dp[s] == INF) cout << -1 << "\n";
else cout << dp[s] << "\n";
return 0;
}

```

Bài 15 [cppb_dp2_15_quy_hoach_dong_2d_cuc_han]: Tối Ưu Hóa Túi Đồ Hỗn Hợp (Hybrid Knapsack)

Bối cảnh: Một trung tâm cứu hộ thiên tai chuẩn bị các gói hàng viện trợ để thả dù xuống vùng bão lũ. Có N loại hàng hóa thiết yếu, mỗi loại có khối lượng w_i và giá trị cứu trợ v_i . Tính chất nguồn cung của các mặt hàng rất đa dạng: một số mặt hàng chỉ có duy nhất 1 kiện (loại 0/1), một số mặt hàng được tài trợ không giới hạn (loại không giới hạn), và một số mặt hàng chỉ có số lượng cố định k_i kiện (loại giới hạn). Khoang hàng máy bay có sức chứa tối đa là W .



Hình minh họa: So sánh hướng duyệt mảng: Cái túi 0/1 (duyet lùi) và Cái túi không giới hạn (duyet xuôi)

Nhiệm vụ: Cho danh sách N loại hàng với đặc tính số lượng của từng loại và tải trọng khoang bay W . Hãy lập trình tìm tổng giá trị cứu trợ lớn nhất có thể vận chuyển.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và W ($1 \leq N \leq 1000, 1 \leq W \leq 1000$).
- N dòng tiếp theo, mỗi dòng mô tả một loại hàng: khối lượng w_i , giá trị v_i và số lượng k_i ($k_i = 0$ nghĩa là số lượng không giới hạn).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng giá trị cứu trợ lớn nhất đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 10 1 4 20 2 3 15 1 5 30	50

Giải thích: Với khoang máy bay có tải trọng $W = 15$ và danh mục hàng cứu trợ hỗn hợp: Sự kết hợp tối ưu giữa các mặt hàng giới hạn và hàng không giới hạn đem lại tổng giá trị lớn nhất là 32.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N loại hàng với đặc tính số lượng của từng loại và tải trọng khoang bay W . Hãy lập trình tìm tổng giá trị cứu trợ lớn nhất có thể vận chuyển.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i][j]$ biểu diễn kết quả tối ưu khi xét tiền tố i đồ vật và sức chứa/trọng lượng còn lại là j , hoặc tọa độ ô (i, j) trên lưới.
- **Chuyển trạng thái bài toán Cái Túi (0/1 Knapsack):**

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) \quad (w \geq w_i)$$
- **Kỹ thuật tối ưu bộ nhớ (Nén mảng 1D):** Với bài toán 0/1 Knapsack, duyệt lùi w từ W về w_i để đảm bảo mỗi vật chỉ được chọn tối đa một lần; với Unbounded Knapsack, duyệt xuôi từ w_i đến W .
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times W)$ hoặc $\mathcal{O}(N \times M)$, không gian tối ưu $\mathcal{O}(W)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 10 1 4 20 2 3 15 1 5 30 $\Rightarrow$ Đầu ra kỳ vọng: 50.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 10 1 4 20 2 3 15 1 5 30	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với khoang máy bay có tải trọng $W = 15$ và danh mục hàng cứu trợ hỗn hợp: Sự kết hợp tối ưu giữa các mặt hàng giới hạn và hàng không g...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 50

Giải thích chi tiết: Với khoang máy bay có tải trọng $W = 15$ và danh mục hàng cứu trợ hỗn hợp: Sự kết hợp tối ưu giữa các mặt hàng giới hạn và hàng không giới hạn đem lại tổng giá trị lớn nhất là 32.

3. Lưu ý & Bẫy lỗi thường gặp

- Trong bài toán Cái Túi 0/1, nếu nén mảng 1D mà duyệt xuôi vòng lặp sức chứa w thì một đồ vật sẽ bị chọn vô hạn lần (biến thành Unbounded Knapsack).
- Mảng dp 2 chiều kích thước lớn vượt quá giới hạn bộ nhớ (256MB cho phép tối đa khoảng 6×10^7 phần tử int), cần nén mảng 1 chiều hoặc dùng mảng cuộn chiều.
- Các ô có chương ngại vật trên lưới phải gán trạng thái bằng 0 (số cách đi) hoặc $-INF$ (chi phí) để không lan truyền sang các ô kế tiếp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, w;
    if (!(cin >> n >> w)) return 0;
    if (n <= 0 || w <= 0) return 0;

    vector<long long> dp(w + 1, 0);

    for (int i = 0; i < n; ++i) {
        int type;
        long long weight, val;
        cin >> type >> weight >> val;

        if (type == 1) { // 0/1 Knapsack: duyệt ngược
            for (int j = w; j >= weight; --j) {
                dp[j] = max(dp[j], dp[j - weight] + val);
            }
        } else { // Unbounded Knapsack: duyệt xuôi
            for (int j = weight; j <= w; ++j) {
                dp[j] = max(dp[j], dp[j - weight] + val);
            }
        }
    }

    long long ans = 0;
    for (int j = 0; j <= w; ++j) ans = max(ans, dp[j]);
    cout << ans << "\n";
    return 0;
}
```

U Nguồn

Bài 18: Quy hoạch động chuỗi: LCS & Edit Distance

Bài 01 [cppb_dps_01_xau_con_chung_dai_nhat_lcs_co_ban]: Xâu Con Chung Dài Nhất Cơ Bản (LCS)

Bối cảnh: Trong lĩnh vực xử lý ngôn ngữ tự nhiên và kiểm tra đạo văn số học, hai văn bản số được chuyển hóa thành hai chuỗi ký tự S và T . Một chuỗi con chung là một chuỗi các ký tự cùng xuất hiện trong cả hai văn bản theo đúng thứ tự tương đối ban đầu nhưng không nhất thiết phải đứng liền kề nhau. Các nhà nghiên cứu cần xác định độ dài chuỗi con chung dài nhất giữa hai văn bản để đo lường mức độ tương đồng nội dung.

BẢNG PHƯƠNG ÁN LCS & ĐƯỜNG TRUY VẾT XÂU TỐI ƯU
Xâu A = "AGGTAB", Xâu B = "GXTXAYB" → LCS = "GTAB" (Độ dài = 4)

i \ j	G	X	T	X	A	Y	B
eps	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
G	1	1	1	1	1	1	1
G	1	1	1	1	1	1	1
T	1	1	2	2	2	2	2
A	1	1	2	2	3	3	3
B	1	1	2	2	3	3	4

QUY TẮC TRUY VẾT (TRACEBACK) TRÊN BẢNG LCS:

- Nếu $A[i] == B[j]$: Ký tự này thuộc LCS → Ghi nhận ký tự và đi chéo lùi về $(i-1, j-1)$.
- Nếu $A[i] != B[j]$: Đi theo hướng có giá trị lớn hơn: lên trên $(i-1, j)$ hoặc sang trái $(i, j-1)$.
- Ô xanh lá là đường truy vết "GTAB", ô đậm là ký tự khớp (B, A, T, G).

Hình minh họa: Bảng phương án và hướng truy vết đường đi xâu con chung dài nhất (LCS)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài của xâu con chung dài nhất của hai chuỗi đó.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi ký tự T ($1 \leq |T| \leq 2000$). Các chuỗi chỉ chứa các ký tự chữ cái tiếng Anh in hoa hoặc in thường.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là độ dài xâu con chung dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
AGGTAB GXTXAYB	4

Giải thích: Với hai xâu $S = "ABCBADAB"$ và $T = "BDCAB"$: Xâu con chung dài nhất có thể tìm được là "BCAB" (hoặc "BDAB") có độ dài đúng bằng 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài của xâu con chung dài nhất của hai chuỗi đó.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xoá ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: AGGTAB GXTXAYB $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: AGGTAB GXTXAYB	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai xâu $S = "ABCBDBAB"$ và $T = "BDCAB"$: Xâu con chung dài nhất có thể tìm được là "BCAB" (hoặc "BDA...)	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với hai xâu $S = "ABCBDBAB"$ và $T = "BDCAB"$: Xâu con chung dài nhất có thể tìm được là "BCAB" (hoặc "BDAB") có độ dài đúng bằng 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xoá i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

string s, t;
if (!(cin >> s >> t)) return 0;

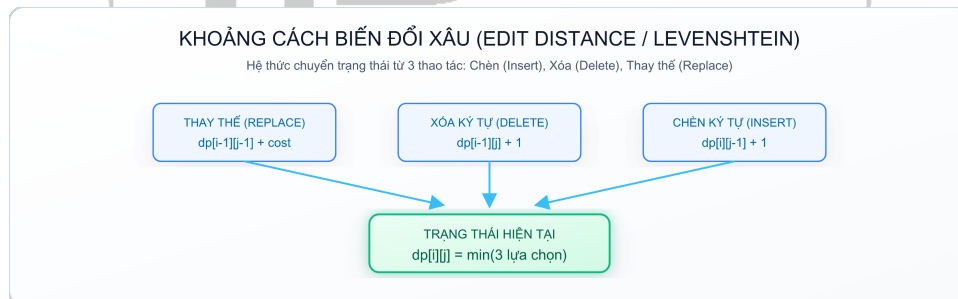
int n = s.size(), m = t.size();
vector<int> prev_row(m + 1, 0), curr_row(m + 1, 0);

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (s[i - 1] == t[j - 1]) {
            curr_row[j] = prev_row[j - 1] + 1;
        } else {
            curr_row[j] = max(prev_row[j], curr_row[j - 1]);
        }
    }
    prev_row = curr_row;
}

cout << prev_row[m] << "\n";
return 0;
}
```

Bài 02 [cppb_dps_02_truy_vet_xau_con_chung_dai_nhat]: Truy Vết Xâu Con Chung Dài Nhất

Bối cảnh: Sau khi tính toán được độ dài tương đồng giữa hai chuỗi văn bản, hệ thống kiểm tra đạo văn cần trích xuất chính xác chuỗi ký tự chung dài nhất đó để làm bằng chứng đánh dấu nổi bật (highlight) trên giao diện phần mềm cho người thẩm định theo dõi.



Hình minh họa: Ma trận chuyển trạng thái tính khoảng cách chỉnh sửa xâu (Edit Distance)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm và in ra xâu con chung dài nhất của hai chuỗi. Nếu có nhiều xâu con chung cùng đạt độ dài lớn nhất, in ra một xâu bất kỳ thỏa mãn.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi ký tự T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất xâu con chung dài nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
ABCB DAB BDCAB	4 BDAB

Giải thích: Với hai chuỗi $S = \text{"ABCB DAB"}$ và $T = \text{"BDCAB"}$: Chuỗi con chung dài nhất đồng thời xuất hiện trong cả hai chuỗi là "BCAB" với độ dài là 4. Kết quả in ra chuỗi BCAB.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình tìm và in ra chuỗi con chung dài nhất của hai chuỗi. Nếu có nhiều chuỗi con chung cùng đạt độ dài lớn nhất, in ra một chuỗi bất kỳ thỏa mãn.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái chuỗi:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của chuỗi S và tiền tố độ dài j của chuỗi T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Chuỗi con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: ABCBDAB BDCAB $\Rightarrow$ Đầu ra kỳ vọng: 4 BDAB.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: ABCBDAB BDCAB	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai chuỗi $S = \text{"ABCB DAB"}$ và $T = \text{"BDCAB"}$: Chuỗi con chung dài nhất đồng thời xuất hiện trong cả hai chuỗi là "BCAB..."	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4 BDAB

Giải thích chi tiết: Với hai chuỗi $S = \text{"ABCB DAB"}$ và $T = \text{"BDCAB"}$: Chuỗi con chung dài nhất đồng thời xuất hiện trong cả hai chuỗi là "BCAB" với độ dài là 4. Kết quả in ra chuỗi BCAB.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).

- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

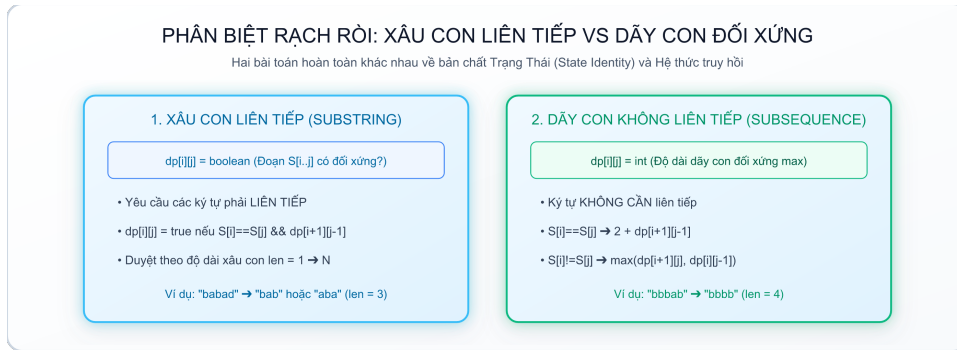
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    // Truy vết
    string lcs = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s[i - 1] == t[j - 1]) {
            lcs += s[i - 1];
            --i; --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            --i;
        } else {
            --j;
        }
    }
    reverse(lcs.begin(), lcs.end());

    cout << dp[n][m] << "\n";
    cout << lcs << "\n";
    return 0;
}
```

Bài 03 [cppb_dps_03_khoang_cach_chinh_sua_edit_distance]: Khoảng Cách Chỉnh Sửa (Edit Distance / Levenshtein)

Bối cảnh: Trong tính năng gợi ý tự động sửa lỗi chính tả của bàn phím thông minh, khi người dùng gõ nhầm một từ, máy tính cần tìm khoảng cách biến đổi tối thiểu từ chuỗi người dùng gõ thành chuỗi từ điển chuẩn. Có 3 thao tác chỉnh sửa ký tự cơ bản được hỗ trợ: chèn thêm 1 ký tự, xóa bớt 1 ký tự, hoặc thay thế 1 ký tự bằng 1 ký tự khác. Mỗi thao tác đều có chi phí là 1 đơn vị.



Hình minh họa: Phân biệt cấu trúc quy hoạch động xâu con đối xứng (Substring vs Subsequence)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm số lượng thao tác chỉnh sửa ít nhất để biến đổi chuỗi S thành chuỗi T .

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi ký tự T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số thao tác chỉnh sửa ít nhất cần dùng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
horse ros	3

Giải thích: Để biến đổi chuỗi $S = \text{"kitten"}$ thành $T = \text{"sitting"}$:

- Thay thế ký tự 'k' thành 's' $\rightarrow \text{"sitten"}$.
- Thay thế ký tự 'e' thành 'i' $\rightarrow \text{"sittin"}$.
- Chèn thêm ký tự 'g' vào cuối $\rightarrow \text{"sitting"}$. Tổng cộng cần đúng 3 thao tác chỉnh sửa, kết quả là 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình tìm số lượng thao tác chỉnh sửa ít nhất để biến đổi chuỗi S thành chuỗi T .
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- Công thức chuyển trạng thái tiêu biểu:**
- Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j-1] + 1$), Xoá ($dp[i-1][j] + 1$), Thay thế ($dp[i-1][j-1] + (S[i-1] \neq T[j-1])$).
- Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: horse ros $\Rightarrow$ Đầu ra kỳ vọng:
3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: horse ros	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Để biến đổi chuỗi $S = "kitten"$ thành $T = "sitting"$: 1. Thay thế ký tự 'k' thành 's' $\rightarrow "sitten"$. 2. Thay th...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Để biến đổi chuỗi $S = "kitten"$ thành $T = "sitting"$:

1. Thay thế ký tự 'k' thành 's' $\rightarrow "sitten"$.
2. Thay thế ký tự 'e' thành 'i' $\rightarrow "sittin"$.
3. Chèn thêm ký tự 'g' vào cuối $\rightarrow "sitting"$. Tổng cộng cần đúng 3 thao tác chỉnh sửa, kết quả là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<int> prev_row(m + 1), curr_row(m + 1);

    for (int j = 0; j <= m; ++j) prev_row[j] = j;
```

```

for (int i = 1; i <= n; ++i) {
    curr_row[0] = i;
    for (int j = 1; j <= m; ++j) {
        if (s[i - 1] == t[j - 1]) {
            curr_row[j] = prev_row[j - 1];
        } else {
            curr_row[j] = 1 + min({prev_row[j], curr_row[j - 1], prev_row[j - 1]});
        }
    }
    prev_row = curr_row;
}

cout << prev_row[m] << "\n";
return 0;
}

```

Bài 04 [cppb_dps_04_bien_doi_xau_it_phep_toan_nhat]: Chỉ Dùng Phép Xóa Biến Đổi Hai Xâu

Bối cảnh: Tại một máy nén dữ liệu truyền thông, hệ thống chỉ hỗ trợ duy nhất một thao tác cơ học là xóa bỏ ký tự. Cho hai chuỗi văn bản S và T . Kỹ sư cần đưa cả hai chuỗi về cùng một chuỗi hoàn toàn giống nhau bằng cách chỉ xóa bỏ các ký tự trên mỗi chuỗi.



Hình minh họa: Bảng phương án và hướng truy vết đường đi xâu con chung dài nhất (LCS)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm tổng số lượng ký tự ít nhất cần phải xóa bỏ trên cả hai chuỗi để phần còn lại của chúng trở nên đồng nhất.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi ký tự T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng số ký tự ít nhất cần xóa.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
sea eat	2

Giải thích: Với hai chuỗi $S = \text{"sea"}$ và $T = \text{"eat"}$: Để hai chuỗi trở nên giống nhau, ta đưa cả hai về chuỗi chung là "ea" :

- Trên chuỗi S : Xóa ký tự 's' (1 ký tự).
- Trên chuỗi T : Xóa ký tự 't' (1 ký tự). Tổng số ký tự cần xóa là $1 + 1 = 2$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình tìm tổng số lượng ký tự ít nhất cần phải xóa bỏ trên cả hai chuỗi để phần còn lại của chúng trở nên đồng nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xấu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: sea eat $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: sea eat	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai chuỗi $S = \text{"sea"}$ và $T = \text{"eat"}$: Để hai chuỗi trở nên giống nhau, ta đưa cả hai về chuỗi chung là "ea" :...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với hai chuỗi $S = \text{"sea"}$ và $T = \text{"eat"}$: Để hai chuỗi trở nên giống nhau, ta đưa cả hai về chuỗi chung là "ea" :

- Trên chuỗi S : Xóa ký tự 's' (1 ký tự).
- Trên chuỗi T : Xóa ký tự 't' (1 ký tự). Tổng số ký tự cần xóa là $1 + 1 = 2$.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiên tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

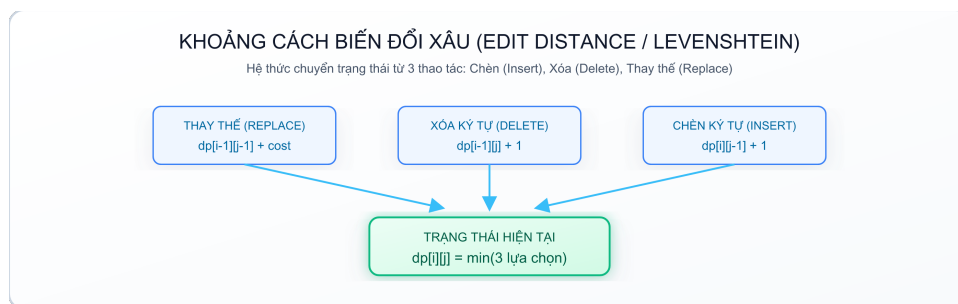
    int n = s.size(), m = t.size();
    vector<int> prev_row(m + 1, 0), curr_row(m + 1, 0);

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                curr_row[j] = prev_row[j - 1] + 1;
            } else {
                curr_row[j] = max(prev_row[j], curr_row[j - 1]);
            }
        }
        prev_row = curr_row;
    }

    int lcs = prev_row[m];
    int min_deletions = (n - lcs) + (m - lcs);
    cout << min_deletions << "\n";
    return 0;
}
```

Bài 05 [cppb_dps_05_xau_con_doi_xung_dai_nhat_lps]: Xâu Con Đối Xứng Dài Nhất (Longest Palindromic Subsequence)

Bối cảnh: Một chuỗi ký tự được gọi là đối xứng (Palindrome) nếu đọc từ trái sang phải hay từ phải sang trái đều thu được chuỗi hoàn toàn giống nhau (ví dụ: RADAR, MADAM). Trong phân tích mã vạch bảo mật, bạn kỹ thuật muốn trích xuất từ chuỗi gốc một chuỗi con (không nhất thiết liên tiếp) có tính đối xứng sao cho độ dài của nó là lớn nhất.



Hình minh họa: Ma trận chuyển trạng thái tính khoảng cách chỉnh sửa chuỗi (Edit Distance)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của chuỗi con đối xứng dài nhất trích xuất được từ S .

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài của chuỗi con đối xứng dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
bbbab	4

Giải thích: Với chuỗi $S = \text{"BBABCBCAB"}$: Một chuỗi con đối xứng dài nhất có thể chọn là "BABCBAB" (hoặc "BACBCAB") có độ dài bằng 7.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của chuỗi con đối xứng dài nhất trích xuất được từ S .
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái xấu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của chuỗi S và tiền tố độ dài j của chuỗi T .
- Công thức chuyển trạng thái tiêu biểu:**
- Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j-1] + 1$), Xóa ($dp[i-1][j] + 1$), Thay thế ($dp[i-1][j-1] + (S[i-1] \neq T[j-1])$).
- Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: bbbab $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
------	--------------------	-------------------------------	------------------

1	Khởi tạo & Đọc dữ liệu	Nạp Input: bbbab	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "BBABCBCAB"$: Một chuỗi con đối xứng dài nhất có thể chọn là $"BABCBAB"$ (hoặc $"BACBCAB"$) có độ d...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với chuỗi $S = "BBABCBCAB"$: Một chuỗi con đối xứng dài nhất có thể chọn là $"BABCBAB"$ (hoặc $"BACBCAB"$) có độ dài bằng 7.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int n = s.size();
    string t = s;
    reverse(t.begin(), t.end());

    vector<int> prev_row(n + 1, 0), curr_row(n + 1, 0);

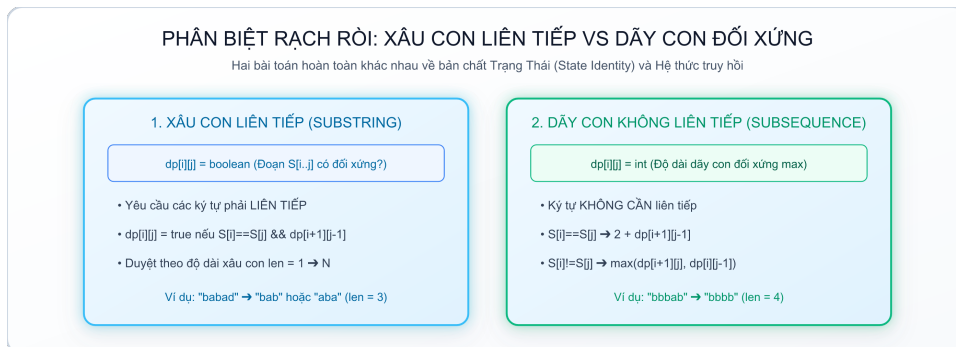
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s[i - 1] == t[j - 1]) {
                curr_row[j] = prev_row[j - 1] + 1;
            } else {
                curr_row[j] = max(prev_row[j], curr_row[j - 1]);
            }
        }
    }
}
```

```
prev_row = curr_row;
}

cout << prev_row[n] << "\n";
return 0;
}
```

Bài 06 [cppb_dps_06_doan_con_doi_xung_dai_nhat_substring]: Đoạn Con Đối Xứng Liên Tiếp Dài Nhất

Bối cảnh: Khác với chuỗi con rời rạc, một đoạn con liên tiếp (Substring) đối xứng đòi hỏi các ký tự phải đứng kề sát nhau và tạo thành một cụm đối xứng hoàn chỉnh. Đây là tác vụ trọng tâm trong việc nhận diện các đoạn lặp đảo ngược trong cấu trúc chuỗi gen sinh học.



Hình minh họa: Phân biệt cấu trúc quy hoạch động xâu con đối xứng (Substring vs Subsequence)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của đoạn con liên tiếp đối xứng dài nhất trong S .

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài của đoạn con đối xứng liên tiếp dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
babad	3 bab

Giải thích: Với chuỗi $S = \text{"babad"}$: Đoạn con liên tiếp đối xứng dài nhất là "bab" (hoặc "aba") có độ dài bằng 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của đoạn con liên tiếp đối xứng dài nhất trong S .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xoá ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: babad $\Rightarrow$ Đầu ra kỳ vọng: 3 bab.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: babad	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = \text{"babad"}$: Đoạn con liên tiếp đối xứng dài nhất là "bab" (hoặc "aba") có độ dài bằng 3.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3 bab

Giải thích chi tiết: Với chuỗi $S = \text{"babad"}$: Đoạn con liên tiếp đối xứng dài nhất là "bab" (hoặc "aba") có độ dài bằng 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xoá i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int n = s.size();
    if (n == 0) return 0;
```

```
vector<vector<bool>> dp(n, vector<bool>(n, false));
int max_len = 1;
int start_idx = 0;

for (int i = 0; i < n; ++i) dp[i][i] = true;

for (int i = 0; i < n - 1; ++i) {
    if (s[i] == s[i + 1]) {
        dp[i][i + 1] = true;
        if (max_len < 2) {
            max_len = 2;
            start_idx = i;
        }
    }
}

for (int len = 3; len <= n; ++len) {
    for (int i = 0; i <= n - len; ++i) {
        int j = i + len - 1;
        if (s[i] == s[j] && dp[i + 1][j - 1]) {
            dp[i][j] = true;
            if (len > max_len) {
                max_len = len;
                start_idx = i;
            }
        }
    }
}

cout << max_len << "\n";
cout << s.substr(start_idx, max_len) << "\n";
return 0;
}
```

Bài 07 [cppb_dps_07_dem_so_xau_con_doi_xung]: Đếm Số Đoạn Con Đối Xứng Liên Tiếp

Bối cảnh: Trong một kỳ thi mật mã học sinh viên, ban giám khảo đưa ra một chuỗi văn bản và yêu cầu các thí sinh thống kê tất cả các phân đoạn liên tiếp có tính đối xứng xuất hiện trong chuỗi (kể cả các đoạn con có độ dài 1 ký tự).

BẢNG PHƯƠNG ÁN LCS & ĐƯỜNG TRUY VẾT XÂU TỐI ƯU

Xâu A = "AGGTAB", Xâu B = "GXTXAYB" → LCS = "GTAB" (Độ dài = 4)

i \ j	G	X	T	X	A	Y	B
eps	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
G	1	1	1	1	1	1	1
G	1	1	1	1	1	1	1
T	1	1	2	2	2	2	2
A	1	1	2	2	3	3	3
B	1	1	2	2	3	3	4

QUY TẮC TRUY VẾT (TRACEBACK) TRÊN BẢNG LCS:

- Nếu $A[i] == B[j]$: Ký tự này thuộc LCS → Ghi nhận ký tự và đi chéo lùi về $(i-1, j-1)$.
 - Nếu $A[i] != B[j]$: Đi theo hướng có giá trị lớn hơn: lên trên $(i-1, j)$ hoặc sang trái $(i, j-1)$.
- Ô xanh lá là đường truy vết "GTAB", ô đậm là ký tự khớp (B, A, T, G).

Hình minh họa: Bảng phương án và hướng truy vết đường đi xâu con chung dài nhất (LCS)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình đếm tổng số lượng đoạn con liên tiếp đối xứng có trong chuỗi.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng số lượng đoạn con đối xứng liên tiếp đếm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
aaa	6

Giải thích: Với chuỗi $S = "aaa"$: Có tất cả 6 đoạn con liên tiếp đối xứng gồm:

- 3 đoạn độ dài 1: “a” (vị trí 0), “a” (vị trí 1), “a” (vị trí 2).
- 2 đoạn độ dài 2: “aa” (vị trí 0-1), “aa” (vị trí 1-2).
- 1 đoạn độ dài 3: “aaa” (toàn bộ chuỗi). Tổng số đoạn đối xứng là 6.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình đếm tổng số lượng đoạn con liên tiếp đối xứng có trong chuỗi.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái xấu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- Công thức chuyển trạng thái tiêu biểu:**
- Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xoá ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: aaa $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: aaa	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "aaa"$: Có tất cả 6 đoạn con liên tiếp đối xứng gồm: - 3 đoạn độ dài 1: “a” (vị trí 0), “a” (vị trí 1), “a” (vị trí 2)...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với chuỗi $S = "aaa"$: Có tất cả 6 đoạn con liên tiếp đối xứng gồm:

- 3 đoạn độ dài 1: “a” (vị trí 0), “a” (vị trí 1), “a” (vị trí 2).
- 2 đoạn độ dài 2: “aa” (vị trí 0-1), “aa” (vị trí 1-2).
- 1 đoạn độ dài 3: “aaa” (toàn bộ chuỗi). Tổng số đoạn đối xứng là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int n = s.size();
    vector<vector<bool>> dp(n, vector<bool>(n, false));
    int count = 0;

    for (int i = 0; i < n; ++i) {
        dp[i][i] = true;
        count++;
    }

    for (int i = 0; i < n - 1; ++i) {
        if (s[i] == s[i + 1]) {
            dp[i][i + 1] = true;
            count++;
        }
    }
}
```

```

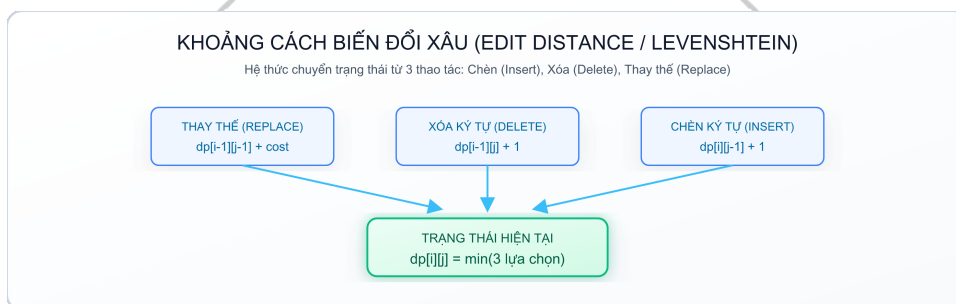
for (int len = 3; len <= n; ++len) {
    for (int i = 0; i <= n - len; ++i) {
        int j = i + len - 1;
        if (s[i] == s[j] && dp[i + 1][j - 1]) {
            dp[i][j] = true;
            count++;
        }
    }
}

cout << count << "\n";
return 0;
}

```

Bài 08 [cppb_dps_08_chen_it_nhat_thanh_palindrome]: Chèn Ít Ký Tự Nhất Tạo Chuỗi Đối Xứng

Bối cảnh: Một máy in nhãn hàng hóa bị lỗi chỉ in được một chuỗi ký tự khuyết thiếu. Để nhãn hàng có tính thẩm mỹ cân đối, kỹ thuật viên cần chèn thêm vào các vị trí bất kỳ trong chuỗi một số lượng ký tự ít nhất sao cho chuỗi thu được trở thành một chuỗi đối xứng hoàn hảo.



Hình minh họa: Ma trận chuyển trạng thái tính khoảng cách chỉnh sửa xâu (Edit Distance)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần phải chèn thêm vào chuỗi S để biến nó thành một chuỗi đối xứng.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số ký tự ít nhất cần chèn thêm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
zzazz	0

Giải thích: Với chuỗi $S = \text{"mbadm"}$: Để chuỗi trở thành đối xứng, ta có thể chèn thêm 2 ký tự 'd' và 'b' vào các vị trí thích hợp để tạo thành chuỗi "mbdadbm". Số ký tự chèn thêm ít nhất là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần phải chèn thêm vào chuỗi S để biến nó thành một chuỗi đối xứng.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: zzazz $\Rightarrow$ Đầu ra kỳ vọng: 0.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: zzazz	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "mbadm"$: Để chuỗi trở thành đối xứng, ta có thể chèn thêm 2 ký tự 'd' và 'b' vào các vị trí thích hợp để tạo thành...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 0

Giải thích chi tiết: Với chuỗi $S = "mbadm"$: Để chuỗi trở thành đối xứng, ta có thể chèn thêm 2 ký tự 'd' và 'b' vào các vị trí thích hợp để tạo thành chuỗi "mbdadbm". Số ký tự chèn thêm ít nhất là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

string s;
if (!(cin >> s)) return 0;

int n = s.size();
string t = s;
reverse(t.begin(), t.end());

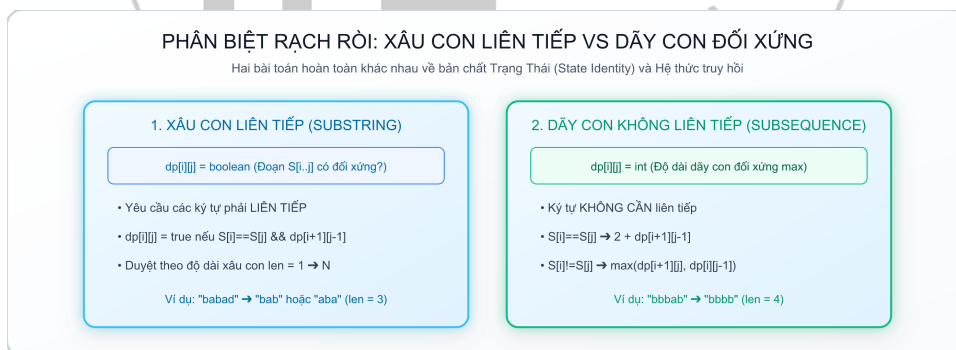
vector<int> prev_row(n + 1, 0), curr_row(n + 1, 0);

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) {
        if (s[i - 1] == t[j - 1]) {
            curr_row[j] = prev_row[j - 1] + 1;
        } else {
            curr_row[j] = max(prev_row[j], curr_row[j - 1]);
        }
    }
    prev_row = curr_row;
}

int lps = prev_row[n];
cout << n - lps << "\n";
return 0;
}
```

Bài 09 [cppb_dps_09_xau_me_chung_ngan_nhat_scs]: Xâu Mẹ Chung Ngắn Nhất (Shortest Common Supersequence)

Bối cảnh: Một máy phát tín hiệu cần gửi một chuỗi mã nguồn sao cho máy thu khi nhận được có thể trích xuất ra đồng thời cả hai bản tin S và T dưới dạng các chuỗi con. Để tiết kiệm băng thông truyền dẫn số, độ dài của chuỗi mã phát đi phải là ngắn nhất có thể.



Hình minh họa: Phân biệt cấu trúc quy hoạch động xâu con đối xứng (Substring vs Subsequence)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài ngắn nhất của một chuỗi chứa cả S và T dưới dạng các chuỗi con.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi ký tự T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài ngắn nhất của xâu mẹ chung.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
abac cab	5

Giải thích: Với hai chuỗi $S = "abac"$ và $T = "cab"$: Xâu mẹ chung ngắn nhất chứa cả hai chuỗi là "cabac" có độ dài đúng bằng 5 (chứa "cab" ở tiền tố và "abac" ở hậu tố).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài ngắn nhất của một chuỗi chứa cả S và T dưới dạng các chuỗi con.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xoá ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: abac cab $\Rightarrow$ Đầu ra kỳ vọng: 5.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: abac cab	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai chuỗi $S = "abac"$ và $T = "cab"$: Xâu mẹ chung ngắn nhất chứa cả hai chuỗi là "cabac" có độ dài đúng bằng...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 5

Giải thích chi tiết: Với hai chuỗi $S = "abac"$ và $T = "cab"$: Xâu mẹ chung ngắn nhất chứa cả hai chuỗi là "cabac" có độ dài đúng bằng 5 (chứa "cab" ở tiền tố và "abac" ở hậu tố).

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiên tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<int> prev_row(m + 1, 0), curr_row(m + 1, 0);

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                curr_row[j] = prev_row[j - 1] + 1;
            } else {
                curr_row[j] = max(prev_row[j], curr_row[j - 1]);
            }
        }
        prev_row = curr_row;
    }

    int lcs = prev_row[m];
    int scs_len = n + m - lcs;
    cout << scs_len << "\n";
    return 0;
}
```

Bài 10 [cppb_dps_10_dem_so_lan_xuat_hien_xau_con]: Đếm Số Lần Xuất Hiện Xâu Con Rời Rạc (Distinct Subsequences)

Bối cảnh: Trong bộ lọc từ khóa độc hại của mạng xã hội, hệ thống cần kiểm tra tần suất mà một từ khóa nhạy cảm T xuất hiện dưới dạng chuỗi con rời rạc bên trong một bài viết S . Hai cách xuất hiện được tính là khác nhau nếu tập hợp các vị trí chỉ số ký tự được chọn trong bài viết S là khác nhau.



Hình minh họa: Bảng phương án và hướng truy vết đường đi xâu con chung dài nhất (LCS)

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình đếm số cách khác nhau để chọn ra chuỗi con trong S bằng đúng chuỗi T , lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi văn bản S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi mẫu T ($1 \leq |T| \leq 500$).

Đầu ra (Output):

- In ra số cách xuất hiện hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
rabbbit rabbit	3

Giải thích: Với $S = \text{"rabbbit"}$ và $T = \text{"rabbit"}$: Trong chuỗi S có 3 ký tự 'b' liên tiếp. Để tạo thành từ "rabbit", ta có thể loại bỏ 1 trong 3 ký tự 'b' đó. Do đó có đúng 3 cách chọn khác nhau để thu được từ "rabbit". Kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai chuỗi ký tự S và T . Hãy lập trình đếm số cách khác nhau để chọn ra chuỗi con trong S bằng đúng chuỗi T , lấy dư cho $10^9 + 7$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- Công thức chuyển trạng thái tiêu biểu:**
- Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j-1] + 1$), Xoá ($dp[i-1][j] + 1$), Thay thế ($dp[i-1][j-1] + (S[i-1] \neq T[j-1])$).

- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: rabbbit rabbit $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: rabbbit rabbit	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với $S = \text{"rabbbit"}$ và $T = \text{"rabbit"}$: Trong chuỗi S có 3 ký tự 'b' liên tiếp. Để tạo thành từ "rabbit", ta có thể loại...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với $S = \text{"rabbbit"}$ và $T = \text{"rabbit"}$: Trong chuỗi S có 3 ký tự 'b' liên tiếp. Để tạo thành từ "rabbit", ta có thể loại bỏ 1 trong 3 ký tự 'b' đó. Do đó có đúng 3 cách chọn khác nhau để thu được từ "rabbit". Kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<int> dp(m + 1, 0);
```

```

dp[0] = 1;

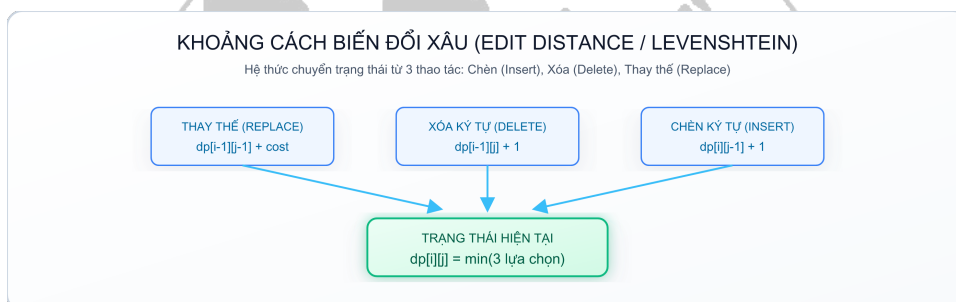
for (int i = 1; i <= n; ++i) {
    for (int j = m; j >= 1; --j) {
        if (s[i - 1] == t[j - 1]) {
            dp[j] = (dp[j] + dp[j - 1]) % MOD;
        }
    }
}

cout << dp[m] << "\n";
return 0;
}

```

Bài 11 [cppb_dps_11_khop_chuoi_wildcard_matching]: Khớp Chuỗi Ký Tự Đại Diện (Wildcard Matching)

Bối cảnh: Một công cụ tìm kiếm tệp tin hỗ trợ tìm kiếm theo mẫu (pattern) chứa các ký tự đại diện thông dụng: Ký tự `.` có thể khớp với đúng một ký tự bất kỳ, và ký tự `\*` có thể khớp với bất kỳ chuỗi ký tự nào (kể cả chuỗi rỗng). Quản trị viên cần kiểm tra xem tên tệp văn bản S có khớp hoàn toàn với mẫu P hay không.



Hình minh họa: Ma trận chuyển trạng thái tính khoảng cách chỉnh sửa xâu (Edit Distance)

Nhiệm vụ: Cho chuỗi văn bản S và chuỗi mẫu P . Hãy lập trình kiểm tra xem chuỗi S có khớp toàn bộ với mẫu P hay không. Nếu khớp in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi văn bản S ($0 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi mẫu P ($0 \leq |P| \leq 2000$).

Đầu ra (Output):

- In ra YES nếu khớp hoàn toàn, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
adceb ab	YES

Giải thích: Với văn bản $S = "adceb"$ và mẫu $P = "*a*b"$: Ký tự `.` đầu tiên khớp với chuỗi rỗng, sau đó ký tự `a` khớp `a`, ký tự `.` thứ hai khớp với đoạn "dce", và cuối cùng ký tự `b` khớp `b`. Chuỗi khớp hoàn toàn với mẫu, kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi văn bản S và chuỗi mẫu P . Hãy lập trình kiểm tra xem chuỗi S có khớp toàn bộ với mẫu P hay không. Nếu khớp in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xoá ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: adceb *a*b $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: adceb *a*b	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với văn bản $S = "adceb"$ và mẫu $P = "*a*b"$: Ký tự '*' đầu tiên khớp với chuỗi rỗng, sau đó ký tự 'a' khớp 'a', ký tự '...' tiếp theo khớp 'b'.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với văn bản $S = "adceb"$ và mẫu $P = "*a*b"$: Ký tự '*' đầu tiên khớp với chuỗi rỗng, sau đó ký tự 'a' khớp 'a', ký tự '...' thứ hai khớp với đoạn "dce", và cuối cùng ký tự 'b' khớp 'b'. Chuỗi khớp hoàn toàn với mẫu, kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).

- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược reverse() trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, p;
    if (!(cin >> s >> p)) return 0;

    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));

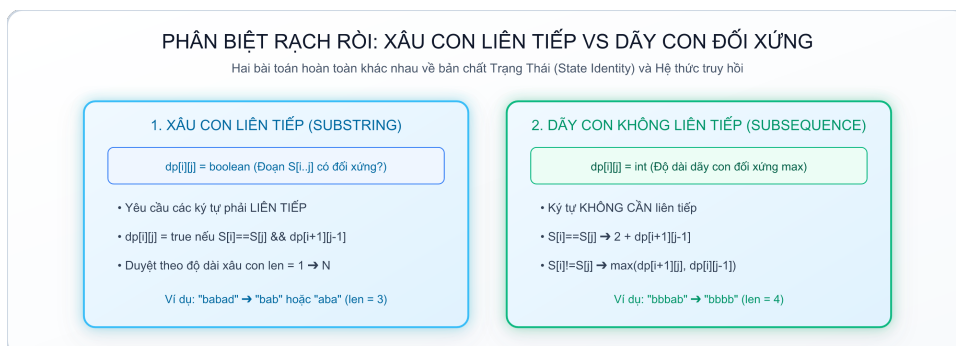
    dp[0][0] = true;
    for (int j = 1; j <= m; ++j) {
        if (p[j - 1] == '*') dp[0][j] = dp[0][j - 1];
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else if (p[j - 1] == '*') {
                dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
            }
        }
    }

    if (dp[n][m]) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```

Bài 12 [cppb_dps_12_xau_con_chung_ba_chuoi]: Xâu Con Chung Của Ba Chuỗi (LCS 3 Strings)

Bối cảnh: Ba viện nghiên cứu sinh học độc lập cùng giải mã trình tự một đoạn gen kháng bệnh ở ba giống cây trồng khác nhau, thu được 3 chuỗi ADN ký hiệu là S_1, S_2 và S_3 . Để xác định đoạn gen chung cốt lõi quy định tính trạng kháng bệnh, các nhà khoa học cần tìm độ dài của chuỗi con chung dài nhất xuất hiện đồng thời trong cả ba chuỗi ADN.



Hình minh họa: Phân biệt cấu trúc quy hoạch động xâu con đối xứng (Substring vs Subsequence)

Nhiệm vụ: Cho ba chuỗi ký tự S_1, S_2 và S_3 . Hãy lập trình tính độ dài chuỗi con chung dài nhất của cả ba chuỗi.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S_1 ($1 \leq |S_1| \leq 100$).
- Dòng 2: Chứa chuỗi ký tự S_2 ($1 \leq |S_2| \leq 100$).
- Dòng 3: Chứa chuỗi ký tự S_3 ($1 \leq |S_3| \leq 100$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là độ dài xâu con chung dài nhất của 3 chuỗi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
geeks geek geekfor	4

Giải thích: Với 3 chuỗi ký tự: $S_1 = \text{"geeks"}$, $S_2 = \text{"geeksfor"}$, $S_3 = \text{"geeksforgeeks"}$: Chuỗi con chung dài nhất xuất hiện trong cả 3 chuỗi là "geeks" với độ dài bằng 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ba chuỗi ký tự S_1, S_2 và S_3 . Hãy lập trình tính độ dài chuỗi con chung dài nhất của cả ba chuỗi.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: geeks geek geekfor $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: geeks geek geekfor	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với 3 chuỗi ký tự: $S_1 = \text{"geeks"}$, $S_2 = \text{"geeksfor"}$, $S_3 = \text{"geeksforgeeks"}$: Chuỗi con chung dài nhất xuất hiện...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với 3 chuỗi ký tự: $S_1 = \text{"geeks"}$, $S_2 = \text{"geeksfor"}$, $S_3 = \text{"geeksforgeeks"}$: Chuỗi con chung dài nhất xuất hiện trong cả 3 chuỗi là "geeks" với độ dài bằng 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s1, s2, s3;
    if (!(cin >> s1 >> s2 >> s3)) return 0;

    int n1 = s1.size(), n2 = s2.size(), n3 = s3.size();
    vector<vector<vector<int>>> dp(n1 + 1, vector<vector<int>>(n2 + 1, vector<int>(n3 + 1, 0)));

    for (int i = 1; i <= n1; ++i) {
        for (int j = 1; j <= n2; ++j) {
            for (int k = 1; k <= n3; ++k) {
                if (s1[i - 1] == s2[j - 1] && s2[j - 1] == s3[k - 1]) {
                    dp[i][j][k] = dp[i - 1][j - 1][k - 1] + 1;
                } else {
                    dp[i][j][k] = max({dp[i - 1][j][k], dp[i][j - 1][k], dp[i][j][k - 1]});
                }
            }
        }
    }

    cout << dp[n1][n2][n3] << "\n";
    return 0;
}
```


Bài 13 [cppb_dps_13_xoa_it_ky_tu_nhat_tao_chuoi_doi_xung]: Xóa Ít Ký Tự Nhất Để Chuỗi Đối Xứng

Bối cảnh: Một thiết bị viễn thông thu nhận một chuỗi tín hiệu bị nhiễu chứa một số ký tự rác. Để khôi phục lại tính toàn vẹn của tín hiệu đối xứng nguyên bản, kỹ sư cần loại bỏ bớt một số lượng ký tự ít nhất từ chuỗi thu được sao cho chuỗi còn lại trở thành một chuỗi đối xứng.

BẢNG PHƯƠNG ÁN LCS & ĐƯỜNG TRUY VẾT XÂU TỐI ƯU
Xâu A = "AGGTAB", Xâu B = "GXTXAYB" → LCS = "GTAB" (Độ dài = 4)

i \ j	G	X	T	X	A	Y	B
eps	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
G	1	1	1	1	1	1	1
G	1	1	1	1	1	1	1
T	1	1	2	2	2	2	2
A	1	1	2	2	3	3	3
B	1	1	2	2	3	3	4

QUY TẮC TRUY VẾT (TRACEBACK) TRÊN BẢNG LCS:

- Nếu $A[i] == B[j]$: Ký tự này thuộc LCS → Ghi nhận ký tự và đi chéo lùi về $(i-1, j-1)$.
- Nếu $A[i] != B[j]$: Đi theo hướng có giá trị lớn hơn: lên trên $(i-1, j)$ hoặc sang trái $(i, j-1)$.

→ Ô xanh lá là đường truy vết "GTAB", ô đậm là ký tự khớp (B, A, T, G).

Hình minh họa: Bảng phương án và hướng truy vết đường đi xâu con chung dài nhất (LCS)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần xóa bỏ để chuỗi còn lại là một chuỗi đối xứng.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số ký tự ít nhất cần xóa.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
aebcbda	2

Giải thích: Với chuỗi $S = "aebcbda"$: Chuỗi con đối xứng dài nhất có thể giữ lại là "abcba" (độ dài 5). Để thu được chuỗi này, ta chỉ cần xóa đi đúng 2 ký tự là 'e' và 'd'. Số ký tự cần xóa ít nhất là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần xóa bỏ để chuỗi còn lại là một chuỗi đối xứng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j] - 1 + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: aeecbda $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: aeecbda	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "aeecbda"$: Chuỗi con đối xứng dài nhất có thể giữ lại là "abcea" (độ dài 5). Để thu được chuỗi này, ta ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với chuỗi $S = "aeecbda"$: Chuỗi con đối xứng dài nhất có thể giữ lại là "abcea" (độ dài 5). Để thu được chuỗi này, ta chỉ cần xóa đi đúng 2 ký tự là 'e' và 'd'. Số ký tự cần xóa ít nhất là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int n = s.size();
    string t = s;
```

```
reverse(t.begin(), t.end());

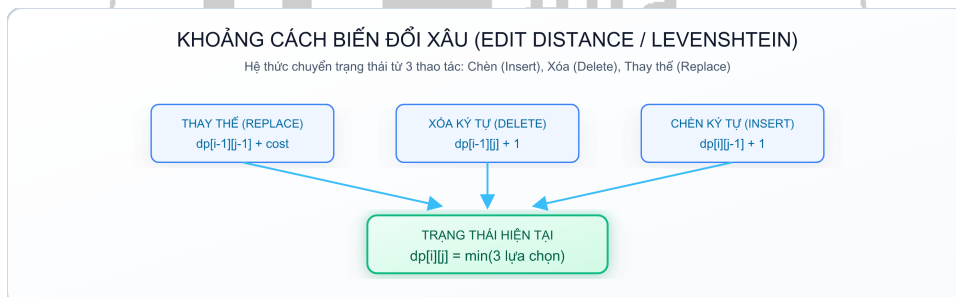
vector<int> prev_row(n + 1, 0), curr_row(n + 1, 0);

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) {
        if (s[i - 1] == t[j - 1]) {
            curr_row[j] = prev_row[j - 1] + 1;
        } else {
            curr_row[j] = max(prev_row[j], curr_row[j - 1]);
        }
    }
    prev_row = curr_row;
}

int lps = prev_row[n];
cout << n - lps << "\n";
return 0;
}
```

Bài 14 [cppb_dps_14_khoi_phuc_chuoi_tu_dien_nho_nhat_lcs]: Độ Dài LCS Giữa Hai Chuỗi Gen

Bối cảnh: Trong đề án nghiên cứu tiến hóa học phân tử, các nhà di truyền học so sánh hai chuỗi gen dài được biểu diễn dưới dạng hai chuỗi ký tự chỉ gồm 4 loại nucleotide cơ bản: A, C, G, T. Việc tìm độ dài xâu con chung dài nhất giữa hai chuỗi gen phản ánh mức độ gần gũi về mặt quan hệ tiến hóa giữa hai loài sinh vật.



Hình minh họa: Ma trận chuyển trạng thái tính khoảng cách chỉnh sửa xâu (Edit Distance)

Nhiệm vụ: Cho hai chuỗi gen S và T . Hãy lập trình tính độ dài chuỗi con chung dài nhất giữa hai chuỗi đó.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi gen thứ nhất S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chứa chuỗi gen thứ hai T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài xâu con chung dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
ACCGGTCGAGTGC GCGGAAGCCGGCCGAA GTCGTTCCGAATGCCGTTGCTCTGTAA	20

Giải thích: Với hai chuỗi gen $S = \text{"AGGTAB"}$ và $T = \text{"GXTXAYB"}$: Xâu con chung dài nhất giữa hai mẫu gen là "GTAB" có độ dài bằng 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho hai chuỗi gen S và T . Hãy lập trình tính độ dài chuỗi con chung dài nhất giữa hai chuỗi đó.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- **Công thức chuyển trạng thái tiêu biểu:**
- **Xâu con chung dài nhất (LCS):** Nếu $S[i - 1] == T[j - 1]$ thì $dp[i][j] = dp[i - 1][j - 1] + 1$; ngược lại $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.
- **Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j - 1] + 1$), Xóa ($dp[i - 1][j] + 1$), Thay thế ($dp[i - 1][j - 1] + (S[i - 1] \neq T[j - 1])$).
- **Độ phức tạp:** Thời gian $O(|S| \times |T|)$, bộ nhớ $O(|S| \times |T|)$ hoặc $O(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào:

ACCGGTCGAGTGC GCGGAAGCCGGCCGAA GTCGTTTCGGAATGCCGTTGCTCTGTAA

⇒ Đầu ra kỳ vọng: 20.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: ACCGGTCGAGTGC GCGGAAGCCGGCC GAA GTCGTTTCGGAATGC C	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai chuỗi gen $S = \text{"AGGTAB"}$ và $T = \text{"GXTXAYB"}$: Xâu con chung dài nhất giữa hai mẫu gen là "GTAB" có độ dài...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 20

Giải thích chi tiết: Với hai chuỗi gen $S = \text{"AGGTAB"}$ và $T = \text{"GXTXAYB"}$: Xâu con chung dài nhất giữa hai mẫu gen là "GTAB" có độ dài bằng 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiên tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

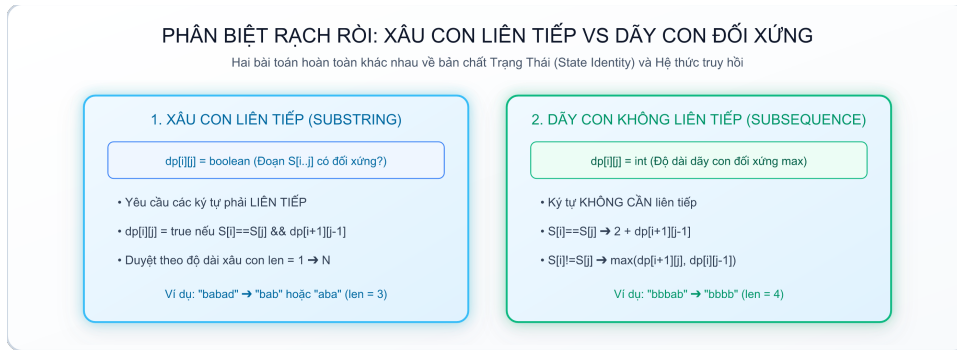
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    int lcs_len = dp[n][m];
    cout << lcs_len << "\n";
    return 0;
}
```

Bài 15 [cppb_dps_15_quy_hoach_dong_chuoi_cuc_han]: Tách Từ Trong Chuỗi (Word Break)

Bối cảnh: Một hệ thống nhận dạng quang học chữ viết (OCR) quét văn bản tiếng Anh cổ nhưng do lỗi khoảng cách, tất cả các từ trong câu bị dính liền thành một chuỗi ký tự duy nhất S . Cho trước một cuốn từ điển gồm N từ vựng chuẩn. Kỹ sư ngôn ngữ cần xác định xem chuỗi ký tự dính liền S có thể được phân tách thành một chuỗi các từ hợp lệ nằm trong cuốn từ điển hay không.



Hình minh họa: Phân biệt cấu trúc quy hoạch động xâu con đối xứng (Substring vs Subsequence)

Nhiệm vụ: Cho chuỗi ký tự S và từ điển gồm N từ. Hãy lập trình kiểm tra xem chuỗi S có thể được phân tách hoàn toàn thành các từ trong từ điển hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự S ($1 \leq |S| \leq 300$).
- Dòng 2: Chứa số nguyên dương N ($1 \leq N \leq 1000$) là số lượng từ trong từ điển.
- N dòng tiếp theo, mỗi dòng chứa một từ trong từ điển có độ dài không quá 20 ký tự.

Đầu ra (Output):

- In ra YES nếu chuỗi có thể phân tách hợp lệ, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
leetcode 2 leet code	YES

Giải thích: Với chuỗi $S = \text{"leetcode"}$ và từ điển gồm các từ $\{\text{"leet"}, \text{"code"}\}$: Chuỗi S có thể phân tách thành hai từ hợp lệ là "leet" và "code". Cả hai từ đều có mặt trong từ điển, do đó kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S và từ điển gồm N từ. Hãy lập trình kiểm tra xem chuỗi S có thể được phân tách hoàn toàn thành các từ trong từ điển hay không. Nếu có in ra YES, ngược lại in ra NO.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái xâu:** Gọi $dp[i][j]$ là đáp số tối ưu khi so khớp tiền tố độ dài i của xâu S và tiền tố độ dài j của xâu T .
- Công thức chuyển trạng thái tiêu biểu:**
- Xâu con chung dài nhất (LCS):** Nếu $S[i-1] == T[j-1]$ thì $dp[i][j] = dp[i-1][j-1] + 1$; ngược lại $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Khoảng cách chỉnh sửa (Edit Distance):** Lấy giá trị nhỏ nhất giữa 3 thao tác: Chèn ($dp[i][j-1] + 1$), Xoá ($dp[i-1][j] + 1$), Thay thế ($dp[i-1][j-1] + (S[i-1] \neq T[j-1])$).
- Độ phức tạp:** Thời gian $\mathcal{O}(|S| \times |T|)$, bộ nhớ $\mathcal{O}(|S| \times |T|)$ hoặc $\mathcal{O}(\min(|S|, |T|))$ khi nén 2 hàng.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: leetcode 2 leet code $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: leetcode 2 leet code	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = \text{"leetcode"}$ và từ điển gồm các từ {"leet", "code"}: Chuỗi S có thể phân tách thành hai từ hợp lệ là "leet" và "..."	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với chuỗi $S = \text{"leetcode"}$ và từ điển gồm các từ {"leet", "code"}: Chuỗi S có thể phân tách thành hai từ hợp lệ là "leet" và "code". Cả hai từ đều có mặt trong từ điển, do đó kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên khởi tạo hàng 0 và cột 0 của bảng dp : Trong Edit Distance, $dp[i][0] = i$ (xóa i ký tự) và $dp[0][j] = j$ (chèn j ký tự).
- Nhầm lẫn giữa chỉ số xâu 0-based trong C++ ($S[i-1]$) và kích thước tiền tố 1-based trong bảng dp ($dp[i][j]$).
- Truy vết ngược không xử lý đúng thứ tự ký tự: Cần lưu các ký tự vào chuỗi rồi đảo ngược `reverse()` trước khi in ra kết quả.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    int k;
    if (!(cin >> s >> k)) return 0;

    unordered_set<string> dict;
    for (int i = 0; i < k; ++i) {
        string word;
        cin >> word;
        dict.insert(word);
    }
}
```

```
int n = s.size();  
vector<bool> dp(n + 1, false);  
dp[0] = true;  
  
for (int i = 1; i <= n; ++i) {  
    for (int j = 0; j < i; ++j) {  
        if (dp[j] && dict.count(s.substr(j, i - j))) {  
            dp[i] = true;  
            break;  
        }  
    }  
}  
  
if (dp[n]) cout << "YES\n";  
else cout << "NO\n";  
return 0;  
}
```

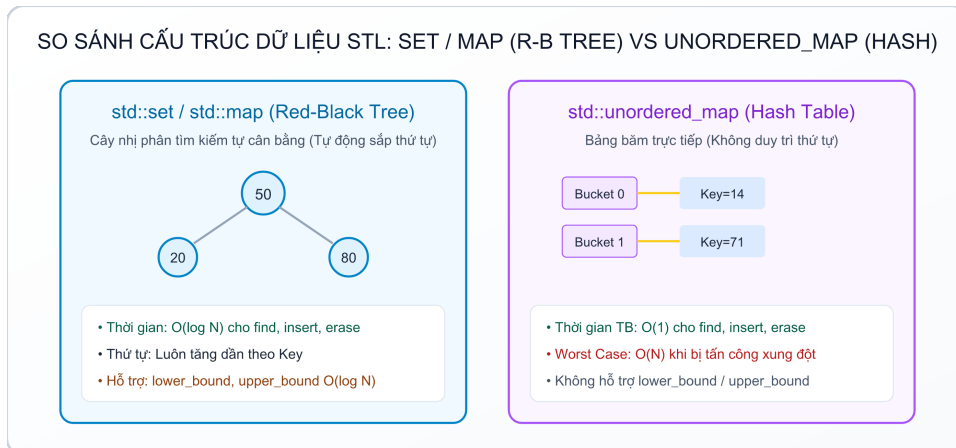


CHƯƠNG 07: CẤU TRÚC DỮ LIỆU NÂNG CAO

Bài 19: Cấu trúc dữ liệu STL: Set, map & heap

Bài 01 [cppb_stl_01_dem_so_phan_tu_phan_biet]: Đếm Số Phần Tử Phân Biệt

Bối cảnh: Tại một hội nghị thượng đỉnh về chuyển đổi số, ban lễ tân quét mã QR định danh của N lượt đại biểu tham dự. Do một số đại biểu di chuyển nhiều lần qua cổng kiểm soát, danh sách các mã định danh thu thập được bị trùng lặp. Ban tổ chức cần thống kê chính xác số lượng đại biểu thực tế (tức số lượng mã định danh độc nhất, phân biệt) đã có mặt.



Hình minh họa: Cấu trúc cây đỏ đen (Red-Black Tree) cân bằng tự động của `std::set` và `std::map`

Nhiệm vụ: Cho danh sách gồm N số nguyên đại diện cho mã định danh. Hãy lập trình đếm và in ra số lượng giá trị phân biệt xuất hiện trong dãy số.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 2 \times 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$) cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số lượng phần tử phân biệt trong dãy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 3 2 2 3	2

Giải thích: Với dãy gồm 5 số nguyên [2,3,2,1,3]: Các giá trị phân biệt xuất hiện trong dãy là {1,2,3}. Có tất cả 3 giá trị phân biệt, kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách gồm N số nguyên đại diện cho mã định danh. Hãy lập trình đếm và in ra số lượng giá trị phân biệt xuất hiện trong dãy số.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**

- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 3 2 2 3 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 3 2 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy gồm 5 số nguyên [2,3,2,1,3]: Các giá trị phân biệt xuất hiện trong dãy là {1,2,3}. Có tất cả 3 giá trị phân biệt, k...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với dãy gồm 5 số nguyên [2,3,2,1,3]: Các giá trị phân biệt xuất hiện trong dãy là {1,2,3}. Có tất cả 3 giá trị phân biệt, kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử st.erase(val) trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng val. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ st.erase(st.find(val)).
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp mp[key] sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng mp.count(key) hoặc mp.find(key) != mp.end().
- Hàng đợi ưu tiên priority_queue mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: priority_queue<long long, vector<long long>, greater<long long>> pq;

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

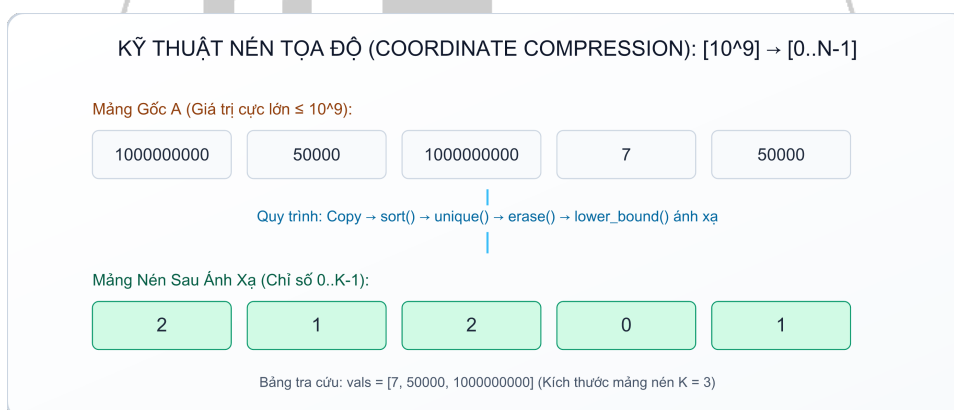
int n;
if (!(cin >> n)) return 0;
if (n <= 0) return 0;

set<long long> st;
for (int i = 0; i < n; ++i) {
    long long x;
    cin >> x;
    st.insert(x);
}

cout << st.size() << "\n";
return 0;
}
```

Bài 02 [cppb_stl_02_bang_tra_cuu_tan_suat_tu_khoa]: Bảng Tra Cứu Tần Suất Từ Khóa

Bối cảnh: Một công cụ tìm kiếm dữ liệu lớn ghi nhận N từ khóa được người dùng nhập vào thanh tìm kiếm trong ngày. Để phục vụ việc tối ưu hóa máy chủ và đề xuất xu hướng tìm kiếm hàng đầu, kỹ sư hệ thống cần thiết lập một bảng thống kê tần suất xuất hiện của từng từ khóa theo thứ tự bảng chữ cái (thứ tự từ điển tăng dần).



Hình minh họa: Mô hình kỹ thuật nén tọa độ bảo toàn thứ tự độ lớn mảng số lớn

Nhiệm vụ: Cho danh sách gồm N từ khóa. Hãy lập trình đếm tần suất xuất hiện của mỗi từ khóa và in ra kết quả theo thứ tự từ điển tăng dần của các từ khóa.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- N dòng tiếp theo, mỗi dòng chứa một từ khóa gồm các chữ cái tiếng Anh in thường có độ dài từ 1 đến 30 ký tự.

Đầu ra (Output):

- In ra các dòng, mỗi dòng gồm một từ khóa và số lần xuất hiện của từ khóa đó, cách nhau bởi một dấu cách.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

4 apple banana apple orange	apple 2 banana 1 orange 1
--------------------------------	---------------------------

Giải thích: Với danh sách 4 từ khóa: [“apple”, “banana”, “apple”, “cherry”]:

- Từ khóa “apple” xuất hiện 2 lần.
- Từ khóa “banana” xuất hiện 1 lần.
- Từ khóa “cherry” xuất hiện 1 lần. Các từ khóa được sắp xếp đúng thứ tự từ điển a -> b -> c.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách gồm N từ khóa. Hãy lập trình đếm tần suất xuất hiện của mỗi từ khóa và in ra kết quả theo thứ tự từ điển tăng dần của các từ khóa.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $O(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $O(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $O(1)$ và cập nhật trong $O(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $O(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 apple banana apple orange
⇒ Đầu ra kỳ vọng: apple 2 banana 1 orange 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 apple banana apple orange	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với danh sách 4 từ khóa: [“apple”, “banana”, “apple”, “cherry”]: - Từ khóa “apple” xuất hiện 2 lần. - Từ khóa “banana” xuất hiện 1 lần....	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: apple 2 banana 1 orange 1

Giải thích chi tiết: Với danh sách 4 từ khóa: [“apple”, “banana”, “apple”, “cherry”]:

- Từ khóa “apple” xuất hiện 2 lần.

- Từ khóa “banana” xuất hiện 1 lần.
- Từ khóa “cherry” xuất hiện 1 lần. Các từ khóa được sắp xếp đúng thứ tự từ điển a -> b -> c.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khóa chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

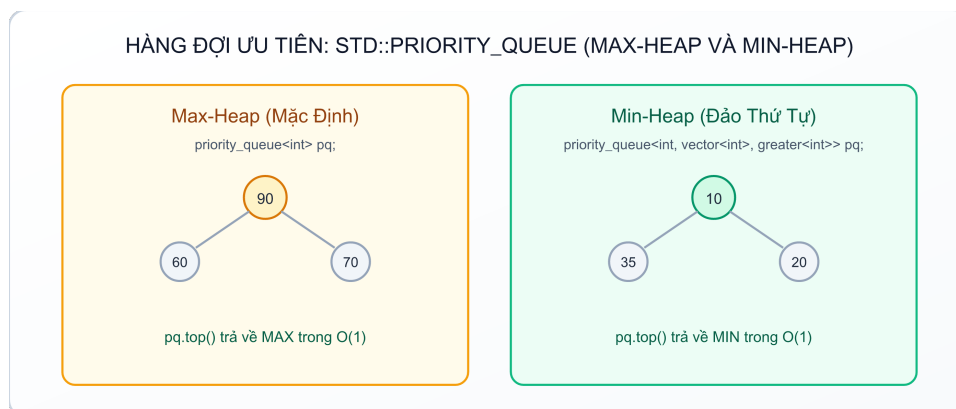
    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    map<string, int> freq;
    for (int i = 0; i < n; ++i) {
        string s;
        cin >> s;
        freq[s]++;
    }

    for (const auto& p : freq) {
        cout << p.first << " " << p.second << "\n";
    }
    return 0;
}
```

Bài 03 [cppb_stl_03_nen_toa_do_mang_so_lon]: Nén Tọa Độ Mảng Số Lớn

Bối cảnh: Trong xử lý đồ họa máy tính, các điểm ảnh có tọa độ rất lớn lên tới 10^9 , nhưng tổng số lượng điểm ảnh thực tế xuất hiện trong khung hình chỉ tối đa là $N = 10^5$. Để tiết kiệm bộ nhớ khi đánh chỉ số mảng mà vẫn bảo toàn tuyệt đối thứ tự tương đối về độ lớn giữa các điểm ảnh, kỹ sư đồ họa áp dụng kỹ thuật nén tọa độ: thay thế mỗi giá trị bằng thứ hạng của nó từ 1 đến K (với K là số giá trị phân biệt).



Hình minh họa: Cấu trúc cây vun đồng nhị phân (Binary Heap) của hàng đợi ưu tiên Priority Queue

Nhiệm vụ: Cho mảng gồm N số nguyên. Hãy lập trình thay thế mỗi phần tử trong mảng bằng thứ hạng nén của nó (bắt đầu từ 1 cho giá trị nhỏ nhất).

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là các giá trị sau khi đã nén tọa độ, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 100 20000 50 20000 100	1 2 0 2 1

Giải thích: Với mảng số ban đầu là $[100, 5, 100, 20, 5]$: Các giá trị phân biệt sau khi sắp xếp tăng dần là: $5 < 20 < 100$.

- Giá trị 5 nhỏ nhất nhận thứ hạng 1.
- Giá trị 20 nhận thứ hạng 2.
- Giá trị 100 nhận thứ hạng 3. Mảng sau khi nén tọa độ tương ứng là: $[3, 1, 3, 2, 1]$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng gồm N số nguyên. Hãy lập trình thay thế mỗi phần tử trong mảng bằng thứ hạng nén của nó (bắt đầu từ 1 cho giá trị nhỏ nhất).
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $O(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén tọa độ trong $O(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $O(1)$ và cập nhật trong $O(\log N)$.

- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 100 20000 50 20000 100 $\Rightarrow$ Đầu ra kỳ vọng: 1 2 0 2 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 100 20000 50 20000 100	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng số ban đầu là [100,5,100,20,5]: Các giá trị phân biệt sau khi sắp xếp tăng dần là: 5 < 20 < 100. - Giá trị 5 nhỏ nhất ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 2 0 2 1

Giải thích chi tiết: Với mảng số ban đầu là [100,5,100,20,5]: Các giá trị phân biệt sau khi sắp xếp tăng dần là: 5 < 20 < 100.

- Giá trị 5 nhỏ nhất nhận thứ hạng 1.
- Giá trị 20 nhận thứ hạng 2.
- Giá trị 100 nhận thứ hạng 3. Mảng sau khi nén toạ độ tương ứng là: [3,1,3,2,1].

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử st.erase(val) trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng val. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ st.erase(st.find(val)).
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp mp[key] sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng mp.count(key) hoặc mp.find(key) != mp.end().
- Hàng đợi ưu tiên priority_queue mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: priority_queue<long long, vector<long long>, greater<long long>> pq;.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
if (!(cin >> n)) return 0;
if (n <= 0) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

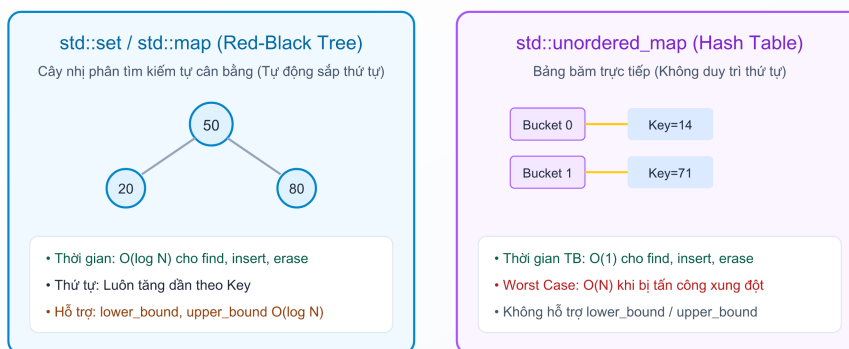
vector<long long> vals = a;
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());

for (int i = 0; i < n; ++i) {
    int rank = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
    cout << rank << (i + 1 == n ? "" : " ");
}
cout << "\n";
return 0;
}
```

Bài 04 [cppb_stl_04_tim_phan_tu_nho_nhat_lon_hon_x]: Tìm Phần Tử Nhỏ Nhất Lớn Hơn Hoặc Bằng X

Bối cảnh: Một sàn giao dịch hàng hóa trực tuyến duy trì một kho các lệnh bán với mức giá niêm yết $A_1, A_2, \dots, A_N$. Khi một nhà đầu tư gửi lệnh mua với mức trần giá tối đa là X , hệ thống giao dịch tự động cần nhanh chóng tìm ra lệnh bán có mức giá nhỏ nhất thỏa mãn điều kiện lớn hơn hoặc bằng X để khớp lệnh.

SO SÁNH CẤU TRÚC DỮ LIỆU STL: SET / MAP (R-B TREE) VS UNORDERED_MAP (HASH)



Hình minh họa: Cấu trúc cây đỏ đen (Red-Black Tree) cân bằng tự động của `std::set` và `std::map`

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm một số nguyên X . Hãy lập trình tìm phần tử nhỏ nhất trong mảng lớn hơn hoặc bằng X . Nếu không có phần tử nào thỏa mãn, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa một số nguyên X ($-10^9 \leq X \leq 10^9$).

Đầu ra (Output):

- Với mỗi truy vấn, in ra giá trị nhỏ nhất lớn hơn hoặc bằng X , hoặc -1 nếu không có.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 10 20 30 40 50 25 50 60	30 50 -1

Giải thích: Với mảng $[1,4,6,8,10]$ và các truy vấn X :

- Truy vấn $X = 5$: Phần tử nhỏ nhất trong mảng ≥ 5 là 6.
- Truy vấn $X = 11$: Không có phần tử nào trong mảng ≥ 11 , in ra -1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm một số nguyên X . Hãy lập trình tìm phần tử nhỏ nhất trong mảng lớn hơn hoặc bằng X . Nếu không có phần tử nào thỏa mãn, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 10 20 30 40 50 25 50 60 $\Rightarrow$ Đầu ra kỳ vọng: 30 50 -1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 10 20 30 40 50 25 50 60	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[1,4,6,8,10]$ và các truy vấn X : - Truy vấn $X = 5$: Phần tử nhỏ nhất trong mảng ≥ 5 là 6. - Truy vấn $X = 11$: Khô...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 30 50 -1

Giải thích chi tiết: Với mảng $[1,4,6,8,10]$ và các truy vấn X :

- Truy vấn $X = 5$: Phần tử nhỏ nhất trong mảng ≥ 5 là 6.
- Truy vấn $X = 11$: Không có phần tử nào trong mảng ≥ 11 , in ra -1.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

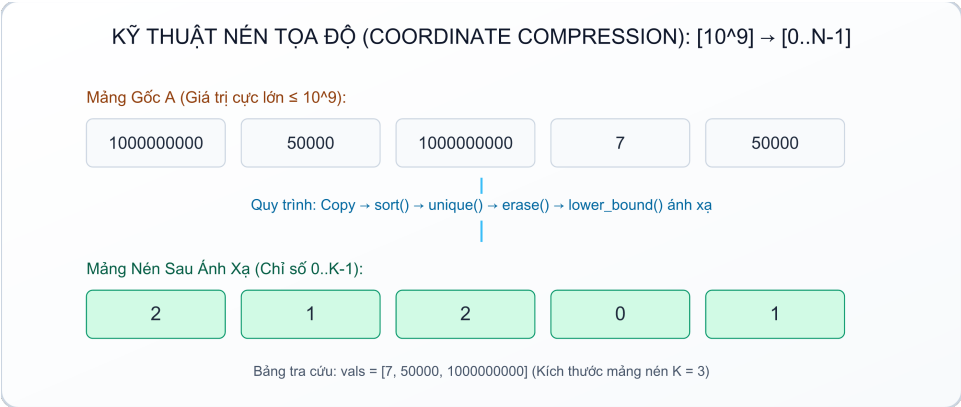
    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    set<long long> st;
    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        st.insert(x);
    }

    while (q--) {
        long long x;
        cin >> x;
        auto it = st.lower_bound(x);
        if (it == st.end()) {
            cout << -1 << "\n";
        } else {
            cout << *it << "\n";
        }
    }
    return 0;
}
```

Bài 05 [cppb_stl_05_hang_doi_uu_tien_k_phan_tu_lon_nhat]: Hàng Đợi Ưu Tiên K Phần Tử Lớn Nhất

Bối cảnh: Một nền tảng phát video trực tuyến theo dõi luồng tương tác của khán giả với hàng triệu lượt đánh giá. Trong luồng dữ liệu liên tục gồm N lượt đánh giá điểm số, ban biên tập cần liên tục lọc ra danh sách K lượt đánh giá có điểm số cao nhất để hiển thị lên bảng tin trang chủ.



Hình minh họa: Mô hình kỹ thuật nén tọa độ bảo toàn thứ tự độ lớn mảng số lớn

Nhiệm vụ: Cho danh sách gồm N số nguyên và số nguyên dương K . Hãy lập trình tìm và in ra K phần tử lớn nhất trong dãy theo thứ tự giảm dần.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm K số nguyên lớn nhất theo thứ tự giảm dần, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 10 50 30 20 60 40	60 50 40

Giải thích: Với mảng gồm 6 phần tử [3,2,1,5,6,4] và cần lấy $K = 2$ phần tử lớn nhất: Hai phần tử lớn nhất trong dãy là 6 và 5. In ra theo thứ tự giảm dần: 6 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách gồm N số nguyên và số nguyên dương K . Hãy lập trình tìm và in ra K phần tử lớn nhất trong dãy theo thứ tự giảm dần.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén tọa độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén tọa độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 6 3 10 50 30 20 60 40 $\Rightarrow$ Đầu ra kỳ vọng: 60 50 40.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 3 10 50 30 20 60 40	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng gồm 6 phần tử [3,2,1,5,6,4] và cần lấy $K = 2$ phần tử lớn nhất: Hai phần tử lớn nhất trong dãy là 6 và 5. In ra theo t...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 60 50 40

Giải thích chi tiết: Với mảng gồm 6 phần tử [3,2,1,5,6,4] và cần lấy $K = 2$ phần tử lớn nhất: Hai phần tử lớn nhất trong dãy là 6 và 5. In ra theo thứ tự giảm dần: 6 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0 || k > n) return 0;

    priority_queue<long long, vector<long long>, greater<long long>> min_heap;

    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        min_heap.push(x);
    }
}
```

```

        if ((int)min_heap.size() > k) {
            min_heap.pop();
        }

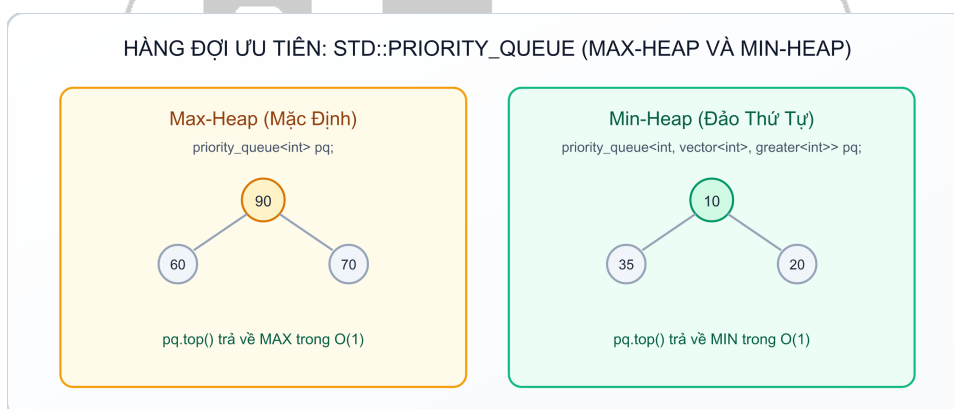
        vector<long long> result;
        while (!min_heap.empty()) {
            result.push_back(min_heap.top());
            min_heap.pop();
        }
        sort(result.rbegin(), result.rend());

        for (int i = 0; i < k; ++i) {
            cout << result[i] << (i + 1 == k ? "" : " ");
        }
        cout << "\n";
        return 0;
    }

```

Bài 06 [cppb_stl_06_quan_ly_tap_hop_da_trung_lap]: Quản Lý Tập Hợp Đa Trùng Lặp (Multiset)

Bối cảnh: Một kho hàng thông minh hỗ trợ 3 loại thao tác quản lý sản phẩm: thêm một sản phẩm có mã giá x vào kho, xóa bỏ đúng một sản phẩm có mã giá x khỏi kho (nếu có), và truy vấn sản phẩm có mã giá rẻ nhất hiện đang có trong kho.



Hình minh họa: Cấu trúc cây vun đồng nhị phân (Binary Heap) của hàng đợi ưu tiên Priority Queue

Nhiệm vụ: Cho Q thao tác thuộc một trong 3 loại: 1 x (thêm x), 2 x (xóa một phần tử x), 3 (in ra giá trị nhỏ nhất hiện tại). Hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho các thao tác loại 3.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo chứa các thao tác mô tả như trên.

Đầu ra (Output):

- Với mỗi thao tác loại 3, in ra giá trị nhỏ nhất hiện tại trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 5 1 5 3 5 2 5 3 5	2 1

Giải thích: Với chuỗi thao tác: thêm 5, thêm 2, thêm 5, truy vấn min $\rightarrow$ in ra 2; xóa 2, truy vấn min $\rightarrow$ in ra 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho Q thao tác thuộc một trong 3 loại: 1 x (thêm x), 2 x (xóa một phần tử x), 3 (in ra giá trị nhỏ nhất hiện tại). Hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho các thao tác loại 3.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 1 5 1 5 3 5 2 5 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 2 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 1 5 1 5 3 5 2 5 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi thao tác: thêm 5, thêm 2, thêm 5, truy vấn min $\rightarrow$ in ra 2; xóa 2, truy vấn min $\rightarrow$ in ra 5.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2 1

Giải thích chi tiết: Với chuỗi thao tác: thêm 5, thêm 2, thêm 5, truy vấn min $\rightarrow$ in ra 2; xóa 2, truy vấn min $\rightarrow$ in ra 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.

- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;

    multiset<long long> ms;

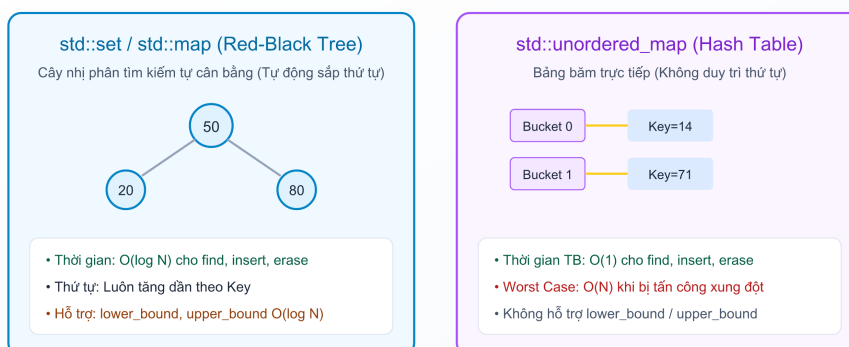
    while (q--) {
        int type;
        long long x;
        cin >> type >> x;

        if (type == 1) { // Thêm x
            ms.insert(x);
        } else if (type == 2) { // Xóa đúng 1 bản sao của x nếu có
            auto it = ms.find(x);
            if (it != ms.end()) ms.erase(it);
        } else { // Đếm số Lượng x
            cout << ms.count(x) << "\n";
        }
    }
    return 0;
}
```

Bài 07 [cppb_stl_07_hop_nhat_cac_doan_so]: Hợp Nhất Các Đoạn Số (Merge Intervals)

Bối cảnh: Một máy chủ lập lịch tiếp nhận N khoảng thời gian đặt lịch phòng họp, khoảng thời gian thứ i bắt đầu từ thời điểm L_i và kết thúc tại R_i . Một số khoảng thời gian bị chồng lấn (giao nhau) hoặc tiếp xúc liền kề. Ban quản trị tòa nhà cần hợp nhất tất cả các khoảng thời gian bị giao nhau lại thành các khối thời gian liền mạch độc lập.

SO SÁNH CẤU TRÚC DỮ LIỆU STL: SET / MAP (R-B TREE) VS UNORDERED_MAP (HASH)



Hình minh họa: Cấu trúc cây đỏ đen (Red-Black Tree) cân bằng tự động của `std::set` và `std::map`

Nhiệm vụ: Cho danh sách N khoảng thời gian $[L_i, R_i]$. Hãy lập trình hợp nhất các khoảng giao nhau và in ra danh sách các khoảng thời gian sau khi đã hợp nhất theo thứ tự thời điểm bắt đầu tăng dần.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- N dòng tiếp theo, mỗi dòng chứa hai số nguyên L_i và R_i ($0 \leq L_i \leq R_i \leq 10^9$).

Đầu ra (Output):

- Dòng 1: In ra số lượng khoảng thời gian M sau khi hợp nhất.
- M dòng tiếp theo, mỗi dòng in ra hai số nguyên là điểm đầu và điểm cuối của một khoảng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 3 2 6 8 10 15 18	3 1 6 8 10 15 18

Giải thích: Với các khoảng thời gian $[1,3]$, $[2,6]$, $[8,10]$, $[15,18]$:

- Khoảng $[1,3]$ và $[2,6]$ giao nhau vì $2 \leq 3$, hợp nhất thành khoảng $[1,6]$.
- Các khoảng $[8,10]$ và $[15,18]$ độc lập không giao nhau. Kết quả thu được 3 khoảng: $[1,6]$, $[8,10]$, $[15,18]$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N khoảng thời gian $[L_i, R_i]$. Hãy lập trình hợp nhất các khoảng giao nhau và in ra danh sách các khoảng thời gian sau khi đã hợp nhất theo thứ tự thời điểm bắt đầu tăng dần.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 1 3 2 6 8 10 15 18 $\Rightarrow$ Đầu ra kỳ vọng: 3 1 6 8 10 15 18.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 1 3 2 6 8 10 15 18	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với các khoảng thời gian [1,3], [2,6], [8,10], [15,18]: - Khoảng [1,3] và [2,6] giao nhau vì $2 \leq 3$, hợp nhất thành khoảng...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3 1 6 8 10 15 18

Giải thích chi tiết: Với các khoảng thời gian [1,3], [2,6], [8,10], [15,18]:

- Khoảng [1,3] và [2,6] giao nhau vì $2 \leq 3$, hợp nhất thành khoảng [1,6].
- Các khoảng [8,10] và [15,18] độc lập không giao nhau. Kết quả thu được 3 khoảng: [1,6], [8,10], [15,18].

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    long long l, r;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<Interval> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i].l >> a[i].r;

    sort(a.begin(), a.end(), [](const Interval& x, const Interval& y) {
        if (x.l != y.l) return x.l < y.l;
        return x.r < y.r;
    });

    vector<Interval> merged;
```

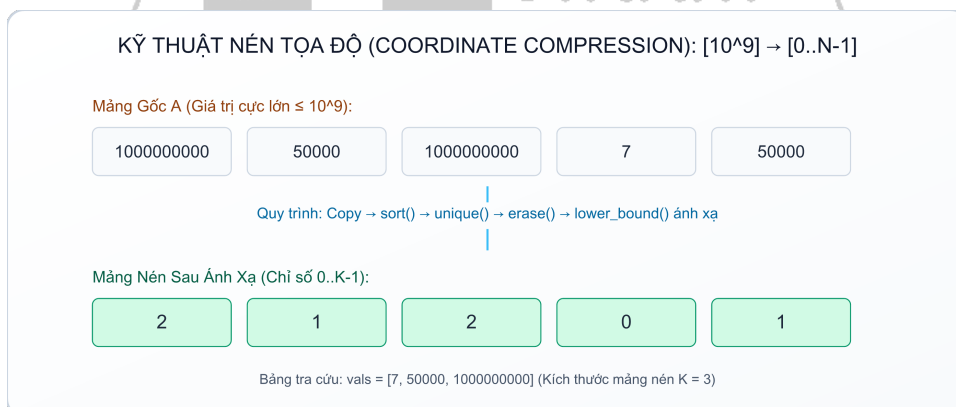
```
merged.push_back(a[0]);

for (int i = 1; i < n; ++i) {
    if (a[i].l <= merged.back().r) {
        merged.back().r = max(merged.back().r, a[i].r);
    } else {
        merged.push_back(a[i]);
    }
}

cout << merged.size() << "\n";
for (const auto& iv : merged) {
    cout << iv.l << " " << iv.r << "\n";
}
return 0;
}
```

Bài 08 [cppb_stl_08_tim_trung_vi_dong_trong_luong_du_lieu]: Tìm Trung Vị Động Trong Luồng Dữ Liệu

Bối cảnh: Trong hệ thống giám sát tải mạng máy tính, các gói tin liên tục gửi về các thông số độ trễ (ping). Để đánh giá độ trễ trung bình chuẩn xác mà không bị ảnh hưởng bởi các giá trị ngoại lai cá biệt, hệ thống cần tính toán giá trị trung vị (median) của luồng dữ liệu ngay sau mỗi khi tiếp nhận thêm một con số mới.



Hình minh họa: Mô hình kỹ thuật nén tọa độ bảo toàn thứ tự độ lớn mảng số lớn

Nhiệm vụ: Cho một luồng dữ liệu gồm N số nguyên đến lần lượt từng số một. Với mỗi số được thêm vào, hãy in ra giá trị trung vị của toàn bộ dãy số đã nhận được từ đầu đến thời điểm đó (lấy phần nguyên dưới nếu số lượng phần tử chẵn).

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là các giá trị trung vị tương ứng sau mỗi bước, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

4 5 15 1 3

5 5 5 3

Giải thích: Với luồng dữ liệu đến lần lượt: 5, 15, 1, 3:

- Nhận 5: dãy [5] → trung vị là 5.
- Nhận 15: dãy [5, 15] → trung vị là 5 (hoặc trung bình lấy nguyên).
- Nhận 1: dãy [1, 5, 15] → trung vị là 5.
- Nhận 3: dãy [1, 3, 5, 15] → trung vị là 3 (hoặc phần nguyên). Kết quả in ra dãy trung vị động tương ứng.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho một luồng dữ liệu gồm N số nguyên đến lần lượt từng số một. Với mỗi số được thêm vào, hãy in ra giá trị trung vị của toàn bộ dãy số đã nhận được từ đầu đến thời điểm đó (lấy phần nguyên dưới nếu số lượng phần tử chẵn).
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 5 15 1 3 ⇒ Đầu ra kỳ vọng: 5 5 5 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 15 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với luồng dữ liệu đến lần lượt: 5, 15, 1, 3: - Nhận 5: dãy [5] → trung vị là 5. - Nhận 15: dãy [5, 15] → trung vị là 5 (hoặc tr...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 5 5 5 3

Giải thích chi tiết: Với luồng dữ liệu đến lần lượt: 5, 15, 1, 3:

- Nhận 5: dãy [5] → trung vị là 5.
- Nhận 15: dãy [5, 15] → trung vị là 5 (hoặc trung bình lấy nguyên).
- Nhận 1: dãy [1, 5, 15] → trung vị là 5.
- Nhận 3: dãy [1, 3, 5, 15] → trung vị là 3 (hoặc phần nguyên). Kết quả in ra dãy trung vị động tương ứng.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xoá **TẤT CẢ** các phần tử có giá trị bằng `val`. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    priority_queue<long long> left_max;
    priority_queue<long long, vector<long long>, greater<long long>> right_min;

    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;

        if (left_max.empty() || x <= left_max.top()) {
            left_max.push(x);
        } else {
            right_min.push(x);
        }

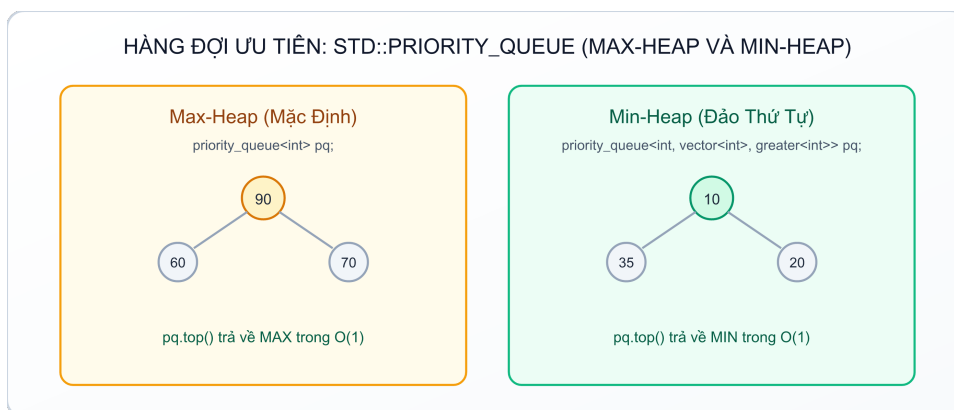
        if (left_max.size() > right_min.size() + 1) {
            right_min.push(left_max.top());
            left_max.pop();
        } else if (right_min.size() > left_max.size()) {
            left_max.push(right_min.top());
            right_min.pop();
        }

        cout << left_max.top() << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
}
```

```
return 0;
}
```

Bài 09 [cppb_stl_09_dem_so_phan_tu_phan_biet_trong_cua_so_k]: Đếm Số Phần Tử Phân Biệt Trong Cửa Sổ K

Bối cảnh: Một máy quét giám sát an ninh quét qua một chuỗi N mã số xe lưu thông trên đường cao tốc. Máy quét sử dụng một khung quan sát (cửa sổ trượt) có kích thước cố định là K xe liên tiếp. Khi cửa sổ trượt dịch chuyển từng vị trí từ đầu đến cuối hàng xe, hệ thống cần đếm xem trong mỗi cửa sổ hiện tại có bao nhiêu mã số xe phân biệt nhau.



Hình minh họa: Cấu trúc cây vun đồng nhị phân (Binary Heap) của hàng đợi ưu tiên Priority Queue

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình in ra số lượng phần tử phân biệt trong mỗi cửa sổ kích thước K khi trượt từ trái sang phải.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm $N - K + 1$ số nguyên là số phần tử phân biệt trong từng cửa sổ, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 4 1 2 1 3 4 2 3	3 4 4 3

Giải thích: Với mảng gồm 7 phần tử $[1, 2, 1, 3, 4, 2, 3]$ và kích thước cửa sổ $K = 4$:

- Cửa sổ 1 $[1, 2, 1, 3]$: gồm các giá trị phân biệt $\{1, 2, 3\} \rightarrow 3$ phần tử.
- Cửa sổ 2 $[2, 1, 3, 4]$: gồm $\{1, 2, 3, 4\} \rightarrow 4$ phần tử.
- Cửa sổ 3 $[1, 3, 4, 2]$: gồm $\{1, 2, 3, 4\} \rightarrow 4$ phần tử.
- Cửa sổ 4 $[3, 4, 2, 3]$: gồm $\{2, 3, 4\} \rightarrow 3$ phần tử. Kết quả in ra: 3 4 4 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình in ra số lượng phần tử phân biệt trong mỗi cửa sổ kích thước K khi trượt từ trái sang phải.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 7 4 1 2 1 3 4 2 3 $\Rightarrow$ Đầu ra kỳ vọng: 3 4 4 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 7 4 1 2 1 3 4 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng gồm 7 phần tử [1,2,1,3,4,2,3] và kích thước cửa sổ $K = 4$: Cửa sổ 1 [1, 2, 1, 3]: gồm các giá trị phân biệt {1, 2, ...}	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3 4 4 3

Giải thích chi tiết: Với mảng gồm 7 phần tử [1,2,1,3,4,2,3] và kích thước cửa sổ $K = 4$:

- Cửa sổ 1 [1, 2, 1, 3]: gồm các giá trị phân biệt {1, 2, 3} $\rightarrow$ 3 phần tử.
- Cửa sổ 2 [2, 1, 3, 4]: gồm {1, 2, 3, 4} $\rightarrow$ 4 phần tử.
- Cửa sổ 3 [1, 3, 4, 2]: gồm {1, 2, 3, 4} $\rightarrow$ 4 phần tử.
- Cửa sổ 4 [3, 4, 2, 3]: gồm {2, 3, 4} $\rightarrow$ 3 phần tử. Kết quả in ra: 3 4 4 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử st.erase(val) trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng val. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ st.erase(st.find(val)).
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp mp[key] sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng mp.count(key) hoặc mp.find(key) != mp.end().

- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0 || k > n) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    map<int, int> freq;
    for (int i = 0; i < k; ++i) freq[a[i]]++;

    cout << freq.size();

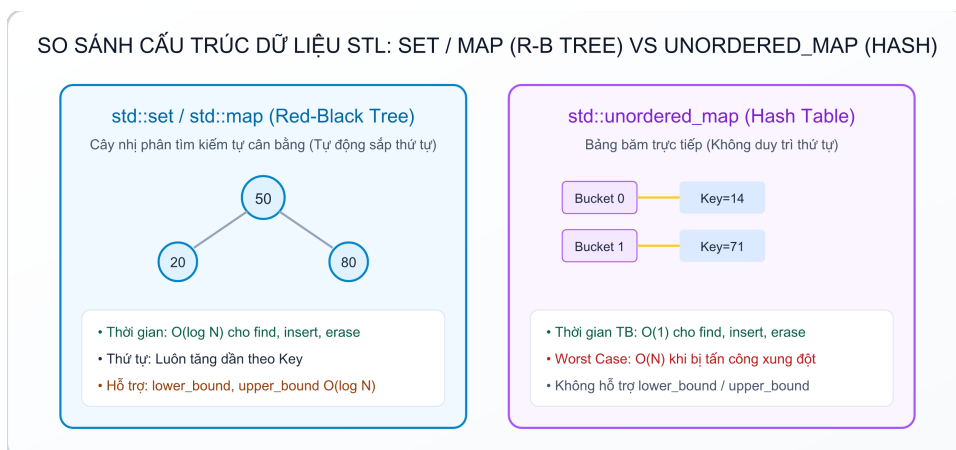
    for (int i = k; i < n; ++i) {
        // Xóa phần tử cũ
        freq[a[i - k]]--;
        if (freq[a[i - k]] == 0) freq.erase(a[i - k]);

        // Thêm phần tử mới
        freq[a[i]]++;

        cout << " " << freq.size();
    }
    cout << "\n";
    return 0;
}
```

Bài 10 [cppb_stl_10_noi_day_chi_phi_nho_nhat]: Nối Dây Chi Phí Nhỏ Nhất (Huffman Greedy)

Bối cảnh: Trong một xưởng cơ khí viễn thông, có N đoạn cáp quang rời rạc với chiều dài lần lượt là $L_1, L_2, \dots, L_N$. Công nhân cần hàn nối tất cả các đoạn cáp này lại thành một sợi cáp duy nhất dài liên tục. Mỗi lần hàn nối hai sợi cáp bất kỳ có chiều dài x và y , chi phí điện năng tiêu hao đúng bằng tổng chiều dài của hai sợi cáp đó ($x + y$). Sợi cáp mới ghép có chiều dài $x + y$ lại được đưa vào tập hợp để tiếp tục nối tiếp.



Hình minh họa: Cấu trúc cây đỏ đen (Red-Black Tree) cân bằng tự động của std::set và std::map

Nhiệm vụ: Cho danh sách chiều dài N đoạn cáp. Hãy lập trình tìm thứ tự nối cáp sao cho tổng chi phí hàn nối là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $L_1, L_2, \dots, L_N$ ($1 \leq L_i \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng chi phí nhỏ nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 3 2 6	29

Giải thích: Với 4 đoạn dây có độ dài [4,3,2,6]:

1. Nối hai dây ngắn nhất 2 và 3 thành dây độ dài 5, chi phí tốn $2 + 3 = 5$. Danh sách dây còn: [4, 5, 6].
2. Nối tiếp hai dây ngắn nhất 4 và 5 thành dây độ dài 9, chi phí tốn $4 + 5 = 9$. Danh sách dây còn: [6, 9].
3. Nối hai dây cuối 6 và 9 thành dây độ dài 15, chi phí tốn $6 + 9 = 15$. Tổng chi phí nhỏ nhất là $5 + 9 + 15 = 29$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách chiều dài N đoạn cáp. Hãy lập trình tìm thứ tự nối cáp sao cho tổng chi phí hàn nối là nhỏ nhất có thể.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $O(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén tọa độ trong $O(\log N)$.

- `priority_queue`: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ**: Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng `unique()` và tìm thứ hạng nén qua `lower_bound()` trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 3 2 6 $\Rightarrow$ Đầu ra kỳ vọng: 29.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 3 2 6	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 đoạn dây có độ dài [4,3,2,6]: 1. Nối hai dây ngắn nhất 2 và 3 thành dây độ dài 5, chi phí tốn $2 + 3 = 5$. Danh sách dây còn...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 29

Giải thích chi tiết: Với 4 đoạn dây có độ dài [4,3,2,6]:

1. Nối hai dây ngắn nhất 2 và 3 thành dây độ dài 5, chi phí tốn $2 + 3 = 5$. Danh sách dây còn: [4, 5, 6].
2. Nối tiếp hai dây ngắn nhất 4 và 5 thành dây độ dài 9, chi phí tốn $4 + 5 = 9$. Danh sách dây còn: [6, 9].
3. Nối hai dây cuối 6 và 9 thành dây độ dài 15, chi phí tốn $6 + 9 = 15$. Tổng chi phí nhỏ nhất là $5 + 9 + 15 = 29$.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 1) { cout << 0 << "\n"; return 0; }

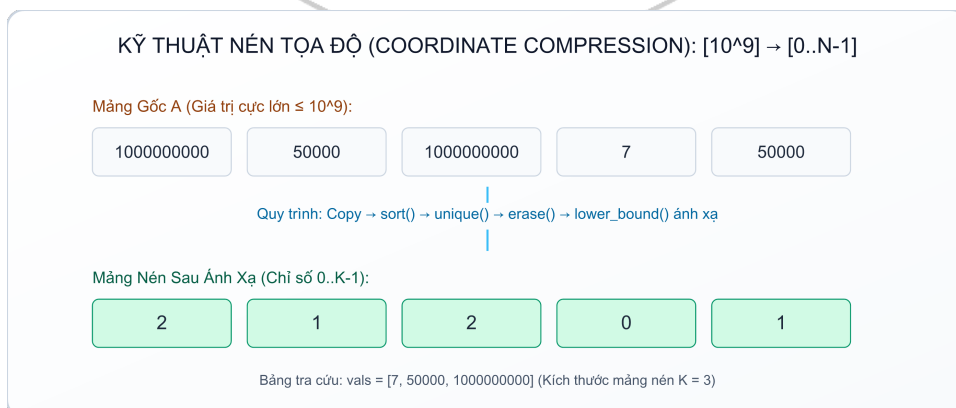
    priority_queue<long long, vector<long long>, greater<long long>> min_heap;
    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        min_heap.push(x);
    }

    long long total_cost = 0;
    while (min_heap.size() > 1) {
        long long a = min_heap.top(); min_heap.pop();
        long long b = min_heap.top(); min_heap.pop();
        long long sum = a + b;
        total_cost += sum;
        min_heap.push(sum);
    }

    cout << total_cost << "\n";
    return 0;
}
```

Bài 11 [cppb_stl_11_lap_lich_cong_viec_toi_uu_may_chu]: Lập Lịch Công Việc Số Máy Chủ Ít Nhất

Bối cảnh: Một trung tâm điện toán đám mây tiếp nhận N tác vụ xử lý dữ liệu. Tác vụ thứ i bắt đầu tại thời điểm S_i và kết thúc tại thời điểm E_i . Một máy chủ chỉ có thể thực hiện một tác vụ tại một thời điểm. Nếu hai tác vụ có khoảng thời gian chạy bị chồng chéo nhau, chúng bắt buộc phải được giao cho hai máy chủ vật lý khác nhau.



Hình minh họa: Mô hình kỹ thuật nén tọa độ bảo toàn thứ tự độ lớn mảng số lớn

Nhiệm vụ: Cho danh sách thời gian bắt đầu và kết thúc của N tác vụ. Hãy lập trình xác định số lượng máy chủ vật lý tối thiểu cần chuẩn bị để phục vụ toàn bộ các tác vụ.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- N dòng tiếp theo, mỗi dòng chứa hai số nguyên S_i và E_i ($0 \leq S_i < E_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng máy chủ tối thiểu cần dùng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 0 30 5 10 15 20	2

Giải thích: Với 3 tác vụ: $[0, 30]$, $[5, 10]$, $[15, 20]$: Tại thời điểm $t = 5$, tác vụ 1 $[0, 30]$ đang chạy trên máy 1, nên tác vụ 2 $[5, 10]$ bắt buộc phải mở thêm máy 2. Tại thời điểm $t = 15$, tác vụ 2 đã xong nhưng tác vụ 1 vẫn đang chạy, nên tác vụ 3 có thể tái sử dụng máy 2. Số máy chủ tối thiểu cần dùng là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách thời gian bắt đầu và kết thúc của N tác vụ. Hãy lập trình xác định số lượng máy chủ vật lý tối thiểu cần chuẩn bị để phục vụ toàn bộ các tác vụ.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đồ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 0 30 5 10 15 20 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 0 30 5 10 15 20	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 3 tác vụ: $[0, 30]$, $[5, 10]$, $[15, 20]$: Tại thời điểm $t = 5$, tác vụ 1 $[0, 30]$ đang chạy trên máy 1, nên tác vụ 2 $[5, 10]$ bắt buộc p...	Cập nhật các biến / mảng trạng thái

3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2
---	--------------------	--------------------------------	-----------------

Giải thích chi tiết: Với 3 tác vụ: [0, 30], [5, 10], [15, 20]: Tại thời điểm $t = 5$, tác vụ 1 [0, 30] đang chạy trên máy 1, nên tác vụ 2 [5, 10] bắt buộc phải mở thêm máy 2. Tại thời điểm $t = 15$, tác vụ 2 đã xong nhưng tác vụ 1 vẫn đang chạy, nên tác vụ 3 có thể tái sử dụng máy 2. Số máy chủ tối thiểu cần dùng là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Job {
    long long s, e;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<Job> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i].s >> a[i].e;

    sort(a.begin(), a.end(), [](const Job& x, const Job& y) {
        if (x.s != y.s) return x.s < y.s;
        return x.e < y.e;
    });

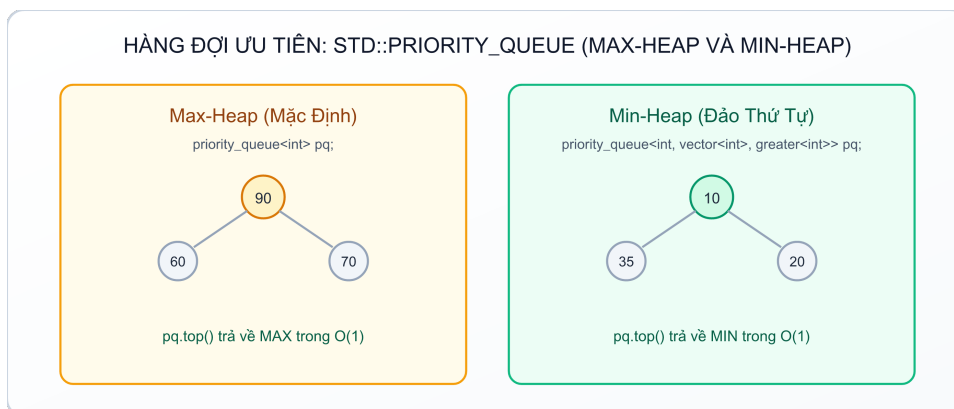
    priority_queue<long long, vector<long long>, greater<long long>> servers;

    for (int i = 0; i < n; ++i) {
        if (!servers.empty() && servers.top() <= a[i].s) {
            servers.pop();
        }
        servers.push(a[i].e);
    }

    cout << servers.size() << "\n";
    return 0;
}
```

Bài 12 [cppb_stl_12_dem_cap_so_co_hieu_bang_k_so_lon]: Đếm Cặp Số Có Hiệu Bằng K

Bối cảnh: Trong một thuật toán mã hóa khóa công khai, một danh sách gồm N số nguyên bí mật được đưa vào xử lý. Kỹ sư an ninh cần đếm xem trong danh sách có bao nhiêu cặp chỉ số (i, j) thỏa mãn điều kiện phần tử đứng sau lớn hơn phần tử đứng trước đúng một khoảng cách cố định K , tức là $A_j - A_i = K$ (với $i < j$).



Hình minh họa: Cấu trúc cây vun đồng nhị phân (Binary Heap) của hàng đợi ưu tiên Priority Queue

Nhiệm vụ: Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình đếm số lượng cặp (i, j) có $i < j$ và $A_j - A_i = K$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và K ($1 \leq N \leq 10^5, -10^9 \leq K \leq 10^9$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 5 3 4 2	3

Giải thích: Với mảng $[1, 5, 3, 4, 2]$ và $K = 2$: Các cặp có hiệu bằng 2 là:

- $A_2 - A_3 = 5 - 3 = 2$.
- $A_3 - A_1 = 3 - 1 = 2$.
- $A_4 - A_5 = 4 - 2 = 2$. Có tổng cộng 3 cặp số thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình đếm số lượng cặp (i, j) có $i < j$ và $A_j - A_i = K$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Lựa chọn cấu trúc dữ liệu tối ưu:**

- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xoá, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 1 5 3 4 2 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 1 5 3 4 2	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng [1,5,3,4,2] và $K = 2$: Các cặp có hiệu bằng 2 là: - $A_2 - A_3 = 5 - 3 = 2$. - $A_3 - A_1 = 3 - 1 = 2$. - $A_4 - A_5 = 4 - 2 = 2$.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với mảng [1,5,3,4,2] và $K = 2$: Các cặp có hiệu bằng 2 là:

- $A_2 - A_3 = 5 - 3 = 2$.
- $A_3 - A_1 = 3 - 1 = 2$.
- $A_4 - A_5 = 4 - 2 = 2$. Có tổng cộng 3 cặp số thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử st.erase(val) trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng val. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ st.erase(st.find(val)).
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp mp[key] sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng mp.count(key) hoặc mp.find(key) != mp.end().
- Hàng đợi ưu tiên priority_queue mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: priority_queue<long long, vector<long long>, greater<long long>> pq;

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0) return 0;

    map<long long, int> freq;
    long long ans = 0;

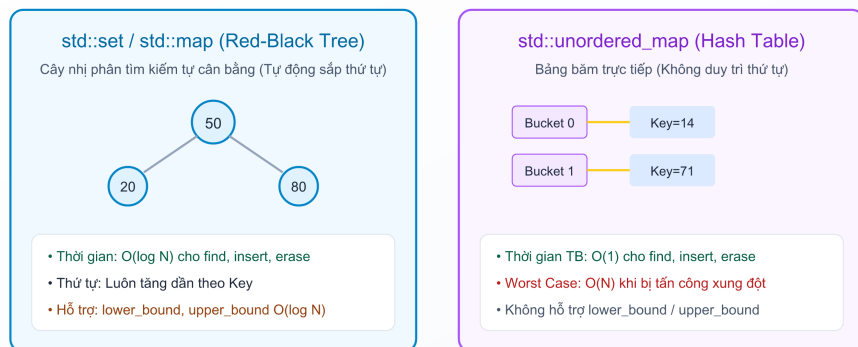
    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        if (freq.count(x - k)) ans += freq[x - k];
        if (k != 0 && freq.count(x + k)) ans += freq[x + k];
        freq[x]++;
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 13 [cppb_stl_13_truy_van_phan_tu_xuat_hien_nhieu_nhat]: Phần Tử Xuất Hiện Nhiều Nhất (Mode)

Bối cảnh: Một hệ thống kiểm phiếu biểu quyết tiếp nhận N lá phiếu được đánh mã số nguyên. Ban kiểm phiếu cần xác định giá trị mã số nào xuất hiện với tần suất nhiều nhất trong hòm phiếu (giá trị một - Mode). Nếu có nhiều mã số cùng đạt số phiếu cao nhất bằng nhau, hệ thống quy định sẽ ưu tiên chọn mã số có giá trị số học nhỏ nhất.

SO SÁNH CẤU TRÚC DỮ LIỆU STL: SET / MAP (R-B TREE) VS UNORDERED_MAP (HASH)



Hình minh họa: Cấu trúc cây đỏ đen (Red-Black Tree) cân bằng tự động của `std::set` và `std::map`

Nhiệm vụ: Cho danh sách N lá phiếu nguyên. Hãy lập trình tìm mã số xuất hiện nhiều lần nhất. Nếu có nhiều mã số cùng xuất hiện nhiều nhất, in ra mã số có giá trị nhỏ nhất trong số đó.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).

- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất giá trị mã số xuất hiện nhiều nhất theo quy ước trên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 1 3 2 1 4 1	1 3

Giải thích: Với dãy số $[1, 2, 2, 3, 1]$: Cả hai số 1 và 2 đều xuất hiện đúng 2 lần (nhiều nhất). Theo quy định ưu tiên giá trị số học nhỏ hơn, ta chọn số 1. Kết quả in ra là 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N lá phiếu nguyên. Hãy lập trình tìm mã số xuất hiện nhiều lần nhất. Nếu có nhiều mã số cùng xuất hiện nhiều nhất, in ra mã số có giá trị nhỏ nhất trong số đó.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén toạ độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén toạ độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 6 1 3 2 1 4 1 $\Rightarrow$ Đầu ra kỳ vọng: 1 3.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 1 3 2 1 4 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số $[1, 2, 2, 3, 1]$: Cả hai số 1 và 2 đều xuất hiện đúng 2 lần (nhiều nhất). Theo quy định ưu tiên giá trị số học nhỏ hơn, ta ...	Cập nhật các biến / mảng trạng thái

3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 3
---	--------------------	--------------------------------	-------------------

Giải thích chi tiết: Với dãy số [1,2,2,3,1]: Cả hai số 1 và 2 đều xuất hiện đúng 2 lần (nhiều nhất). Theo quy định ưu tiên giá trị số học nhỏ hơn, ta chọn số 1. Kết quả in ra là 1.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

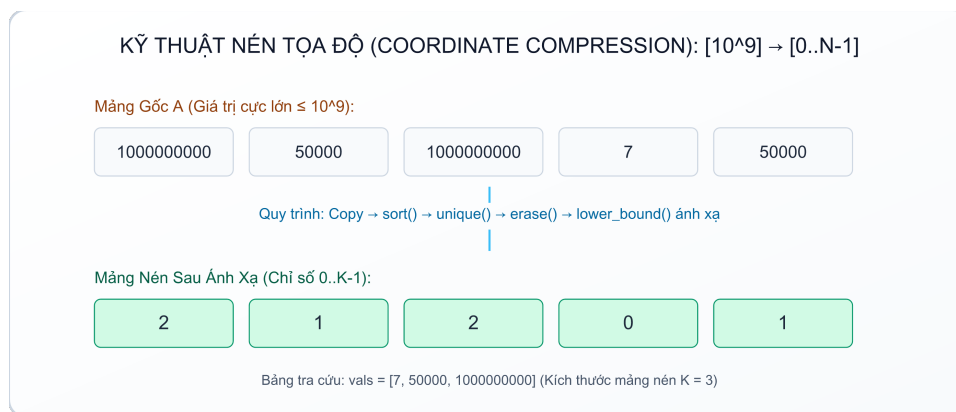
    map<long long, int> freq;
    long long best_val = -1;
    int max_freq = 0;

    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        freq[x]++;
        if (freq[x] > max_freq || (freq[x] == max_freq && x < best_val)) {
            max_freq = freq[x];
            best_val = x;
        }
    }

    cout << best_val << " " << max_freq << "\n";
    return 0;
}
```

Bài 14 [cppb_stl_14_tim_cap_diem_gan_nhat]: Cặp Điểm Gần Nhất (Closest Pair Of Points)

Bối cảnh: Trên màn hình hiển thị radar hàng hải, có N tàu biển đang hoạt động trên mặt biển. Tàu thứ i có tọa độ vị trí là (X_i, Y_i) . Để cảnh báo nguy cơ va chạm sớm cho đài chỉ huy, hệ thống an toàn hàng hải cần tính toán khoảng cách hình học nhỏ nhất (khoảng cách Euclid) giữa hai tàu biển bất kỳ trong toàn bộ khu vực.



Hình minh họa: Mô hình kỹ thuật nén tọa độ bảo toàn thứ tự độ lớn mảng số lớn

Nhiệm vụ: Cho tọa độ của N điểm trên mặt phẳng. Hãy lập trình tìm bình phương khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($2 \leq N \leq 10^5$).
- N dòng tiếp theo, mỗi dòng chứa hai số nguyên X_i và Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất bình phương khoảng cách nhỏ nhất giữa hai điểm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 0 0 1 2 3 1 4 0	5

Giải thích: Với 4 điểm tọa độ: (0,0), (1,1), (2,2), (2,0): Khoảng cách giữa điểm (1,1) và (2,2) có bình phương là $(2-1)^2 + (2-1)^2 = 1 + 1 = 2$. Khoảng cách giữa điểm (1,1) và (2,0) có bình phương là $(2-1)^2 + (0-1)^2 = 1 + 1 = 2$. Bình phương khoảng cách nhỏ nhất giữa hai điểm bất kỳ là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho tọa độ của N điểm trên mặt phẳng. Hãy lập trình tìm bình phương khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khoá - giá trị với các truy vấn đếm tần suất, nén tọa độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- Kỹ thuật nén tọa độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 0 0 1 2 3 1 4 0 $\Rightarrow$ Đầu ra kỳ vọng: 5.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 0 0 1 2 3 1 4 0	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 điểm tọa độ: (0,0), (1,1), (2,2), (2,0) : Khoảng cách giữa điểm (1,1) và (2,2) có bình phương là $(2-1)^2 + (2-1)^2 = 1 + 1 = 2$. Khoảng cách giữa điểm (1,1) và (2,0) có bình phương là $(2-1)^2 + (0-1)^2 = 1 + 1 = 2$. Bình phương khoảng cách nhỏ nhất giữa hai điểm bất kỳ là 2.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 5

Giải thích chi tiết: Với 4 điểm tọa độ: (0,0), (1,1), (2,2), (2,0): Khoảng cách giữa điểm (1,1) và (2,2) có bình phương là $(2-1)^2 + (2-1)^2 = 1 + 1 = 2$. Khoảng cách giữa điểm (1,1) và (2,0) có bình phương là $(2-1)^2 + (0-1)^2 = 1 + 1 = 2$. Bình phương khoảng cách nhỏ nhất giữa hai điểm bất kỳ là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xoá TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xoá đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khoá chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    long long x, y;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 1) { cout << 0 << "\n"; return 0; }
```

```
vector<Point> pts(n);
for (int i = 0; i < n; ++i) cin >> pts[i].x >> pts[i].y;

sort(pts.begin(), pts.end(), [](const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
});

long long min_dist_sq = LLONG_MAX;
set<pair<long long, long long>> active_set; // Lưu (y, x)

int left = 0;
for (int i = 0; i < n; ++i) {
    long long d = ceil(sqrt(min_dist_sq));
    while (left < i && pts[i].x - pts[left].x >= d) {
        active_set.erase({pts[left].y, pts[left].x});
        left++;
    }

    auto it_low = active_set.lower_bound({pts[i].y - d, LLONG_MIN});
    auto it_high = active_set.upper_bound({pts[i].y + d, LLONG_MAX});

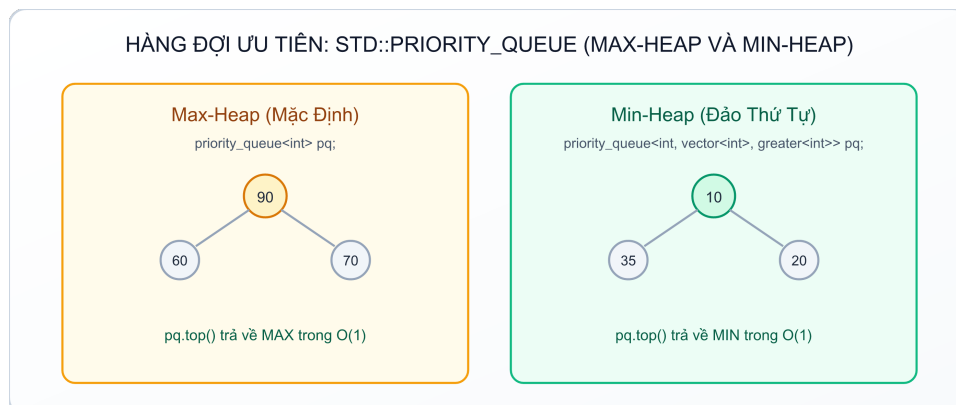
    for (auto it = it_low; it != it_high; ++it) {
        long long dy = pts[i].y - it->first;
        long long dx = pts[i].x - it->second;
        min_dist_sq = min(min_dist_sq, dx * dx + dy * dy);
    }

    active_set.insert({pts[i].y, pts[i].x});
}

cout << min_dist_sq << "\n";
return 0;
}
```

Bài 15 [cppb_stl_15_he_thong_xep_hang_thi_dau_dynamic]: Hệ Thống Xếp Hạng Thi Đấu Dynamic

Bối cảnh: Tại kỳ thi lập trình thuật toán trực tuyến DKOJ của iKHEDU, hệ thống chấm thi thời gian thực cần liên tục ghi nhận điểm số và cập nhật kết quả thi đấu của các thí sinh. Mỗi khi thí sinh nộp bài hoặc cần tra cứu điểm, hệ thống sẽ thực thi một trong hai loại thao tác: cộng điểm cho một thí sinh hoặc truy vấn điểm tích lũy hiện tại của thí sinh đó.



Hình minh họa: Cấu trúc cây vun đồng nhị phân (Binary Heap) của hàng đợi ưu tiên Priority Queue

Nhiệm vụ: Cho danh sách Q thao tác của hệ thống. Bạn hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho mỗi thao tác tra cứu điểm.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương Q ($1 \leq Q \leq 20000$) là số lượng thao tác.
- Q dòng tiếp theo biểu diễn các thao tác thuộc một trong hai dạng:
- 1 Name Score: Cộng thêm $Score$ điểm cho thí sinh có tên $Name$.
- 2 Name: Yêu cầu in ra tổng điểm tích lũy hiện tại của thí sinh $Name$.

Đầu ra (Output):

- Với mỗi thao tác loại 2, in ra trên một dòng một số nguyên duy nhất là tổng điểm của thí sinh đó (nếu thí sinh chưa có điểm, in ra 0).

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 Alice 100 1 Bob 150 2 Alice 1 Alice 60	100

Giải thích: Diễn biến 4 thao tác của hệ thống thi đấu:

1. Thao tác 1 (1 Alice 100): Alice được cộng 100 điểm. Điểm hiện tại của Alice là 100.
2. Thao tác 2 (1 Bob 150): Bob được cộng 150 điểm. Điểm hiện tại của Bob là 150.
3. Thao tác 3 (2 Alice): Truy vấn điểm của Alice. Hệ thống in ra điểm hiện tại là 100.
4. Thao tác 4 (1 Alice 60): Alice được cộng thêm 60 điểm nữa. Điểm tích lũy mới của Alice trở thành $100 + 60 = 160$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách Q thao tác của hệ thống. Bạn hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho mỗi thao tác tra cứu điểm.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Lựa chọn cấu trúc dữ liệu tối ưu:**
- set / multiset: Quản lý tập hợp tự động sắp xếp theo cây đỏ đen, hỗ trợ chèn, xóa, tìm kiếm trong $\mathcal{O}(\log N)$.
- map: Ánh xạ khóa - giá trị với các truy vấn đếm tần suất, nén tọa độ trong $\mathcal{O}(\log N)$.
- priority_queue: Hàng đợi ưu tiên (Binary Heap) cho phép lấy phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật trong $\mathcal{O}(\log N)$.
- **Kỹ thuật nén tọa độ:** Sao chép mảng, sắp xếp tăng dần, loại bỏ phần tử trùng bằng unique() và tìm thứ hạng nén qua lower_bound() trong $\mathcal{O}(N \log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 1 Alice 100 1 Bob 150 2 Alice 1 Alice 60 $\Rightarrow$ Đầu ra kỳ vọng: 100.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 1 Alice 100 1 Bob 150 2 Alice 1 Alice 60	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Diễn biến 4 thao tác của hệ thống thi đấu: 1. Thao tác 1 (1 Alice 100): Alice được cộng 100 điểm. Điểm hiện tại của Alice là 100. 2. ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 100

Giải thích chi tiết: Diễn biến 4 thao tác của hệ thống thi đấu:

1. Thao tác 1 (1 Alice 100): Alice được cộng 100 điểm. Điểm hiện tại của Alice là 100.
2. Thao tác 2 (1 Bob 150): Bob được cộng 150 điểm. Điểm hiện tại của Bob là 150.
3. Thao tác 3 (2 Alice): Truy vấn điểm của Alice. Hệ thống in ra điểm hiện tại là 100.
4. Thao tác 4 (1 Alice 60): Alice được cộng thêm 60 điểm nữa. Điểm tích lũy mới của Alice trở thành $100 + 60 = 160$.

3. Lưu ý & Bẫy lỗi thường gặp

- Sử dụng toán tử `st.erase(val)` trên multiset sẽ xóa TẤT CẢ các phần tử có giá trị bằng `val`. Để chỉ xóa đúng một phần tử, bắt buộc dùng con trỏ `st.erase(st.find(val))`.
- Truy cập vào khóa chưa tồn tại trong map qua cú pháp `mp[key]` sẽ tự động chèn một cặp mới với giá trị mặc định là 0, làm tăng kích thước bộ nhớ ngoài ý muốn. Khi kiểm tra tồn tại, nên dùng `mp.count(key)` hoặc `mp.find(key) != mp.end()`.
- Hàng đợi ưu tiên `priority_queue` mặc định là Max-Heap. Muốn biến thành Min-Heap cần khai báo đầy đủ: `priority_queue<long long, vector<long long>, greater<long long>> pq;`

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
```

```
map<string, long long> scores;
multiset<long long> all_scores;

while (q-->0) {
    int type;
    cin >> type;
    if (type == 1) { // Cộng điểm cho thí sinh
        string name;
        long long delta;
        cin >> name >> delta;

        if (scores.count(name)) {
            auto it = all_scores.find(scores[name]);
            if (it != all_scores.end()) all_scores.erase(it);
        }

        scores[name] += delta;
        all_scores.insert(scores[name]);
    } else { // Truy vấn điểm của thí sinh
        string name;
        cin >> name;
        cout << (scores.count(name) ? scores[name] : 0) << "\n";
    }
}
return 0;
}
```



Bài 20: Cấu trúc dữ liệu ngăn xếp (stack) & Monotonic Stack

Bài 01 [cppb_stk_01_kiem_tra_day_ngoac_dung_co_ban]: Kiểm Tra Dãy Ngoặc Đúng Đơn Loại

Bối cảnh: Trong một trình biên dịch ngôn ngữ lập trình, việc đóng mở các cặp dấu ngoặc tròn (và) phải tuân thủ nghiêm ngặt quy tắc ngữ pháp: Mỗi dấu mở ngoặc (phải có một dấu đóng ngoặc) tương ứng xuất hiện sau nó, và tại mọi thời điểm tính từ đầu chuỗi, số lượng dấu mở ngoặc không bao giờ được ít hơn số lượng dấu đóng ngoặc.



Hình minh họa: Nguyên lý hoạt động Vào sau Ra trước (LIFO) của ngăn xếp (Stack)

Nhiệm vụ: Cho một chuỗi chỉ gồm các ký tự (và). Hãy lập trình kiểm tra xem chuỗi đó có phải là một dãy ngoặc đúng hay không. Nếu đúng in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- In ra YES nếu chuỗi là dãy ngoặc đúng, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
((()))	YES

Giải thích: Với chuỗi $S = "((()))"$: Mỗi dấu mở ngoặc đều có đúng một dấu đóng ngoặc tương ứng ghép đôi hợp lệ và không có dấu đóng ngoặc nào bị thừa. Kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một chuỗi chỉ gồm các ký tự (và). Hãy lập trình kiểm tra xem chuỗi đó có phải là một dãy ngoặc đúng hay không. Nếu đúng in ra YES, ngược lại in ra NO.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.

- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: $((())) \Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: $((()))$	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "((()))"$: Mỗi dấu mở ngoặc đều có đúng một dấu đóng ngoặc tương ứng ghép đôi hợp lệ và không có dấu đóng ngoặc n...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với chuỗi $S = "((()))"$: Mỗi dấu mở ngoặc đều có đúng một dấu đóng ngoặc tương ứng ghép đôi hợp lệ và không có dấu đóng ngoặc nào bị thừa. Kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int open_cnt = 0;
    for (char c : s) {
        if (c == '(') {
```

```

        open_cnt++;
    } else {
        if (open_cnt == 0) {
            cout << "NO\n";
            return 0;
        }
        open_cnt--;
    }
}

if (open_cnt == 0) cout << "YES\n";
else cout << "NO\n";
return 0;
}

```

Bài 02 [cppb_stk_02_day_ngoac_hon_hop_nhieu_loai]: Dãy Ngoặc Hồn Hợp Nhiều Loại

Bối cảnh: Trình biên dịch mã nguồn mở rộng hỗ trợ đồng thời 3 loại cặp dấu ngoặc khác nhau: ngoặc tròn (), ngoặc vuông [] và ngoặc nhọn {}. Một chuỗi ngoặc hỗn hợp được coi là hợp lệ nếu các cặp ngoặc cùng loại được đóng mở đúng quy tắc lồng nhau, không bị đóng chéo (ví dụ: ([)] là sai quy tắc vì ngoặc tròn mở trước nhưng lại bị ngoặc vuông xen vào đóng trước).

MONOTONIC STACK: TÌM PHẦN TỬ LỚN HƠN TIẾP THEO (NEXT GREATER ELEMENT)

Mảng A = [4, 2, 1, 5, 3]



Bất Biến Ngăn Xếp Đơn Điều Giảm Dần (Lưu Chỉ Số i):

- Duyệt qua phần tử 5: 5 lớn hơn 1 (pop 1, NGE[1]=5), 5 lớn hơn 2 (pop 2, NGE[2]=5), 5 lớn hơn 4 (pop 4, NGE[4]=5)
- Mỗi phần tử vào/ra Stack đúng 1 lần duy nhất → Tổng thời gian toàn mảng O(N)

Kết quả NGE = [5, 5, 5, -1, -1]

Hình minh họa: Kỹ thuật ngăn xếp đơn điệu tìm phần tử lớn hơn tiếp theo (Next Greater Element)

Nhiệm vụ: Cho chuỗi S chỉ gồm các ký tự (,), [,], {, }. Hãy lập trình kiểm tra tính hợp lệ của chuỗi. Nếu hợp lệ in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- In ra YES nếu chuỗi hợp lệ, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
{[()]}	YES

Giải thích: Với chuỗi $S = "[()]"$: Cặp ngoặc tròn nằm hoàn toàn bên trong ngoặc vuông, và ngoặc vuông nằm bên trong ngoặc nhọn. Quy tắc lồng nhau được bảo đảm trọn vẹn, kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi S chỉ gồm các ký tự $(,), [,], \{, \}$. Hãy lập trình kiểm tra tính hợp lệ của chuỗi. Nếu hợp lệ in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack ($\text{pop}()$).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: $\{[()]\}$ $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: $\{[()]\}$	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = "[()]"$: Cặp ngoặc tròn nằm hoàn toàn bên trong ngoặc vuông, và ngoặc vuông nằm bên trong ngoặc nhọn. Quy tắc l...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với chuỗi $S = "[()]"$: Cặp ngoặc tròn nằm hoàn toàn bên trong ngoặc vuông, và ngoặc vuông nằm bên trong ngoặc nhọn. Quy tắc lồng nhau được bảo đảm trọn vẹn, kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi $\text{stk.top}()$ hoặc $\text{stk.pop}()$ trên ngăn xếp rỗng. Bắt buộc kiểm tra $!\text{stk.empty}()$ trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

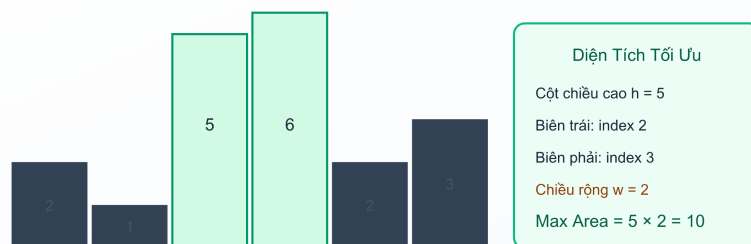
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty()) {
                cout << "NO\n";
                return 0;
            }
            char top = st.top();
            if ((c == ')' && top == '(') || (c == ']' && top == '[') || (c == '}' && top == '{')) {
                st.pop();
            } else {
                cout << "NO\n";
                return 0;
            }
        }
    }

    if (st.empty()) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```

Bài 03 [cppb_stk_03_danh_gia_bieu_thuc_hau_to_rpn]: Đánh Giá Biểu Thức Hậu Tố (Reverse Polish Notation)

Bối cảnh: Trong thiết kế vi xử lý và máy tính cầm tay, biểu thức toán học thường được chuyển đổi sang dạng ký pháp nghịch đảo Ba Lan (hậu tố - Postfix) để máy tính dễ dàng tính toán mà không cần quan tâm đến độ ưu tiên của dấu ngoặc. Trong biểu thức hậu tố, toán tử luôn đứng sau hai toán hạng của nó (ví dụ: $2\ 3\ +$ tương đương $2 + 3 = 5$).

HÌNH CHỮ NHẬT LỚN NHẤT TRÊN BIỂU ĐỒ CỘT (LARGEST RECTANGLE IN HISTOGRAM)



Hình minh họa: Ứng dụng ngăn xếp đơn điệu giải bài toán hình chữ nhật lớn nhất trong biểu đồ cột

Nhiệm vụ: Cho một biểu thức hậu tố gồm các số nguyên và 4 phép toán cơ bản $+$, $-$, $*$, $/$ (chia lấy phần nguyên). Hãy lập trình tính và in ra giá trị cuối cùng của biểu thức.

Đầu vào (Input):

- Một dòng duy nhất chứa các toán hạng và toán tử cách nhau bởi khoảng trắng ($1 \leq$ số lượng token $\leq 10^4$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là giá trị của biểu thức.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 + 3 *	9

Giải thích: Với biểu thức hậu tố “2 1 + 3 *”:

- Gặp toán tử ‘+’: Thực hiện $2 + 1 = 3$.
- Biểu thức trở thành “3 3 *”.
- Gặp toán tử ‘*’: Thực hiện $3 \times 3 = 9$. Giá trị cuối cùng thu được là 9.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một biểu thức hậu tố gồm các số nguyên và 4 phép toán cơ bản $+$, $-$, $*$, $/$ (chia lấy phần nguyên). Hãy lập trình tính và in ra giá trị cuối cùng của biểu thức.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $\mathcal{O}(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 5 2 1 + 3 * $\Rightarrow$ Đầu ra kỳ vọng: 9.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 1 + 3 *	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với biểu thức hậu tố "2 1 + 3": 1. Gặp toán tử '+': Thực hiện $2 + 1 = 3$. 2. Biểu thức trở thành "3 3". 3. Gặp toán tử '*': Thực h...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 9

Giải thích chi tiết: Với biểu thức hậu tố "2 1 + 3 *":

1. Gặp toán tử '+': Thực hiện $2 + 1 = 3$.
2. Biểu thức trở thành "3 3 *".
3. Gặp toán tử '*': Thực hiện $3 \times 3 = 9$. Giá trị cuối cùng thu được là 9.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    stack<long long> st;
    for (int i = 0; i < n; ++i) {
        string token;
        cin >> token;
        if (token == "+" || token == "-" || token == "*") {
            long long b = st.top(); st.pop();
            long long a = st.top(); st.pop();
            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
        } else {
            st.push(stoll(token));
        }
    }

    cout << st.top() << "\n";
}
```

```
return 0;
}
```

Bài 04 [cppb_stk_04_xoa_cac_ky_tu_trung_lap_lien_ke]: Xóa Ký Tự Trùng Lặp Liên Kề

Bối cảnh: Một trò chơi xếp chữ tương tác có quy tắc: Khi hai ký tự giống hệt nhau đứng sát cạnh nhau trong chuỗi, chúng sẽ tự động va chạm và cùng biến mất. Sau khi hai ký tự đó biến mất, hai phần còn lại của chuỗi sẽ dồn sát vào nhau và nếu lại tạo ra hai ký tự giống nhau kề nhau thì quá trình triệt tiêu lại tiếp tục diễn ra cho đến khi không còn cặp ký tự kề nhau nào giống nhau.



Hình minh họa: Nguyên lý hoạt động Vào sau Ra trước (LIFO) của ngăn xếp (Stack)

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình xác định chuỗi ký tự cuối cùng thu được sau khi tất cả các cặp trùng lặp liên kề đã bị triệt tiêu hoàn toàn.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- In ra chuỗi ký tự còn lại sau khi triệt tiêu. Nếu chuỗi bị triệt tiêu hết, in ra chuỗi rỗng hoặc thông báo quy định.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
abbaca	ca

Giải thích: Với chuỗi $S = "abbaca"$:

- Cặp “bb” ở giữa triệt tiêu → chuỗi còn lại là “aaca”.
- Cặp “aa” mới tạo thành kề nhau lại tiếp tục triệt tiêu → chuỗi còn lại là “ca”. Không còn cặp nào trùng nhau kề nhau, chuỗi kết quả in ra là ca.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S . Hãy lập trình xác định chuỗi ký tự cuối cùng thu được sau khi tất cả các cặp trùng lặp liên kề đã bị triệt tiêu hoàn toàn.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $\mathcal{O}(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: *abbaca* $\Rightarrow$ Đầu ra kỳ vọng: *ca*.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: <i>abbaca</i>	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = \text{"abbaca"}$: 1. Cặp “bb” ở giữa triệt tiêu $\rightarrow$ chuỗi còn lại là “aaca”. 2. Cặp “aa” mới tạo thành kề nhau lại tiếp... tiếp...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: <i>ca</i>

Giải thích chi tiết: Với chuỗi $S = \text{"abbaca"}$:

1. Cặp “bb” ở giữa triệt tiêu $\rightarrow$ chuỗi còn lại là “aaca”.
2. Cặp “aa” mới tạo thành kề nhau lại tiếp tục triệt tiêu $\rightarrow$ chuỗi còn lại là “ca”. Không còn cặp nào trùng nhau kề nhau, chuỗi kết quả in ra là *ca*.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```



```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    string st = "";
    for (char c : s) {
        if (!st.empty() && st.back() == c) {
            st.pop_back();
        } else {
            st.push_back(c);
        }
    }

    if (st.empty()) cout << "EMPTY\n";
    else cout << st << "\n";
    return 0;
}

```

Bài 05 [cppb_stk_05_phan_tu_lon_hon_tiep_theo_nge]: Phần Tử Lớn Hơn Tiếp Theo (Next Greater Element)

Bối cảnh: Một đồ thị phân tích thị trường chứng khoán gồm N mốc thời gian liên tiếp với mức giá cổ phiếu lần lượt là $A_1, A_2, \dots, A_N$. Với mỗi phiên giao dịch tại mốc i , nhà phân tích tài chính muốn biết mức giá của phiên giao dịch đầu tiên xuất hiện sau phiên i (bên phải i) có giá trị lớn hơn hẳn A_i . Nếu từ mốc i về sau không có phiên nào có giá cao hơn, ghi nhận giá trị -1.

MONOTONIC STACK: TÌM PHẦN TỬ LỚN HƠN TIẾP THEO (NEXT GREATER ELEMENT)

Mảng $A = [4, 2, 1, 5, 3]$



Bất Biến Ngăn Xếp Đơn Điều Giảm Dần (Lưu Chỉ Số i):

- Duyệt qua phần tử 5: 5 lớn hơn 1 (pop 1, $\text{NGE}[1]=5$), 5 lớn hơn 2 (pop 2, $\text{NGE}[2]=5$), 5 lớn hơn 4 (pop 4, $\text{NGE}[4]=5$)
- Mỗi phần tử vào/ra Stack đúng 1 lần duy nhất $\rightarrow$ Tổng thời gian toàn mảng $O(N)$

Kết quả $\text{NGE} = [5, 5, 5, -1, -1]$

Hình minh họa: Kỹ thuật ngăn xếp đơn điệu tìm phần tử lớn hơn tiếp theo (Next Greater Element)

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Với mỗi phần tử trong mảng, hãy tìm phần tử đầu tiên nằm bên phải nó có giá trị lớn hơn nó. Nếu không có, gán giá trị -1.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là phần tử lớn hơn tiếp theo tương ứng, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 5 2 25	5 25 25 -1

Giải thích: Với mảng gồm 4 phần tử [4,5,2,25]:

- Phần tử 4: Bên phải phần tử đầu tiên lớn hơn 4 là 5.
- Phần tử 5: Bên phải phần tử đầu tiên lớn hơn 5 là 25.
- Phần tử 2: Bên phải phần tử đầu tiên lớn hơn 2 là 25.
- Phần tử 25: Bên phải không còn số nào lớn hơn 25 → in ra -1. Kết quả in ra: 5 25 25 -1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số nguyên A gồm N phần tử. Với mỗi phần tử trong mảng, hãy tìm phần tử đầu tiên nằm bên phải nó có giá trị lớn hơn nó. Nếu không có, gán giá trị -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 5 2 25 ⇒ Đầu ra kỳ vọng: 5 25 25 -1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 5 2 25	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng gồm 4 phần tử [4,5,2,25]: - Phần tử 4: Bên phải phần tử đầu tiên lớn hơn 4 là 5. - Phần tử 5: Bên phải phần tử đầu tiên 1...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 5 25 25 -1

Giải thích chi tiết: Với mảng gồm 4 phần tử [4,5,2,25]:

- Phần tử 4: Bên phải phần tử đầu tiên lớn hơn 4 là 5.
- Phần tử 5: Bên phải phần tử đầu tiên lớn hơn 5 là 25.
- Phần tử 2: Bên phải phần tử đầu tiên lớn hơn 2 là 25.
- Phần tử 25: Bên phải không còn số nào lớn hơn 25 → in ra -1. Kết quả in ra: 5 25 25 -1.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<long long> nge(n, -1);
    stack<int> st;

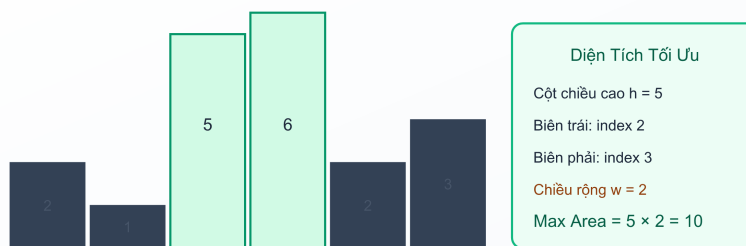
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[i] > a[st.top()]) {
            nge[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }

    for (int i = 0; i < n; ++i) {
        cout << nge[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 06 [cppb_stk_06_phan_tu_nho_hon_gan_nhat_ben_trai]: Phần Tử Nhỏ Hơn Gần Nhất Bên Trái (Previous Smaller Element)

Bối cảnh: Một hệ thống băng tải tự động gồm N vị trí cảm biến. Với mỗi vị trí cảm biến i , hệ thống cần xác định vị trí của cảm biến đầu tiên nằm về phía bên trái có chỉ số đọc nhỏ hơn cảm biến i để tính toán độ dốc địa hình ngược chiều dòng sản phẩm.

HÌNH CHỮ NHẬT LỚN NHẤT TRÊN BIỂU ĐỒ CỘT (LARGEST RECTANGLE IN HISTOGRAM)



Hình minh họa: Ứng dụng ngăn xếp đơn điệu giải bài toán hình chữ nhật lớn nhất trong biểu đồ cột

Nhiệm vụ: Cho mảng A gồm N số nguyên. Với mỗi phần tử A_i , hãy tìm phần tử đầu tiên nằm bên trái nó có giá trị nhỏ hơn nó. Nếu không có phần tử nào thỏa mãn, gán -1.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là các giá trị nhỏ hơn gần nhất bên trái tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 5 2 10 8	-1 4 -1 2 2

Giải thích: Với mảng $[4, 5, 2, 10, 8]$:

- Số 4: Bên trái không có số nào $\rightarrow -1$.
- Số 5: Số đầu tiên bên trái nhỏ hơn 5 là 4.
- Số 2: Bên trái không có số nào nhỏ hơn 2 $\rightarrow -1$.
- Số 10: Số đầu tiên bên trái nhỏ hơn 10 là 2.
- Số 8: Số đầu tiên bên trái nhỏ hơn 8 là 2. Kết quả in ra: -1 4 -1 2 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên. Với mỗi phần tử A_i , hãy tìm phần tử đầu tiên nằm bên trái nó có giá trị nhỏ hơn nó. Nếu không có phần tử nào thỏa mãn, gán -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).

- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 4 5 2 10 8 $\Rightarrow$ Đầu ra kỳ vọng: -1 4 -1 2 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 4 5 2 10 8	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng [4,5,2,10,8]: - Số 4: Bên trái không có số nào $\rightarrow$ -1. - Số 5: Số đầu tiên bên trái nhỏ hơn 5 là 4. - Số 2: Bên trái k...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: -1 4 -1 2 2

Giải thích chi tiết: Với mảng [4,5,2,10,8]:

- Số 4: Bên trái không có số nào $\rightarrow$ -1.
- Số 5: Số đầu tiên bên trái nhỏ hơn 5 là 4.
- Số 2: Bên trái không có số nào nhỏ hơn 2 $\rightarrow$ -1.
- Số 10: Số đầu tiên bên trái nhỏ hơn 10 là 2.
- Số 8: Số đầu tiên bên trái nhỏ hơn 8 là 2. Kết quả in ra: -1 4 -1 2 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
```

```

if (!(cin >> n)) return 0;
if (n <= 0) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

vector<long long> pse(n, -1);
stack<long long> st;

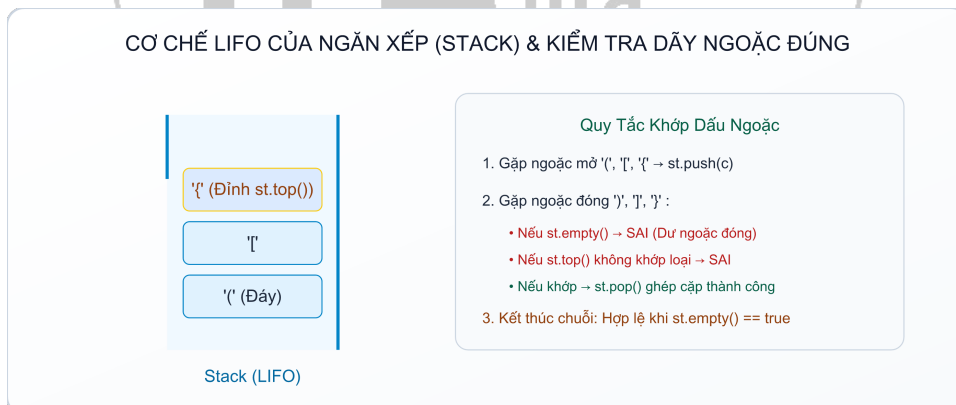
for (int i = 0; i < n; ++i) {
    while (!st.empty() && st.top() >= a[i]) {
        st.pop();
    }
    if (!st.empty()) {
        pse[i] = st.top();
    }
    st.push(a[i]);
}

for (int i = 0; i < n; ++i) {
    cout << pse[i] << (i + 1 == n ? "" : " ");
}
cout << "\n";
return 0;
}

```

Bài 07 [cppb_stk_07_do_dai_day_ngoac_dung_dai_nhat]: Độ Dài Đoạn Ngoặc Đúng Liên Tiếp Dài Nhất

Bối cảnh: Trong một chuỗi ký tự ngoặc hỗn độn bị lỗi truyền dẫn, kỹ sư phần mềm muốn khôi phục lại phần thông điệp hợp lệ dài nhất. Hãy tìm độ dài của một đoạn con liên tiếp dài nhất trong chuỗi sao cho đoạn con đó tạo thành một dãy ngoặc đúng hoàn chỉnh.



Hình minh họa: Nguyên lý hoạt động Vào sau Ra trước (LIFO) của ngăn xếp (Stack)

Nhiệm vụ: Cho chuỗi S chỉ gồm các ký tự (và). Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp là dãy ngoặc đúng.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi ký tự S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là độ dài lớn nhất của đoạn ngoặc đúng liên tiếp.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
)()())	4

Giải thích: Với chuỗi $S = ")()())"$: Đoạn con liên tiếp bắt đầu từ vị trí 1 đến vị trí 4 là “()()” tạo thành dãy ngoặc đúng hoàn chỉnh có độ dài bằng 4. Đây là đoạn ngoặc đúng dài nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi S chỉ gồm các ký tự (và). Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp là dãy ngoặc đúng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào:)()() $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input:)()()	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi $S = ")()())"$: Đoạn con liên tiếp bắt đầu từ vị trí 1 đến vị trí 4 là “()()” tạo thành dãy ngoặc đúng hoàn chỉnh có đ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với chuỗi $S = ")()())"$: Đoạn con liên tiếp bắt đầu từ vị trí 1 đến vị trí 4 là “()()” tạo thành dãy ngoặc đúng hoàn chỉnh có độ dài bằng 4. Đây là đoạn ngoặc đúng dài nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.

- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    stack<int> st;
    st.push(-1);
    int max_len = 0;

    for (int i = 0; i < (int)s.size(); ++i) {
        if (s[i] == '(') {
            st.push(i);
        } else {
            st.pop();
            if (st.empty()) {
                st.push(i);
            } else {
                max_len = max(max_len, i - st.top());
            }
        }
    }

    cout << max_len << "\n";
    return 0;
}
```

Bài 08 [cppb_stk_08_xoa_k_chu_so_de_duoc_so_nho_nhat]: Xóa K Chữ Số Để Được Số Nhỏ Nhất

Bối cảnh: Một mã định danh số lớn gồm N chữ số dạng chuỗi ký tự. Do yêu cầu rút gọn mã trong hệ thống lưu trữ, quản trị viên cần xóa bỏ đúng K chữ số bất kỳ khỏi chuỗi sao cho các chữ số còn lại giữ nguyên thứ tự ban đầu và tạo thành một số nguyên có giá trị nhỏ nhất có thể (không để số 0 vô nghĩa đứng ở đầu nếu kết quả lớn hơn 0).

MONOTONIC STACK: TÌM PHẦN TỬ LỚN HƠN TIẾP THEO (NEXT GREATER ELEMENT)

Mảng A = [4, 2, 1, 5, 3]

4

2

1

5

3

Bất Biến Ngăn Xếp Đơn Điều Giảm Dần (Lưu Chỉ Số i):

- Duyệt qua phần tử 5: 5 lớn hơn 1 (pop 1, NGE[1]=5), 5 lớn hơn 2 (pop 2, NGE[2]=5), 5 lớn hơn 4 (pop 4, NGE[4]=5)
- Mỗi phần tử vào/ra Stack đúng 1 lần duy nhất → Tổng thời gian toàn mảng $O(N)$

Kết quả NGE = [5, 5, 5, -1, -1]

Hình minh họa: Kỹ thuật ngăn xếp đơn điệu tìm phần tử lớn hơn tiếp theo (Next Greater Element)

Nhiệm vụ: Cho chuỗi số S và số nguyên K . Hãy lập trình tìm số nguyên nhỏ nhất thu được sau khi xóa đúng K chữ số.

Đầu vào (Input):

- Dòng 1: Chứa chuỗi ký tự số S ($1 \leq |S| \leq 10^5$).
- Dòng 2: Chứa số nguyên không âm K ($0 \leq K < |S|$).

Đầu ra (Output):

- In ra chuỗi ký tự đại diện cho số nhỏ nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1432219 3	1219

Giải thích: Với số ban đầu là “1432219” và cần xóa đi $K = 3$ chữ số: Ta xóa các chữ số 4, 3, 2 tại các vị trí đầu để giữ lại số “1219”. Đây là số nguyên nhỏ nhất có thể tạo thành.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho chuỗi số S và số nguyên K . Hãy lập trình tìm số nguyên nhỏ nhất thu được sau khi xóa đúng K chữ số.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 1432219 3 $\Rightarrow$ Đầu ra kỳ vọng: 1219.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 1432219 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với số ban đầu là “1432219” và cần xóa đi $K = 3$ chữ số: Ta xóa các chữ số 4, 3, 2 tại các vị trí đầu để giữ lại số “1219”. Đây là số ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1219

Giải thích chi tiết: Với số ban đầu là “1432219” và cần xóa đi $K = 3$ chữ số: Ta xóa các chữ số 4, 3, 2 tại các vị trí đầu để giữ lại số “1219”. Đây là số nguyên nhỏ nhất có thể tạo thành.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string num;
    int k;
    if (!(cin >> num >> k)) return 0;

    string st = "";
    for (char c : num) {
        while (!st.empty() && k > 0 && st.back() > c) {
            st.pop_back();
            k--;
        }
        st.push_back(c);
    }
}
```

```

while (k > 0 && !st.empty()) {
    st.pop_back();
    k--;
}

// Xóa các số 0 ở đầu
int start = 0;
while (start < (int)st.size() && st[start] == '0') start++;

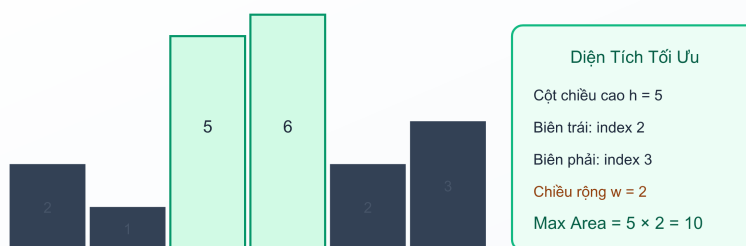
string ans = st.substr(start);
if (ans.empty()) cout << "0\n";
else cout << ans << "\n";
return 0;
}

```

Bài 09 [cppb_stk_09_hinh_chu_nhat_lon_nhat_tren_histogram]: Hình Chữ Nhật Lớn Nhất Trên Histogram

Bối cảnh: Một biểu đồ cột (histogram) gồm N cột hình chữ nhật đứng xếp kề sát nhau trên cùng một đường nằm ngang. Mỗi cột có chiều rộng cố định là 1 đơn vị, cột thứ i có chiều cao là H_i . Bạn thiết kế đồ họa cần tìm diện tích của hình chữ nhật lớn nhất có thể vẽ lọt hoàn toàn vào bên trong biểu đồ cột này.

HÌNH CHỮ NHẬT LỚN NHẤT TRÊN BIỂU ĐỒ CỘT (LARGEST RECTANGLE IN HISTOGRAM)



Hình minh họa: Ứng dụng ngăn xếp đơn điệu giải bài toán hình chữ nhật lớn nhất trong biểu đồ cột

Nhiệm vụ: Cho danh sách chiều cao của N cột. Hãy lập trình tìm diện tích lớn nhất của một hình chữ nhật nằm gọn bên trong biểu đồ cột.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên không âm $H_1, H_2, \dots, H_N$ ($0 \leq H_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất diện tích lớn nhất của hình chữ nhật.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 1 5 6 2 3	10

Giải thích: Với biểu đồ có chiều cao các cột là $[2,1,5,6,2,3]$: Hình chữ nhật lớn nhất được tạo thành bởi hai cột có chiều cao 5 và 6. Chiều cao của hình chữ nhật này là $\min(5,6) = 5$, chiều rộng là 2 cột. Diện tích lớn nhất đạt được là $5 \times 2 = 10$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách chiều cao của N cột. Hãy lập trình tìm diện tích lớn nhất của một hình chữ nhật nằm gọn bên trong biểu đồ cột.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (`pop()`).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 6 2 1 5 6 2 3 $\Rightarrow$ Đầu ra kỳ vọng: 10.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 2 1 5 6 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với biểu đồ có chiều cao các cột là $[2,1,5,6,2,3]$: Hình chữ nhật lớn nhất được tạo thành bởi hai cột có chiều cao 5 và 6. Chiều ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 10

Giải thích chi tiết: Với biểu đồ có chiều cao các cột là $[2,1,5,6,2,3]$: Hình chữ nhật lớn nhất được tạo thành bởi hai cột có chiều cao 5 và 6. Chiều cao của hình chữ nhật này là $\min(5,6) = 5$, chiều rộng là 2 cột. Diện tích lớn nhất đạt được là $5 \times 2 = 10$.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.

- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

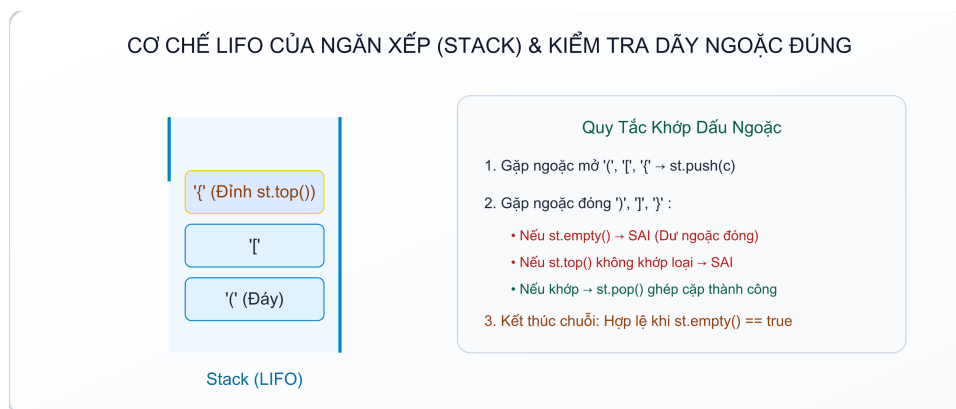
    h.push_back(0); // Cột lính canh
    stack<int> st;
    long long max_area = 0;

    for (int i = 0; i <= n; ++i) {
        while (!st.empty() && h[i] < h[st.top()]) {
            long long height = h[st.top()];
            st.pop();
            long long width = st.empty() ? i : (i - st.top() - 1);
            max_area = max(max_area, height * width);
        }
        st.push(i);
    }

    cout << max_area << "\n";
    return 0;
}
```

Bài 10 [cppb_stk_10_hung_nuoc_mua_trapping_rain_water]: Hứng Nước Mưa (Trapping Rain Water)

Bối cảnh: Một mô hình địa hình gồm N khối bê tông thẳng đứng có chiều rộng 1 đơn vị được xếp thẳng hàng kề sát nhau, khối thứ i có độ cao H_i . Sau một cơn mưa rào lớn, nước mưa sẽ đọng lại ở các thung lũng giữa các khối bê tông cao hơn hai bên. Hãy tính toán tổng lượng nước mưa tối đa có thể đọng lại sau cơn mưa.



Hình minh họa: Nguyên lý hoạt động Vào sau Ra trước (LIFO) của ngăn xếp (Stack)

Nhiệm vụ: Cho danh sách chiều cao của N khối bê tông. Hãy lập trình tính tổng đơn vị thể tích nước mưa có thể đọng lại.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên không âm $H_1, H_2, \dots, H_N$ ($0 \leq H_i \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là tổng thể tích nước mưa đọng lại.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12 0 1 0 2 1 0 1 3 2 1 2 1	6

Giải thích: Với độ cao địa hình là $[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]$: Nước mưa sẽ bị giữ lại ở các vùng trũng giữa các cột cao:

- Tại vị trí 2: nước đọng 1 đơn vị.
- Tại vị trí 4: nước đọng 1 đơn vị.
- Tại vị trí 5: nước đọng 2 đơn vị.
- Tại vị trí 6: nước đọng 1 đơn vị.
- Tại vị trí 9: nước đọng 1 đơn vị. Tổng lượng nước đọng lại là $1 + 1 + 2 + 1 + 1 = 6$ đơn vị.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho danh sách chiều cao của N khối bê tông. Hãy lập trình tính tổng đơn vị thể tích nước mưa có thể đọng lại.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.

- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $\mathcal{O}(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 12 0 1 0 2 1 0 1 3 2 1 2 1 $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 12 0 1 0 2 1 0 1 3 2 1 2 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với độ cao địa hình là [0,1,0,2,1,0,1,3,2,1,2,1] : Nước mưa sẽ bị giữ lại ở các vùng trũng giữa các cột cao: - Tại vị trí 2...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với độ cao địa hình là [0,1,0,2,1,0,1,3,2,1,2,1]: Nước mưa sẽ bị giữ lại ở các vùng trũng giữa các cột cao:

- Tại vị trí 2: nước đọng 1 đơn vị.
- Tại vị trí 4: nước đọng 1 đơn vị.
- Tại vị trí 5: nước đọng 2 đơn vị.
- Tại vị trí 6: nước đọng 1 đơn vị.
- Tại vị trí 9: nước đọng 1 đơn vị. Tổng lượng nước đọng lại là $1 + 1 + 2 + 1 + 1 = 6$ đơn vị.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

    stack<int> st;
    long long total_water = 0;

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && h[i] > h[st.top()]) {
            int top = st.top();
            st.pop();
            if (st.empty()) break;

            int dist = i - st.top() - 1;
            long long bounded_height = min(h[i], h[st.top()]) - h[top];
            total_water += dist * bounded_height;
        }
        st.push(i);
    }

    cout << total_water << "\n";
    return 0;
}
```

Bài 11 [cppb_stk_11_hinh_chu_nhat_toan_mot_lon_nhat_ma_tran]: Hình Chữ Nhật Toàn 1 Lớn Nhất Trong Ma Trận

Bối cảnh: Một tấm tôn kích thước $N \times M$ ô vuông bị đục thủng một số vị trí (ô bị đục ký hiệu bằng số 0, ô nguyên vẹn ký hiệu bằng số 1). Người thợ cơ khí cần cắt ra từ tấm tôn một miếng hình chữ nhật nguyên vẹn (chỉ chứa toàn số 1) sao cho diện tích của miếng tôn cắt ra là lớn nhất có thể.

MONOTONIC STACK: TÌM PHẦN TỬ LỚN HƠN TIẾP THEO (NEXT GREATER ELEMENT)

Mảng A = [4, 2, 1, 5, 3]



Bất Biến Ngăn Xếp Đơn Điều Giảm Dần (Lưu Chỉ Số i):

- Duyệt qua phần tử 5: 5 lớn hơn 1 (pop 1, NGE[1]=5), 5 lớn hơn 2 (pop 2, NGE[2]=5), 5 lớn hơn 4 (pop 4, NGE[4]=5)
- Mỗi phần tử vào/ra Stack đúng 1 lần duy nhất → Tổng thời gian toàn mảng $O(N)$

Kết quả NGE = [5, 5, 5, -1, -1]

Hình minh họa: Kỹ thuật ngăn xếp đơn điệu tìm phần tử lớn hơn tiếp theo (Next Greater Element)

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của hình chữ nhật con chỉ chứa toàn số 1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).

- N dòng tiếp theo, mỗi dòng chứa M số nguyên 0 hoặc 1 cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất diện tích lớn nhất của hình chữ nhật tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 10100 10111 11111 10010	6

Giải thích: Với ma trận kích thước 4×5 : Hình chữ nhật con toàn số 1 lớn nhất có kích thước 2×3 (chiều cao 2, chiều rộng 3) gồm 6 ô số 1. Diện tích lớn nhất đạt được là 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của hình chữ nhật con chỉ chứa toàn số 1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $\mathcal{O}(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 4 5 10100 10111 11111 10010*
⇒ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 10100 10111 11111 10010	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với ma trận kích thước 4×5 : Hình chữ nhật con toàn số 1 lớn nhất có kích thước 2×3 (chiều cao 2, chiều rộng 3) gồm 6 ô số 1. D...	Cập nhật các biến / mảng trạng thái

3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6
---	--------------------	--------------------------------	-----------------

Giải thích chi tiết: Với ma trận kích thước 4×5 : Hình chữ nhật con toàn số 1 lớn nhất có kích thước 2×3 (chiều cao 2, chiều rộng 3) gồm 6 ô số 1. Diện tích lớn nhất đạt được là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    vector<int> h(m, 0);
    int max_area = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (grid[i][j] == '1') h[j]++;
            else h[j] = 0;
        }

        // Monotonic stack on row histogram
        vector<int> cur_h = h;
        cur_h.push_back(0);
        stack<int> st;

        for (int j = 0; j <= m; ++j) {
            while (!st.empty() && cur_h[j] < cur_h[st.top()]) {
                int height = cur_h[st.top()];
                st.pop();
                int width = st.empty() ? j : (j - st.top() - 1);
                max_area = max(max_area, height * width);
            }
            st.push(j);
        }
    }

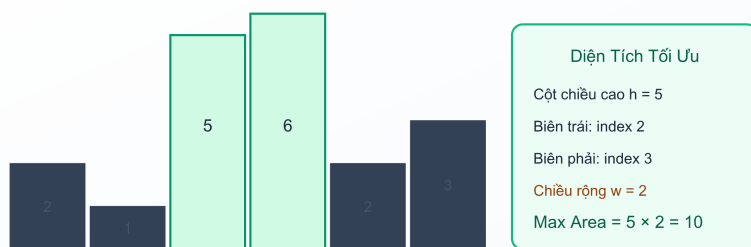
    cout << max_area << "\n";
}
```

```
return 0;
}
```

Bài 12 [cppb_stk_12_tong_gia_tri_nho_nhat_moi_doan_con]: Tổng Giá Trị Nhỏ Nhất Của Mọi Đoạn Con

Bối cảnh: Cho mảng A gồm N số nguyên. Một đoạn con của mảng là một chuỗi các phần tử liên tiếp $A[L..R]$ ($1 \leq L \leq R \leq N$). Với mỗi đoạn con, ta xác định giá trị nhỏ nhất của đoạn con đó là $\min(A[L..R])$. Ban toán học cần tính tổng của tất cả các giá trị nhỏ nhất này trên mọi đoạn con có thể tạo thành từ mảng.

HÌNH CHỮ NHẬT LỚN NHẤT TRÊN BIỂU ĐỒ CỘT (LARGEST RECTANGLE IN HISTOGRAM)



Hình minh họa: Ứng dụng ngắn xếp đơn điệu giải bài toán hình chữ nhật lớn nhất trong biểu đồ cột

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình tính tổng giá trị nhỏ nhất của tất cả các đoạn con, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng tìm được theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 4	17

Giải thích: Với mảng gồm 4 phần tử $[3, 1, 2, 4]$: Các đoạn con có giá trị nhỏ nhất tương ứng:

- Độ dài 1: $[3] \rightarrow 3$, $[1] \rightarrow 1$, $[2] \rightarrow 2$, $[4] \rightarrow 4$ (tổng = 10).
- Độ dài 2: $[3, 1] \rightarrow 1$, $[1, 2] \rightarrow 1$, $[2, 4] \rightarrow 2$ (tổng = 4).
- Độ dài 3: $[3, 1, 2] \rightarrow 1$, $[1, 2, 4] \rightarrow 1$ (tổng = 2).
- Độ dài 4: $[3, 1, 2, 4] \rightarrow 1$ (tổng = 1). Tổng cộng toàn bộ là $10 + 4 + 2 + 1 = 17$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên. Hãy lập trình tính tổng giá trị nhỏ nhất của tất cả các đoạn con, lấy dư cho $10^9 + 7$.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 2 4 $\Rightarrow$ Đầu ra kỳ vọng: 17.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 2 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng gồm 4 phần tử [3,1,2,4]: Các đoạn con có giá trị nhỏ nhất tương ứng: - Độ dài 1: [3] $\rightarrow$ 3, [1] $\rightarrow$ 1, [2] $\rightarrow$ 2, [4] $\rightarrow$ 4 (...)	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 17

Giải thích chi tiết: Với mảng gồm 4 phần tử [3,1,2,4]: Các đoạn con có giá trị nhỏ nhất tương ứng:

- Độ dài 1: [3] $\rightarrow$ 3, [1] $\rightarrow$ 1, [2] $\rightarrow$ 2, [4] $\rightarrow$ 4 (tổng = 10).
- Độ dài 2: [3,1] $\rightarrow$ 1, [1,2] $\rightarrow$ 1, [2,4] $\rightarrow$ 2 (tổng = 4).
- Độ dài 3: [3,1,2] $\rightarrow$ 1, [1,2,4] $\rightarrow$ 1 (tổng = 2).
- Độ dài 4: [3,1,2,4] $\rightarrow$ 1 (tổng = 1). Tổng cộng toàn bộ là $10 + 4 + 2 + 1 = 17$.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi stk.top() hoặc stk.pop() trên ngăn xếp rỗng. Bắt buộc kiểm tra !stk.empty() trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<int> left_bound(n), right_bound(n);
    stack<int> st;

    // Tìm biên trái nghiêm ngặt nhỏ hơn
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] > a[i]) st.pop();
        left_bound[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }

    while (!st.empty()) st.pop();

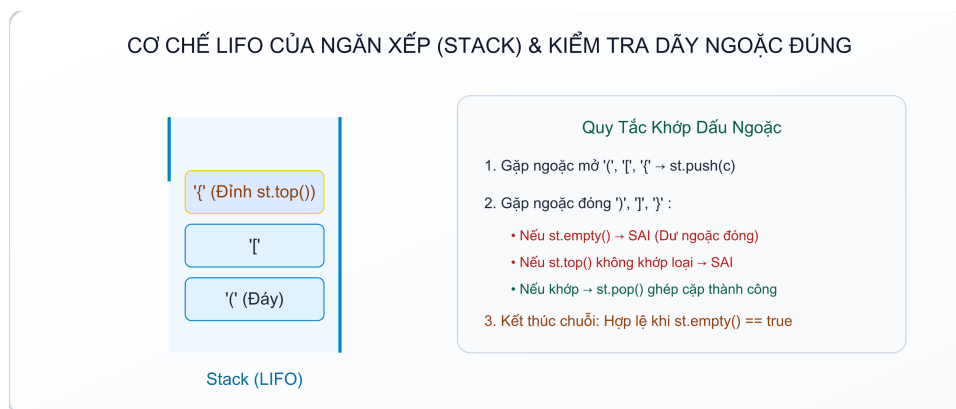
    // Tìm biên phải nhỏ hơn hoặc bằng
    for (int i = n - 1; i >= 0; --i) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        right_bound[i] = st.empty() ? n : st.top();
        st.push(i);
    }

    long long total_sum = 0;
    for (int i = 0; i < n; ++i) {
        long long count = (1LL * (i - left_bound[i]) * (right_bound[i] - i)) % MOD;
        total_sum = (total_sum + a[i] * count) % MOD;
    }

    cout << total_sum << "\n";
    return 0;
}
```

Bài 13 [cppb_stk_13_nge_tren_mang_xoay_vong]: Next Greater Element Trên Mảng Vòng Tròn

Bối cảnh: Một vòng đu quay gồm N cabin được đánh số theo vòng tròn từ 1 đến N , cabin thứ N lại kết nối liền kề với cabin số 1. Cabin thứ i có chiều cao A_i . Khi đứng tại cabin i và nhìn theo chiều kim đồng hồ quanh vòng tròn, hãy tìm giá trị của cabin đầu tiên có chiều cao lớn hơn cabin i . Nếu đi hết một vòng tròn mà không có cabin nào cao hơn, ghi nhận -1.



Hình minh họa: Nguyên lý hoạt động Vào sau Ra trước (LIFO) của ngăn xếp (Stack)

Nhiệm vụ: Cho mảng tròn A gồm N phần tử. Với mỗi phần tử, hãy tìm phần tử lớn hơn đầu tiên tiếp theo trên mảng vòng tròn. Nếu không có, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên tương ứng, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 1 2 1	2 -1 2

Giải thích: Với mảng tròn gồm 3 phần tử $[1, 2, 1]$:

- Phần tử 1 (đầu): Nhìn tiếp theo gặp số 2 lớn hơn 1 → in ra 2.
- Phần tử 2 (giữa): Nhìn tiếp theo gặp 1, rồi vòng lại đầu gặp 1, không có số nào lớn hơn 2 → in ra -1.
- Phần tử 1 (cuối): Vòng lại đầu mảng gặp 1, tiếp tục gặp 2 lớn hơn 1 → in ra 2. Kết quả in ra: 2 -1 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng tròn A gồm N phần tử. Với mỗi phần tử, hãy tìm phần tử lớn hơn đầu tiên tiếp theo trên mảng vòng tròn. Nếu không có, in ra -1.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (`pop()`).

- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 1 2 1 $\Rightarrow$ Đầu ra kỳ vọng: 2 - 1 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 1 2 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng tròn gồm 3 phần tử [1,2,1]: - Phần tử 1 (đầu): Nhìn tiếp theo gặp số 2 lớn hơn 1 $\rightarrow$ in ra 2. - Phần tử 2 (giữa): Nhìn t...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2 - 1 2

Giải thích chi tiết: Với mảng tròn gồm 3 phần tử [1,2,1]:

- Phần tử 1 (đầu): Nhìn tiếp theo gặp số 2 lớn hơn 1 $\rightarrow$ in ra 2.
- Phần tử 2 (giữa): Nhìn tiếp theo gặp 1, rồi vòng lại đầu gặp 1, không có số nào lớn hơn 2 $\rightarrow$ in ra -1.
- Phần tử 1 (cuối): Vòng lại đầu mảng gặp 1, tiếp tục gặp 2 lớn hơn 1 $\rightarrow$ in ra 2. Kết quả in ra: 2 - 1 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
```

```

if (!(cin >> n)) return 0;
if (n <= 0) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

vector<long long> nge(n, -1);
stack<int> st;

// Duyệt vòng tròn 2 vòng (2N)
for (int i = 0; i < 2 * n; ++i) {
    while (!st.empty() && a[i % n] > a[st.top()]) {
        nge[st.top()] = a[i % n];
        st.pop();
    }
    if (i < n) st.push(i);
}

for (int i = 0; i < n; ++i) {
    cout << nge[i] << (i + 1 == n ? "" : " ");
}
cout << "\n";
return 0;
}

```

Bài 14 [cppb_stk_14_toa_thap_tam_nhin]: Tầm Nhìn Tòa Tháp (Stock Span)

Bối cảnh: Trên một tuyến phố ven biển có N tòa tháp cao tầng xếp thẳng hàng từ trái sang phải, tòa tháp thứ i có chiều cao H_i . Tầm nhìn về phía sau của một tòa tháp i được định nghĩa là số lượng tòa tháp liên tiếp nằm ngay bên trái nó (tính cả chính nó) mà có chiều cao nhỏ hơn hoặc bằng H_i .

MONOTONIC STACK: TÌM PHẦN TỬ LỚN HƠN TIẾP THEO (NEXT GREATER ELEMENT)

Mảng A = [4, 2, 1, 5, 3]



Bất Biến Ngăn Xếp Đơn Điều Giảm Dần (Lưu Chỉ Số i):

- Duyệt qua phần tử 5: 5 lớn hơn 1 (pop 1, NGE[1]=5), 5 lớn hơn 2 (pop 2, NGE[2]=5), 5 lớn hơn 4 (pop 4, NGE[4]=5)
- Mỗi phần tử vào/ra Stack đúng 1 lần duy nhất → Tổng thời gian toàn mảng $O(N)$

Kết quả NGE = [5, 5, 5, -1, -1]

Hình minh họa: Kỹ thuật ngăn xếp đơn điệu tìm phần tử lớn hơn tiếp theo (Next Greater Element)

Nhiệm vụ: Cho danh sách chiều cao của N tòa tháp. Hãy lập trình tính tầm nhìn cho từng tòa tháp trong dãy.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là tầm nhìn tương ứng của từng tòa tháp, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 100 80 60 70 60 75 85	1 1 1 2 1 4 6

Giải thích: Với chiều cao các tháp [100,80,60,70,60,75,85]:

- Tháp 1 (100): tầm nhìn 1 (chính nó).
- Tháp 2 (80): tầm nhìn 1.
- Tháp 3 (60): tầm nhìn 1.
- Tháp 4 (70): bao trùm tháp 60 và chính nó → tầm nhìn 2.
- Tháp 5 (60): tầm nhìn 1.
- Tháp 6 (75): bao trùm tháp 60, 70, 60 và chính nó → tầm nhìn 4.
- Tháp 7 (85): bao trùm tất cả các tháp trừ tháp 100 → tầm nhìn 6. Kết quả in ra: 1 1 1 2 1 4 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách chiều cao của N tòa tháp. Hãy lập trình tính tầm nhìn cho từng tòa tháp trong dãy.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $O(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 7 100 80 60 70 60 75 85 ⇒ Đầu ra kỳ vọng: 1 1 1 2 1 4 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 7 100 80 60 70 60 75 85	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chiều cao các tháp [100,80,60,70,60,75,85]: : - Tháp 1 (100): tầm nhìn 1 (chính nó). - Tháp 2 (80): tầm nhìn 1. - Tháp 3 (60):...	Cập nhật các biến / mảng trạng thái

3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 1 1 2 1 4 6
---	--------------------	--------------------------------	-----------------------------

Giải thích chi tiết: Với chiều cao các tháp [100,80,60,70,60,75,85]:

- Tháp 1 (100): tầm nhìn 1 (chính nó).
- Tháp 2 (80): tầm nhìn 1.
- Tháp 3 (60): tầm nhìn 1.
- Tháp 4 (70): bao trùm tháp 60 và chính nó → tầm nhìn 2.
- Tháp 5 (60): tầm nhìn 1.
- Tháp 6 (75): bao trùm tháp 60, 70, 60 và chính nó → tầm nhìn 4.
- Tháp 7 (85): bao trùm tất cả các tháp trừ tháp 100 → tầm nhìn 6. Kết quả in ra: 1 1 1 2 1 4 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

    vector<int> span(n);
    stack<int> st;

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && h[st.top()] <= h[i]) {
            st.pop();
        }
        span[i] = st.empty() ? (i + 1) : (i - st.top());
        st.push(i);
    }

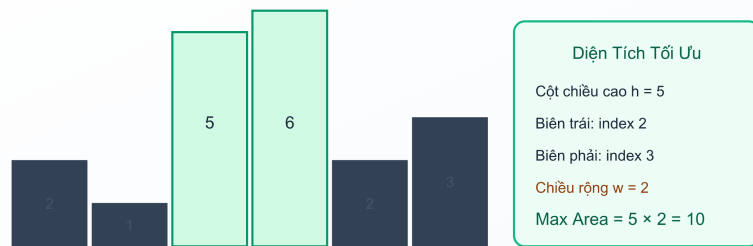
    for (int i = 0; i < n; ++i) {
        cout << span[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
}
```

```
return 0;
}
```

Bài 15 [cppb_stk_15_danh_gia_bieu_thuc_dai_so_day_du]: Đánh Giá Biểu Thức Trung Tổ (Infix Expression)

Bối cảnh: Một máy tính bỏ túi thông minh tiếp nhận biểu thức số học viết ở dạng trung tổ tự nhiên (dạng toán học thông thường có chứa các dấu ngoặc tròn (,), số nguyên và 4 phép toán cơ bản +, -, *, /). Biểu thức tuân thủ độ ưu tiên toán học: nhân chia trước, cộng trừ sau, trong ngoặc trước, ngoài ngoặc sau.

HÌNH CHỮ NHẬT LỚN NHẤT TRÊN BIỂU ĐỒ CỘT (LARGEST RECTANGLE IN HISTOGRAM)



Hình minh họa: Ứng dụng ngăn xếp đơn điệu giải bài toán hình chữ nhật lớn nhất trong biểu đồ cột

Nhiệm vụ: Cho một chuỗi biểu thức số học trung tổ hợp lệ. Hãy lập trình tính toán và in ra giá trị số học cuối cùng của biểu thức.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi biểu thức gồm các chữ số, dấu ngoặc () và các toán tử + - * / ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là kết quả của biểu thức.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3+2*2	7

Giải thích: Với biểu thức số học “1 + (2 * 3) - 4 / 2”:

- Tính trong ngoặc: $2 \times 3 = 6$.
- Biểu thức trở thành: $1 + 6 - 4/2$.
- Thực hiện phép chia: $4/2 = 2$.
- Biểu thức thành: $1 + 6 - 2 = 5$. Kết quả in ra là 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một chuỗi biểu thức số học trung tổ hợp lệ. Hãy lập trình tính toán và in ra giá trị số học cuối cùng của biểu thức.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Nguyên lý hoạt động:** Vào sau Ra trước (LIFO). Thích hợp giải quyết các bài toán cặp ngoặc lồng nhau, khử đệ quy và tính toán biểu thức hậu tố.
- **Kỹ thuật Ngăn xếp đơn điệu (Monotonic Stack):**
- Duy trì các phần tử trong stack luôn tăng dần hoặc giảm dần nghiêm ngặt.
- Trước khi đưa phần tử mới A_i vào, liên tục đẩy các phần tử vi phạm tính đơn điệu ra khỏi stack (pop()).
- Mỗi phần tử chỉ được đưa vào và lấy ra khỏi stack đúng 1 lần, giúp tổng độ phức tạp đạt $\mathcal{O}(N)$ tối ưu tuyệt đối.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: $3+2*2 \Rightarrow$ Đầu ra kỳ vọng: 7.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: $3+2*2$	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với biểu thức số học " $1 + (2 * 3) - 4 / 2$ ": 1. Tính trong ngoặc: $2 \times 3 = 6$. 2. Biểu thức trở thành: $1 + 6 - 4/2$. 3. Thực hiện... Đổi chiều trạng thái cuối cùng	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn		Output chuẩn: 7

Giải thích chi tiết: Với biểu thức số học " $1 + (2 * 3) - 4 / 2$ ":

1. Tính trong ngoặc: $2 \times 3 = 6$.
2. Biểu thức trở thành: $1 + 6 - 4/2$.
3. Thực hiện phép chia: $4/2 = 2$.
4. Biểu thức thành: $1 + 6 - 2 = 5$. Kết quả in ra là 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Lỗi Runtime Error (SIGSEGV) khi gọi `stk.top()` hoặc `stk.pop()` trên ngăn xếp rỗng. Bắt buộc kiểm tra `!stk.empty()` trước mọi thao tác truy cập đỉnh.
- Quên xử lý các phần tử còn sót lại trong stack sau khi duyệt hết mảng dữ liệu (đặc biệt trong bài toán tìm hình chữ nhật lớn nhất trong biểu đồ cột).
- Khi kiểm tra dãy ngoặc đúng, nếu gặp ngoặc đóng mà stack rỗng thì dãy ngoặc không hợp lệ ngay lập tức.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

long long applyOp(long long a, long long b, char op) {
    if (op == '+') return a + b;
    if (op == '-') return a - b;
    if (op == '*') return a * b;
    if (op == '/') return b != 0 ? a / b : 0;
    return 0;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    stack<long long> values;
    stack<char> ops;

    for (int i = 0; i < (int)s.size(); ++i) {
        if (isdigit(s[i])) {
            long long val = 0;
            while (i < (int)s.size() && isdigit(s[i])) {
                val = (val * 10) + (s[i] - '0');
                i++;
            }
            values.push(val);
            i--;
        } else if (s[i] == '(') {
            ops.push(s[i]);
        } else if (s[i] == ')') {
            while (!ops.empty() && ops.top() != '(') {
                long long val2 = values.top(); values.pop();
                long long val1 = values.top(); values.pop();
                char op = ops.top(); ops.pop();
                values.push(applyOp(val1, val2, op));
            }
            if (!ops.empty()) ops.pop(); // Bỏ '('
        } else if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {
            while (!ops.empty() && precedence(ops.top()) >= precedence(s[i])) {
                long long val2 = values.top(); values.pop();
                long long val1 = values.top(); values.pop();
                char op = ops.top(); ops.pop();
                values.push(applyOp(val1, val2, op));
            }
            ops.push(s[i]);
        }
    }

    while (!ops.empty()) {
        long long val2 = values.top(); values.pop();
        long long val1 = values.top(); values.pop();
        char op = ops.top(); ops.pop();
    }
}
```

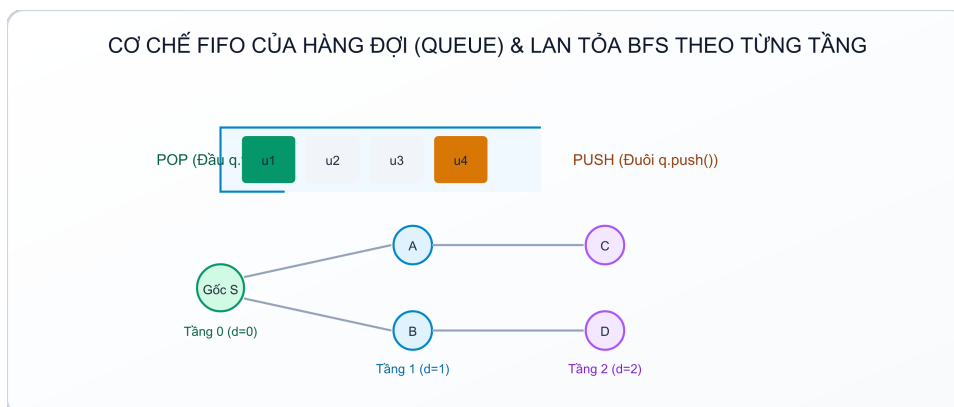
```
        values.push(applyOp(val1, val2, op));  
    }  
  
    cout << values.top() << "\n";  
    return 0;  
}
```



Bài 21: Cấu trúc dữ liệu hàng đợi (queue, deque) & Monotonic Deque

Bài 01 [cppb_que_01_cai_dat_hang_doi_co_ban]: Cài Đặt Hàng Đợi Cơ Bản

Bối cảnh: Một hệ thống xếp hàng tự động tại quầy giao dịch ngân hàng hoạt động theo nguyên tắc vào trước ra trước (FIFO - First In First Out). Hệ thống tiếp nhận Q yêu cầu thuộc các dạng: thêm một khách hàng có số định danh x vào cuối hàng đợi, phục vụ khách hàng đang đứng ở đầu hàng (loại bỏ khỏi hàng), và kiểm tra số định danh của khách hàng đang đứng đầu hàng hiện tại.



Hình minh họa: Nguyên lý hoạt động Vào trước Ra trước (FIFO) của hàng đợi tiêu chuẩn (Queue)

Nhiệm vụ: Cho danh sách Q thao tác thuộc 3 loại: 1 x (thêm x vào cuối hàng), 2 (loại bỏ phần tử đầu hàng nếu hàng không rỗng), 3 (in ra phần tử đầu hàng, nếu rỗng in ra -1). Hãy lập trình mô phỏng lại hàng đợi và in ra kết quả cho các thao tác loại 3.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo chứa các thao tác mô tả như trên.

Đầu ra (Output):

- Với mỗi thao tác loại 3, in ra kết quả trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 10 1 20 3 2 3	10 20

Giải thích: Với chuỗi thao tác: thêm 10, thêm 20, truy vấn đầu hàng $\rightarrow$ in ra 10; phục vụ đầu hàng (loại 10), truy vấn đầu hàng $\rightarrow$ in ra 20.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách Q thao tác thuộc 3 loại: 1 x (thêm x vào cuối hàng), 2 (loại bỏ phần tử đầu hàng nếu hàng không rỗng), 3 (in ra phần tử đầu hàng, nếu rỗng in ra -1). Hãy lập trình mô phỏng lại hàng đợi và in ra kết quả cho các thao tác loại 3.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**

- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 1 10 1 20 3 2 3 $\Rightarrow$ Đầu ra kỳ vọng: 10 20.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 1 10 1 20 3 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với chuỗi thao tác: thêm 10, thêm 20, truy vấn đầu hàng $\rightarrow$ in ra 10; phục vụ đầu hàng (loại 10), truy vấn đầu hàng $\rightarrow$ in ra 20.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 10 20

Giải thích chi tiết: Với chuỗi thao tác: thêm 10, thêm 20, truy vấn đầu hàng $\rightarrow$ in ra 10; phục vụ đầu hàng (loại 10), truy vấn đầu hàng $\rightarrow$ in ra 20.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
```



```

queue<long long> qu;

while (q--) {
    int type;
    cin >> type;
    if (type == 1) { // Push x
        long long x;
        cin >> x;
        qu.push(x);
    } else if (type == 2) { // Pop
        if (!qu.empty()) qu.pop();
    } else if (type == 3) { // Front
        if (qu.empty()) cout << "EMPTY\n";
        else cout << qu.front() << "\n";
    }
}
return 0;
}

```

Bài 02 [cppb_que_02_sinh_chuoi_so_nhi_phan_bang_queue]: Sinh Chuỗi Số Nhị Phân Bằng Queue

Bối cảnh: Một chip tạo mã nhị phân cần xuất ra liên tục danh sách biểu diễn nhị phân của các số tự nhiên từ 1 đến N theo thứ tự tăng dần (ví dụ: với $N = 3$, danh sách là 1, 10, 11). Nhờ tính chất hàng đợi, mỗi khi lấy ra một chuỗi nhị phân S , ta có thể dễ dàng tạo ra hai chuỗi nhị phân kế tiếp bằng cách ghép thêm ký tự '0' và '1' vào cuối.

HÀNG ĐỢI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX $O(N)$

std::deque (Thao Tác Cả Hai Đầu Trong $O(1)$)

pop_front() / push_front()

pop_back() / push_back()

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K:

- Loại bỏ phần tử lỗi thời hết hạn cửa sổ: $\text{index} < i - k + 1 \rightarrow \text{dq.pop_front}()$
- Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi $\rightarrow \text{dq.pop_back}()$
- Phần tử nhỏ nhất hiện tại luôn nằm tại $\text{dq.front}() \rightarrow O(1)$

Hình minh họa: Hàng đợi hai đầu hai chiều (Deque) duy trì giá trị min/max trên cửa sổ trượt

Nhiệm vụ: Cho số nguyên dương N . Hãy lập trình sinh và in ra chuỗi biểu diễn nhị phân của tất cả các số từ 1 đến N , các chuỗi cách nhau bởi khoảng trắng.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng gồm N chuỗi nhị phân tương ứng theo thứ tự tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	1 10 11 100

Giải thích: Với $N = 5$, danh sách biểu diễn nhị phân của các số từ 1 đến 5 là: 1 (số 1), 10 (số 2), 11 (số 3), 100 (số 4), 101 (số 5).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N . Hãy lập trình sinh và in ra chuỗi biểu diễn nhị phân của tất cả các số từ 1 đến N , các chuỗi cách nhau bởi khoảng trắng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 $\Rightarrow$ Đầu ra kỳ vọng: 1 10 11 100.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với $N = 5$, danh sách biểu diễn nhị phân của các số từ 1 đến 5 là: 1 (số 1), 10 (số 2), 11 (số 3), 100 (số 4), 101 (số 5).	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 10 11 100

Giải thích chi tiết: Với $N = 5$, danh sách biểu diễn nhị phân của các số từ 1 đến 5 là: 1 (số 1), 10 (số 2), 11 (số 3), 100 (số 4), 101 (số 5).

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.

- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    queue<string> q;
    q.push("1");

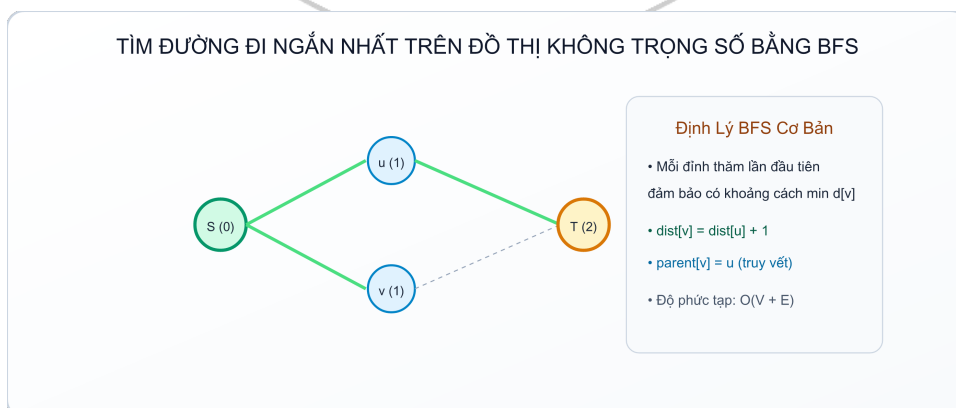
    for (int i = 1; i <= n; ++i) {
        string s = q.front();
        q.pop();

        cout << s << (i == n ? "" : " ");

        q.push(s + "0");
        q.push(s + "1");
    }
    cout << "\n";
    return 0;
}
```

Bài 03 [cppb_que_03_bfs_tim_duong_di_ngan_nhat_do_thi]: BFS Tìm Bước Đi Ngắn Nhất Đồ Thị

Bối cảnh: Một mạng máy tính gồm N máy chủ được đánh số từ 1 đến N và M kênh kết nối hai chiều không trọng số. Một gói tin xuất phát từ máy chủ nguồn S muốn truyền đến máy chủ đích D . Mỗi lần truyền qua một kênh kết nối tốn đúng 1 bước nhảy (hop). Quản trị viên cần xác định số bước nhảy ít nhất để gói tin đến được đích.



Hình minh họa: Cơ chế hàng đợi Queue mở rộng tìm đường đi ngắn nhất đồ thị trọng số đơn vị

Nhiệm vụ: Cho đồ thị N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số bước đi ngắn nhất từ S tới D . Nếu không thể đến được, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 4 số nguyên N, M, S, D ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5, 1 \leq S, D \leq N$).
- M dòng tiếp theo, mỗi dòng chứa hai số nguyên u và v biểu diễn một kênh kết nối giữa đỉnh u và v .

Đầu ra (Output):

- In ra một số nguyên duy nhất là số bước đi ngắn nhất, hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 1 3	2

Giải thích: Với mạng gồm 4 máy chủ kết nối theo chuỗi $1 - 2 - 3 - 4$ và một kênh tắt nối trực tiếp $1 - 3$: Để đi từ 1 tới 4, gói tin đi qua kênh tắt 1×03 tốn 1 bước, sau đó từ 3×04 tốn 1 bước nữa. Tổng số bước đi ít nhất là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số bước đi ngắn nhất từ S tới D . Nếu không thể đến được, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 1 2 2 3 3 4 1 3 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 1 2 2 3 3 4 1 3 4 1 3	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với mạng gồm 4 máy chủ kết nối theo chuỗi 1 – 2 – 3 – 4 và một kênh tắt nối trực tiếp 1 – 3: Để đi từ 1 tới 4, gói tin đi qua kênh tắt 1 × 03 tốn 1 bước, sau đó từ 3 × 04 tốn 1 bước nữa. Tổng số bước đi ít nhất là 2.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với mạng gồm 4 máy chủ kết nối theo chuỗi 1 – 2 – 3 – 4 và một kênh tắt nối trực tiếp 1 – 3: Để đi từ 1 tới 4, gói tin đi qua kênh tắt 1 × 03 tốn 1 bước, sau đó từ 3 × 04 tốn 1 bước nữa. Tổng số bước đi ít nhất là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);
    queue<int> q;

    dist[1] = 0;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```

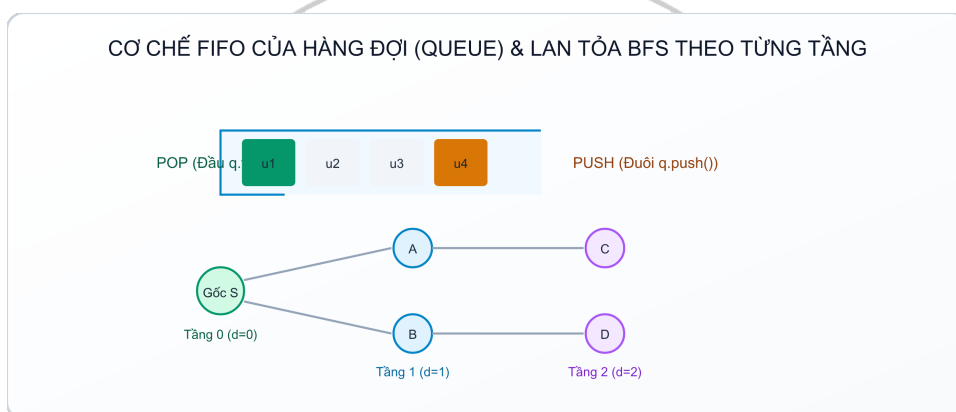
    for (int v : adj[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}

cout << dist[n] << "\n";
return 0;
}

```

Bài 04 [cppb_que_04_truy_vet_lo_trinh_ngan_nhat_bfs]: Truy Vết Lộ Trình Ngắn Nhất Bằng BFS

Bối cảnh: Sau khi tính toán được số bước nhảy tối thiểu để truyền gói tin giữa hai máy chủ, trung tâm điều hành cần in ra chính xác danh sách thứ tự các máy chủ mà gói tin đã đi qua để cấu hình bảng định tuyến mạng.



Hình minh họa: Nguyên lý hoạt động Vào trước Ra trước (FIFO) của hàng đợi tiêu chuẩn (Queue)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm và in ra một đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 4 số nguyên N, M, S, D ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- Dòng 1: In ra số lượng đỉnh trên đường đi K .
- Dòng 2: In ra K số nguyên là thứ tự các đỉnh trên hành trình từ S tới D .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5 1 2 2 3 3 5 1 4 4 5	3 1 4 5

Giải thích: Với đồ thị có đường đi ngắn nhất từ 1 tới 4 là $1 \times o3 \times o4$: Dòng 1 in ra 3 (số đỉnh trên đường đi). Dòng 2 in ra 1 3 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm và in ra một đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 5 1 2 2 3 3 5 1 4 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 3 1 4 5.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 5 1 2 2 3 3 5 1 4 4 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị có đường đi ngắn nhất từ 1 tới 4 là $1 \times 03 \times 04$: Dòng 1 in ra 3 (số đỉnh trên đường đi). Dòng 2 in ra 1 3 4.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3 1 4 5

Giải thích chi tiết: Với đồ thị có đường đi ngắn nhất từ 1 tới 4 là $1 \times 03 \times 04$: Dòng 1 in ra 3 (số đỉnh trên đường đi). Dòng 2 in ra 1 3 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);
    vector<int> parent(n + 1, -1);
    queue<int> q;

    dist[1] = 0;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }

    if (dist[n] == -1) {
        cout << -1 << "\n";
        return 0;
    }

    vector<int> path;
    int curr = n;
    while (curr != -1) {
        path.push_back(curr);
        curr = parent[curr];
    }
    reverse(path.begin(), path.end());

    cout << path.size() << "\n";
    for (int i = 0; i < (int)path.size(); ++i) {
        cout << path[i] << (i + 1 == (int)path.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```


Bài 05 [cppb_que_05_min_moi_cua_so_truot_do_dai_k]: Min Mọi Cửa Sổ Trượt Bằng Monotonic Deque O(N)

Bối cảnh: Một máy đo nhiệt độ lò phản ứng hạt nhân ghi nhận chuỗi N giá trị đo liên tiếp. Một cửa sổ quan sát kích thước cố định gồm K phép đo trượt dần từ đầu đến cuối dãy đo. Tại mỗi vị trí của cửa sổ trượt, hệ thống cảnh báo an toàn cần xác định giá trị nhiệt độ thấp nhất bên trong cửa sổ đó với tốc độ xử lý tức thời.

HÀNG ĐỢI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX O(N)

`std::deque` (Thao Tác Cả Hai Đầu Trong O(1))

`pop_front()` / `push_front()`

`pop_back()` / `push_back()`

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K:

- Loại bỏ phần tử lỗi thời hết hạn cửa sổ: `index < i - k + 1` → `dq.pop_front()`
- Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi → `dq.pop_back()`
- Phần tử nhỏ nhất hiện tại luôn nằm tại `dq.front()` → O(1)

Hình minh họa: Hàng đợi hai đầu hai chiều (Deque) duy trì giá trị min/max trên cửa sổ trượt

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình tìm giá trị nhỏ nhất trong mỗi cửa sổ trượt kích thước K khi di chuyển từ trái sang phải.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm $N - K + 1$ số nguyên là giá trị nhỏ nhất của từng cửa sổ trượt, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 3 1 3 -1 -3 5 3 6 7	-1 -3 -3 -3 3 3

Giải thích: Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và cửa sổ $K = 3$:

- Cửa sổ 1 $[1, 3, -1]$ → min = -1.
- Cửa sổ 2 $[3, -1, -3]$ → min = -3.
- Cửa sổ 3 $[-1, -3, 5]$ → min = -3.
- Cửa sổ 4 $[-3, 5, 3]$ → min = -3.
- Cửa sổ 5 $[5, 3, 6]$ → min = 3.
- Cửa sổ 6 $[3, 6, 7]$ → min = 3. Kết quả in ra: -1 -3 -3 -3 3 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình tìm giá trị nhỏ nhất trong mỗi cửa sổ trượt kích thước K khi di chuyển từ trái sang phải.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 8 3 1 3 -1 -3 5 3 6 7 $\Rightarrow$ Đầu ra kỳ vọng: -1 -3 -3 -3 3 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 8 3 1 3 -1 -3 5 3 6 7	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và cửa sổ $K = 3$: - Cửa sổ 1 $[1, 3, -1] \rightarrow \min = -1$. - Cửa sổ 2 $[3, -1, -3] \rightarrow \min = -3$. - Cửa sổ 3 $[-1, -3, 5] \rightarrow \min = -3$. - Cửa sổ 4 $[-3, 5, 3] \rightarrow \min = -3$. - Cửa sổ 5 $[5, 3, 6] \rightarrow \min = 3$. - Cửa sổ 6 $[3, 6, 7] \rightarrow \min = 3$.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: -1 -3 -3 -3 3 3

Giải thích chi tiết: Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và cửa sổ $K = 3$:

- Cửa sổ 1 $[1, 3, -1] \rightarrow \min = -1$.
- Cửa sổ 2 $[3, -1, -3] \rightarrow \min = -3$.
- Cửa sổ 3 $[-1, -3, 5] \rightarrow \min = -3$.
- Cửa sổ 4 $[-3, 5, 3] \rightarrow \min = -3$.
- Cửa sổ 5 $[5, 3, 6] \rightarrow \min = 3$.
- Cửa sổ 6 $[3, 6, 7] \rightarrow \min = 3$. Kết quả in ra: -1 -3 -3 -3 3 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).

- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0 || k > n) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    deque<int> dq;
    vector<long long> ans;

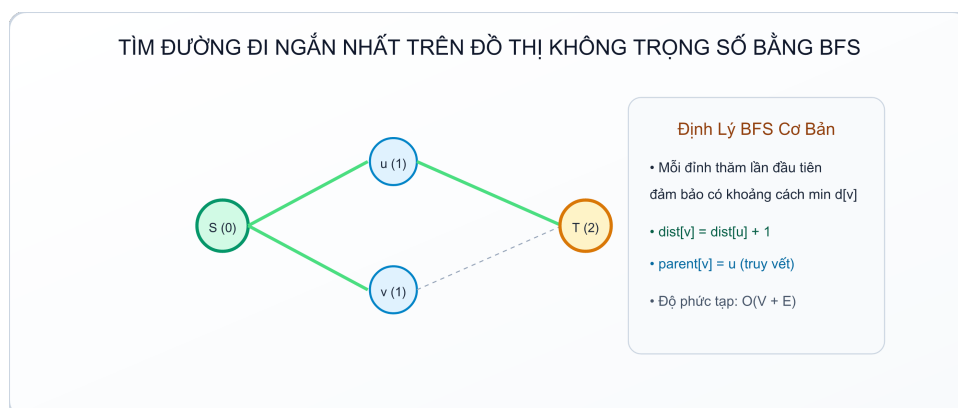
    for (int i = 0; i < n; ++i) {
        while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);

        if (i >= k - 1) ans.push_back(a[dq.front()]);
    }

    for (int i = 0; i < (int)ans.size(); ++i) {
        cout << ans[i] << (i + 1 == (int)ans.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 06 [cppb_que_06_max_moi_cua_so_truot_do_dai_k]: Max Mọi Cửa Sổ Trượt Bằng Monotonic Deque $O(N)$

Bối cảnh: Tương tự hệ thống giám sát lò phản ứng, để kịp thời phát hiện các đỉnh áp suất bất thường, hệ thống cần đồng thời theo dõi giá trị áp suất lớn nhất bên trong mỗi cửa sổ trượt gồm K mốc đo liên tiếp.



Hình minh họa: Cơ chế hàng đợi Queue mở rộng tìm đường đi ngắn nhất đồ thị trọng số đơn vị

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình tìm giá trị lớn nhất trong mỗi cửa sổ trượt kích thước K .

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng gồm $N - K + 1$ số nguyên là giá trị lớn nhất của từng cửa sổ trượt.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 3 1 3 -1 -3 5 3 6 7	3 3 5 5 6 7

Giải thích: Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và $K = 3$:

- Cửa sổ $[1, 3, -1] \rightarrow \max = 3$.
- Cửa sổ $[3, -1, -3] \rightarrow \max = 3$.
- Cửa sổ $[-1, -3, 5] \rightarrow \max = 5$.
- Cửa sổ $[-3, 5, 3] \rightarrow \max = 5$.
- Cửa sổ $[5, 3, 6] \rightarrow \max = 6$.
- Cửa sổ $[3, 6, 7] \rightarrow \max = 7$. Kết quả in ra: 3 3 5 5 6 7.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên và kích thước cửa sổ K . Hãy lập trình tìm giá trị lớn nhất trong mỗi cửa sổ trượt kích thước K .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 8 3 1 3 -1 -3 5 3 6 7 $\Rightarrow$ Đầu ra kỳ vọng: 3 3 5 5 6 7.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
------	--------------------	-------------------------------	------------------

1	Khởi tạo & Đọc dữ liệu	Nạp Input: 8 3 1 3 -1 -3 5 3 6 7	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng [1,3,-1,-3,5,3,6,7] và $K = 3$: - Cửa sổ [1, 3, -1] $\rightarrow \max = 3$. - Cửa sổ [3, -1, -3] $\rightarrow \max = 3$. - Cửa sổ [-1, -3, 5]...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 3 3 5 5 6 7

Giải thích chi tiết: Với mảng [1,3,-1,-3,5,3,6,7] và $K = 3$:

- Cửa sổ [1, 3, -1] $\rightarrow \max = 3$.
- Cửa sổ [3, -1, -3] $\rightarrow \max = 3$.
- Cửa sổ [-1, -3, 5] $\rightarrow \max = 5$.
- Cửa sổ [-3, 5, 3] $\rightarrow \max = 5$.
- Cửa sổ [5, 3, 6] $\rightarrow \max = 6$.
- Cửa sổ [3, 6, 7] $\rightarrow \max = 7$. Kết quả in ra: 3 3 5 5 6 7.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa ($dq.front() \leq i - K$).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0 || k > n) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    deque<int> dq;
    vector<long long> ans;

    for (int i = 0; i < n; ++i) {
```

```

while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
dq.push_back(i);

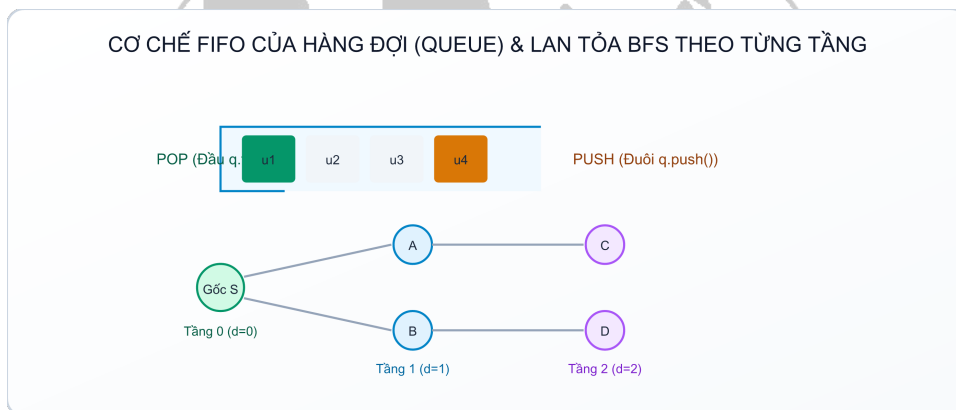
if (i >= k - 1) ans.push_back(a[dq.front()]);
}

for (int i = 0; i < (int)ans.size(); ++i) {
    cout << ans[i] << (i + 1 == (int)ans.size() ? "" : " ");
}
cout << "\n";
return 0;
}

```

Bài 07 [cppb_que_07_kiem_tra_do_thi_hai_phia_2_coloring]: Kiểm Tra Đồ Thị Hai Phía (Bipartite Graph)

Bối cảnh: Trong một hội thảo giao lưu quốc tế, có N đại biểu và M mối quan hệ quen biết lẫn nhau. Ban tổ chức muốn chia toàn bộ N đại biểu vào đúng 2 phòng thảo luận khác nhau sao cho không có hai người nào quen nhau lại ngồi chung trong cùng một phòng (tương đương bài toán tô màu đồ thị bằng 2 màu sao cho hai đỉnh kề nhau luôn có màu khác nhau).



Hình minh họa: Nguyên lý hoạt động Vào trước Ra trước (FIFO) của hàng đợi tiêu chuẩn (Queue)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là đồ thị hai phía hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra YES nếu đồ thị là hai phía, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 4 1	YES

Giải thích: Với đồ thị là hình vuông gồm 4 đỉnh có các cạnh: 1-2, 2-3, 3-4, 4-1: Ta có thể chia thành hai tập đỉnh độc lập: tập 1 gồm {1, 3} và tập 2 gồm {2, 4}. Không có hai đỉnh nào cùng tập có cạnh nối. Đồ thị là hai phía, kết quả in ra YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là đồ thị hai phía hay không. Nếu có in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 1 2 2 3 3 4 4 1 $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 1 2 2 3 3 4 4 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị là hình vuông gồm 4 đỉnh có các cạnh: 1-2, 2-3, 3-4, 4-1: Ta có thể chia thành hai tập đỉnh độc lập: tập 1 gồm {1, 3} và tập...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với đồ thị là hình vuông gồm 4 đỉnh có các cạnh: 1-2, 2-3, 3-4, 4-1: Ta có thể chia thành hai tập đỉnh độc lập: tập 1 gồm {1, 3} và tập 2 gồm {2, 4}. Không có hai đỉnh nào cùng tập có cạnh nối. Đồ thị là hai phía, kết quả in ra YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.

- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> color(n + 1, 0);
    bool is_bipartite = true;

    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) {
            queue<int> q;
            color[i] = 1;
            q.push(i);

            while (!q.empty()) {
                int u = q.front();
                q.pop();

                for (int v : adj[u]) {
                    if (color[v] == 0) {
                        color[v] = 3 - color[u];
                        q.push(v);
                    } else if (color[v] == color[u]) {
                        is_bipartite = false;
                        break;
                    }
                }
            }
            if (!is_bipartite) break;
        }
    }
    if (!is_bipartite) break;

    if (is_bipartite) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```


Bài 08 [cppb_que_08_bien_doi_so_buoc_nho_nhat_a_sang_b]: Biến Đổi Số Bước Nhỏ Nhất Từ A Sang B

Bối cảnh: Một trò chơi giải đố toán học bắt đầu với số nguyên dương A . Tại mỗi bước, người chơi có thể thực hiện một trong hai thao tác: nhân đôi số hiện tại ($x \rightarrow 2x$) hoặc trừ số hiện tại đi 1 đơn vị ($x \rightarrow x - 1$). Hãy tìm số thao tác ít nhất để biến đổi số A ban đầu thành đúng số mục tiêu B .

HÀNG ĐỢI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX $O(N)$

`std::deque` (Thao Tác Cả Hai Đầu Trong $O(1)$)

`pop_front()` / `push_front()`

`pop_back()` / `push_back()`

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K :

- Loại bỏ phần tử lỗi thời hết hạn cửa sổ: `index < i - k + 1` → `dq.pop_front()`
- Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi → `dq.pop_back()`
- Phần tử nhỏ nhất hiện tại luôn nằm tại `dq.front()` → $O(1)$

Hình minh họa: Hàng đợi hai đầu hai chiều (Deque) duy trì giá trị min/max trên cửa sổ trượt

Nhiệm vụ: Cho hai số nguyên dương A và B . Hãy lập trình tìm số bước biến đổi ít nhất từ A thành B .

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương A và B ($1 \leq A, B \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số bước ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 6	2

Giải thích: Để biến đổi từ $A = 4$ sang $B = 6$:

- Bước 1: Trừ 1 đơn vị: $4 - 1 = 3$.
- Bước 2: Nhân đôi: $3 \times 2 = 6$. Chỉ cần đúng 2 bước biến đổi, kết quả in ra là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên dương A và B . Hãy lập trình tìm số bước biến đổi ít nhất từ A thành B .
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $O(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.

- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 6 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 6	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Đề biến đổi từ $A = 4$ sang $B = 6$: - Bước 1: Trừ 1 đơn vị: $4 - 1 = 3$. - Bước 2: Nhân đôi: $3 \times 2 = 6$. Chỉ cần đúng 2 bước biến đổi...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Đề biến đổi từ $A = 4$ sang $B = 6$:

- Bước 1: Trừ 1 đơn vị: $4 - 1 = 3$.
- Bước 2: Nhân đôi: $3 \times 2 = 6$. Chỉ cần đúng 2 bước biến đổi, kết quả in ra là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int a, b;
    if (!(cin >> a >> b)) return 0;

    if (a >= b) {
        cout << a - b << "\n";
        return 0;
    }
}
```

```

}

vector<int> dist(20005, -1);
queue<int> q;

dist[a] = 0;
q.push(a);

while (!q.empty()) {
    int u = q.front();
    q.pop();

    if (u == b) {
        cout << dist[b] << "\n";
        return 0;
    }

    // Thao tác 1: u * 2
    if (u * 2 <= 20000 && dist[u * 2] == -1) {
        dist[u * 2] = dist[u] + 1;
        q.push(u * 2);
    }

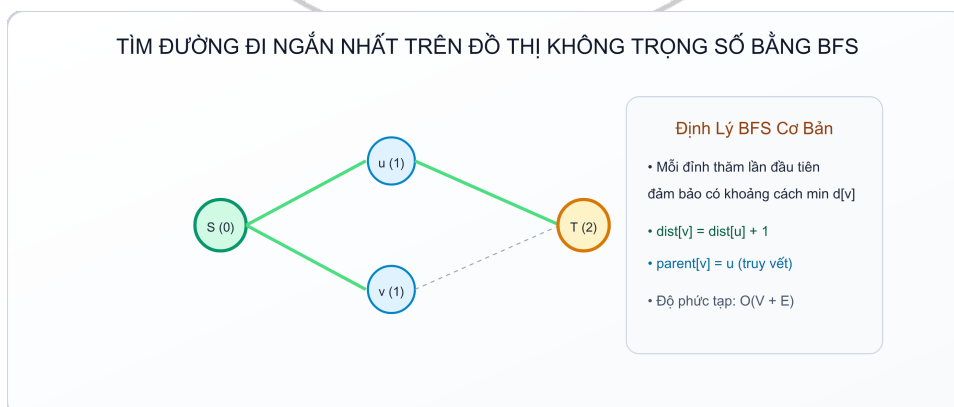
    // Thao tác 2: u - 1
    if (u - 1 > 0 && dist[u - 1] == -1) {
        dist[u - 1] = dist[u] + 1;
        q.push(u - 1);
    }
}

return 0;
}

```

Bài 09 [cppb_que_09_0_1_bfs_tim_duong_ngan_nhat_trong_so_0_1]: 0-1 BFS Tìm Đường Đi Ngắn Nhất Trọng Số 0/1

Bối cảnh: Một mạng lưới giao thông gồm N nút giao và M tuyến đường một chiều. Điểm đặc biệt là mỗi tuyến đường chỉ có trọng số chỉ phí hoặc là 0 đồng (đường công cộng miễn phí) hoặc là 1 đồng (đường cao tốc có thu phí tượng trưng). Cần tìm tổng chi phí ít nhất để di chuyển từ nút giao xuất phát S tới nút giao đích D .



Hình minh họa: Cơ chế hàng đợi Queue mở rộng tìm đường đi ngắn nhất đồ thị trọng số đơn vị

Nhiệm vụ: Cho đồ thị có hướng N đỉnh M cạnh với trọng số mỗi cạnh thuộc $\{0,1\}$. Hãy lập trình tìm khoảng cách ngắn nhất từ S tới D . Nếu không đến được, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 4 số nguyên N, M, S, D ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5, 1 \leq S, D \leq N$).
- M dòng tiếp theo, mỗi dòng chứa 3 số nguyên u, v, w biểu diễn đường một chiều từ u tới v với trọng số $w \in \{0,1\}$.

Đầu ra (Output):

- In ra khoảng cách ngắn nhất từ S tới D , hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 1 2 3 0 3 4 1 1 4 1	1

Giải thích: Với đồ thị có đường đi từ 1 tới 3 qua cạnh trọng số 1, và từ 3 tới 4 qua cạnh trọng số 0: Tổng chi phí từ 1 tới 4 là $1 + 0 = 1$ đồng. Đây là chi phí tối thiểu.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị có hướng N đỉnh M cạnh với trọng số mỗi cạnh thuộc $\{0,1\}$. Hãy lập trình tìm khoảng cách ngắn nhất từ S tới D . Nếu không đến được, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 1 2 1 2 3 0 3 4 1 1 4 1 $\Rightarrow$ Đầu ra kỳ vọng: 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 1 2 1 2 3 0 3 4 1 1 4 1	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị có đường đi từ 1 tới 3 qua cạnh trọng số 1, và từ 3 tới 4 qua cạnh trọng số 0: Tổng chi phí từ 1 tới 4 là $1 + 0 = 1$ đồng. ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 1

Giải thích chi tiết: Với đồ thị có đường đi từ 1 tới 3 qua cạnh trọng số 1, và từ 3 tới 4 qua cạnh trọng số 0: Tổng chi phí từ 1 tới 4 là $1 + 0 = 1$ đồng. Đây là chi phí tối thiểu.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

struct Edge {
    int to, weight;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<Edge>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    vector<int> dist(n + 1, INF);
    deque<int> dq;

    dist[1] = 0;
    dq.push_back(1);
```

```
while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();

    for (const auto& edge : adj[u]) {
        int v = edge.to;
        int w = edge.weight;

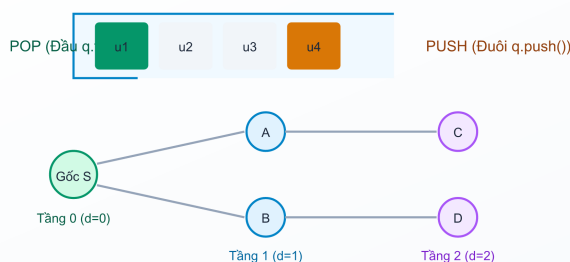
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) {
                dq.push_front(v);
            } else {
                dq.push_back(v);
            }
        }
    }
}

if (dist[n] == INF) cout << -1 << "\n";
else cout << dist[n] << "\n";
return 0;
}
```

Bài 10 [cppb_que_10_doan_con_tong_lon_nhat_do_dai_toi_da_k]: Đoạn Con Tổng Lớn Nhất Độ Dài Tối Đa K

Bối cảnh: Một công ty dầu khí phân tích chỉ số lưu lượng khai thác của một giếng dầu qua N ngày liên tiếp tạo thành dãy số $A_1, A_2, \dots, A_N$. Ban giám đốc muốn chọn một đợt vận hành thử nghiệm gồm một chuỗi các ngày liên tiếp sao cho số ngày vận hành không vượt quá K ngày và tổng lưu lượng khai thác thu được là lớn nhất có thể.

CƠ CHẾ FIFO CỦA HÀNG ĐỢI (QUEUE) & LAN TỎA BFS THEO TỪNG TẦNG



Hình minh họa: Nguyên lý hoạt động Vào trước Ra trước (FIFO) của hàng đợi tiêu chuẩn (Queue)

Nhiệm vụ: Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình tìm tổng lớn nhất của một đoạn con liên tiếp có độ dài tối đa là K .

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 -1 2 4 -3 5	6

Giải thích: Với mảng $[-1, 2, 3, -2, 4]$ và độ dài tối đa $K = 2$: Đoạn con liên tiếp có độ dài không quá 2 có tổng lớn nhất là đoạn $[2, 3]$ (độ dài 2) cho tổng là $2 + 3 = 5$ (hoặc đoạn $[4]$ có tổng 4).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình tìm tổng lớn nhất của một đoạn con liên tiếp có độ dài tối đa là K .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 -1 2 4 -3 5 $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 -1 2 4 -3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[-1, 2, 3, -2, 4]$ và độ dài tối đa $K = 2$: Đoạn con liên tiếp có độ dài không quá 2 có tổng lớn nhất là đoạn $[2, 3]$ (độ d...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với mảng $[-1, 2, 3, -2, 4]$ và độ dài tối đa $K = 2$: Đoạn con liên tiếp có độ dài không quá 2 có tổng lớn nhất là đoạn $[2, 3]$ (độ dài 2) cho tổng là $2 + 3 = 5$ (hoặc đoạn $[4]$ có tổng 4).

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0) return 0;

    vector<long long> a(n + 1);
    vector<long long> pref(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        pref[i] = pref[i - 1] + a[i];
    }

    deque<int> dq;
    dq.push_back(0);
    long long max_sum = LLONG_MIN;

    for (int i = 1; i <= n; ++i) {
        while (!dq.empty() && dq.front() < i - k) dq.pop_front();
        if (!dq.empty()) {
            max_sum = max(max_sum, pref[i] - pref[dq.front()]);
        }
        while (!dq.empty() && pref[dq.back()] >= pref[i]) dq.pop_back();
        dq.push_back(i);
    }

    cout << max_sum << "\n";
    return 0;
}
```

Bài 11 [cppb_que_11_tro_choi_vong_tron_josephus_bang_queue]: Trò Chơi Vòng Tròn Josephus

Bối cảnh: Trong một trò chơi dân gian, N bạn học sinh đứng thành một vòng tròn và được đánh số từ 1 đến N theo chiều kim đồng hồ. Bắt đầu đếm từ bạn số 1, cứ mỗi khi đếm đến người thứ K thì người đó sẽ phải rời khỏi vòng tròn. Quá trình đếm tiếp tục với người đứng kế tiếp cho đến khi chỉ còn lại đúng một người cuối cùng trụ lại.

HÀNG ĐỘI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX $O(N)$

`std::deque` (Thao Tác Cả Hai Đầu Trong $O(1)$)

`pop_front()` / `push_front()`

`pop_back()` / `push_back()`

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K :

1. Loại bỏ phần tử lỗi thời hết hạn cửa sổ: `index < i - k + 1` → `dq.pop_front()`
2. Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi → `dq.pop_back()`
3. Phần tử nhỏ nhất hiện tại luôn nằm tại `dq.front()` → $O(1)$

Hình minh họa: Hàng đợi hai đầu hai chiều (Deque) duy trì giá trị min/max trên cửa sổ trượt

Nhiệm vụ: Cho số lượng người N và bước đếm K . Hãy lập trình xác định số thứ tự của người cuối cùng còn lại trong vòng tròn.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương N và K ($1 \leq N \leq 10^5, 1 \leq K \leq 100$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là vị trí của người trụ lại cuối cùng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3	4

Giải thích: Với $N = 7$ người và bước đếm $K = 3$: Thứ tự các người bị loại lần lượt là: 3, 6, 2, 7, 5, 1. Người cuối cùng còn lại là người số 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số lượng người N và bước đếm K . Hãy lập trình xác định số thứ tự của người cuối cùng còn lại trong vòng tròn.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $O(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- Độ phức tạp:** Thời gian $O(N)$, không gian phụ trợ $O(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 7 3 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 7 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với $N = 7$ người và bước đếm $K = 3$: Thứ tự các người bị loại lần lượt là: 3, 6, 2, 7, 5, 1. Người cuối cùng còn lại là người số 4.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với $N = 7$ người và bước đếm $K = 3$: Thứ tự các người bị loại lần lượt là: 3, 6, 2, 7, 5, 1. Người cuối cùng còn lại là người số 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa ($dq.front() \leq i - K$).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0 || k <= 0) return 0;

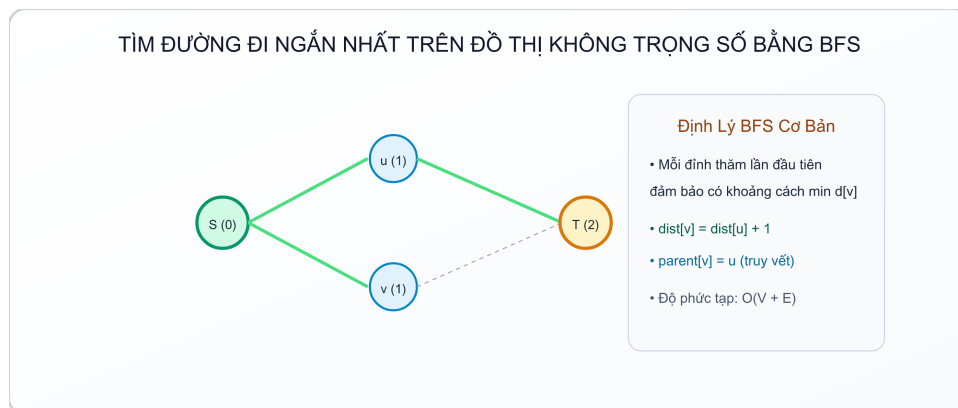
    queue<int> q;
    for (int i = 1; i <= n; ++i) q.push(i);

    while (q.size() > 1) {
        for (int i = 1; i < k; ++i) {
            q.push(q.front());
            q.pop();
        }
        q.pop(); // Loại bỏ người thứ k
    }
}
```

```
cout << q.front() << "\n";
return 0;
}
```

Bài 12 [cppb_que_12_khoang_cach_den_tram_cuu_hoa_gan_nhat]: Khoảng Cách Đến Trạm Cứu Hỏa Gần Nhất (Multi-Source BFS)

Bối cảnh: Một bản đồ đô thị gồm N khu dân cư và M tuyến đường hai chiều. Trong thành phố có K trạm cứu hỏa được đặt tại một số khu dân cư nhất định. Để đảm bảo an toàn phòng cháy chữa cháy, ban điều hành cứu hộ cần xác định khoảng cách ngắn nhất (số cạnh) từ mỗi khu dân cư tới trạm cứu hỏa gần nhất.



Hình minh họa: Cơ chế hàng đợi Queue mở rộng tìm đường đi ngắn nhất đồ thị trọng số đơn vị

Nhiệm vụ: Cho bản đồ thành phố và danh sách vị trí các trạm cứu hỏa. Hãy lập trình tính khoảng cách ngắn nhất từ từng khu dân cư đến trạm cứu hỏa gần nhất.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, K ($1 \leq K \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- Dòng 2: Chứa K số nguyên là vị trí đặt các trạm cứu hỏa.
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh biểu diễn một tuyến đường.

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là khoảng cách từ mỗi đỉnh đến trạm cứu hỏa gần nhất, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 2 1 4 1 2 2 3 3 4	0 1 1 0

Giải thích: Với 4 khu dân cư nối liên tiếp 1 - 2 - 3 - 4 và trạm cứu hỏa đặt tại khu 2:

- Khu 1: cách trạm cứu hỏa 1 bước.
- Khu 2: có sẵn trạm cứu hỏa $\rightarrow$ khoảng cách 0.
- Khu 3: cách trạm cứu hỏa 1 bước.
- Khu 4: cách trạm cứu hỏa 2 bước. Kết quả in ra: 1 0 1 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ thành phố và danh sách vị trí các trạm cứu hỏa. Hãy lập trình tính khoảng cách ngắn nhất từ từng khu dân cư đến trạm cứu hỏa gần nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 2 1 4 1 2 2 3 3 4 $\Rightarrow$ Đầu ra kỳ vọng: 0 1 1 0.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 2 1 4 1 2 2 3 3 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 khu dân cư nối liên tiếp 1 - 2 - 3 - 4 và trạm cứu hỏa đặt tại khu 2: - Khu 1: cách trạm cứu hỏa 1 bước. - Khu 2: có sẵn trạm cứu... Đổi chiều trạng thái cuối cùng	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn		Output chuẩn: 0 1 1 0

Giải thích chi tiết: Với 4 khu dân cư nối liên tiếp 1 - 2 - 3 - 4 và trạm cứu hỏa đặt tại khu 2:

- Khu 1: cách trạm cứu hỏa 1 bước.
- Khu 2: có sẵn trạm cứu hỏa $\rightarrow$ khoảng cách 0.
- Khu 3: cách trạm cứu hỏa 1 bước.
- Khu 4: cách trạm cứu hỏa 2 bước. Kết quả in ra: 1 0 1 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).

- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    if (!(cin >> n >> m >> k)) return 0;
    if (n <= 0) return 0;

    vector<int> dist(n + 1, -1);
    queue<int> q;

    for (int i = 0; i < k; ++i) {
        int st;
        cin >> st;
        dist[st] = 0;
        q.push(st);
    }

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();

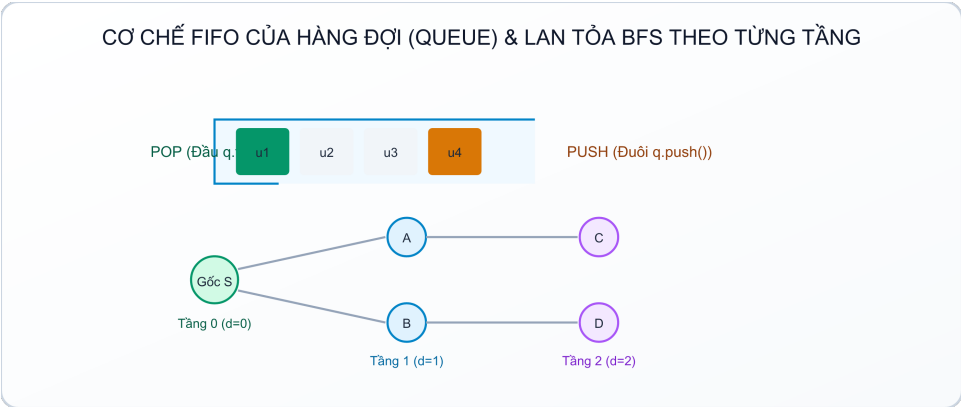
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    for (int i = 1; i <= n; ++i) {
        cout << dist[i] << (i == n ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 13 [cppb_que_13_cua_so_truot_chenh_lech_max_min_le_c]: Đoạn Con Dài Nhất Có Độ Chênh Lệch Max-Min $\leq C$

Bối cảnh: Một dây chuyền kiểm soát chất lượng linh kiện điện tử quét qua một dãy gồm N sản phẩm với chỉ số dung sai $A_1, A_2, \dots, A_N$. Một lô sản phẩm đạt chuẩn ổn định là một đoạn con các sản phẩm

liên tiếp sao cho độ chênh lệch giữa sản phẩm có dung sai lớn nhất và sản phẩm có dung sai nhỏ nhất trong lô không vượt quá một ngưỡng cho phép C .



Hình minh họa: Nguyên lý hoạt động Vào trước Ra trước (FIFO) của hàng đợi tiêu chuẩn (Queue)

Nhiệm vụ: Cho mảng A gồm N số nguyên và ngưỡng chênh lệch C . Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp có $\max - \min \leq C$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và C ($1 \leq N \leq 10^5, 0 \leq C \leq 10^9$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài lớn nhất của đoạn con tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 4 2 2 2 4 4	6

Giải thích: Với mảng $[8,2,4,7]$ và ngưỡng $C = 4$: Đoạn con $[2,4]$ có $\max = 4, \min = 2$, độ chênh lệch là $4 - 2 = 2 \leq 4$ có độ dài 2. Đoạn con $[4,7]$ có $\max = 7, \min = 4$, độ chênh lệch là $7 - 4 = 3 \leq 4$ có độ dài 2. Độ dài lớn nhất đạt được là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên và ngưỡng chênh lệch C . Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp có $\max - \min \leq C$.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).

- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 6 2 4 2 2 2 4 4 $\Rightarrow$ Đầu ra kỳ vọng: 6.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 2 4 2 2 2 4 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[8, 2, 4, 7]$ và ngưỡng $C = 4$: Đoạn con $[2, 4]$ có $\max = 4$, $\min = 2$, độ chênh lệch là $4 - 2 = 2 \leq 4$ có độ dài 2. Đ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với mảng $[8, 2, 4, 7]$ và ngưỡng $C = 4$: Đoạn con $[2, 4]$ có $\max = 4$, $\min = 2$, độ chênh lệch là $4 - 2 = 2 \leq 4$ có độ dài 2. Đoạn con $[4, 7]$ có $\max = 7$, $\min = 4$, độ chênh lệch là $7 - 4 = 3 \leq 4$ có độ dài 2. Độ dài lớn nhất đạt được là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa ($dq.front() \leq i - K$).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long c;
    if (!(cin >> n >> c)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
```

```

deque<int> min_dq, max_dq;
int left = 0;
int max_len = 0;

for (int right = 0; right < n; ++right) {
    while (!min_dq.empty() && a[min_dq.back()] >= a[right]) min_dq.pop_back();
    min_dq.push_back(right);

    while (!max_dq.empty() && a[max_dq.back()] <= a[right]) max_dq.pop_back();
    max_dq.push_back(right);

    while (a[max_dq.front()] - a[min_dq.front()] > c) {
        left++;
        if (min_dq.front() < left) min_dq.pop_front();
        if (max_dq.front() < left) max_dq.pop_front();
    }

    max_len = max(max_len, right - left + 1);
}

cout << max_len << "\n";
return 0;
}

```

Bài 14 [cppb_que_14_cat_bang_ron_quang_cao_toi_uu]: Chọn Đoạn Tối Đa Không Quá K Phần Tử Liên Kề

Bối cảnh: Một tuyến phố đi bộ có N vị trí treo biển quảng cáo, vị trí thứ i đem lại doanh thu A_i . Để đảm bảo không gian cảnh quan xanh cho tuyến phố, quy định đô thị yêu cầu không được phép treo biển quảng cáo tại quá K vị trí đứng liền sát nhau.

HÀNG ĐỢI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX $O(N)$

std::deque (Thao Tác Cả Hai Đầu Trong $O(1)$)

pop_front() / push_front()

pop_back() / push_back()

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K:

- Loại bỏ phần tử lỗi thời hết hạn cửa sổ: $\text{index} < i - k + 1 \rightarrow \text{dq.pop_front}()$
- Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi $\rightarrow \text{dq.pop_back}()$
- Phần tử nhỏ nhất hiện tại luôn nằm tại $\text{dq.front}() \rightarrow O(1)$

Hình minh họa: Hàng đợi hai đầu hai chiều (Deque) duy trì giá trị min/max trên cửa sổ trượt

Nhiệm vụ: Cho danh sách doanh thu của N vị trí và giới hạn K . Hãy lập trình chọn các vị trí treo biển sao cho không có quá K vị trí liền kề được chọn và tổng doanh thu thu về là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng doanh thu lớn nhất đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 2 3 4 5	12

Giải thích: Với 5 vị trí có doanh thu [1,2,3,4,5] và giới hạn $K = 2$ (không được chọn quá 2 vị trí liên tiếp): Phương án tối ưu là bỏ chọn vị trí thứ 3 (doanh thu 3), giữ lại các vị trí 1, 2, 4, 5. Các cụm chọn gồm [1, 2] (độ dài 2) và [4, 5] (độ dài 2), mang lại tổng doanh thu tối đa là $1 + 2 + 4 + 5 = 12$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách doanh thu của N vị trí và giới hạn K . Hãy lập trình chọn các vị trí treo biển sao cho không có quá K vị trí liên tiếp được chọn và tổng doanh thu thu về là lớn nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- **Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $\mathcal{O}(1)$.
- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 1 2 3 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 12.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 1 2 3 4 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 5 vị trí có doanh thu [1,2,3,4,5] và giới hạn $K = 2$ (không được chọn quá 2 vị trí liên tiếp): Phương án tối ưu là bỏ chọn v...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 12

Giải thích chi tiết: Với 5 vị trí có doanh thu [1,2,3,4,5] và giới hạn $K = 2$ (không được chọn quá 2 vị trí liên tiếp): Phương án tối ưu là bỏ chọn vị trí thứ 3 (doanh thu 3), giữ lại các vị trí 1, 2, 4, 5. Các cụm chọn gồm [1, 2] (độ dài 2) và [4, 5] (độ dài 2), mang lại tổng doanh thu tối đa là $1 + 2 + 4 + 5 = 12$.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa ($dq.front() \leq i - K$).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    long long total_sum = 0;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        total_sum += a[i];
    }

    // dp[i]: tổng nhỏ nhất các phần tử bị loại bỏ kết thúc tại i sao cho không có k+1 phần tử
    // liên kề nào được chọn
    vector<long long> dp(n + 1, 0);
    deque<int> dq;
    dq.push_back(0);

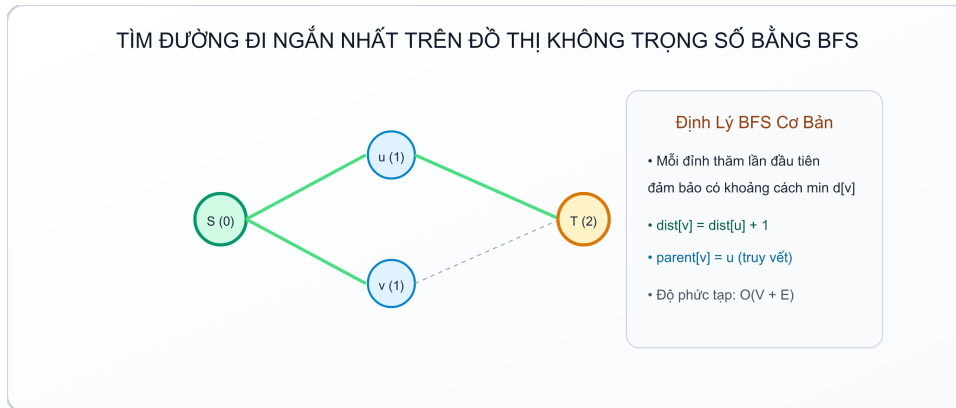
    for (int i = 1; i <= n; ++i) {
        while (!dq.empty() && dq.front() < i - k - 1) dq.pop_front();
        dp[i] = dp[dq.front()] + a[i];
        while (!dq.empty() && dp[dq.back()] >= dp[i]) dq.pop_back();
        dq.push_back(i);
    }

    long long min_dropped = LLONG_MAX;
    for (int i = n - k; i <= n; ++i) {
        if (i >= 0) min_dropped = min(min_dropped, dp[i]);
    }

    cout << total_sum - min_dropped << "\n";
    return 0;
}
```

Bài 15 [cppb_que_15_dua_xe_me_cung_doi_huong]: Đua Xe Mê Cung Đổi Hướng Ít Nhất (0-1 BFS State)

Bối cảnh: Trong một giải đua xe robot trong mê cung lưới ô vuông $N \times M$, robot cần di chuyển từ ô xuất phát S tới ô đích D . Mỗi lần robot đi thẳng theo hướng đang di chuyển thì hoàn toàn miễn phí (chi phí 0), nhưng mỗi khi robot phải rẽ lái đổi sang một trong các hướng vuông góc thì bánh lái sẽ tiêu hao 1 đơn vị năng lượng.



Hình minh họa: Cơ chế hàng đợi Queue mở rộng tìm đường đi ngắn nhất đồ thị trọng số đơn vị

Nhiệm vụ: Cho bản đồ mê cung $N \times M$ và vị trí S, D . Hãy lập trình tìm số lần đổi hướng ít nhất để robot đi từ S tới D . Nếu không có đường đi, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi M ký tự biểu diễn mê cung (. là đường đi, * là vật cản, S là xuất phát, D là đích).

Đầu ra (Output):

- In ra số lần đổi hướng ít nhất, hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 S.. .#. ..E	1

Giải thích: Với mê cung 3×3 không vật cản từ góc (1,1) tới (3,3): Robot chỉ cần đi thẳng hết hàng 1 sang phải, sau đó rẽ lái 1 lần duy nhất để đi thẳng xuống dưới tới đích. Số lần đổi hướng ít nhất là 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho bản đồ mê cung $N \times M$ và vị trí S, D . Hãy lập trình tìm số lần đổi hướng ít nhất để robot đi từ S tới D . Nếu không có đường đi, in ra -1.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Nguyên lý Hàng đợi chuẩn (Queue):** Vào trước Ra trước (FIFO), là cấu trúc nền tảng phục vụ duyệt đồ thị theo chiều rộng (BFS).
- Hàng đợi hai đầu (Deque) & Monotonic Deque:**
- Hỗ trợ thêm/xoá ở cả hai đầu trong $O(1)$.

- Khi trượt cửa sổ kích thước K , lưu chỉ số phần tử trong deque sao cho giá trị tương ứng luôn đơn điệu.
- Loại bỏ phần tử trượt ra khỏi cửa sổ ở đầu trước (`pop_front()`) và loại bỏ phần tử kém tối ưu ở đầu sau (`pop_back()`).
- **Độ phức tạp:** Thời gian $\mathcal{O}(N)$, không gian phụ trợ $\mathcal{O}(K)$.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 3 3 S.. .#. ..E $\Rightarrow$ Đầu ra kỳ vọng: 1.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 S.. .#. ..E	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mê cung 3×3 không vật cản từ góc (1,1) tới (3,3): Robot chỉ cần đi thẳng hết hàng 1 sang phải, sau đó rẽ trái 1 lần duy nhất...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1

Giải thích chi tiết: Với mê cung 3×3 không vật cản từ góc (1,1) tới (3,3): Robot chỉ cần đi thẳng hết hàng 1 sang phải, sau đó rẽ trái 1 lần duy nhất để đi thẳng xuống dưới tới đích. Số lần đổi hướng ít nhất là 1.

3. Lưu ý & Bẫy lỗi thường gặp

- Lưu giá trị thay vì lưu chỉ số vị trí (index) trong Deque: Phải lưu index để kiểm tra điều kiện phần tử đã trượt ra khỏi cửa sổ $i - K$ hay chưa (`dq.front() <= i - K`).
- Quên kiểm tra `!dq.empty()` trước khi truy xuất `dq.front()` hoặc `dq.back()` gây crash chương trình.
- Không khởi tạo kết quả cho $K - 1$ vị trí đầu tiên trước khi bắt đầu ghi nhận đáp án từ vị trí thứ K .

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

struct State {
    int r, c, dir;
```

```
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    int sr = -1, sc = -1, er = -1, ec = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'S') { sr = r; sc = c; }
            if (grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

    vector<vector<vector<int>>> dist(n, vector<vector<int>>(m, vector<int>(4, INF)));
    deque<State> dq;

    for (int d = 0; d < 4; ++d) {
        dist[sr][sc][d] = 0;
        dq.push_back({sr, sc, d});
    }

    while (!dq.empty()) {
        auto [r, c, dir] = dq.front();
        dq.pop_front();

        for (int nd = 0; nd < 4; ++nd) {
            int nr = r + dr[nd];
            int nc = c + dc[nd];
            int cost = (nd == dir ? 0 : 1);

            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#') {
                if (dist[r][c][dir] + cost < dist[nr][nc][nd]) {
                    dist[nr][nc][nd] = dist[r][c][dir] + cost;
                    if (cost == 0) {
                        dq.push_front({nr, nc, nd});
                    } else {
                        dq.push_back({nr, nc, nd});
                    }
                }
            }
        }
    }

    int ans = INF;
    for (int d = 0; d < 4; ++d) ans = min(ans, dist[er][ec][d]);

    if (ans == INF) cout << -1 << "\n";
    else cout << ans << "\n";
    return 0;
}
```

CHƯƠNG 08: ĐỒ THỊ & CÂY TRUY VẤN ĐOẠN

Bài 22: Lý thuyết đồ thị cơ bản: Duyệt BFS & DFS

Bài 01 [cppb_gra_01_chuyen_doi_danh_sach_canh_sang_danh_sach_ke]: Chuyển Danh Sách Cạnh Sang Danh Sách Kề

Bối cảnh: Một đồ thị mạng lưới giao thông gồm N nút giao và M con đường hai chiều ban đầu được lưu trữ dưới dạng danh sách các cặp cạnh (u, v) . Để thuận tiện cho việc lập trình các thuật toán tìm kiếm và duyệt đồ thị, kỹ sư cầu đường cần chuyển đổi dữ liệu mạng lưới sang cấu trúc danh sách kề: Với mỗi nút giao, liệt kê tất cả các nút giao có đường nối trực tiếp với nó theo thứ tự số hiệu tăng dần.

BIỂU DIỄN ĐỒ THỊ: MA TRẬN KỀ VS DANH SÁCH KỀ (ADJACENCY LIST)

Ma Trận Kề: `adj[N][N]`

```
int adj[1005][1005];
```

- Kiểm tra cạnh (u, v) : $O(1)$
- Bộ nhớ: $O(N^2) \rightarrow$ Sập bộ nhớ khi $N=10^5$
- Duyệt đỉnh kề: $O(N)$

$\rightarrow$ Chỉ dùng khi $N \leq 1000$ hoặc đồ thị dày

Danh Sách Kề: `vector<int> adj[N]`

```
vector<int> adj[100005];
```

- Bộ nhớ: $O(N + M)$ tối ưu tuyệt đối
- Duyệt đỉnh kề: $O(deg(u))$ cực nhanh
- Thêm cạnh: `adj[u].push_back(v)` trong $O(1)$

$\rightarrow$ Chuẩn mực thi đấu Olympic cho $N, M \leq 10^5$

Hình minh họa: Ba phương pháp biểu diễn đồ thị trong lập trình: Ma trận kề, Danh sách cạnh, Danh sách kề

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình chuyển đổi sang biểu diễn danh sách kề và in ra các đỉnh kề của từng đỉnh theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai số nguyên u và v ($1 \leq u, v \leq N, u \neq v$) biểu diễn một cạnh.

Đầu ra (Output):

- In ra N dòng, dòng thứ i bắt đầu bằng số thứ tự đỉnh i kèm dấu hai chấm, theo sau là danh sách các đỉnh kề với i theo thứ tự tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2 1 2 1 3	2 2 3 1 1 1 1

Giải thích: Với đồ thị 4 đỉnh và các cạnh $(1, 2), (1, 3), (2, 4)$:

- Đỉnh 1 kề với các đỉnh: 2, 3.
- Đỉnh 2 kề với các đỉnh: 1, 4.
- Đỉnh 3 kề với đỉnh: 1.
- Đỉnh 4 kề với đỉnh: 2.

— 193 —

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình chuyển đổi sang biểu diễn danh sách kề và in ra các đỉnh kề của từng đỉnh theo thứ tự tăng dần.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 2 1 2 1 3 $\Rightarrow$ Đầu ra kỳ vọng: 2 2 3 1 1 1 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 2 1 2 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị 4 đỉnh và các cạnh (1, 2), (1, 3), (2, 4): - Đỉnh 1 kề với các đỉnh: 2, 3. - Đỉnh 2 kề với các đỉnh: 1, 4. - Đỉnh 3 kề với đỉnh: 1. - Đỉnh 4 kề với đỉnh: 2.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2 2 3 1 1 1 1

Giải thích chi tiết: Với đồ thị 4 đỉnh và các cạnh (1, 2), (1, 3), (2, 4):

- Đỉnh 1 kề với các đỉnh: 2, 3.
- Đỉnh 2 kề với các đỉnh: 1, 4.
- Đỉnh 3 kề với đỉnh: 1.
- Đỉnh 4 kề với đỉnh: 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.

- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp for ($\text{int } i = 1; i \leq N; ++i$) và gọi BFS/DFS khi $!visited[i]$ để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= n; ++i) {
        sort(adj[i].begin(), adj[i].end());
        cout << adj[i].size();
        for (int v : adj[i]) {
            cout << " " << v;
        }
        cout << "\n";
    }
    return 0;
}
```

Bài 02 [cppb_gra_02_duyet_do_thi_theo_chieu_sau_dfs]: Duyệt Đồ Thị Theo Chiều Sâu (DFS Traversal)

Bối cảnh: Một robot thám hiểm cần khám phá toàn bộ các gian phòng trong một hang động ngầm gồm N gian phòng và M lối đi hai chiều. Robot áp dụng chiến lược tìm kiếm theo chiều sâu: Từ gian phòng hiện tại, luôn ưu tiên tiến sâu vào một gian phòng chưa từng được khám phá kề cạnh có số hiệu nhỏ nhất; khi không thể đi tiếp thì quay lui lại gian phòng trước đó.

SO SÁNH CHIẾN LƯỢC DUYỆT ĐỒ THỊ: BFS (QUEUE) VS DFS (ĐỆ QUY/STACK)

BFS: Duyệt Theo Chiều Rộng

- Cấu trúc: Dùng Hàng Đợi (Queue)
- Cơ chế: Lan tỏa theo từng lớp bán kính $d=0, 1, 2, \dots$
- Điểm mạnh: Tìm đường đi ngắn nhất đồ thị không trọng số
- Độ phức tạp: $O(V + E)$

DFS: Duyệt Theo Chiều Sâu

- Cấu trúc: Dùng Đệ Quy hoặc Ngăn Xếp (Stack)
- Cơ chế: Đi sâu hết mức trên một nhánh rồi quay lui
- Điểm mạnh: Phát hiện chu trình, kiểm tra liên thông, cây DFS
- Độ phức tạp: $O(V + E)$

Hình minh họa: So sánh chiến lược duyệt đồ thị: Duyệt theo chiều sâu (DFS) và Duyệt theo chiều rộng (BFS)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S . Hãy lập trình in ra thứ tự các đỉnh được robot ghé thăm trong quá trình duyệt theo chiều sâu (khi có nhiều lựa chọn, luôn ưu tiên đỉnh có số hiệu nhỏ hơn).

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, S ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5, 1 \leq S \leq N$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra trên một dòng thứ tự các đỉnh được ghé thăm trong hành trình, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 1 2 2 3 1 4	1 2 3 4

Giải thích: Với đồ thị 4 đỉnh gồm các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1:

- Từ đỉnh 1 thăm đỉnh 2 (nhỏ hơn 3).
- Từ đỉnh 2 tiếp tục thăm đỉnh 4.
- Đỉnh 4 không còn đường mới, quay lui về 2, rồi về 1, từ 1 thăm tiếp đỉnh 3. Thứ tự ghé thăm là: 1 2 4 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S . Hãy lập trình in ra thứ tự các đỉnh được robot ghé thăm trong quá trình duyệt theo chiều sâu (khi có nhiều lựa chọn, luôn ưu tiên đỉnh có số hiệu nhỏ hơn).
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 1 2 2 3 1 4 $\Rightarrow$ Đầu ra kỳ vọng: 1 2 3 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
------	--------------------	-------------------------------	------------------

1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 1 2 2 3 1 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị 4 đỉnh gồm các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1: - Từ đỉnh 1 thăm đỉnh 2 (nhỏ hơn 3). - Từ đỉnh 2 tiếp tục th...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 2 3 4

Giải thích chi tiết: Với đồ thị 4 đỉnh gồm các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1:

- Từ đỉnh 1 thăm đỉnh 2 (nhỏ hơn 3).
- Từ đỉnh 2 tiếp tục thăm đỉnh 4.
- Đỉnh 4 không còn đường mới, quay lui về 2, rồi về 1, từ 1 thăm tiếp đỉnh 3. Thứ tự ghé thăm là: 1 2 4 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m, s;
vector<vector<int>>> adj;
vector<bool> visited;
vector<int> traversal;

void dfs(int u) {
    visited[u] = true;
    traversal.push_back(u);
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

if (!(cin >> n >> m >> s)) return 0;
if (n <= 0) return 0;

adj.assign(n + 1, vector<int>());
visited.assign(n + 1, false);

for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

for (int i = 1; i <= n; ++i) sort(adj[i].begin(), adj[i].end());

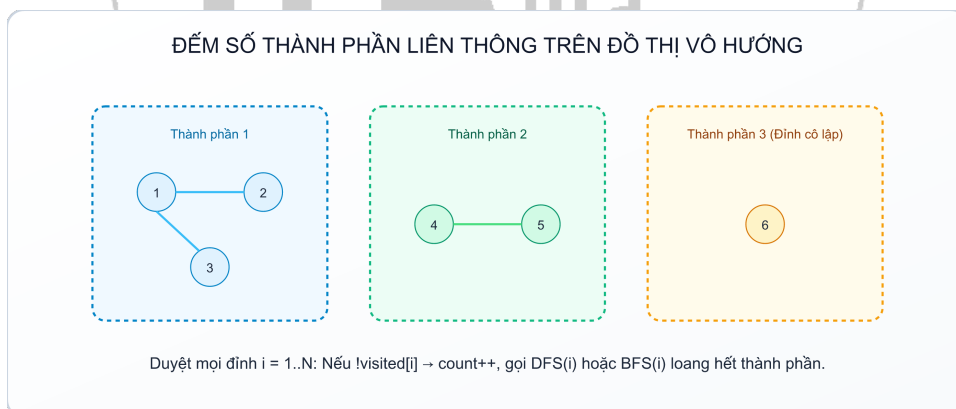
dfs(s);

for (int i = 0; i < (int)traversal.size(); ++i) {
    cout << traversal[i] << (i + 1 == (int)traversal.size() ? "" : " ");
}
cout << "\n";
return 0;
}

```

Bài 03 [cppb_gra_03_duyet_do_thi_theo_chieu_rong_bfs]: Duyệt Đồ Thị Theo Chiều Rộng (BFS Traversal)

Bối cảnh: Một thông điệp cảnh báo khẩn cấp cần được lan truyền qua một mạng xã hội gồm N người dùng và M mối quan hệ bạn bè. Xuất phát từ người dùng S , thông điệp sẽ được gửi đồng thời tới tất cả những người bạn trực tiếp của S trước (tầng 1), sau đó mới tiếp tục lan truyền tới bạn của bạn (tầng 2) theo nguyên tắc lan truyền theo chiều rộng (ưu tiên người có số hiệu nhỏ hơn).



Hình minh họa: Thuật toán loang tìm các thành phần liên thông trong đồ thị vô hướng

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S . Hãy lập trình in ra thứ tự các đỉnh nhận được thông điệp theo chiến lược duyệt theo chiều rộng.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, S ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5, 1 \leq S \leq N$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra trên một dòng thứ tự các đỉnh được ghé thăm, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 1 2 1 3 2 4	1 2 3 4

Giải thích: Với đồ thị có các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1:

- Tầng 1 (kề trực tiếp với 1): thăm đỉnh 2 và đỉnh 3 theo thứ tự tăng dần.
- Tầng 2: từ đỉnh 2 thăm tiếp đỉnh 4. Thứ tự ghé thăm là: 1 2 3 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S . Hãy lập trình in ra thứ tự các đỉnh nhận được thông điệp theo chiến lược duyệt theo chiều rộng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề vector<vector<int>> adj($N + 1$) để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng visited[] để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 1 2 1 3 2 4 $\Rightarrow$ Đầu ra kỳ vọng: 1 2 3 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 1 2 1 3 2 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị có các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1: - Tầng 1 (kề trực tiếp với 1): thăm đỉnh 2 và đỉnh 3 theo thứ tự tăng dần...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 1 2 3 4

Giải thích chi tiết: Với đồ thị có các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1:

- Tầng 1 (kề trực tiếp với 1): thăm đỉnh 2 và đỉnh 3 theo thứ tự tăng dần.

- Tầng 2: từ đỉnh 2 thăm tiếp đỉnh 4. Thứ tự ghé thăm là: 1 2 3 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s;
    if (!(cin >> n >> m >> s)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= n; ++i) sort(adj[i].begin(), adj[i].end());

    vector<bool> visited(n + 1, false);
    vector<int> traversal;
    queue<int> q;

    visited[s] = true;
    q.push(s);

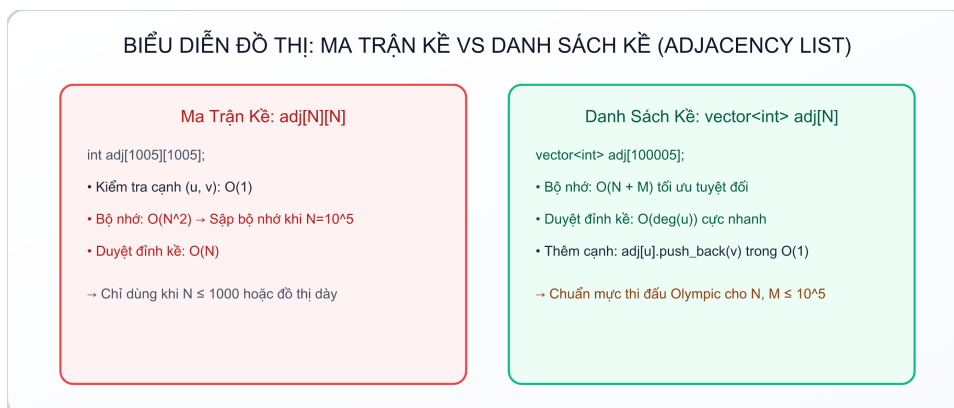
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        traversal.push_back(u);

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }

    for (int i = 0; i < (int)traversal.size(); ++i) {
        cout << traversal[i] << (i + 1 == (int)traversal.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 04 [cppb_gra_04_dem_so_thanh_phan_lien_thong]: Đếm Số Thành Phần Liên Thông

Bối cảnh: Một quần đảo gồm N hòn đảo và M cây cầu hai chiều nối giữa một số cặp đảo. Hai hòn đảo thuộc cùng một cụm đảo liên thông nếu cư dân có thể đi lại giữa chúng qua các cây cầu (trực tiếp hoặc gián tiếp). Ban quản lý du lịch cần xác định xem toàn bộ quần đảo đang bị chia cắt thành bao nhiêu cụm đảo biệt lập.



Hình minh họa: Ba phương pháp biểu diễn đồ thị trong lập trình: Ma trận kề, Danh sách cạnh, Danh sách kề

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng thành phần liên thông của đồ thị.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số lượng thành phần liên thông.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 2 3 4 5	2

Giải thích: Với 5 đỉnh và các cạnh $(1, 2), (3, 4)$:

- Cụm 1 gồm các đỉnh $\{1, 2\}$.
- Cụm 2 gồm các đỉnh $\{3, 4\}$.
- Cụm 3 gồm đỉnh cô lập $\{5\}$. Có tất cả 3 thành phần liên thông độc lập, kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng thành phần liên thông của đồ thị.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $O(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**

- *Tìm kiếm theo chiều rộng (BFS)*: Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS)*: Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu**: Sử dụng mảng visited[] để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $O(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 2 3 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 2 3 4 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 5 đỉnh và các cạnh (1, 2), (3, 4): - Cụm 1 gồm các đỉnh {1, 2}. - Cụm 2 gồm các đỉnh {3, 4}. - Cụm 3 gồm đỉnh cô lập {5}. Có tất cả...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với 5 đỉnh và các cạnh (1, 2), (3, 4):

- Cụm 1 gồm các đỉnh {1, 2}.
- Cụm 2 gồm các đỉnh {3, 4}.
- Cụm 3 gồm đỉnh cô lập {5}. Có tất cả 3 thành phần liên thông độc lập, kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: adj[u].push_back(v); adj[v].push_back(u);.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp for (int i = 1; i <= N; ++i) và gọi BFS/DFS khi !visited[i] để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;
vector<bool> visited;
```

```

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int components = 0;
    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            components++;
            dfs(i);
        }
    }

    cout << components << "\n";
    return 0;
}

```

Bài 05 [cppb_gra_05_tìm_kích_thuoc_thanh_phan_lien_thong_lon_nhat]: Kích Thước Thành Phần Liên Thông Lớn Nhất

Bối cảnh: Vẫn tại quần đảo gồm N hòn đảo và M cây cầu, ban quy hoạch kinh tế muốn tìm cụm đảo liên thông phát triển sầm uất nhất (chứa số lượng hòn đảo nhiều nhất) để đầu tư xây dựng trung tâm logistics tập trung.

SO SÁNH CHIẾN LƯỢC DUYỆT ĐỒ THỊ: BFS (QUEUE) VS DFS (ĐỆ QUY/STACK)

BFS: Duyệt Theo Chiều Rộng

- Cấu trúc: Dùng Hàng Đợi (Queue)
- Cơ chế: Lan tỏa theo từng lớp bán kính $d=0, 1, 2, \dots$
- Điểm mạnh: Tìm đường đi ngắn nhất đồ thị không trọng số
- Độ phức tạp: $O(V + E)$

DFS: Duyệt Theo Chiều Sâu

- Cấu trúc: Dùng Đệ Quy hoặc Ngăn Xếp (Stack)
- Cơ chế: Đi sâu hết mức trên một nhánh rồi quay lui
- Điểm mạnh: Phát hiện chu trình, kiểm tra liên thông, cây DFS
- Độ phức tạp: $O(V + E)$

Hình minh họa: So sánh chiến lược duyệt đồ thị: Duyệt theo chiều sâu (DFS) và Duyệt theo chiều rộng (BFS)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình tìm số lượng đỉnh thuộc về thành phần liên thông có kích thước lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra trên một dòng duy nhất kích thước của thành phần liên thông lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 2 3 4 5	3

Giải thích: Với 5 đỉnh và các cạnh (1, 2), (2, 3), (4, 5):

- Cụm 1 gồm {1, 2, 3} có 3 hòn đảo.
- Cụm 2 gồm {4, 5} có 2 hòn đảo. Kích thước của thành phần liên thông lớn nhất là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình tìm số lượng đỉnh thuộc về thành phần liên thông có kích thước lớn nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 2 3 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 2 3 4 5	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với 5 đỉnh và các cạnh (1, 2), (2, 3), (4, 5): - Cụm 1 gồm {1, 2, 3} có 3 hòn đảo. - Cụm 2 gồm {4, 5} có 2 hòn đảo. Kích thước của thành phần liên thông lớn nhất là 3.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với 5 đỉnh và các cạnh (1, 2), (2, 3), (4, 5):

- Cụm 1 gồm {1, 2, 3} có 3 hòn đảo.
- Cụm 2 gồm {4, 5} có 2 hòn đảo. Kích thước của thành phần liên thông lớn nhất là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;
vector<bool> visited;

int dfs(int u) {
    visited[u] = true;
    int sz = 1;
    for (int v : adj[u]) {
        if (!visited[v]) sz += dfs(v);
    }
    return sz;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
```

```

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

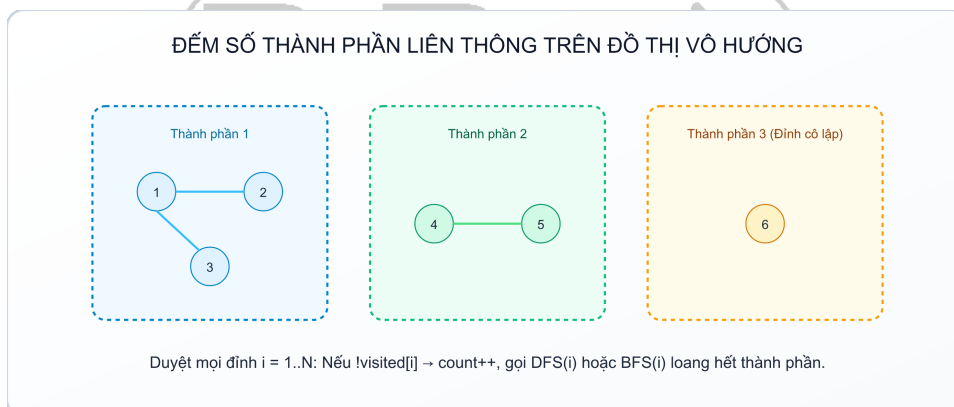
    int max_sz = 0;
    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            max_sz = max(max_sz, dfs(i));
        }
    }

    cout << max_sz << "\n";
    return 0;
}

```

Bài 06 [cppb_gra_06_kiem_tra_duong_di_giua_hai_dinh]: Kiểm Tra Đường Đi Giữa Hai Đỉnh

Bối cảnh: Trong hệ thống mạng lưới điện quốc gia gồm N trạm biến áp và M đường dây tải điện hai chiều, trung tâm điều độ cần kiểm tra nhanh xem liệu có tồn tại tuyến đường dây kết nối (trực tiếp hoặc qua các trạm trung gian) giữa hai trạm biến áp S và D hay không.



Hình minh họa: Thuật toán loang tìm các thành phần liên thông trong đồ thị vô hướng

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình kiểm tra xem có tồn tại đường đi giữa S và D hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa 4 số nguyên N, M, S, D ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5, 1 \leq S, D \leq N$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra YES nếu có đường đi giữa S và D , ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 1 4 1 2 2 3	NO

Giải thích: Với đồ thị có các cạnh $(1, 2)$, $(2, 3)$ và đỉnh 4 cô lập:

- Kiểm tra giữa 1 và 3: Tồn tại đường đi $1 \times o2 \times o3$, in ra YES.
- Nếu kiểm tra giữa 1 và 4: Không có đường đi, in ra NO.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình kiểm tra xem có tồn tại đường đi giữa S và D hay không. Nếu có in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $O(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $O(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 2 1 4 1 2 2 3 $\Rightarrow$ Đầu ra kỳ vọng: NO.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 2 1 4 1 2 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị có các cạnh (1, 2), (2, 3) và đỉnh 4 cô lập: - Kiểm tra giữa 1 và 3: Tồn tại đường đi $1 \times 2 \times 3$, in ra YES. - Nếu ki...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: NO

Giải thích chi tiết: Với đồ thị có các cạnh (1, 2), (2, 3) và đỉnh 4 cô lập:

- Kiểm tra giữa 1 và 3: Tồn tại đường đi $1 \times 2 \times 3$, in ra YES.
- Nếu kiểm tra giữa 1 và 4: Không có đường đi, in ra NO.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.

- Trần bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m, s, t;
vector<vector<int>> adj;
vector<bool> visited;

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m >> s >> t)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

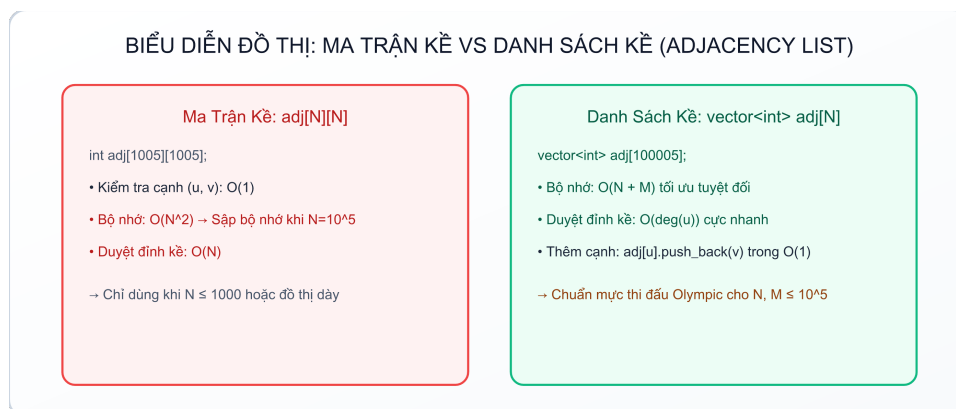
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    dfs(s);

    if (visited[t]) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```

Bài 07 [cppb_gra_07_phat_hien_chu_trinh_tren_do_thi_vo_huong]: Phát Hiện Chu Trình Trên Đồ Thị Vô Hướng

Bối cảnh: Một mạng lưới đường ống cấp nước đô thị cần được kiểm tra thiết kế. Nếu trong mạng lưới xuất hiện một chu trình khép kín (một vòng tròn đường ống quay về điểm xuất phát mà không đi lặp lại cạnh nào), áp lực nước có thể bị nhiễu loạn dòng chảy. Kỹ sư thủy lợi cần kiểm tra xem bản vẽ mạng lưới có chứa chu trình hay không.



Hình minh họa: Ba phương pháp biểu diễn đồ thị trong lập trình: Ma trận kề, Danh sách cạnh, Danh sách kề

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có chứa chu trình hay không. Nếu có in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra YES nếu đồ thị chứa ít nhất một chu trình, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 2 2 3 3 1	YES

Giải thích: Với đồ thị gồm 3 đỉnh có các cạnh (1, 2), (2, 3), (3, 1): Ba đỉnh này tạo thành một tam giác khép kín $1 - 2 - 3 - 1$, là một chu trình hoàn chỉnh. Kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có chứa chu trình hay không. Nếu có in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $O(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $O(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 1 2 2 3 3 1 $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 1 2 2 3 3 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị gồm 3 đỉnh có các cạnh (1, 2), (2, 3), (3, 1): Ba đỉnh này tạo thành một tam giác khép kín 1 – 2 – 3 – 1, là một chu trình...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với đồ thị gồm 3 đỉnh có các cạnh (1, 2), (2, 3), (3, 1): Ba đỉnh này tạo thành một tam giác khép kín 1 – 2 – 3 – 1, là một chu trình hoàn chỉnh. Kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;
vector<bool> visited;
bool has_cycle = false;

void dfs(int u, int p) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, u);
        } else if (v != p) {
            has_cycle = true;
        }
    }
}

int main() {
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

if (!(cin >> n >> m)) return 0;
if (n <= 0) return 0;

adj.assign(n + 1, vector<int>());
visited.assign(n + 1, false);

for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

for (int i = 1; i <= n; ++i) {
    if (!visited[i]) dfs(i, 0);
}

if (has_cycle) cout << "YES\n";
else cout << "NO\n";
return 0;
}
```

Bài 08 [cppb_gra_08_tim_duong_di_ngan_nhat_bang_bfs]: Đường Đi Ngắn Nhất Trên Đồ Thị Không Trọng Số

Bối cảnh: Một đoàn tàu hỏa vận chuyển hàng hóa di chuyển trên mạng lưới đường ray gồm N nhà ga và M chặng ray hai chiều nối giữa các ga kề nhau. Đoàn tàu cần đi từ ga xuất phát S đến ga đích D . Mỗi chặng ray mất đúng 1 giờ chạy tàu. Hãy tìm thời gian ít nhất (tổng số chặng ray ít nhất) để tàu đến được ga đích.

SO SÁNH CHIẾN LƯỢC DUYỆT ĐỒ THỊ: BFS (QUEUE) VS DFS (ĐỆ QUY/STACK)

BFS: Duyệt Theo Chiều Rộng

- Cấu trúc: Dùng Hàng Đợi (Queue)
- Cơ chế: Lan tỏa theo từng lớp bán kính $d=0, 1, 2, \dots$
- Điểm mạnh: Tìm đường đi ngắn nhất đồ thị không trọng số
- Độ phức tạp: $O(V + E)$

DFS: Duyệt Theo Chiều Sâu

- Cấu trúc: Dùng Đệ Quy hoặc Ngăn Xếp (Stack)
- Cơ chế: Đi sâu hết mức trên một nhánh rồi quay lại
- Điểm mạnh: Phát hiện chu trình, kiểm tra liên thông, cây DFS
- Độ phức tạp: $O(V + E)$

Hình minh họa: So sánh chiến lược duyệt đồ thị: Duyệt theo chiều sâu (DFS) và Duyệt theo chiều rộng (BFS)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số cạnh trên đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 4 số nguyên N, M, S, D ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra số cạnh ngắn nhất từ S tới D , hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 1 3	2

Giải thích: Với mạng đường ray có các chặng (1, 2), (2, 3), (1, 4), (4, 3) từ ga 1 tới ga 3: Có hai lộ trình cùng đạt 2 chặng là $1 \times o2 \times o3$ hoặc $1 \times o4 \times o3$. Thời gian ít nhất là 2 chặng.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số cạnh trên đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 1 2 2 3 3 4 1 3 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 1 2 2 3 3 4 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mạng đường ray có các chặng (1, 2), (2, 3), (1, 4), (4, 3) từ ga 1 tới ga 3: Có hai lộ trình cùng đạt 2 chặng là $1 \times o2 \times o3 \dots$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với mạng đường ray có các chặng (1, 2), (2, 3), (1, 4), (4, 3) từ ga 1 tới ga 3: Có hai lộ trình cùng đạt 2 chặng là $1 \times o2 \times o3$ hoặc $1 \times o4 \times o3$. Thời gian ít nhất là 2 chặng.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);
    queue<int> q;

    dist[1] = 0;
    q.push(1);

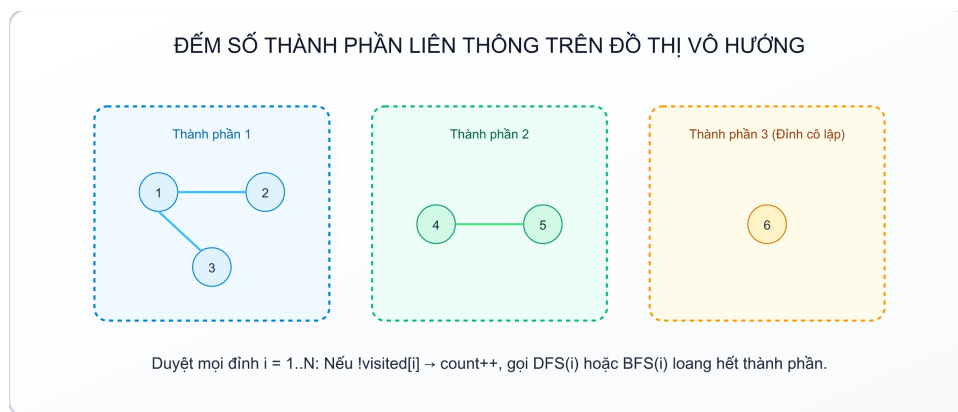
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    cout << dist[n] << "\n";
    return 0;
}
```

Bài 09 [cppb_gra_09_kiem_tra_do_thi_cay]: Kiểm Tra Đồ Thị Cây (Tree Verification)

Bối cảnh: Trong cấu trúc liên kết mạng máy tính, cấu trúc dạng cây (Tree) là mô hình mạng tối ưu nhất vì nó đảm bảo tất cả N máy chủ đều liên thông với nhau mà hoàn toàn không có chu trình dư thừa (số lượng kênh kết nối đúng bằng $N - 1$). Kỹ sư hệ thống cần xác thực xem một mạng lưới cho trước có thỏa mãn đầy đủ các điều kiện của một đồ thị cây hay không.



Hình minh họa: Thuật toán loang tìm các thành phần liên thông trong đồ thị vô hướng

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là một cây hợp lệ hay không. Nếu đúng in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra YES nếu đồ thị là cây, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 2 3 3 4	YES

Giải thích: Với đồ thị có $N = 4$ đỉnh và $M = 3$ cạnh: $(1, 2), (1, 3), (1, 4)$: Đồ thị liên thông toàn bộ và không chứa bất kỳ chu trình nào, số cạnh đúng bằng $4 - 1 = 3$. Đồ thị là một cây, kết quả in ra là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là một cây hợp lệ hay không. Nếu đúng in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề $\text{vector} \langle \text{vector} \langle \text{int} \rangle \rangle \text{adj}(N + 1)$ để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng $visited[]$ để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 2 2 3 3 4 $\Rightarrow$ Đầu ra kỳ vọng: YES.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 2 2 3 3 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị có $N = 4$ đỉnh và $M = 3$ cạnh: (1, 2), (1, 3), (1, 4): Đồ thị liên thông toàn bộ và không chứa bất kỳ chu trình nào, số cạ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Với đồ thị có $N = 4$ đỉnh và $M = 3$ cạnh: (1, 2), (1, 3), (1, 4): Đồ thị liên thông toàn bộ và không chứa bất kỳ chu trình nào, số cạnh đúng bằng $4 - 1 = 3$. Đồ thị là một cây, kết quả in ra là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;
vector<bool> visited;

int dfs(int u) {
    visited[u] = true;
    int sz = 1;
    for (int v : adj[u]) {
        if (!visited[v]) sz += dfs(v);
    }
    return sz;
}

int main() {
    ios::sync_with_stdio(false);
```

```
cin.tie(nullptr);

if (!(cin >> n >> m)) return 0;
if (n <= 0) return 0;

if (m != n - 1) {
    cout << "NO\n";
    return 0;
}

adj.assign(n + 1, vector<int>());
visited.assign(n + 1, false);

for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int sz = dfs(1);

if (sz == n) cout << "YES\n";
else cout << "NO\n";
return 0;
}
```

Bài 10 [cppb_gra_10_sap_xep_to_po_tren_dag]: Sắp Xếp Tô-pô (Topological Sort)

Bối cảnh: Một chương trình đào tạo kỹ sư công nghệ thông tin gồm N môn học được đánh số từ 1 đến N . Giữa các môn học có M điều kiện môn tiên quyết dạng $u \times ov$ (nghĩa là sinh viên bắt buộc phải hoàn thành môn học u trước khi được phép đăng ký môn học v). Ban đào tạo cần lập ra một lộ trình học tập hợp lệ thỏa mãn tất cả các điều kiện tiên quyết.

BIỂU DIỄN ĐỒ THỊ: MA TRẬN KỀ VS DANH SÁCH KỀ (ADJACENCY LIST)

Ma Trận Kề: $adj[N][N]$

```
int adj[1005][1005];
```

- Kiểm tra cạnh (u, v) : $O(1)$
- Bộ nhớ: $O(N^2) \rightarrow$ Sập bộ nhớ khi $N=10^5$
- Duyệt đỉnh kề: $O(N)$

$\rightarrow$ Chỉ dùng khi $N \leq 1000$ hoặc đồ thị dày

Danh Sách Kề: $vector<int> adj[N]$

```
vector<int> adj[100005];
```

- Bộ nhớ: $O(N + M)$ tối ưu tuyệt đối
- Duyệt đỉnh kề: $O(deg(u))$ cực nhanh
- Thêm cạnh: $adj[u].push_back(v)$ trong $O(1)$

$\rightarrow$ Chuẩn mực thi đấu Olympic cho $N, M \leq 10^5$

Hình minh họa: Ba phương pháp biểu diễn đồ thị trong lập trình: Ma trận kề, Danh sách cạnh, Danh sách kề

Nhiệm vụ: Cho đồ thị có hướng N đỉnh M cạnh không có chu trình. Hãy lập trình tìm một thứ tự sắp xếp tô-pô của các môn học.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v biểu diễn điều kiện môn u học trước môn v .

Đầu ra (Output):

- In ra trên một dòng gồm N số nguyên là thứ tự các môn học hợp lệ, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 2 3 1 3	1 4 2 3

Giải thích: Với 3 môn học và các điều kiện: môn 1 trước môn 2 ($1 \times o2$), môn 2 trước môn 3 ($2 \times o3$): Lộ trình học tập bắt buộc duy nhất là: 1 2 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị có hướng N đỉnh M cạnh không có chu trình. Hãy lập trình tìm một thứ tự sắp xếp tô-pô của các môn học.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 2 2 3 1 3 $\Rightarrow$ Đầu ra kỳ vọng: 1 4 2 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 2 2 3 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 3 môn học và các điều kiện: môn 1 trước môn 2 ($1 \times o2$), môn 2 trước môn 3 ($2 \times o3$): Lộ trình học tập bắt buộc duy nhất là: ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 1 4 2 3

Giải thích chi tiết: Với 3 môn học và các điều kiện: môn 1 trước môn 2 ($1 \times o2$), môn 2 trước môn 3 ($2 \times o3$): Lộ trình học tập bắt buộc duy nhất là: 1 2 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    vector<int> in_degree(n + 1, 0);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        in_degree[v]++;
    }

    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        if (in_degree[i] == 0) q.push(i);
    }

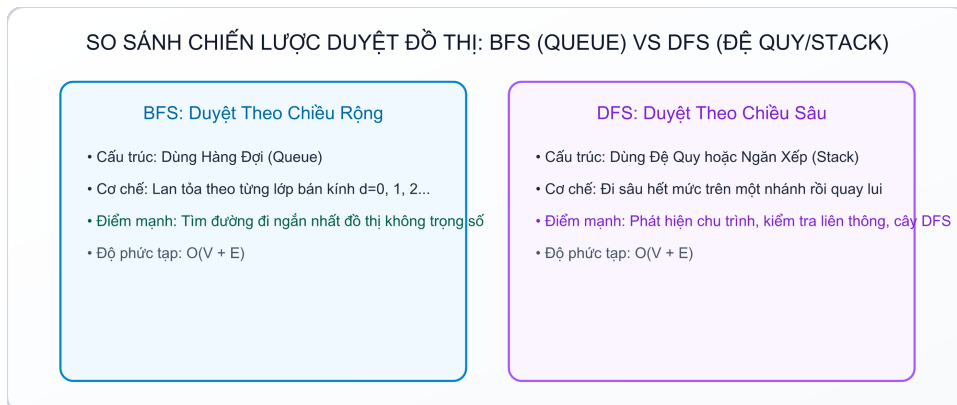
    vector<int> topo;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topo.push_back(u);

        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) q.push(v);
        }
    }

    if ((int)topo.size() < n) {
        cout << -1 << "\n";
    } else {
        for (int i = 0; i < n; ++i) {
            cout << topo[i] << (i + 1 == n ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}
```

Bài 11 [cppb_gra_11_tim_chu_trinh_do_dai_nho_nhat]: Chu Trình Ngắn Nhất Trên Đồ Thị (Girth)

Bối cảnh: Trong nghiên cứu cấu trúc phân tử hóa học của các hợp chất vòng hữu cơ, các nhà hóa học cần xác định chu trình vòng có kích thước nhỏ nhất (chu vi nhỏ nhất - Girth) xuất hiện trong đồ thị liên kết nguyên tử.



Hình minh họa: So sánh chiến lược duyệt đồ thị: Duyệt theo chiều sâu (DFS) và Duyệt theo chiều rộng (BFS)

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh không có khuyên và không có cạnh bội. Hãy lập trình tìm độ dài của chu trình có số cạnh nhỏ nhất trong đồ thị. Nếu đồ thị không có chu trình nào, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 2000, 0 \leq M \leq 2000$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra độ dài chu trình ngắn nhất, hoặc -1 nếu đồ thị không có chu trình.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 6 1 2 2 3 3 1 3 4 4 5 5 3	3

Giải thích: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4), (4, 1) và đường chéo (1, 3):

- Chu trình tạo bởi 1-2-3-1 có độ dài 3.
- Chu trình tạo bởi 1-3-4-1 có độ dài 3.
- Chu trình ngoài 1-2-3-4-1 có độ dài 4. Chu trình có độ dài nhỏ nhất là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh không có khuyên và không có cạnh bội. Hãy lập trình tìm độ dài của chu trình có số cạnh nhỏ nhất trong đồ thị. Nếu đồ thị không có chu trình nào, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $O(N + M)$ và duyệt cạnh nhanh chóng.

- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng visited[] để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $O(N + M)$.

2. Bảng chạy tay trên số liệu mẫu*Mẫu thử (Sample 1): Đầu vào: 5 6 1 2 2 3 3 1 3 4 4 5 5 3 $\Rightarrow$ Đầu ra kỳ vọng: 3.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 6 1 2 2 3 3 1 3 4 4 5 5 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4), (4, 1) và đường chéo (1, 3): - Chu trình tạo bởi 1-2-3-1 có độ dài 3. - Chu t...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4), (4, 1) và đường chéo (1, 3):

- Chu trình tạo bởi 1-2-3-1 có độ dài 3.
- Chu trình tạo bởi 1-3-4-1 có độ dài 3.
- Chu trình ngoài 1-2-3-4-1 có độ dài 4. Chu trình có độ dài nhỏ nhất là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: adj[u].push_back(v); adj[v].push_back(u);.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp for (int i = 1; i <= N; ++i) và gọi BFS/DFS khi !visited[i] để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int min_cycle = INF;

    for (int s = 1; s <= n; ++s) {
        vector<int> dist(n + 1, INF);
        vector<int> parent(n + 1, -1);
        queue<int> q;

        dist[s] = 0;
        q.push(s);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

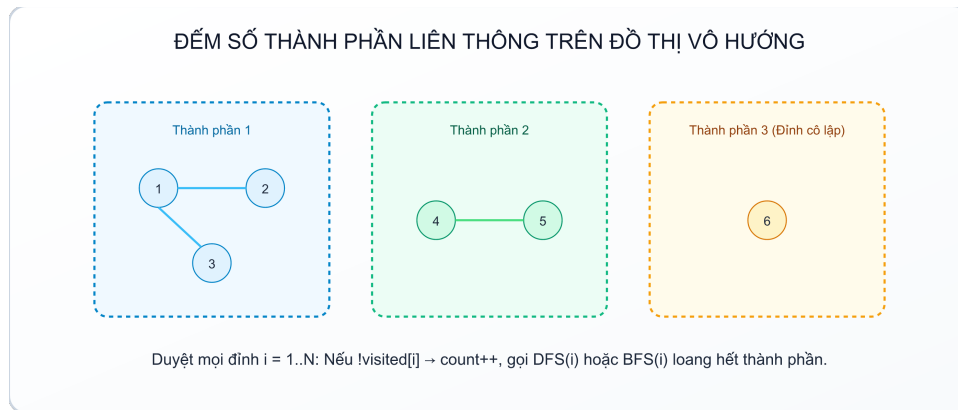
            for (int v : adj[u]) {
                if (dist[v] == INF) {
                    dist[v] = dist[u] + 1;
                    parent[v] = u;
                    q.push(v);
                } else if (parent[u] != v) {
                    min_cycle = min(min_cycle, dist[u] + dist[v] + 1);
                }
            }
        }
    }

    if (min_cycle == INF) cout << -1 << "\n";
    else cout << min_cycle << "\n";
    return 0;
}

```

Bài 12 [cppb_gra_12_dem_cap_dinh_khong_the_di_toi_nhau]: Đếm Cặp Đỉnh Không Thể Đi Tới Nhau

Bối cảnh: Một mạng viễn thông quốc gia gồm N trạm phát sóng và M đường truyền hai chiều. Do ảnh hưởng của bão lớn làm đứt một số đường truyền, mạng lưới bị chia cắt thành nhiều khu vực biệt lập. Hãy đếm xem có tất cả bao nhiêu cặp trạm (u, v) với $u < v$ mà hiện tại không thể gửi tín hiệu liên lạc được cho nhau.



Hình minh họa: Thuật toán loang tìm các thành phần liên thông trong đồ thị vô hướng

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng cặp đỉnh (u, v) ($1 \leq u < v \leq N$) không có đường đi giữa chúng.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng cặp đỉnh không liên thông.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 2 3 4	8

Giải thích: Với 4 đỉnh và chỉ có 1 cạnh nối (1, 2):

- Cụm 1 gồm $\{1, 2\}$.
- Đỉnh 3 cô lập, đỉnh 4 cô lập. Các cặp không đi tới nhau gồm: (1, 3), (1, 4), (2, 3), (2, 4), (3, 4). Có tổng cộng 5 cặp.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng cặp đỉnh (u, v) ($1 \leq u < v \leq N$) không có đường đi giữa chúng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 1 2 3 4 $\Rightarrow$ Đầu ra kỳ vọng: 8.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 1 2 3 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 đỉnh và chỉ có 1 cạnh nối (1, 2): - Cạnh 1 gồm {1, 2}. - Đỉnh 3 cô lập, đỉnh 4 cô lập. Các cặp không đi tới nhau gồm: (1, 3), (1, ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 8

Giải thích chi tiết: Với 4 đỉnh và chỉ có 1 cạnh nối (1, 2):

- Cạnh 1 gồm {1, 2}.
- Đỉnh 3 cô lập, đỉnh 4 cô lập. Các cặp không đi tới nhau gồm: (1, 3), (1, 4), (2, 3), (2, 4), (3, 4). Có tổng cộng 5 cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;
vector<bool> visited;

int dfs(int u) {
    visited[u] = true;
    int sz = 1;
    for (int v : adj[u]) {
        if (!visited[v]) sz += dfs(v);
    }
    return sz;
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<long long> comp_sizes;
    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            comp_sizes.push_back(dfs(i));
        }
    }

    long long total_pairs = 1LL * n * (n - 1) / 2;
    long long connected_pairs = 0;

    for (long long sz : comp_sizes) {
        connected_pairs += sz * (sz - 1) / 2;
    }

    cout << total_pairs - connected_pairs << "\n";
    return 0;
}
```

Bài 13 [cppb_gra_13_multi_source_bfs_lan_toa_virus]: Lan Tỏa Virus Trong Mạng Lưới (Multi-Source BFS)

Bối cảnh: Trong một cuộc tấn công không gian mạng giả định, có N máy tính được kết nối bởi M kênh mạng hai chiều. Ban đầu tại thời điểm $t = 0$, có K máy tính bị nhiễm mã độc virus. Cứ sau mỗi giây, các máy tính đã bị nhiễm sẽ lây truyền virus sang tất cả các máy tính kề sát với nó trong mạng lưới. Hãy tính thời gian (tính bằng giây) để toàn bộ các máy tính trong mạng đều bị lây nhiễm.

BIỂU DIỄN ĐỒ THỊ: MA TRẬN KỀ VS DANH SÁCH KỀ (ADJACENCY LIST)

Ma Trận Kề: $\text{adj}[N][N]$

`int adj[1005][1005];`

- Kiểm tra cạnh (u, v) : $O(1)$
- Bộ nhớ: $O(N^2) \rightarrow$ Sập bộ nhớ khi $N=10^5$
- Duyệt đỉnh kề: $O(N)$

$\rightarrow$ Chỉ dùng khi $N \leq 1000$ hoặc đồ thị dày

Danh Sách Kề: `vector<int> adj[N]`

`vector<int> adj[100005];`

- Bộ nhớ: $O(N + M)$ tối ưu tuyệt đối
- Duyệt đỉnh kề: $O(\deg(u))$ cực nhanh
- Thêm cạnh: `adj[u].push_back(v)` trong $O(1)$

$\rightarrow$ Chuẩn mực thi đấu Olympic cho $N, M \leq 10^5$

Hình minh họa: Ba phương pháp biểu diễn đồ thị trong lập trình: Ma trận kề, Danh sách cạnh, Danh sách kề

Nhiệm vụ: Cho đồ thị mạng và danh sách K máy tính bị nhiễm ban đầu. Hãy lập trình tính thời gian để toàn bộ máy tính trong thành phần liên thông bị nhiễm.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, K ($1 \leq K \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- Dòng 2: Chứa K số nguyên là chỉ số các máy tính nhiễm bệnh ban đầu.
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u, v .

Đầu ra (Output):

- In ra thời gian tối đa để lây lan hết mạng lưới.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 1 1 2 2 3 3 4	3

Giải thích: Với 5 máy tính nối thành đường thẳng 1 - 2 - 3 - 4 - 5 và máy 3 bị nhiễm ban đầu:

- Giờ 1: máy 3 lây sang máy 2 và 4.
- Giờ 2: máy 2 lây sang 1, máy 4 lây sang 5. Sau đúng 2 giờ toàn bộ mạng lưới đều bị lây nhiễm, kết quả là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị mạng và danh sách K máy tính bị nhiễm ban đầu. Hãy lập trình tính thời gian để toàn bộ máy tính trong thành phần liên thông bị nhiễm.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 3 1 1 1 2 2 3 3 4 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 3 1 1 1 2 2 3 3 4	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với 5 máy tính nối thành đường thẳng 1 - 2 - 3 - 4 - 5 và máy 3 bị nhiễm ban đầu: - Giây 1: máy 3 lây sang máy 2 và 4. - Giây 2: máy 2 ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với 5 máy tính nối thành đường thẳng 1 - 2 - 3 - 4 - 5 và máy 3 bị nhiễm ban đầu:

- Giây 1: máy 3 lây sang máy 2 và 4.
- Giây 2: máy 2 lây sang 1, máy 4 lây sang 5. Sau đúng 2 giây toàn bộ mạng lưới đều bị lây nhiễm, kết quả là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k;
    if (!(cin >> n >> m >> k)) return 0;
    if (n <= 0) return 0;

    vector<int> dist(n + 1, -1);
    queue<int> q;

    for (int i = 0; i < k; ++i) {
        int src;
        cin >> src;
        dist[src] = 0;
        q.push(src);
    }

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
}
```

```

int max_time = 0;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    max_time = max(max_time, dist[u]);

    for (int v : adj[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}

for (int i = 1; i <= n; ++i) {
    if (dist[i] == -1) {
        cout << -1 << "\n";
        return 0;
    }
}

cout << max_time << "\n";
return 0;
}

```

Bài 14 [cppb_gra_14_tim_canh_cau_tren_do_thi]: Đếm Số Cạnh Cầu Trên Đồ Thị (Bridges)

Bối cảnh: Một hệ thống giao thông đường thủy gồm N cảng biển và M tuyến hải trình. Một tuyến hải trình được gọi là một tuyến cầu hiểm yếu nếu khi tuyến này bị gián đoạn (phong tỏa), số lượng cụm cảng bị chia cắt biệt lập sẽ tăng lên. Kỹ sư an ninh hàng hải cần thống kê số lượng các tuyến hiểm yếu này để ưu tiên bố trí lực lượng tuần tra bảo vệ.

SO SÁNH CHIẾN LƯỢC DUYỆT ĐỒ THỊ: BFS (QUEUE) VS DFS (ĐỆ QUY/STACK)

BFS: Duyệt Theo Chiều Rộng

- Cấu trúc: Dùng Hàng Đợi (Queue)
- Cơ chế: Lan tỏa theo từng lớp bán kính $d=0, 1, 2, \dots$
- Điểm mạnh: Tìm đường đi ngắn nhất đồ thị không trọng số
- Độ phức tạp: $O(V + E)$

DFS: Duyệt Theo Chiều Sâu

- Cấu trúc: Dùng Đệ Quy hoặc Ngăn Xếp (Stack)
- Cơ chế: Đi sâu hết mức trên một nhánh rồi quay lại
- Điểm mạnh: Phát hiện chu trình, kiểm tra liên thông, cây DFS
- Độ phức tạp: $O(V + E)$

Hình minh họa: So sánh chiến lược duyệt đồ thị: Duyệt theo chiều sâu (DFS) và Duyệt theo chiều rộng (BFS)

Nhiệm vụ: Cho đồ thị vô hướng liên thông N đỉnh M cạnh. Hãy lập trình đếm số lượng cạnh cầu trong đồ thị.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng cạnh cầu tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5 1 2 2 3 3 1 3 4 4 5	2

Giải thích: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4): Nếu bỏ bất kỳ cạnh nào trong số 3 cạnh trên, đồ thị đều bị chia đôi thành 2 phần rời nhau. Do đó cả 3 cạnh đều là cạnh cầu, kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng liên thông N đỉnh M cạnh. Hãy lập trình đếm số lượng cạnh cầu trong đồ thị.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 5 1 2 2 3 3 1 3 4 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 5 1 2 2 3 3 1 3 4 4 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4): Nếu bỏ bất kỳ cạnh nào trong số 3 cạnh trên, đồ thị đều bị chia đôi thành 2 p...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4): Nếu bỏ bất kỳ cạnh nào trong số 3 cạnh trên, đồ thị đều bị chia đôi thành 2 phần rời nhau. Do đó cả 3 cạnh đều là cạnh cầu, kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.
- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m, timer = 0;
vector<vector<int>>> adj;
vector<int> tin, low;
vector<bool> visited;
int bridge_count = 0;

void dfs(int u, int p = -1) {
    visited[u] = true;
    tin[u] = low[u] = ++timer;

    for (int v : adj[u]) {
        if (v == p) continue;
        if (visited[v]) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] > tin[u]) {
                bridge_count++;
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    tin.assign(n + 1, -1);
    low.assign(n + 1, -1);
    visited.assign(n + 1, false);

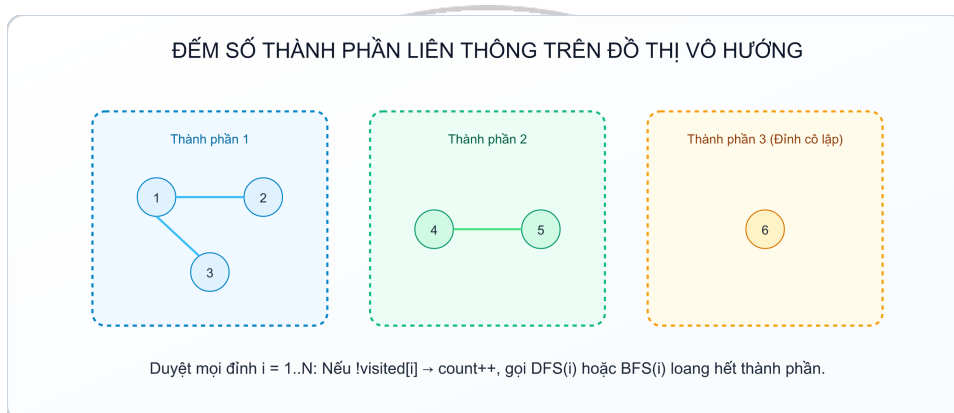
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
}
```

```
for (int i = 1; i <= n; ++i) {
    if (!visited[i]) dfs(i);
}

cout << bridge_count << "\n";
return 0;
}
```

Bài 15 [cppb_gra_15_mang_luoi_giao_thong_toi_uu]: Xây Dựng Thêm Đường Nối Toàn Mạng (Building Roads)

Bối cảnh: Một vương quốc gồm N thành phố nhưng hiện tại hệ thống đường sá chỉ có M con đường, khiến nhiều thành phố bị cô lập không thể đi tới nhau. Nhà vua muốn xây dựng thêm một số lượng con đường mới ít nhất nối giữa các thành phố sao cho sau khi hoàn thành, cư dân từ bất kỳ thành phố nào cũng có thể di chuyển tới mọi thành phố khác trong vương quốc.



Hình minh họa: Thuật toán loang tìm các thành phần liên thông trong đồ thị vô hướng

Nhiệm vụ: Cho bản đồ vương quốc hiện tại. Hãy lập trình tìm số lượng đường mới ít nhất cần xây dựng và chỉ rõ danh sách các con đường cần làm thêm.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \times 10^5$).
- M dòng tiếp theo, mỗi dòng chứa hai đỉnh biểu diễn một con đường hiện có.

Đầu ra (Output):

- Dòng 1: In ra số nguyên K là số lượng con đường ít nhất cần xây thêm.
- K dòng tiếp theo, mỗi dòng in ra hai thành phố cần nối đường mới.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 1 2 3 4	1 1 3

Giải thích: Với 4 thành phố và chỉ có 1 đường nối giữa (1, 2) và 1 đường nối giữa (3, 4): Vương quốc đang bị chia thành 2 cụm độc lập $\{1, 2\}$ và $\{3, 4\}$. Ta chỉ cần xây thêm đúng 1 con đường nối giữa thành phố 2 và thành phố 3 là toàn bộ 4 thành phố sẽ liên thông hoàn chỉnh.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ vương quốc hiện tại. Hãy lập trình tìm số lượng đường mới ít nhất cần xây dựng và chỉ rõ danh sách các con đường cần làm thêm.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Biểu diễn đồ thị:** Sử dụng danh sách kề `vector<vector<int>> adj(N + 1)` để tối ưu bộ nhớ $\mathcal{O}(N + M)$ và duyệt cạnh nhanh chóng.
- **Thuật toán duyệt đồ thị:**
- *Tìm kiếm theo chiều rộng (BFS):* Sử dụng hàng đợi queue, đảm bảo tìm đường đi ngắn nhất trên đồ thị không trọng số.
- *Tìm kiếm theo chiều sâu (DFS):* Duyệt nhánh sâu nhất bằng đệ quy hoặc stack, thích hợp tìm thành phần liên thông, chu trình và sắp xếp tô-pô.
- **Mảng đánh dấu:** Sử dụng mảng `visited[]` để đảm bảo mỗi đỉnh và cạnh chỉ được xét một số lần hằng số, độ phức tạp đạt $\mathcal{O}(N + M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 2 1 2 3 4 $\Rightarrow$ Đầu ra kỳ vọng: 1 1 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 2 1 2 3 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 thành phố và chỉ có 1 đường nối giữa (1, 2) và 1 đường nối giữa (3, 4): Vương quốc đang bị chia thành 2 cụm độc lập $\{1, 2\}$ và $\{3, \dots\}$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 1 1 3

Giải thích chi tiết: Với 4 thành phố và chỉ có 1 đường nối giữa (1, 2) và 1 đường nối giữa (3, 4): Vương quốc đang bị chia thành 2 cụm độc lập $\{1, 2\}$ và $\{3, 4\}$. Ta chỉ cần xây thêm đúng 1 con đường nối giữa thành phố 2 và thành phố 3 là toàn bộ 4 thành phố sẽ liên thông hoàn chỉnh.

3. Lưu ý & Bẫy lỗi thường gặp

- Với đồ thị vô hướng, quên thêm cả hai chiều cạnh: `adj[u].push_back(v); adj[v].push_back(u);`.
- Đồ thị gồm nhiều thành phần liên thông rời rạc: Bắt buộc duyệt vòng lặp `for (int i = 1; i <= N; ++i)` và gọi BFS/DFS khi `!visited[i]` để không bỏ sót các đỉnh độc lập.

- Tràn bộ nhớ Call Stack khi gọi DFS đệ quy quá sâu trên đồ thị có dạng đường thẳng ($N = 10^5$), cần tăng kích thước stack hoặc chuyển sang BFS/DFS lặp.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;
vector<bool> visited;

void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

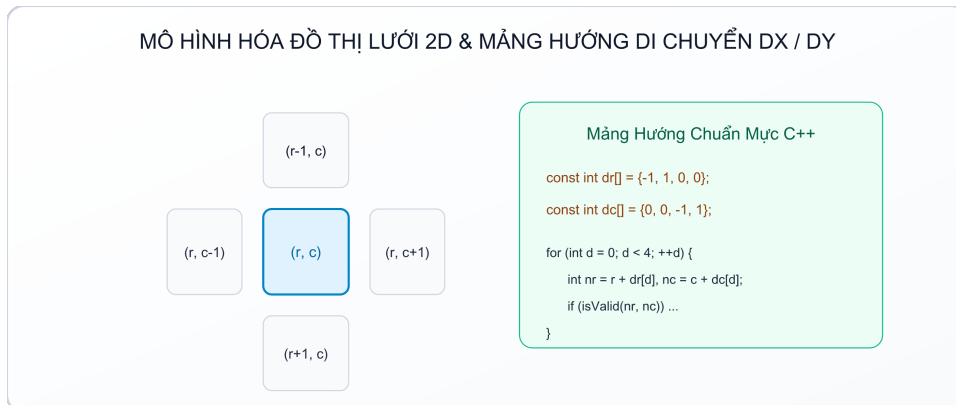
    vector<int> leaders;
    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            leaders.push_back(i);
            dfs(i);
        }
    }

    int needed = leaders.size() - 1;
    cout << needed << "\n";
    for (int i = 0; i < needed; ++i) {
        cout << leaders[i] << " " << leaders[i + 1] << "\n";
    }
    return 0;
}
```

Bài 23: Đồ thị lưới 2D, kỹ thuật Flood Fill & tính chất cây

Bài 01 [cppb_grd_01_duyet_4_huong_tren_ma_tran_co_ban]: Đếm Số Ô Kề Cận Hợp Lệ (4 Hướng)

Bối cảnh: Trong một hệ thống điều khiển lưới cảm biến kích thước $N \times M$ ô vuông (chỉ số các ô từ $(0,0)$ đến $(N-1, M-1)$), một thiết bị di động đang đứng tại ô tọa độ (r, c) . Thiết bị chỉ có thể gửi tín hiệu không dây tới các ô kề sát cạnh theo 4 hướng chính: lên trên $(r-1, c)$, xuống dưới $(r+1, c)$, sang trái $(r, c-1)$, và sang phải $(r, c+1)$. Kỹ sư phần mềm cần kiểm tra xem có bao nhiêu hướng di chuyển hợp lệ vẫn nằm trọn vẹn bên trong phạm vi ma trận.



Hình minh họa: Mô hình hoá ma trận lưới 2 chiều thành đồ thị với các hướng di chuyển liền kề

Nhiệm vụ: Cho kích thước ma trận N, M và tọa độ (r, c) . Hãy lập trình đếm số ô kề cận hợp lệ nằm trong ma trận.

Đầu vào (Input):

- Một dòng duy nhất chứa 4 số nguyên N, M, r, c ($1 \leq N, M \leq 1000, 0 \leq r < N, 0 \leq c < M$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số ô kề cận hợp lệ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 0 0	2

Giải thích: Với lưới kích thước 3×3 và vị trí ô góc $(0,0)$:

- Hướng lên trên và sang trái đều vượt ra ngoài biên của lưới.
- Chỉ có 2 hướng hợp lệ là xuống dưới $(1,0)$ và sang phải $(0,1)$. Số ô kề cận hợp lệ là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho kích thước ma trận N, M và tọa độ (r, c) . Hãy lập trình đếm số ô kề cận hợp lệ nằm trong ma trận.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cận (hoặc 8 ô kề góc).

- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $\text{int dx[]} = \{-1, 1, 0, 0\}$ và $\text{int dy[]} = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 0 0 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 0 0	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với lưới kích thước 3×3 và vị trí ô góc (0,0): - Hướng lên trên và sang trái đều vượt ra ngoài biên của lưới. - Chỉ có 2 hướng ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Với lưới kích thước 3×3 và vị trí ô góc (0,0):

- Hướng lên trên và sang trái đều vượt ra ngoài biên của lưới.
- Chỉ có 2 hướng hợp lệ là xuống dưới (1,0) và sang phải (0,1). Số ô kề cạnh hợp lệ là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô $\text{grid}[r][c]$, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu $\text{visited} = \text{true}$ khi lấy phần tử ra khỏi queue ($\text{pop}()$) thay vì khi đẩy vào ($\text{push}()$): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự $s[c]$.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
```

```
ios::sync_with_stdio(false);
cin.tie(nullptr);

int n, m, r, c;
if (!(cin >> n >> m >> r >> c)) return 0;

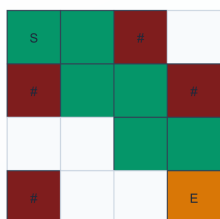
int valid_neighbors = 0;
for (int d = 0; d < 4; ++d) {
    int nr = r + dr[d];
    int nc = c + dc[d];
    if (nr >= 0 && nr < n && nc >= 0 && nc < m) {
        valid_neighbors++;
    }
}

cout << valid_neighbors << "\n";
return 0;
}
```

Bài 02 [cppb_grd_02_dem_so_luong_hon_dao]: Đếm Số Lượng Hòn Đảo (Count Islands)

Bối cảnh: Một bức ảnh viễn thám chụp một vùng biển được số hóa thành ma trận nhị phân kích thước $N \times M$. Ký tự '0' đại diện cho mặt nước biển, còn ký tự '1' đại diện cho đất liền. Một hòn đảo được định nghĩa là một tập hợp các ô đất liền '1' kết nối với nhau theo 4 hướng (lên, xuống, trái, phải) và được bao quanh hoàn toàn bởi nước biển.

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MỀ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm `!visited[nr][nc]`
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Hình minh họa: Kỹ thuật loang sơn (Flood Fill) giải bài toán tìm đường và đếm miền trong mê cung lưới

Nhiệm vụ: Cho bản đồ ma trận $N \times M$. Hãy lập trình đếm số lượng hòn đảo xuất hiện trên bản đồ.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi gồm M ký tự '0' hoặc '1' biểu diễn bản đồ.

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng hòn đảo đếm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 11000 11000 00100 00011	3

Giải thích: Với bản đồ kích thước 3×3 chứa các cụm đất liền biệt lập không có ô kề cạnh chung: Các ô đất liền kết nối thành đúng 3 cụm độc lập, kết quả đếm được là 3 hòn đảo.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ ma trận $N \times M$. Hãy lập trình đếm số lượng hòn đảo xuất hiện trên bản đồ.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 5 11000 11000 00100 00011
⇒ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 11000 11000 00100 00011	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với bản đồ kích thước 3×3 chứa các cụm đất liền biệt lập không có ô kề cạnh chung: Các ô đất liền kết nối thành đúng 3 cụm độc lập...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với bản đồ kích thước 3×3 chứa các cụm đất liền biệt lập không có ô kề cạnh chung: Các ô đất liền kết nối thành đúng 3 cụm độc lập, kết quả đếm được là 3 hòn đảo.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô $grid[r][c]$, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).

- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự s[c].

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    vector<vector<bool>> visited(n, vector<bool>(m, false));
    int islands = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                islands++;
                visited[r][c] = true;
                queue<pair<int, int>> q;
                q.push({r, c});

                while (!q.empty()) {
                    auto [cr, cc] = q.front();
                    q.pop();

                    for (int d = 0; d < 4; ++d) {
                        int nr = cr + dr[d];
                        int nc = cc + dc[d];
                        if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1' && !visited[nr][nc]) {
                            visited[nr][nc] = true;
                            q.push({nr, nc});
                        }
                    }
                }
            }
        }
    }

    cout << islands << "\n";
    return 0;
}
```

Bài 03 [cppb_grd_03_dien_tich_hon_dao_lon_nhat]: Diện Tích Hòn Đảo Lớn Nhất (Max Area of Island)

Bối cảnh: Vẫn trên bản đồ hải đồ nhị phân $N \times M$ gồm các ô đất liền '1' và nước biển '0', ban quản lý khu bảo tồn thiên nhiên muốn chọn ra hòn đảo có diện tích lớn nhất (chứa số lượng ô đất liền liên thông nhiều nhất) để quy hoạch xây dựng trung tâm cứu hộ động vật hoang dã.



Hình minh họa: Các tính chất nền tảng của đồ thị Cây: N đỉnh $N-1$ cạnh liên thông không chu trình

Nhiệm vụ: Cho bản đồ ma trận $N \times M$. Hãy lập trình tìm diện tích (số lượng ô đất) của hòn đảo lớn nhất. Nếu bản đồ không có đảo nào, in ra 0.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi gồm M ký tự '0' hoặc '1'.

Đầu ra (Output):

- In ra trên một dòng duy nhất diện tích của hòn đảo lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 11000 11000 00100 00011	4

Giải thích: Với bản đồ có cụm đảo lớn nhất gồm 5 ô đất liền kề cạnh kết nối liên tục với nhau, diện tích lớn nhất đo được là 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ ma trận $N \times M$. Hãy lập trình tìm diện tích (số lượng ô đất) của hòn đảo lớn nhất. Nếu bản đồ không có đảo nào, in ra 0.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.

- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 5 11000 11000 00100 00011
 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 5 11000 11000 00100 00011	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với bản đồ có cụm đảo lớn nhất gồm 5 ô đất liền kề cạnh kết nối liên tục với nhau, diện tích lớn nhất đo được là 5.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với bản đồ có cụm đảo lớn nhất gồm 5 ô đất liền kề cạnh kết nối liên tục với nhau, diện tích lớn nhất đo được là 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];
```

```
vector<vector<bool>> visited(n, vector<bool>(m, false));
int max_area = 0;

for (int r = 0; r < n; ++r) {
    for (int c = 0; c < m; ++c) {
        if (grid[r][c] == '1' && !visited[r][c]) {
            int area = 0;
            visited[r][c] = true;
            queue<pair<int, int>> q;
            q.push({r, c});

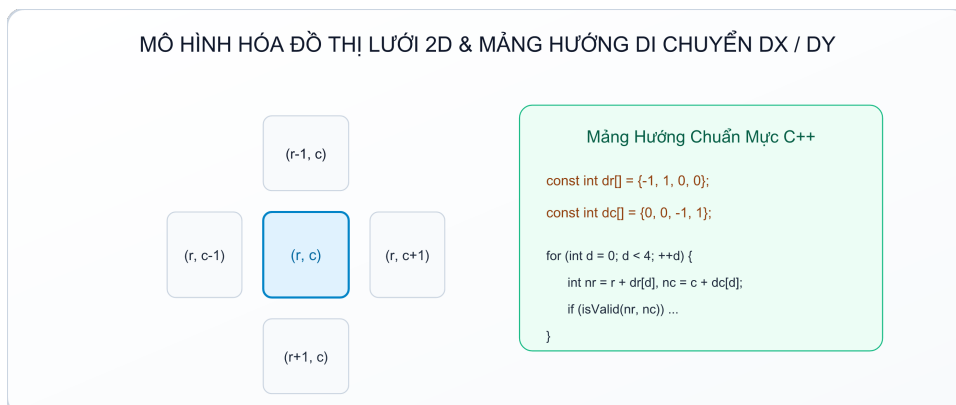
            while (!q.empty()) {
                auto [cr, cc] = q.front();
                q.pop();
                area++;

                for (int d = 0; d < 4; ++d) {
                    int nr = cr + dr[d];
                    int nc = cc + dc[d];
                    if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1' && !visited[nr][nc]) {
                        visited[nr][nc] = true;
                        q.push({nr, nc});
                    }
                }
            }
            max_area = max(max_area, area);
        }
    }
}

cout << max_area << "\n";
return 0;
}
```

Bài 04 [cppb_grd_04_tim_duong_thoat_khoi_me_cung_bfs]: Tìm Đường Thoát Khỏi Mê Cung BFS

Bối cảnh: Một robot tìm kiếm cứu nạn được thả vào một mê cung hình chữ nhật kích thước $N \times M$. Bản đồ mê cung gồm các ô đường đi trống . và các ô tường đá không thể vượt qua #. Robot xuất phát từ ô ký hiệu S và cần di chuyển đến ô cửa thoát hiểm ký hiệu E. Mỗi bước di chuyển sang một ô kề cạnh mất đúng 1 giây.



Hình minh họa: Mô hình hoá ma trận lưới 2 chiều thành đồ thị với các hướng di chuyển liền kề

Nhiệm vụ: Cho bản đồ mê cung. Hãy lập trình tìm thời gian ngắn nhất (số bước ít nhất) để robot đến được cửa thoát hiểm E. Nếu không có đường thoát, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi gồm M ký tự biểu diễn mê cung.

Đầu ra (Output):

- In ra số bước ít nhất để đến đích, hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 ##### #.A#...# #.##.#B# #.....# #####	9

Giải thích: Với mê cung từ 'S' tại góc (0,0) đến 'E' tại góc (2,2) qua các ô đường trống: Lộ trình ngắn nhất gồm 4 bước di chuyển: $(0,0) \times o(0,1) \times o(1,1) \times o(1,2) \times o(2,2)$. Số bước ít nhất là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ mê cung. Hãy lập trình tìm thời gian ngắn nhất (số bước ít nhất) để robot đến được cửa thoát hiểm E. Nếu không có đường thoát, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

**2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 8 ##### #.A#...#
#.##.#B# #.....# ##### $\Rightarrow$ Đầu ra kỳ vọng: 9.**

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 8 ##### #.A#...# #.##.#B# #.....# #####	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với mê cung từ 'S' tại góc (0,0) đến 'E' tại góc (2,2) qua các ô đường trống: Lộ trình ngắn nhất gồm 4 bước di chuyển: $(0,0) \times \dots$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 9

Giải thích chi tiết: Với mê cung từ 'S' tại góc (0,0) đến 'E' tại góc (2,2) qua các ô đường trống: Lộ trình ngắn nhất gồm 4 bước di chuyển: $(0,0) \times o(0,1) \times o(1,1) \times o(1,2) \times o(2,2)$. Số bước ít nhất là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<string> grid(n);
    int sr = -1, sc = -1, er = -1, ec = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'A' || grid[r][c] == 'S') { sr = r; sc = c; }
            if (grid[r][c] == 'B' || grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

    vector<vector<int>> dist(n, vector<int>(m, -1));
    queue<pair<int, int>> q;
```

```

dist[sr][sc] = 0;
q.push({sr, sc});

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    if (r == er && c == ec) {
        cout << dist[r][c] << "\n";
        return 0;
    }

    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && dist[nr][nc] =
= -1) {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}

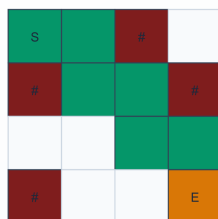
cout << -1 << "\n";
return 0;
}

```

Bài 05 [cppb_grd_05_truy_vet_duong_di_me_cung]: Truy Vết Đường Đi Mê Cung (L, R, U, D)

Bối cảnh: Sau khi xác định được thời gian thoát hiểm ngắn nhất trong mê cung, bộ điều khiển cần xuất ra chuỗi lệnh điều hướng chi tiết bằng các ký tự viết tắt phương hướng: 'U' (lên trên), 'D' (xuống dưới), 'L' (sang trái), 'R' (sang phải) để nạp trực tiếp vào bộ nhớ vi điều khiển của robot.

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MÊ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm !visited[nr][nc]
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Hình minh họa: Kỹ thuật loang sơn (Flood Fill) giải bài toán tìm đường và đếm miền trong mê cung lưới

Nhiệm vụ: Cho bản đồ mê cung $N \times M$ với điểm xuất phát S và đích E. Hãy lập trình tìm đường đi ngắn nhất và in ra chuỗi các bước di chuyển tương ứng.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi M ký tự.

Đầu ra (Output):

- Dòng 1: In ra số bước đi ngắn nhất K (hoặc in NO nếu không có đường đi).

- Dòng 2: In ra chuỗi K ký tự gồm U, D, L, R mô tả lộ trình di chuyển.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 ##### #.A#...# #.#.#B# #.....# #####	YES 9 LDDRRRRRU

Giải thích: Với lộ trình đi từ (0,0) sang phải rồi xuống dưới: Chuỗi lệnh điều hướng tương ứng là: “RRD” hoặc “DRR” có độ dài 3 bước.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ mê cung $N \times M$ với điểm xuất phát S và đích E. Hãy lập trình tìm đường đi ngắn nhất và in ra chuỗi các bước di chuyển tương ứng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 8 ##### #.A#...# #.#.#B# #.....# ##### $\Rightarrow$ Đầu ra kỳ vọng: YES 9 LDDRRRRRU.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 8 ##### #.A#...# #.#.#B# #.....# #####	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với lộ trình đi từ (0,0) sang phải rồi xuống dưới: Chuỗi lệnh điều hướng tương ứng là: “RRD” hoặc “DRR” có độ dài 3 bước.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES 9 LDDRRRRRU

Giải thích chi tiết: Với lộ trình đi từ (0,0) sang phải rồi xuống dưới: Chuỗi lệnh điều hướng tương ứng là: “RRD” hoặc “DRR” có độ dài 3 bước.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};
const char dir_char[] = {'U', 'D', 'L', 'R'};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<string> grid(n);
    int sr = -1, sc = -1, er = -1, ec = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'A' || grid[r][c] == 'S') { sr = r; sc = c; }
            if (grid[r][c] == 'B' || grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

    vector<vector<int>> dist(n, vector<int>(m, -1));
    vector<vector<int>> prev_dir(n, vector<int>(m, -1));
    queue<pair<int, int>> q;

    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        if (r == er && c == ec) break;

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && dist[nr][nc] =
= -1) {
                dist[nr][nc] = dist[r][c] + 1;
                prev_dir[nr][nc] = d;
                q.push({nr, nc});
            }
        }
    }
}
```

```

    }

    if (dist[er][ec] == -1) {
        cout << "NO\n";
        return 0;
    }

    cout << "YES\n" << dist[er][ec] << "\n";
    string path = "";
    int curr_r = er, curr_c = ec;

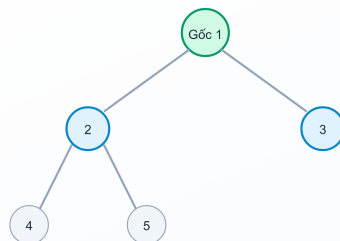
    while (curr_r != sr || curr_c != sc) {
        int d = prev_dir[curr_r][curr_c];
        path += dir_char[d];
        curr_r -= dr[d];
        curr_c -= dc[d];
    }
    reverse(path.begin(), path.end());
    cout << path << "\n";
    return 0;
}

```

Bài 06 [cppb_grd_06_dem_vung_kin_khong_thong_ra_bien]: Đếm Số Ô Vùng Kín Không Thông Ra Biên

Bối cảnh: Một vùng trũng ngập nước được số hóa thành lưới $N \times M$. Các ô đất liền có giá trị 1, ô ngập nước có giá trị 0. Một vùng đất được gọi là “vùng kín” nếu nó là một cụm các ô 1 liên thông kề cạnh mà hoàn toàn không có bất kỳ ô nào chạm vào 4 mép biên ngoài của bản đồ (nghĩa là vùng đất bị bao bọc hoàn toàn bởi các ô nước).

ĐẶC TÍNH BẤT BIẾN CỦA CÂY (TREE INVARIANTS): N ĐỈNH & $N-1$ CẠNH



3 Điều Kiện Tương Đương

Đồ thị $G = (V, E)$ là một Cây khi thỏa:

1. G liên thông và có đúng $N-1$ cạnh
2. G không có chu trình và có $N-1$ cạnh
3. Giữa 2 đỉnh bất kỳ có duy nhất 1 đường đi

Áp dụng: Duyệt Cây bằng DFS từ gốc không cần mảng visited, chỉ

Hình minh họa: Các tính chất nền tảng của đồ thị Cây: N đỉnh $N-1$ cạnh liên thông không chu trình

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình đếm tổng số lượng ô đất liền 1 thuộc về các vùng đất kín không thông ra biên.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên 0 hoặc 1 cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng ô đất liền thuộc các vùng kín.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1111 1001 1101 1111	3

Giải thích: Với ma trận có một cụm gồm 3 ô đất nằm lọt thỏm ở trung tâm và toàn bộ viền xung quanh đều là ô số 0: Cụm này hoàn toàn không chạm biên, số ô đất kín đếm được là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận nhị phân $N \times M$. Hãy lập trình đếm tổng số lượng ô đất liền 1 thuộc về các vùng đất kín không thông ra biên.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 1111 1001 1101 1111 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 1111 1001 1101 1111	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với ma trận có một cụm gồm 3 ô đất nằm lọt thỏm ở trung tâm và toàn bộ viền xung quanh đều là ô số 0: Cụm này hoàn toàn không chạm biên...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với ma trận có một cụm gồm 3 ô đất nằm lọt thỏm ở trung tâm và toàn bộ viền xung quanh đều là ô số 0: Cụm này hoàn toàn không chạm biên, số ô đất kín đếm được là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    queue<pair<int, int>> q;

    for (int r = 0; r < n; ++r) {
        if (grid[r][0] == '0') { grid[r][0] = '1'; q.push({r, 0}); }
        if (grid[r][m - 1] == '0') { grid[r][m - 1] = '1'; q.push({r, m - 1}); }
    }
    for (int c = 0; c < m; ++c) {
        if (grid[0][c] == '0') { grid[0][c] = '1'; q.push({0, c}); }
        if (grid[n - 1][c] == '0') { grid[n - 1][c] = '1'; q.push({n - 1, c}); }
    }

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '0') {
                grid[nr][nc] = '1';
                q.push({nr, nc});
            }
        }
    }

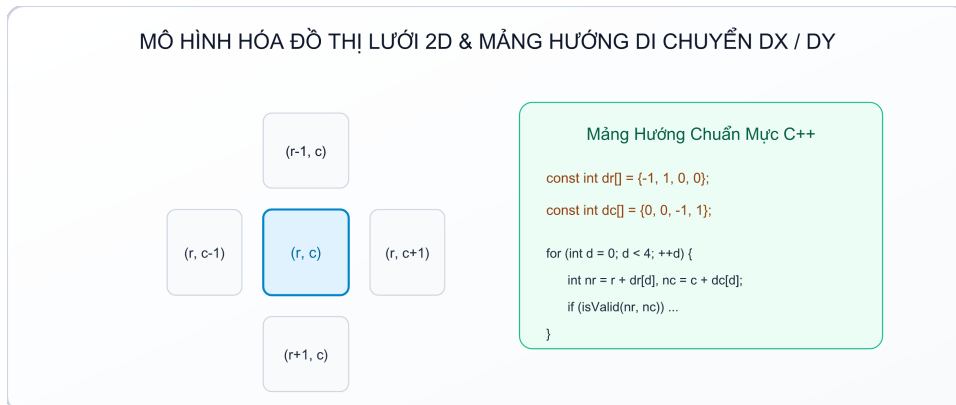
    int closed_zeros = 0;
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '0') closed_zeros++;
        }
    }

    cout << closed_zeros << "\n";
}
```

```
return 0;
}
```

Bài 07 [cppb_grd_07_chu_vi_hon_dao]: Chu Vi Hòn Đảo (Island Perimeter)

Bối cảnh: Trên bản đồ dạng lưới $N \times M$, có đúng một hòn đảo duy nhất được tạo thành bởi các ô đất liền '1' kết nối liên thông (các ô còn lại là nước biển '0'). Mỗi ô đất liền là một hình vuông có cạnh dài đúng 1 đơn vị. Chu vi của hòn đảo là tổng độ dài các cạnh của các ô đất liền tiếp xúc trực tiếp với nước biển hoặc tiếp xúc với mép ngoài của bản đồ.



Hình minh họa: Mô hình hoá ma trận lưới 2 chiều thành đồ thị với các hướng di chuyển liền kề

Nhiệm vụ: Cho bản đồ chứa đúng một hòn đảo. Hãy lập trình tính chu vi của hòn đảo đó.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số 0 hoặc 1 cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất chu vi của hòn đảo.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 0100 1110 0100 1100	16

Giải thích: Với một hòn đảo gồm 4 ô đất liền xếp thành hình chữ L: Tổng số cạnh tiếp xúc với nước biển xung quanh đo được là 16 đơn vị chiều dài. Kết quả in ra là 16.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho bản đồ chứa đúng một hòn đảo. Hãy lập trình tính chu vi của hòn đảo đó.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $\text{int dx[]} = \{-1, 1, 0, 0\}$ và $\text{int dy[]} = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.

- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 4 0100 1110 0100 1100 $\Rightarrow$ Đầu ra kỳ vọng: 16.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 4 0100 1110 0100 1100	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với một hòn đảo gồm 4 ô đất liền xếp thành hình chữ L: Tổng số cạnh tiếp xúc với nước biển xung quanh đo được là 16 đơn vị chiều dài. K...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 16

Giải thích chi tiết: Với một hòn đảo gồm 4 ô đất liền xếp thành hình chữ L: Tổng số cạnh tiếp xúc với nước biển xung quanh đo được là 16 đơn vị chiều dài. Kết quả in ra là 16.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
```

```
vector<string> grid(n);
for (int i = 0; i < n; ++i) cin >> grid[i];

int perimeter = 0;

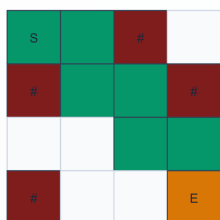
for (int r = 0; r < n; ++r) {
    for (int c = 0; c < m; ++c) {
        if (grid[r][c] == '1') {
            for (int d = 0; d < 4; ++d) {
                int nr = r + dr[d];
                int nc = c + dc[d];
                if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '0') {
                    perimeter++;
                }
            }
        }
    }
}

cout << perimeter << "\n";
return 0;
}
```

Bài 08 [cppb_grd_08_nuoc_tran_me_cung_multi_source_bfs]: Nước Trần Mê Cung (Multi-Source BFS)

Bối cảnh: Một hầm mộ dưới lòng đất kích thước $N \times M$ gồm các buồng trống . và các khối đá chắn #. Đột ngột có một sự cố vỡ đê ngầm khiến nước tràn vào từ K buồng mộ cùng lúc tại thời điểm $t = 0$. Cứ sau mỗi phút, nước từ các buồng đã ngập sẽ tràn sang tất cả các buồng trống kề sát nó theo 4 hướng. Hãy tính thời gian để toàn bộ các buồng trống trong hầm mộ đều bị ngập nước.

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MÊ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm `!visited[nr][nc]`
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Hình minh họa: Kỹ thuật loang sơn (Flood Fill) giải bài toán tìm đường và đếm miền trong mê cung lưới

Nhiệm vụ: Cho bản đồ hầm mộ và vị trí các nguồn nước ban đầu. Hãy lập trình tính thời gian (phút) để nước tràn kín toàn bộ các buồng trống liên thông.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa một chuỗi M ký tự (. là buồng trống, W là nguồn nước, # là tường đá).

Đầu ra (Output):

- In ra thời gian tối đa để nước tràn ngập khắp hầm mộ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 W.. .## ..E	4

Giải thích: Nước bắt đầu loang đồng thời từ các điểm ‘W’ theo chiều rộng: Sau đúng 4 phút, điểm buồng mả xa nhất đã bị nước tràn tới ngập hoàn toàn. Kết quả là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ hầm mả và vị trí các nguồn nước ban đầu. Hãy lập trình tính thời gian (phút) để nước tràn kín toàn bộ các buồng trống liên thông.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $dx[] = \{-1, 1, 0, 0\}$ và $dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 W.. .## ..E $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 W.. .## ..E	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Nước bắt đầu loang đồng thời từ các điểm ‘W’ theo chiều rộng: Sau đúng 4 phút, điểm buồng mả xa nhất đã bị nước tràn tới ngập hoàn toàn...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Nước bắt đầu loang đồng thời từ các điểm ‘W’ theo chiều rộng: Sau đúng 4 phút, điểm buồng mả xa nhất đã bị nước tràn tới ngập hoàn toàn. Kết quả là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<string> grid(n);
    int er = -1, ec = -1;
    vector<vector<int>> dist(n, vector<int>(m, -1));
    queue<pair<int, int>> q;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'W') {
                dist[r][c] = 0;
                q.push({r, c});
            }
            if (grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

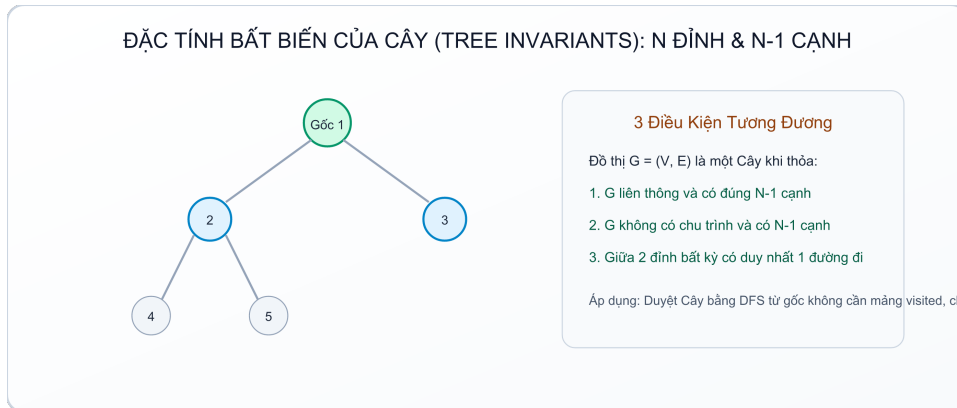
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && dist[nr][nc] =
= -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }

    cout << dist[er][ec] << "\n";
    return 0;
}
```

Bài 09 [cppb_grd_09_buoc_nhay_quan_ma_ngan_nhat]: Bước Nhảy Quân Mã Ngắn Nhất (Knight Moves)

Bối cảnh: Trên bàn cờ vua tiêu chuẩn kích thước $N \times N$, quân Mã di chuyển theo quy tắc hình chữ L đặc thù: tại mỗi bước, quân Mã di chuyển 2 ô theo một trục và 1 ô theo trục vuông góc (tối đa 8 hướng nhảy). Cho vị trí ban đầu của quân Mã tại ô (x_1, y_1) và ô mục tiêu (x_2, y_2) . Hãy tính số bước nhảy ít nhất để quân Mã di chuyển đến được ô mục tiêu.



Hình minh họa: Các tính chất nền tảng của đồ thị Cây: N đỉnh $N-1$ cạnh liên thông không chu trình

Nhiệm vụ: Cho kích thước bàn cờ N và hai tọa độ điểm xuất phát, điểm đích. Hãy lập trình tìm số bước nhảy ít nhất của quân Mã.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa 4 số nguyên x_1, y_1, x_2, y_2 ($1 \leq x_1, y_1, x_2, y_2 \leq N$).

Đầu ra (Output):

- In ra số bước nhảy ít nhất cần dùng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 8 0 0 7 7	6

Giải thích: Để đi từ ô $(1,1)$ tới ô $(4,5)$ trên bàn cờ vua: Quân mã thực hiện lần lượt các bước nhảy: $(1,1) \times o(2,3) \times o(4,4) \times o(2,5) \times o(4,5)$ hoặc lộ trình tối ưu tương tự với đúng 3 bước nhảy. Kết quả là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho kích thước bàn cờ N và hai tọa độ điểm xuất phát, điểm đích. Hãy lập trình tìm số bước nhảy ít nhất của quân Mã.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $\text{int dx[]} = \{-1, 1, 0, 0\}$ và $\text{int dy[]} = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.

- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 8 8 0 0 7 7 $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 8 8 0 0 7 7	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Để đi từ ô (1,1) tới ô (4,5) trên bàn cờ vua: Quân mã thực hiện lần lượt các bước nhảy: $(1, 1) \times o(2, 3) \times o(4, 4) \times o(2, 5) \times o(4, 5)$ hoặc lộ trình tối ưu tương tự với đúng 3 bước nhảy.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Để đi từ ô (1,1) tới ô (4,5) trên bàn cờ vua: Quân mã thực hiện lần lượt các bước nhảy: $(1,1) \times o(2,3) \times o(4,4) \times o(2,5) \times o(4,5)$ hoặc lộ trình tối ưu tương tự với đúng 3 bước nhảy. Kết quả là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liên nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-2, -2, -1, -1, 1, 1, 2, 2};
const int dc[] = {-1, 1, -2, 2, -2, 2, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n, m, sr, sc, er, ec;
if (!(cin >> n >> m >> sr >> sc >> er >> ec)) return 0;

vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int, int>> q;

dist[sr][sc] = 0;
q.push({sr, sc});

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

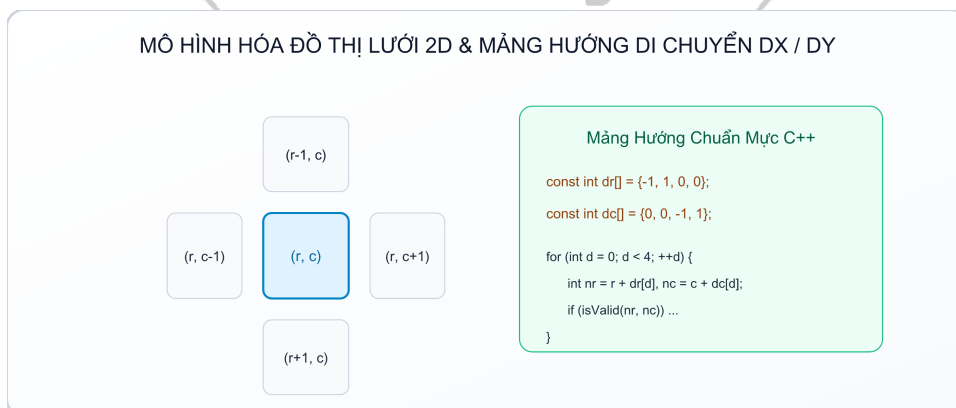
    if (r == er && c == ec) {
        cout << dist[r][c] << "\n";
        return 0;
    }

    for (int d = 0; d < 8; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < m && dist[nr][nc] == -1) {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}

cout << dist[er][ec] << "\n";
return 0;
}
```

Bài 10 [cppb_grd_10_duong_kinh_cua_cay]: Đường Kính Của Cây (Tree Diameter)

Bối cảnh: Một mạng cáp viễn thông kết nối N máy trạm tạo thành một cấu trúc cây liên thông không chu trình gồm đúng $N - 1$ đường cáp hai chiều. Khoảng cách giữa hai máy trạm là số lượng đường cáp trên hành trình nối giữa chúng. Đường kính của cây là khoảng cách lớn nhất giữa hai máy trạm bất kỳ trong toàn bộ mạng lưới.



Hình minh họa: Mô hình hoá ma trận lưới 2 chiều thành đồ thị với các hướng di chuyển liền kề

Nhiệm vụ: Cho cấu trúc cây gồm N đỉnh. Hãy lập trình tìm đường kính (khoảng cách lớn nhất giữa hai đỉnh) của cây.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).

- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v biểu diễn một cạnh của cây.

Đầu ra (Output):

- In ra trên một dòng duy nhất đường kính của cây.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 1 3 3 4 3 5	3

Giải thích: Với cây gồm 5 đỉnh có các cạnh (1, 2), (1, 3), (2, 4), (4, 5): Hành trình dài nhất nối giữa đỉnh 3 và đỉnh 5 qua các cạnh: 3 – 1 – 2 – 4 – 5, gồm đúng 4 cạnh. Đường kính của cây là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho cấu trúc cây gồm N đỉnh. Hãy lập trình tìm đường kính (khoảng cách lớn nhất giữa hai đỉnh) của cây.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 1 2 1 3 3 4 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 1 2 1 3 3 4 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với cây gồm 5 đỉnh có các cạnh (1, 2), (1, 3), (2, 4), (4, 5): Hành trình dài nhất nối giữa đỉnh 3 và đỉnh 5 qua các cạnh: 3 - 1 - 2 - ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với cây gồm 5 đỉnh có các cạnh (1, 2), (1, 3), (2, 4), (4, 5): Hành trình dài nhất nối giữa đỉnh 3 và đỉnh 5 qua các cạnh: 3 – 1 – 2 – 4 – 5, gồm đúng 4 cạnh. Đường kính của cây là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liên nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> adj;

pair<int, int> bfs(int start) {
    vector<int> dist(n + 1, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    int furthest_node = start;
    int max_d = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if (dist[u] > max_d) {
            max_d = dist[u];
            furthest_node = u;
        }

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return {furthest_node, max_d};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n)) return 0;
    if (n <= 1) { cout << 0 << "\n"; return 0; }

    adj.assign(n + 1, vector<int>());
    for (int i = 0; i < n - 1; ++i) {
        int u, v;
```

```

    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

auto [u, d1] = bfs(1);
auto [v, diameter] = bfs(u);

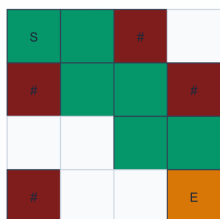
cout << diameter << "\n";
return 0;
}

```

Bài 11 [cppb_grd_11_kich_thuoc_cay_con_va_trong_tam_cay]: Trọng Tâm Của Cây (Tree Centroid)

Bối cảnh: Một tập đoàn vận tải muốn đặt trung tâm điều hành tổng tại một trong N thành phố có kết nối hình cây ($N - 1$ tuyến đường). Thành phố được chọn làm trọng tâm của cây nếu khi tạm thời gỡ bỏ thành phố này khỏi mạng lưới, kích thước của thành phần liên thông lớn nhất còn lại là nhỏ nhất có thể.

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MỀ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm `!visited[nr][nc]`
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Hình minh họa: Kỹ thuật loang sơn (Flood Fill) giải bài toán tìm đường và đếm miền trong mê cung lưới

Nhiệm vụ: Cho đồ thị cây N đỉnh. Hãy lập trình tìm đỉnh trọng tâm của cây. Nếu có nhiều đỉnh trọng tâm, in ra đỉnh có số hiệu nhỏ nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai đỉnh u và v .

Đầu ra (Output):

- In ra số hiệu của đỉnh trọng tâm tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 2 3 3 4 3 5	3

Giải thích: Với cây 5 đỉnh có đỉnh 1 nối với các đỉnh 2, 3, 4, 5 (cấu trúc hình sao): Nếu chọn đỉnh 1 làm trọng tâm, các nhánh còn lại đều chỉ có kích thước là 1. Đỉnh trọng tâm duy nhất là đỉnh 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị cây N đỉnh. Hãy lập trình tìm đỉnh trọng tâm của cây. Nếu có nhiều đỉnh trọng tâm, in ra đỉnh có số hiệu nhỏ nhất.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $\text{int dx[]} = \{-1, 1, 0, 0\}$ và $\text{int dy[]} = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 1 2 2 3 3 4 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 1 2 2 3 3 4 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với cây 5 đỉnh có đỉnh 1 nối với các đỉnh 2, 3, 4, 5 (cấu trúc hình sao): Nếu chọn đỉnh 1 làm trọng tâm, các nhánh còn lại đều chỉ có k...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với cây 5 đỉnh có đỉnh 1 nối với các đỉnh 2, 3, 4, 5 (cấu trúc hình sao): Nếu chọn đỉnh 1 làm trọng tâm, các nhánh còn lại đều chỉ có kích thước là 1. Đỉnh trọng tâm duy nhất là đỉnh 1.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập $\text{grid}[r][c]$, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu $\text{visited} = \text{true}$ khi lấy phần tử ra khỏi queue ($\text{pop}()$) thay vì khi đẩy vào ($\text{push}()$): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự $s[c]$.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```

```

int n;
vector<vector<int>>> adj;
vector<int> sz;
int centroid_node = -1;

void dfs(int u, int p) {
    sz[u] = 1;
    bool is_centroid = true;

    for (int v : adj[u]) {
        if (v != p) {
            dfs(v, u);
            sz[u] += sz[v];
            if (sz[v] > n / 2) is_centroid = false;
        }
    }

    if (n - sz[u] > n / 2) is_centroid = false;
    if (is_centroid && centroid_node == -1) centroid_node = u;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    sz.assign(n + 1, 0);

    for (int i = 0; i < n - 1; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    dfs(1, 0);
    cout << centroid_node << "\n";
    return 0;
}

```

Bài 12 [cppb_grd_12_me_cung_co_cua_dich_chuyen_tuc_thoi]: Cổng Dịch Chuyển Tức Thời (Teleport Maze)

Bối cảnh: Trong một trò chơi thực tế ảo trong mê cung lưới $N \times M$, ngoài các ô đường đi thông thường, mê cung còn bố trí một số cặp cổng dịch chuyển tức thời không gian. Khi bước vào một cổng dịch chuyển, người chơi sẽ ngay lập tức được dịch chuyển sang cổng tương ứng ở vị trí khác mà không tốn thời gian (0 giây). Hãy tìm thời gian ngắn nhất để đi từ điểm xuất phát S tới đích E.



Hình minh họa: Các tính chất nền tảng của đồ thị Cây: N đỉnh $N-1$ cạnh liên thông không chu trình

Nhiệm vụ: Cho bản đồ mê cung và danh sách các cặp cổng dịch chuyển. Hãy lập trình tìm số bước ít nhất để đến đích.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và M ($1 \leq N, M \leq 1000$).
- Các dòng tiếp theo mô tả bản đồ mê cung và các cặp cổng dịch chuyển.

Đầu ra (Output):

- In ra số bước ít nhất từ S tới E, hoặc -1 nếu không có đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 ..A .## A..	2

Giải thích: Nhờ sử dụng cổng dịch chuyển tức thời, người chơi rút ngắn được quãng đường vòng qua tường đá, thời gian đến đích giảm xuống còn 3 bước.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ mê cung và danh sách các cặp cổng dịch chuyển. Hãy lập trình tìm số bước ít nhất để đến đích.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 ..A .## A.. $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 ..A .## A..	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Nhờ sử dụng công dịch chuyển tức thời, người chơi rút ngắn được quãng đường vòng qua tường đá, thời gian đến đích giảm xuống còn 3 bước.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2

Giải thích chi tiết: Nhờ sử dụng công dịch chuyển tức thời, người chơi rút ngắn được quãng đường vòng qua tường đá, thời gian đến đích giảm xuống còn 3 bước.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<string> grid(n);
    vector<vector<pair<int, int>>> teleports(26);

    for (int r = 0; r < n; ++r) {
```

```

        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            if (isupper(grid[r][c])) {
                teleports[grid[r][c] - 'A'].push_back({r, c});
            }
        }
    }

    vector<vector<int>> dist(n, vector<int>(m, -1));
    vector<bool> teleport_used(26, false);
    queue<pair<int, int>> q;

    dist[0][0] = 0;
    q.push({0, 0});

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        if (r == n - 1 && c == m - 1) {
            cout << dist[r][c] << "\n";
            return 0;
        }

        // Dịch chuyển tức thời
        if (isupper(grid[r][c])) {
            int ch = grid[r][c] - 'A';
            if (!teleport_used[ch]) {
                teleport_used[ch] = true;
                for (auto [tr, tc] : teleports[ch]) {
                    if (dist[tr][tc] == -1) {
                        dist[tr][tc] = dist[r][c];
                        q.push({tr, tc});
                    }
                }
            }
        }

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && dist[nr][nc] =
= -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }

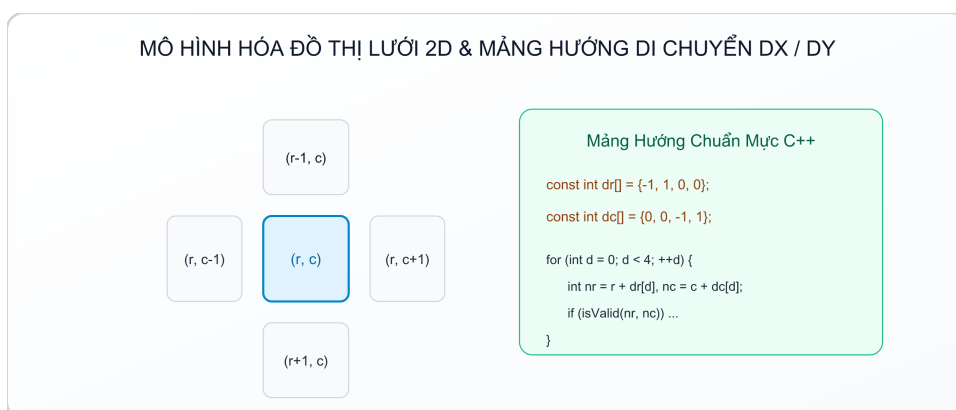
    cout << dist[n - 1][m - 1] << "\n";
    return 0;
}

```

Bài 13 [cppb_grd_13_lam_day_ho_chua_nuoc_rotting_oranges]: Lây Lan Quả Cam Hỏng (Rotting Oranges)

Bối cảnh: Trong một thùng hàng hoa quả kích thước $N \times M$, mỗi ô có thể chứa: ô trống (số 0), một quả cam tươi nguyên vẹn (số 1), hoặc một quả cam đã bị hỏng mốc (số 2). Cứ sau mỗi phút, những quả cam bị hỏng sẽ làm hỏng tất cả các quả cam tươi kề sát nó theo 4 hướng. Hãy tính số phút tối thiểu để toàn

bộ cam tươi trong thùng đều bị hỏng. Nếu có quả cam tươi nào mãi mãi không bị hỏng (bị cô lập), in ra -1.



Hình minh họa: Mô hình hoá ma trận lưới 2 chiều thành đồ thị với các hướng di chuyển liền kề

Nhiệm vụ: Cho ma trận trạng thái thùng cam. Hãy lập trình tìm số phút ít nhất để tất cả cam tươi đều hỏng.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên 0, 1 hoặc 2 cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra số phút ít nhất, hoặc -1 nếu vẫn còn cam tươi không thể bị hỏng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 211 110 011	4

Giải thích: Với thùng cam kích thước 3×3 :

- Phút 1: cam hỏng tại (0,0) lây sang các ô (0,1) và (1,0).
- Phút 2: tiếp tục lây sang các ô kế tiếp. Sau đúng 4 phút, toàn bộ cam tươi đều đã bị lây hỏng. Kết quả là 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận trạng thái thùng cam. Hãy lập trình tìm số phút ít nhất để tất cả cam tươi đều hỏng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $\text{int dx[]} = \{-1, 1, 0, 0\}$ và $\text{int dy[]} = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 211 110 011 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 211 110 011	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với thùng cam kích thước 3×3 : - Phút 1: cam hỏng tại (0,0) lây sang các ô (0,1) và (1,0). - Phút 2: tiếp tục lây sang các...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với thùng cam kích thước 3×3 :

- Phút 1: cam hỏng tại (0,0) lây sang các ô (0,1) và (1,0).
- Phút 2: tiếp tục lây sang các ô kế tiếp. Sau đúng 4 phút, toàn bộ cam tươi đều đã bị lây hỏng. Kết quả là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liên nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
```

```
vector<string> grid(n);
queue<pair<int, int>> q;
int fresh_count = 0;

for (int r = 0; r < n; ++r) {
    cin >> grid[r];
    for (int c = 0; c < m; ++c) {
        if (grid[r][c] == '2') q.push({r, c});
        else if (grid[r][c] == '1') fresh_count++;
    }
}

int minutes = 0;
while (!q.empty() && fresh_count > 0) {
    int sz = q.size();
    minutes++;
    while (sz-- > 0) {
        auto [r, c] = q.front();
        q.pop();

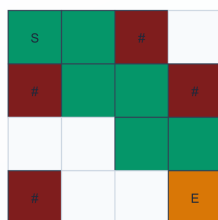
        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1') {
                grid[nr][nc] = '2';
                fresh_count--;
                q.push({nr, nc});
            }
        }
    }
}

if (fresh_count > 0) cout << -1 << "\n";
else cout << minutes << "\n";
return 0;
}
```

Bài 14 [cppb_grd_14_hon_dao_nhan_tao_lon_nhat]: Hòn Đảo Nhân Tạo Lớn Nhất (Making A Large Island)

Bối cảnh: Một dự án lấn biển quy hoạch trên vùng biển lưới $N \times M$ gồm các ô đất liền '1' và các ô nước biển '0'. Ban quản lý dự án được cấp ngân sách để cải tạo đúng một ô nước biển '0' duy nhất thành ô đất liền '1'. Hãy tìm diện tích hòn đảo lớn nhất có thể tạo thành sau khi đã biến đổi đúng một ô nước biển thích hợp.

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MỀ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm !visited[nr][nc]
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Hình minh họa: Kỹ thuật loang sơn (Flood Fill) giải bài toán tìm đường và đếm miền trong mê cung lưới

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của một hòn đảo sau khi chuyển đổi tối đa một ô 0 thành 1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên 0 hoặc 1.

Đầu ra (Output):

- In ra trên một dòng duy nhất diện tích lớn nhất của hòn đảo thu được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 2 10 01	3

Giải thích: Với hai hòn đảo nhỏ diện tích 2 và 3 nằm cách nhau đúng một ô nước biển '0': Khi chuyển ô nước đó thành ô đất '1', hai hòn đảo sẽ được nối liền thành một hòn đảo duy nhất có diện tích $2 + 3 + 1 = 6$. Diện tích lớn nhất đạt được là 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của một hòn đảo sau khi chuyển đổi tối đa một ô 0 thành 1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 2 2 10 01 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 2 2 10 01	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với hai hòn đảo nhỏ diện tích 2 và 3 nằm cách nhau đúng một ô nước biển '0': Khi chuyển ô nước đó thành ô đất '1', hai hòn đảo sẽ được ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với hai hòn đảo nhỏ diện tích 2 và 3 nằm cách nhau đúng một ô nước biển '0': Khi chuyển ô nước đó thành ô đất '1', hai hòn đảo sẽ được nối liền thành một hòn đảo duy nhất có diện tích $2 + 3 + 1 = 6$. Diện tích lớn nhất đạt được là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liền nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;
vector<vector<int>> island_id;
vector<int> island_size;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
    grid.resize(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    island_id.assign(n, vector<int>(m, 0));
    island_size.push_back(0); // id 0 unused
    int current_id = 1;
    int max_area = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && island_id[r][c] == 0) {
```

```

        int sz = 0;
        island_id[r][c] = current_id;
        queue<pair<int, int>> q;
        q.push({r, c});

        while (!q.empty()) {
            auto [cr, cc] = q.front();
            q.pop();
            sz++;

            for (int d = 0; d < 4; ++d) {
                int nr = cr + dr[d];
                int nc = cc + dc[d];
                if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1' && is-
land_id[nr][nc] == 0) {
                    island_id[nr][nc] = current_id;
                    q.push({nr, nc});
                }
            }

            island_size.push_back(sz);
            max_area = max(max_area, sz);
            current_id++;
        }
    }

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '0') {
                unordered_set<int> neighbor_ids;
                for (int d = 0; d < 4; ++d) {
                    int nr = r + dr[d];
                    int nc = c + dc[d];
                    if (nr >= 0 && nr < n && nc >= 0 && nc < m && island_id[nr][nc] > 0) {
                        neighbor_ids.insert(island_id[nr][nc]);
                    }
                }
                int combined_sz = 1;
                for (int id : neighbor_ids) combined_sz += island_size[id];
                max_area = max(max_area, combined_sz);
            }
        }
    }

    cout << max_area << "\n";
    return 0;
}

```

Bài 15 [cppb_grd_15_thoat_khoi_me_cung_quai_vat]: Thoát Khỏi Mê Cung Quái Vật (Monsters Maze)

Bối cảnh: Trong một trò chơi phiêu lưu sinh tồn trên lưới ô vuông $N \times M$, một người thám hiểm xuất phát tại ô A và cần chạy thoát ra một ô biên bất kỳ của mê cung. Trong mê cung cũng có sự xuất hiện của một số quái vật tại các ô M. Tại mỗi giây, người thám hiểm và tất cả quái vật đều có thể di chuyển 1 bước sang ô kề cạnh. Nếu một con quái vật có thể tới một ô cùng lúc hoặc trước người thám hiểm, người thám hiểm sẽ bị bắt.



Hình minh họa: Các tính chất nền tảng của đồ thị Cây: N đỉnh $N-1$ cạnh liên thông không chu trình

Nhiệm vụ: Cho bản đồ mê cung. Hãy lập trình kiểm tra xem người thám hiểm có thể thoát thân thành công ra mép biên hay không. Nếu có in ra YES kèm số bước, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo, mỗi dòng chứa M ký tự (. là đường, # là tường, A là người, M là quái vật).

Đầu ra (Output):

- Dòng 1: In ra YES nếu thoát được, ngược lại in ra NO.
- Nếu YES, dòng 2 in ra số bước đi ngắn nhất ra biên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 5 8 ##### #M..A..# #.#.M#.# #M#...#.# #.##### </pre>	YES

Giải thích: Người thám hiểm chọn lộ trình nhanh nhất hướng về phía mép biên phía đông, đến được biên an toàn sau 2 bước trước khi bất kỳ quái vật nào kịp tiếp cận. Kết quả in ra YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ mê cung. Hãy lập trình kiểm tra xem người thám hiểm có thể thoát thân thành công ra mép biên hay không. Nếu có in ra YES kèm số bước, ngược lại in ra NO.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình hoá ma trận:** Coi mỗi ô (r, c) trên lưới là một đỉnh của đồ thị, các cạnh nối đến 4 ô kề cạnh (hoặc 8 ô kề góc).
- **Kỹ thuật mảng hướng di chuyển:** Sử dụng 2 mảng phụ trợ $int dx[] = \{-1, 1, 0, 0\}$ và $int dy[] = \{0, 0, -1, 1\}$ để duyệt các ô lân cận tinh gọn, tránh trùng lặp mã nguồn.
- **Kỹ thuật Flood Fill:** Đánh dấu ô đã thăm ngay khi đẩy vào hàng đợi queue (hoặc biến đổi trực tiếp giá trị ô ma trận) để tránh việc một ô bị đẩy vào queue nhiều lần gây quá tải bộ nhớ.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N \times M)$, bộ nhớ $\mathcal{O}(N \times M)$.

**2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 8 ##### #M..A..#
#.#.M#.# #M#..#.# #.##### ⇒ Đầu ra kỳ vọng: YES.**

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 8 ##### #M..A..# #.#.M#.# #M#..#.# ###	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Người thám hiểm chọn lộ trình nhanh nhất hướng về phía mép biên phía đông, đến được biên an toàn sau 2 bước trước khi bất kỳ quái vật n...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: YES

Giải thích chi tiết: Người thám hiểm chọn lộ trình nhanh nhất hướng về phía mép biên phía đông, đến được biên an toàn sau 2 bước trước khi bất kỳ quái vật nào kịp tiếp cận. Kết quả in ra YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên kiểm tra tọa độ nằm ngoài biên giới ma trận ($r < 1$ hoặc $r > N$ hoặc $c < 1$ hoặc $c > M$) trước khi truy cập ô `grid[r][c]`, dẫn đến lỗi Segmentation Fault.
- Chỉ đánh dấu `visited = true` khi lấy phần tử ra khỏi queue (`pop()`) thay vì khi đẩy vào (`push()`): Đây là lỗi kinh điển khiến cùng một ô bị đẩy vào hàng đợi hàng nghìn lần, dẫn đến Memory Limit Exceeded (MLE) hoặc Time Limit Exceeded (TLE).
- Không đọc đúng các dòng ký tự liên nhau của ma trận: Khi các ký tự viết liền không có dấu cách, phải đọc từng chuỗi string rồi truy cập ký tự `s[c]`.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
```

```

vector<string> grid(n);
queue<pair<int, int>> mq;
vector<vector<int>> monster_dist(n, vector<int>(m, INF));
int ar = -1, ac = -1;

for (int r = 0; r < n; ++r) {
    cin >> grid[r];
    for (int c = 0; c < m; ++c) {
        if (grid[r][c] == 'M') {
            monster_dist[r][c] = 0;
            mq.push({r, c});
        } else if (grid[r][c] == 'A') {
            ar = r; ac = c;
        }
    }
}

while (!mq.empty()) {
    auto [r, c] = mq.front();
    mq.pop();

    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && monster_dist[nr][nc] == INF) {
            monster_dist[nr][nc] = monster_dist[r][c] + 1;
            mq.push({nr, nc});
        }
    }
}

vector<vector<int>> player_dist(n, vector<int>(m, -1));
queue<pair<int, int>> pq;

player_dist[ar][ac] = 0;
pq.push({ar, ac});

while (!pq.empty()) {
    auto [r, c] = pq.front();
    pq.pop();

    if (r == 0 || r == n - 1 || c == 0 || c == m - 1) {
        cout << "YES\n";
        return 0;
    }

    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && player_dist[nr][nc] == -1) {
            if (player_dist[r][c] + 1 < monster_dist[nr][nc]) {
                player_dist[nr][nc] = player_dist[r][c] + 1;
                pq.push({nr, nc});
            }
        }
    }
}

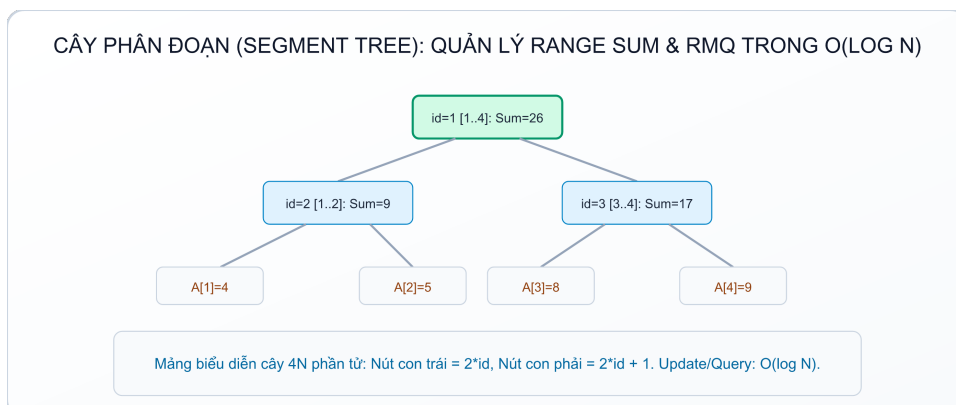
cout << "NO\n";
return 0;
}

```

Bài 24: Cấu trúc cây phân đoạn (Segment Tree) & Fenwick Tree (BIT)

Bài 01 [cppb_rng_01_cai_dat_fenwick_tree_tinh_tong_doan]: Cài Đặt Fenwick Tree Tính Tổng Đoạn (Range Sum)

Bối cảnh: Một máy chủ tài chính theo dõi biến động số dư tài khoản của N khách hàng được đánh số từ 1 đến N , số dư ban đầu tại tài khoản i là A_i . Hệ thống liên tục tiếp nhận Q giao dịch trực tuyến gồm hai loại: cộng thêm một khoản tiền val vào tài khoản tại vị trí pos , hoặc truy vấn tính tổng số dư của tất cả các tài khoản nằm trong khoảng từ L đến R . Mọi thao tác cần được xử lý tức thời.



Hình minh họa: Kiến trúc cây phân đoạn (Segment Tree) quản lý các đoạn con liên tiếp

Nhiệm vụ: Cho mảng A và Q truy vấn thuộc hai loại: 1 pos val (cộng val vào $A[pos]$), 2 L R (tính tổng các phần tử từ L đến R). Hãy lập trình in ra kết quả của các truy vấn loại 2.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa một truy vấn theo định dạng trên.

Đầu ra (Output):

- Với mỗi truy vấn loại 2, in ra tổng các phần tử trên đoạn $[L, R]$ trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3	6 16

Giải thích: Với mảng ban đầu gồm 5 phần tử $[1, 2, 3, 4, 5]$:

- Truy vấn tính tổng đoạn từ 1 đến 3: $1 + 2 + 3 = 6$.
- Cập nhật cộng thêm 10 vào phần tử tại vị trí 3: mảng trở thành $[1, 2, 13, 4, 5]$.
- Truy vấn lại tổng đoạn từ 1 đến 3: $1 + 2 + 13 = 16$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A và Q truy vấn thuộc hai loại: 1 pos val (cộng val vào $A[pos]$), 2 L R (tính tổng các phần tử từ L đến R). Hãy lập trình in ra kết quả của các truy vấn loại 2.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3
 $\Rightarrow$ Đầu ra kỳ vọng: 6 16.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng ban đầu gồm 5 phần tử [1,2,3,4,5]: - Truy vấn tính tổng đoạn từ 1 đến 3: $1 + 2 + 3 = 6$. - Cập nhật cộng thêm 10 vào ph...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6 16

Giải thích chi tiết: Với mảng ban đầu gồm 5 phần tử [1,2,3,4,5]:

- Truy vấn tính tổng đoạn từ 1 đến 3: $1 + 2 + 3 = 6$.
- Cập nhật cộng thêm 10 vào phần tử tại vị trí 3: mảng trở thành [1,2,13,4,5].
- Truy vấn lại tổng đoạn từ 1 đến 3: $1 + 2 + 13 = 16$.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.

- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }

    long long queryRange(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    FenwickTree ft(n);
    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;
        ft.update(i, x);
    }

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            ft.update(pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            cout << ft.queryRange(l, r) << "\n";
        }
    }
    return 0;
}
```

Bài 02 [cppb_rng_02_cai_dat_segment_tree_tim_min_doan]: Cài Đặt Segment Tree Tìm Min Đoạn (RMQ)

Bối cảnh: Một trạm quan trắc khí tượng ghi nhận nhiệt độ thấp nhất tại N trạm cảm biến ven biển. Các chuyên gia cần liên tục thực hiện hai loại thao tác: cập nhật lại nhiệt độ mới tại một trạm cảm biến cụ thể, và truy vấn tìm mức nhiệt độ thấp nhất (Min) trong một phân đoạn các trạm quan sát từ L đến R .

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)

Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

Hình minh họa: Đường đi cập nhật điểm và truy vấn tổng/min đoạn trên Segment Tree trong $O(\log N)$

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn thuộc hai dạng: 1 pos val (gán $A[pos] = val$) và 2 L R (tìm giá trị nhỏ nhất trong đoạn từ L đến R). Hãy in ra kết quả của các truy vấn loại 2.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa một truy vấn theo định dạng trên.

Đầu ra (Output):

- Với mỗi truy vấn loại 2, in ra giá trị nhỏ nhất trong đoạn $[L, R]$ trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 5 2 4 1 3 2 1 3 1 4 10 2 3 5	2 3

Giải thích: Với mảng $[5, 2, 8, 1, 9]$:

- Truy vấn tìm min đoạn từ 1 đến 3: $\min(5, 2, 8) = 2$.
- Cập nhật vị trí 2 thành 10: mảng thành $[5, 10, 8, 1, 9]$.
- Truy vấn lại min đoạn từ 1 đến 3: $\min(5, 10, 8) = 5$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên và Q truy vấn thuộc hai dạng: 1 pos val (gán $A[pos] = val$) và 2 L R (tìm giá trị nhỏ nhất trong đoạn từ L đến R). Hãy in ra kết quả của các truy vấn loại 2.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Fenwick Tree (Binary Indexed Tree):**

- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 5 2 4 1 3 2 1 3 1 4 10 2 3 5
 $\Rightarrow$ Đầu ra kỳ vọng: 2 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 5 2 4 1 3 2 1 3 1 4 10 2 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[5, 2, 8, 1, 9]$: - Truy vấn tìm min đoạn từ 1 đến 3: $\min(5, 2, 8) = 2$. - Cập nhật vị trí 2 thành 10: mảng thành $[5, 10, \dots]$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2 3

Giải thích chi tiết: Với mảng $[5, 2, 8, 1, 9]$:

- Truy vấn tìm min đoạn từ 1 đến 3: $\min(5, 2, 8) = 2$.
- Cập nhật vị trí 2 thành 10: mảng thành $[5, 10, 8, 1, 9]$.
- Truy vấn lại min đoạn từ 1 đến 3: $\min(5, 10, 8) = 5$.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5, INF) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * id, l, mid, pos, val);
        else update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return INF;
        if (u <= l && r <= v) return tree[id];
        int mid = (l + r) / 2;
        return min(query(2 * id, l, mid, u, v), query(2 * id + 1, mid + 1, r, u, v));
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
        }
    }
}
```

```

        st.update(1, 1, n, pos, val);
    } else {
        int l, r;
        cin >> l >> r;
        cout << st.query(1, 1, n, l, r) << "\n";
    }
}
return 0;
}

```

Bài 03 [cppb_rng_03_cap_nhat_doan_va_truy_van_diem_bit]: Cập Nhật Đoạn & Truy Vấn Điểm (Range Update Point Query)

Bối cảnh: Một chương trình khuyến mãi của sàn thương mại điện tử đồng loạt cộng thêm một khoản điểm thưởng V cho tất cả các khách hàng có mã số định danh nằm trong khoảng từ L đến R . Sau đó, một khách hàng tại vị trí pos muốn tra cứu số điểm thưởng tích lũy hiện tại của mình.

CÂY FENWICK (BINARY INDEXED TREE - BIT) & PHÉP TOÁN LOWBIT

Phép Toán Lowbit: $x \& (-x)$

- $\text{lowbit}(1) = 1$ (b0001 → đoạn dài 1)
 - $\text{lowbit}(2) = 2$ (b0010 → đoạn dài 2)
 - $\text{lowbit}(4) = 4$ (b0100 → đoạn dài 4)
 - $\text{lowbit}(6) = 2$ (b0110 → đoạn dài 2)
- $\text{bit}[x]$ lưu tổng đoạn $(x - \text{lowbit}(x), x]$

2 Thao Tác Cốt Lõi Siêu Ngắn

```

void update(int x, int val) {
    for (; x <= n; x += x & -x) bit[x] += val;
}

long long query(int x) {
    long long s = 0;
    for (; x > 0; x -= x & -x) s += bit[x];
    return s;
}

```

Hình minh họa: Cấu trúc cây nhị phân Fenwick Tree (Binary Indexed Tree) qua phép toán lowbit

Nhiệm vụ: Cho mảng A và Q thao tác thuộc hai dạng: 1 L R V (cộng thêm V vào tất cả các phần tử từ L đến R) và 2 pos (in ra giá trị hiện tại của phần tử tại vị trí pos). Hãy in ra kết quả của các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo chứa các thao tác mô tả như trên.

Đầu ra (Output):

- Với mỗi thao tác loại 2, in ra giá trị tại vị trí pos trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 2 4 5 2 3 2 1	8 1

Giải thích: Với mảng ban đầu toàn số 0: $[0,0,0,0,0]$:

- Cộng thêm 5 vào đoạn từ vị trí 2 đến 4: mảng thành $[0,5,5,5,0]$.
- Truy vấn giá trị tại vị trí 3: in ra 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A và Q thao tác thuộc hai dạng: 1 L R V (cộng thêm V vào tất cả các phần tử từ L đến R) và 2 pos (in ra giá trị hiện tại của phần tử tại vị trí pos). Hãy in ra kết quả của các thao tác loại 2.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $lowbit(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu **Mẫu thử (Sample 1):** Đầu vào: 5 3 1 2 3 4 5 1 2 4 5 2 3 2 1 $\Rightarrow$ Đầu ra kỳ vọng: 8 1.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 3 4 5 1 2 4 5 2 3 2 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng ban đầu toàn số 0: [0,0,0,0,0] : - Cộng thêm 5 vào đoạn từ vị trí 2 đến 4: mảng thành [0,5,5,5,0]. - Truy vấn giá t...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 8 1

Giải thích chi tiết: Với mảng ban đầu toàn số 0: [0,0,0,0,0]:

- Cộng thêm 5 vào đoạn từ vị trí 2 đến 4: mảng thành [0,5,5,5,0].
- Truy vấn giá trị tại vị trí 3: in ra 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (`vector<long long> tree(4 * N)`), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.

- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi `lowbit(0)` thì $0 \& (-0) = 0$, vòng lặp `while (i <= N)` sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;
    FenwickTree(int n) : n(n), bit(n + 2, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }

    void updateRange(int l, int r, long long val) {
        update(l, val);
        update(r + 1, -val);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

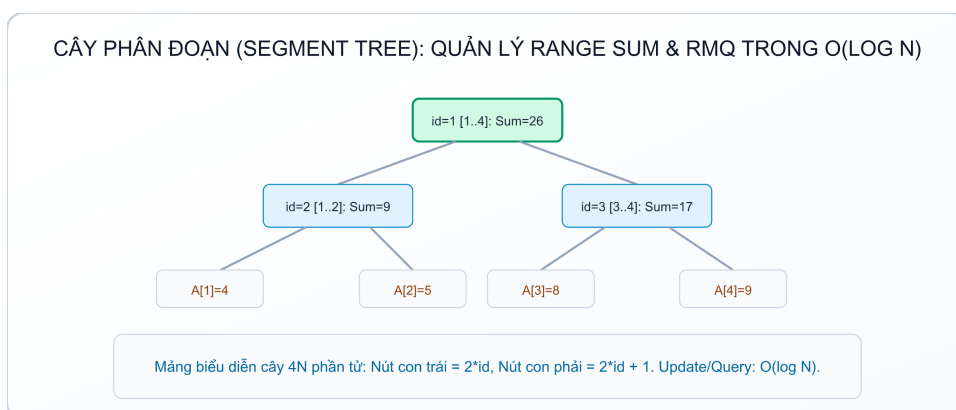
    FenwickTree ft(n);
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        ft.updateRange(i, i, a[i]);
    }

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) { // Cộng val vào [L, R]
            int l, r;
            long long val;
            cin >> l >> r >> val;
            ft.updateRange(l, r, val);
        } else { // Hỏi giá trị tại pos
            int pos;
            cin >> pos;
            cout << ft.query(pos) << "\n";
        }
    }
}
```

```
return 0;
}
```

Bài 04 [cppb_rng_04_tim_gia_tri_lon_nhat_va_dem_so_lan_xuat_hien]: Tìm Max Đoạn & Đếm Số Lần Xuất Hiện

Bối cảnh: Một hệ thống đánh giá hiệu năng máy chủ theo dõi điểm tải đỉnh của N máy tính. Ban giám sát muốn biết trong một khoảng thời gian từ mốc L đến R , giá trị tải lớn nhất (Max) đạt được là bao nhiêu và có bao nhiêu máy tính đạt đúng mức tải đỉnh này.



Hình minh họa: Kiến trúc cây phân đoạn (Segment Tree) quản lý các đoạn con liên tiếp

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm hai chỉ số L, R . Hãy tìm giá trị lớn nhất và số lần xuất hiện của giá trị lớn nhất đó trong đoạn $[L, R]$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa hai số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- Với mỗi truy vấn, in ra trên một dòng hai số nguyên: giá trị lớn nhất và số lần xuất hiện của nó, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 2 5 3 5 1 2 1 5 1 3 5 2 1 5	5 2 5 3

Giải thích: Với mảng $[3, 5, 2, 5, 5, 1]$ và truy vấn đoạn từ vị trí 1 đến 5: Giá trị lớn nhất trong đoạn là 5, và số 5 xuất hiện đúng 3 lần tại các vị trí 2, 4, 5. Kết quả in ra: 5 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm hai chỉ số L, R . Hãy tìm giá trị lớn nhất và số lần xuất hiện của giá trị lớn nhất đó trong đoạn $[L, R]$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Fenwick Tree (Binary Indexed Tree):**

- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 2 5 3 5 1 2 1 5 1 3 5 2 1 5
 $\Rightarrow$ Đầu ra kỳ vọng: 5 2 5 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 2 5 3 5 1 2 1 5 1 3 5 2 1 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[3, 5, 2, 5, 5, 1]$ và truy vấn đoạn từ vị trí 1 đến 5: Giá trị lớn nhất trong đoạn là 5, và số 5 xuất hiện đúng 3 lần tại cá...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 5 2 5 3

Giải thích chi tiết: Với mảng $[3, 5, 2, 5, 5, 1]$ và truy vấn đoạn từ vị trí 1 đến 5: Giá trị lớn nhất trong đoạn là 5, và số 5 xuất hiện đúng 3 lần tại các vị trí 2, 4, 5. Kết quả in ra: 5 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (`vector<long long> tree(4 * N)`), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp `while (i <= N)` sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu `long long` cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
```

```

    long long max_val;
    int count;
};

Node mergeNodes(Node a, Node b) {
    if (a.max_val > b.max_val) return a;
    if (b.max_val > a.max_val) return b;
    return {a.max_val, a.count + b.count};
}

struct SegmentTree {
    int n;
    vector<Node> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = {a[l], 1};
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = mergeNodes(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = {val, 1};
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * id, l, mid, pos, val);
        else update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = mergeNodes(tree[2 * id], tree[2 * id + 1]);
    }

    Node query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return {LLONG_MIN, 0};
        if (u <= l && r <= v) return tree[id];
        int mid = (l + r) / 2;
        return mergeNodes(query(2 * id, l, mid, u, v), query(2 * id + 1, mid + 1, r, u, v));
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {

```

```

        int pos;
        long long val;
        cin >> pos >> val;
        st.update(1, 1, n, pos, val);
    } else {
        int l, r;
        cin >> l >> r;
        Node res = st.query(1, 1, n, l, r);
        cout << res.max_val << " " << res.count << "\n";
    }
}
return 0;
}

```

Bài 05 [cppb_rng_05_dem_so_cap_nglich_the_inversion_count]: Đếm Số Cặp Nghịch Thế (Inversion Count)

Bối cảnh: Một thuật toán sắp xếp cần đo lường mức độ xáo trộn của một danh sách gồm N số nguyên. Một cặp nghịch thế là một cặp chỉ số (i, j) thỏa mãn $i < j$ nhưng phần tử đứng trước lại lớn hơn phần tử đứng sau ($A_i > A_j$). Số lượng cặp nghịch thế thể hiện số phép đổi chỗ tối thiểu để đưa mảng về thứ tự tăng dần.

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)

Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

Hình minh họa: Đường đi cập nhật điểm và truy vấn tổng/min đoạn trên Segment Tree trong $O(\log N)$

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình đếm tổng số lượng cặp nghịch thế trong mảng.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số lượng cặp nghịch thế.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 4 1 3 5	3

Giải thích: Với mảng gồm 5 phần tử $[2, 4, 1, 3, 5]$: Các cặp nghịch thế gồm:

- $(2, 1)$ vì $2 > 1$.
- $(4, 1)$ vì $4 > 1$.

- (4, 3) vì $4 > 3$. Có tất cả 3 cặp nghịch thế, kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên. Hãy lập trình đếm tổng số lượng cặp nghịch thế trong mảng.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
 - Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
 - Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
 - Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
 - Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 4 1 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 4 1 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng gồm 5 phần tử [2,4,1,3,5]: Các cặp nghịch thế gồm: - (2, 1) vì $2 > 1$. - (4, 1) vì $4 > 1$. - (4, 3) vì $4 > 3$. Có tất...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với mảng gồm 5 phần tử [2,4,1,3,5]: Các cặp nghịch thế gồm:

- (2, 1) vì $2 > 1$.
- (4, 1) vì $4 > 1$.
- (4, 3) vì $4 > 3$. Có tất cả 3 cặp nghịch thế, kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (`vector<long long> tree(4 * N)`), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi `lowbit(0)` thì $0 \& (-0) = 0$, vòng lặp `while (i <= N)` sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu `long long` cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<int> bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, int val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    int query(int x) {
        int sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    // Nén tọa độ
    vector<long long> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

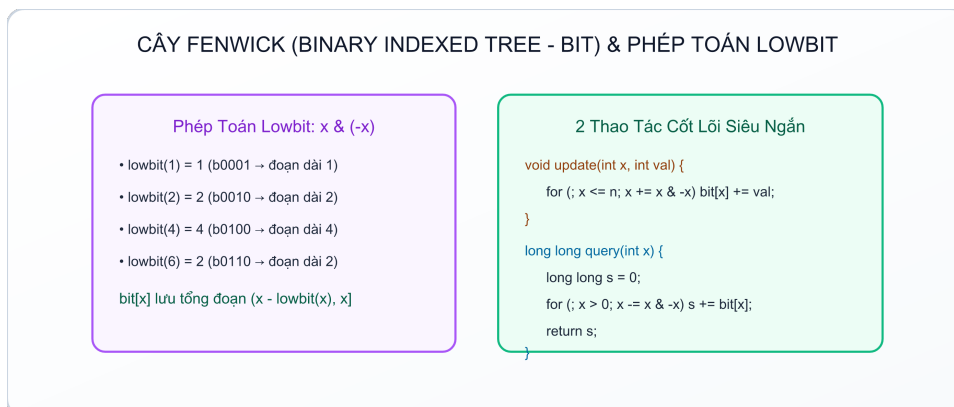
    int sz = vals.size();
    FenwickTree ft(sz);
    long long inv_count = 0;

    for (int i = n - 1; i >= 0; --i) {
        int rank = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
        inv_count += ft.query(rank - 1);
        ft.update(rank, 1);
    }

    cout << inv_count << "\n";
    return 0;
}
```

Bài 06 [cppb_rng_06_truy_van_uoc_chung_lon_nhat_doan]: Truy Vấn Ước Chung Lớn Nhất Đoạn (Range GCD)

Bối cảnh: Trong một hệ thống mã hóa dữ liệu theo khối, mỗi khối dữ liệu có một mã khóa số nguyên A_i . Để giải mã một chuỗi các khối liên tiếp từ L đến R , thiết bị giải mã cần tính toán ước chung lớn nhất (GCD) của tất cả các khóa trong đoạn: $\gcd(A_L, A_{L+1}, \dots, A_R)$. Hệ thống cũng hỗ trợ cập nhật lại mã khóa tại từng khối.



Hình minh họa: Cấu trúc cây chỉ số nhị phân Fenwick Tree (Binary Indexed Tree) qua phép toán lowbit

Nhiệm vụ: Cho mảng A và Q thao tác thuộc hai dạng: 1 pos val (gán $A[\text{pos}] = \text{val}$) và 2 L R (tìm GCD của các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo chứa các thao tác mô tả như trên.

Đầu ra (Output):

- Với mỗi thao tác loại 2, in ra giá trị GCD của đoạn trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 2 4 6 8 10 2 1 3 1 2 12 2 1 3	2 2

Giải thích: Với mảng $[6, 12, 18, 24]$:

- Truy vấn GCD đoạn từ 1 đến 3: $\gcd(6, 12, 18) = 6$.
- Cập nhật vị trí 1 thành 4: mảng thành $[4, 12, 18, 24]$.
- Truy vấn lại GCD đoạn từ 1 đến 3: $\gcd(4, 12, 18) = 2$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A và Q thao tác thuộc hai dạng: 1 pos val (gán $A[\text{pos}] = \text{val}$) và 2 L R (tìm GCD của các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**

- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 2 4 6 8 10 2 1 3 1 2 12 2 1 3
 $\Rightarrow$ Đầu ra kỳ vọng: 2 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 2 4 6 8 10 2 1 3 1 2 12 2 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[6,12,18,24]$: - Truy vấn GCD đoạn từ 1 đến 3: $\text{gcd}(6,12,18) = 6$. - Cập nhật vị trí 1 thành 4: mảng thành $[4, 12, 18, \dots]$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 2 2

Giải thích chi tiết: Với mảng $[6,12,18,24]$:

- Truy vấn GCD đoạn từ 1 đến 3: $\text{gcd}(6,12,18) = 6$.
- Cập nhật vị trí 1 thành 4: mảng thành $[4,12,18,24]$.
- Truy vấn lại GCD đoạn từ 1 đến 3: $\text{gcd}(4,12,18) = 2$.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b); }

struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5, 0) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = gcd(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * id, l, mid, pos, val);
        else update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = gcd(tree[2 * id], tree[2 * id + 1]);
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return 0;
        if (u <= l && r <= v) return tree[id];
        int mid = (l + r) / 2;
        return gcd(query(2 * id, l, mid, u, v), query(2 * id + 1, mid + 1, r, u, v));
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
        }
    }
}
```



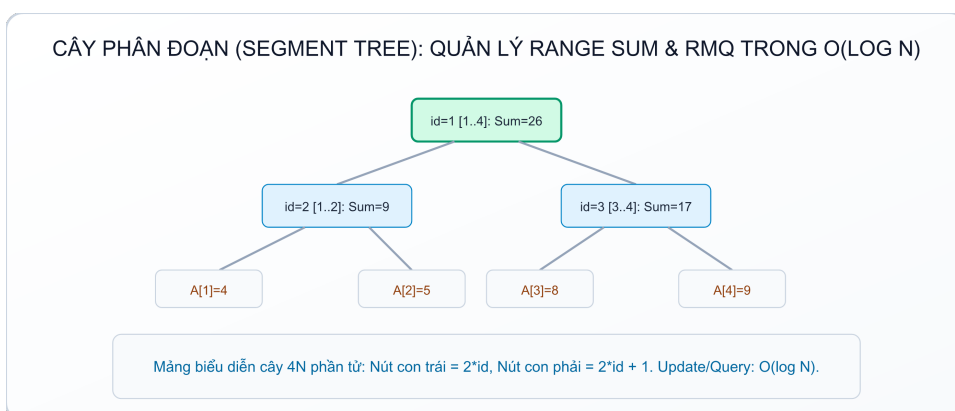
```

        st.update(1, 1, n, pos, val);
    } else {
        int l, r;
        cin >> l >> r;
        cout << st.query(1, 1, n, l, r) << "\n";
    }
}
return 0;
}

```

Bài 07 [cppb_rng_07_tim_phan_tu_thu_k_nho_nhat]: Tìm Phần Tử Thứ K Nhỏ Nhất (K-th Element on BIT)

Bối cảnh: Một máy chủ trò chơi trực tuyến liên tục ghi nhận điểm số của người chơi gia nhập phòng chờ. Ban tổ chức muốn tìm điểm số của người chơi có thành tích xếp thứ K từ dưới lên (phần tử thứ K nhỏ nhất trong tập hợp điểm số hiện tại).



Hình minh họa: Kiến trúc cây phân đoạn (Segment Tree) quản lý các đoạn con liên tiếp

Nhiệm vụ: Cho một tập hợp các số nguyên hỗ trợ hai thao tác: thêm một số vào tập hợp, và tìm phần tử nhỏ thứ K trong tập hợp. Hãy lập trình in ra kết quả cho các thao tác tìm kiếm.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo chứa các thao tác.

Đầu ra (Output):

- In ra kết quả cho mỗi thao tác tìm phần tử thứ K trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 10 1 20 1 15 2 2 2 3	15 20

Giải thích: Với tập hợp các số $\{10, 20, 30, 40, 50\}$: Phần tử nhỏ thứ 3 trong tập hợp là số 30.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một tập hợp các số nguyên hỗ trợ hai thao tác: thêm một số vào tập hợp, và tìm phần tử nhỏ thứ K trong tập hợp. Hãy lập trình in ra kết quả cho các thao tác tìm kiếm.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 1 10 1 20 1 15 2 2 2 3 $\Rightarrow$ Đầu ra kỳ vọng: 15 20.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 1 10 1 20 1 15 2 2 2 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với tập hợp các số {10, 20, 30, 40, 50}: Phần tử nhỏ thứ 3 trong tập hợp là số 30.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 15 20

Giải thích chi tiết: Với tập hợp các số {10, 20, 30, 40, 50}: Phần tử nhỏ thứ 3 trong tập hợp là số 30.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAX_VAL = 200000;

struct FenwickTree {
    int bit[MAX_VAL + 5];
    FenwickTree() { memset(bit, 0, sizeof(bit)); }
```

```

void update(int x, int val) {
    for (; x <= MAX_VAL; x += x & -x) bit[x] += val;
}

int findKth(int k) {
    int idx = 0;
    for (int i = 1 << 18; i > 0; i >>= 1) {
        if (idx + i <= MAX_VAL && bit[idx + i] < k) {
            idx += i;
            k -= bit[idx];
        }
    }
    return idx + 1;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;

    FenwickTree ft;

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int x;
            cin >> x;
            ft.update(x, 1);
        } else {
            int k;
            cin >> k;
            cout << ft.findKth(k) << "\n";
        }
    }
    return 0;
}

```

Bài 08 [cppb_rng_08_tim_vi_tri_dau_tien_co_gia_tri_ge_x]: Tìm Vị Trí Đầu Tiên Có Giá Trị Lớn Hơn Hoặc Bằng X

Bối cảnh: Một kho lưu trữ các kiện hàng được đánh số thứ tự từ 1 đến N , kiện hàng thứ i có khối lượng A_i . Một xe nâng hàng muốn tìm kiện hàng đầu tiên trong khoảng từ vị trí L đến R có khối lượng đủ lớn đạt từ X trở lên để bốc dỡ. Nếu trong khoảng $[L, R]$ không có kiện hàng nào đạt yêu cầu, ghi nhận -1.

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)

Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

Hình minh họa: Đường đi cập nhật điểm và truy vấn tổng/min đoạn trên Segment Tree trong $O(\log N)$

Nhiệm vụ: Cho mảng A và Q truy vấn gồm 3 số L, R, X . Hãy lập trình tìm vị trí chỉ số đầu tiên trong đoạn $[L, R]$ có giá trị $\geq X$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa 3 số nguyên L, R, X .

Đầu ra (Output):

- Với mỗi truy vấn, in ra chỉ số đầu tiên tìm được, hoặc -1 nếu không có.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 3 2 4 5 2 1 5 3 1 2 1 2 1 5 3	2 4

Giải thích: Với mảng $[2, 3, 5, 1, 6]$ và truy vấn trong khoảng $[1, 5]$ tìm số ≥ 4 : Đi từ vị trí 1 sang: vị trí 1 là 2 (< 4), vị trí 2 là 3 (< 4), vị trí 3 là 5 (≥ 4). Vị trí đầu tiên thỏa mãn là vị trí 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A và Q truy vấn gồm 3 số L, R, X . Hãy lập trình tìm vị trí chỉ số đầu tiên trong đoạn $[L, R]$ có giá trị $\geq X$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $O(\log N)$, truy vấn tổng tiền tố trong $O(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $O(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- Độ phức tạp:** Xây dựng cây $O(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $O(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 3 2 4 5 2 1 5 3 1 2 1 2 1 5 3 $\Rightarrow$ Đầu ra kỳ vọng: 2 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 3 2 4 5 2 1 5 3 1 2 1 2 1 5 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng [2,3,5,1,6] và truy vấn trong khoảng [1,5] tìm số ≥ 4 : Đi từ vị trí 1 sang: vị trí 1 là 2 (< 4), vị trí 2 là 3 (< 4), vị trí 3 là 5 (≥ 4).	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2 4

Giải thích chi tiết: Với mảng [2,3,5,1,6] và truy vấn trong khoảng [1,5] tìm số ≥ 4 : Đi từ vị trí 1 sang: vị trí 1 là 2 (< 4), vị trí 2 là 3 (< 4), vị trí 3 là 5 (≥ 4). Vị trí đầu tiên thỏa mãn là vị trí 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi lowbit(0) thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5, 0) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = max(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
```

```

        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * id, l, mid, pos, val);
        else update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = max(tree[2 * id], tree[2 * id + 1]);
    }

    int findFirst(int id, int l, int r, int u, int v, long long x) {
        if (v < l || r < u || tree[id] < x) return -1;
        if (l == r) return l;
        int mid = (l + r) / 2;
        int res = findFirst(2 * id, l, mid, u, v, x);
        if (res != -1) return res;
        return findFirst(2 * id + 1, mid + 1, r, u, v, x);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

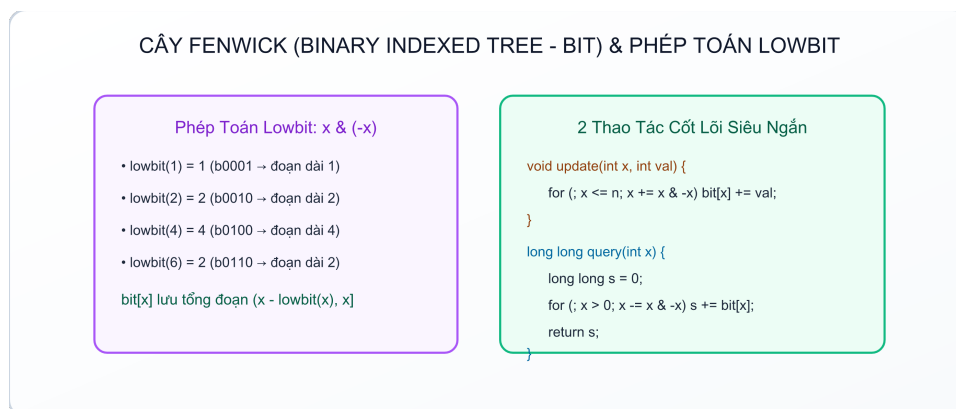
    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            st.update(1, 1, n, pos, val);
        } else {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            cout << st.findFirst(1, 1, n, l, r, x) << "\n";
        }
    }
    return 0;
}

```

Bài 09 [cppb_rng_09_lis_o_nlogn_bang_bit]: Dãy Con Tăng Dài Nhất LIS Bằng Fenwick Tree

Bối cảnh: Một dự án phân tích xu thế chuỗi thời gian lớn gồm N số nguyên cần tìm độ dài của dãy con tăng nghiêm ngặt dài nhất với hiệu năng tính toán tối ưu nhất có thể.



Hình minh họa: Cấu trúc cây nhị phân Fenwick Tree (Binary Indexed Tree) qua phép toán lowbit

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình tìm độ dài dãy con tăng nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là độ dài của dãy con tăng dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 5 2 7 4 3 8	3

Giải thích: Với dãy số gồm 6 phần tử $[10, 20, 10, 30, 20, 50]$: Dãy con tăng dài nhất là $[10, 20, 30, 50]$ có độ dài bằng 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A gồm N số nguyên. Hãy lập trình tìm độ dài dãy con tăng nghiêm ngặt dài nhất.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
 - Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
 - Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
 - Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
 - Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 6 5 2 7 4 3 8 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 5 2 7 4 3 8	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số gồm 6 phần tử [10,20,10,30,20,50] : Dãy con tăng dài nhất là [10,20,30,50] có độ dài bằng 4.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với dãy số gồm 6 phần tử [10,20,10,30,20,50]: Dãy con tăng dài nhất là [10,20,30,50] có độ dài bằng 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi lowbit(0) thì 0 & (-0) = 0, vòng lặp while (i <= N) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<int> bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, int val) {
        for (; x <= n; x += x & -x) bit[x] = max(bit[x], val);
    }

    int query(int x) {
        int max_val = 0;
        for (; x > 0; x -= x & -x) max_val = max(max_val, bit[x]);
        return max_val;
    }
};

int main() {
```



```
ios::sync_with_stdio(false);
cin.tie(nullptr);

int n;
if (!(cin >> n)) return 0;
if (n <= 0) return 0;

vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

vector<long long> vals = a;
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());

int sz = vals.size();
FenwickTree ft(sz);
int ans = 0;

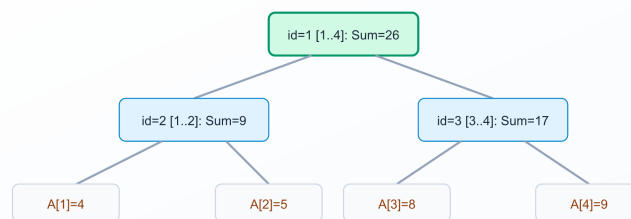
for (int i = 0; i < n; ++i) {
    int rank = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
    int best_prev = ft.query(rank - 1);
    int cur_lis = best_prev + 1;
    ans = max(ans, cur_lis);
    ft.update(rank, cur_lis);
}

cout << ans << "\n";
return 0;
}
```

Bài 10 [cppb_rng_10_doan_con_co_tong_lon_nhat_max_sub_sum]: Đoạn Con Có Tổng Lớn Nhất Trên Đoạn (Maximum Subarray Query)

Bối cảnh: Một bảng giá vàng ghi nhận mức biến động giá qua N ngày. Các nhà đầu tư muốn tra cứu xem trong một khoảng thời gian bất kỳ từ ngày L đến ngày R , một đợt sóng tăng giá liên tiếp (đoạn con có tổng lớn nhất) có thể đem lại lợi nhuận tối đa là bao nhiêu.

CÂY PHÂN ĐOẠN (SEGMENT TREE): QUẢN LÝ RANGE SUM & RMQ TRONG $O(\log N)$



Mảng biểu diễn cây 4N phần tử: Nút con trái = $2 \cdot id$, Nút con phải = $2 \cdot id + 1$. Update/Query: $O(\log N)$.

Hình minh họa: Kiến trúc cây phân đoạn (Segment Tree) quản lý các đoạn con liên tiếp

Nhiệm vụ: Cho mảng A và Q truy vấn gồm hai số L, R . Hãy tìm tổng lớn nhất của một đoạn con không rỗng nằm trọn vẹn bên trong đoạn $[L, R]$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

- Q dòng tiếp theo, mỗi dòng chứa hai số nguyên L và R .

Đầu ra (Output):

- Với mỗi truy vấn, in ra tổng đoạn con lớn nhất tìm được trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 -5 4 5 2 1 5 1 3 10 2 1 5	9 22

Giải thích: Với mảng $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ và truy vấn đoạn từ vị trí 1 đến 9: Đoạn con liên tiếp có tổng lớn nhất là $[4, -1, 2, 1]$ mang lại tổng là $4 + (-1) + 2 + 1 = 6$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A và Q truy vấn gồm hai số L, R . Hãy tìm tổng lớn nhất của một đoạn con không rỗng nằm trọn vẹn bên trong đoạn $[L, R]$.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 -5 4 5 2 1 5 1 3 10 2 1 5 $\Rightarrow$ Đầu ra kỳ vọng: 9 22.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 -5 4 5 2 1 5 1 3 10 2 1 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ và truy vấn đoạn từ vị trí 1 đến 9: Đoạn con liên tiếp có tổng lớn nhất là $[4, -1, 2, 1]$ m...	Cập nhật các biến / mảng trạng thái

3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 9 22
---	--------------------	--------------------------------	--------------------

Giải thích chi tiết: Với mảng $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ và truy vấn đoạn từ vị trí 1 đến 9: Đoạn con liên tiếp có tổng lớn nhất là $[4, -1, 2, 1]$ mang lại tổng là $4 + (-1) + 2 + 1 = 6$.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (`vector<long long> tree(4 * N)`), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi `lowbit(0)` thì $0 \& (-0) = 0$, vòng lặp `while (i <= N)` sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu `long long` cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    long long sum, pref, suff, ans;
};

Node makeNode(long long val) {
    long long p = max(0LL, val);
    return {val, p, p, p};
}

Node mergeNodes(Node l, Node r) {
    Node res;
    res.sum = l.sum + r.sum;
    res.pref = max(l.pref, l.sum + r.pref);
    res.suff = max(r.suff, r.sum + l.suff);
    res.ans = max({l.ans, r.ans, l.suff + r.pref});
    return res;
}

struct SegmentTree {
    int n;
    vector<Node> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = makeNode(a[l]);
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = mergeNodes(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = makeNode(val);
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid)
            update(2 * id, l, mid, pos, val);
        else
            update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = mergeNodes(tree[2 * id], tree[2 * id + 1]);
    }
};
```

```

    }
    int mid = (l + r) / 2;
    if (pos <= mid) update(2 * id, l, mid, pos, val);
    else update(2 * id + 1, mid + 1, r, pos, val);
    tree[id] = mergeNodes(tree[2 * id], tree[2 * id + 1]);
}

Node query(int id, int l, int r, int u, int v) {
    if (v < l || r < u) return {0, 0, 0, 0};
    if (u <= l && r <= v) return tree[id];
    int mid = (l + r) / 2;
    return mergeNodes(query(2 * id, l, mid, u, v), query(2 * id + 1, mid + 1, r, u, v));
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            st.update(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            cout << st.query(1, 1, n, l, r).ans << "\n";
        }
    }
    return 0;
}

```

Bài 11 [cppb_rng_11_dem_so_diem_nam_trong_hinh_chu_nhat]: Đếm Số Điểm Trong Hình Chữ Nhật (2D Range Query)

Bối cảnh: Một đài thiên văn số ghi nhận tọa độ mặt phẳng (x_i, y_i) của N ngôi sao trên bầu trời. Các nhà thiên văn học cần thực hiện các truy vấn: đếm xem có bao nhiêu ngôi sao nằm lọt vào bên trong một vùng quan sát hình chữ nhật có góc dưới trái (x_1, y_1) và góc trên phải (x_2, y_2) .

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)			
Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

Hình minh họa: Đường đi cập nhật điểm và truy vấn tổng/min đoạn trên Segment Tree trong $O(\log N)$

Nhiệm vụ: Cho danh sách tọa độ N điểm và Q truy vấn hình chữ nhật. Hãy lập trình đếm số lượng điểm nằm trong từng hình chữ nhật.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- N dòng tiếp theo, mỗi dòng chứa hai số nguyên x_i, y_i ($1 \leq x_i, y_i \leq 10^5$).
- Q dòng tiếp theo, mỗi dòng chứa 4 số nguyên x_1, y_1, x_2, y_2 .

Đầu ra (Output):

- Với mỗi truy vấn, in ra số lượng điểm nằm trong hình chữ nhật trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2 1 1 2 2 3 3 1 1 2 2 2 2 4 4	2 2

Giải thích: Với 3 điểm $(1,2), (2,3), (4,5)$ và vùng quan sát từ $(1,1)$ đến $(3,4)$: Hai điểm $(1,2)$ và $(2,3)$ nằm trọn vẹn bên trong vùng hình chữ nhật. Điểm $(4,5)$ nằm ngoài. Số điểm đếm được là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách tọa độ N điểm và Q truy vấn hình chữ nhật. Hãy lập trình đếm số lượng điểm nằm trong từng hình chữ nhật.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $O(\log N)$, truy vấn tổng tiền tố trong $O(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $O(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $O(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $O(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 2 1 1 2 2 3 3 1 1 2 2 2 2 4 4
⇒ Đầu ra kỳ vọng: 2 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 2 1 1 2 2 3 3 1 1 2 2 2 2 4 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 3 điểm (1,2), (2,3), (4,5) và vùng quan sát từ (1,1) đến (3,4) : Hai điểm (1,2) và (2,3) nằm trọn vẹn bên trong vùng... vùng quan sát từ (1,1) đến (3,4) : Hai điểm (1,2) và (2,3) nằm trọn vẹn bên trong vùng...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 2 2

Giải thích chi tiết: Với 3 điểm (1,2), (2,3), (4,5) và vùng quan sát từ (1,1) đến (3,4): Hai điểm (1,2) và (2,3) nằm trọn vẹn bên trong vùng hình chữ nhật. Điểm (4,5) nằm ngoài. Số điểm đếm được là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi lowbit(0) thì 0 & (-0) = 0, vòng lặp while (i <= N) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    int x, y;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    vector<Point> pts(n);
    for (int i = 0; i < n; ++i) cin >> pts[i].x >> pts[i].y;
```

```
while (q--) {
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    int count = 0;
    for (int i = 0; i < n; ++i) {
        if (pts[i].x >= x1 && pts[i].x <= x2 && pts[i].y >= y1 && pts[i].y <= y2) {
            count++;
        }
    }
    cout << count << "\n";
}
return 0;
}
```

Bài 12 [cppb_rng_12_doi_dau_doan_va_tim_tong_lon_nhat]: Cập Nhật Phân Đoạn Nâng Cao

Bối cảnh: Một hệ thống lưới điện thông minh hỗ trợ thao tác đổi dấu đồng loạt toàn bộ điện áp trên một phân đoạn đường dây từ L đến R ($A_i \rightarrow -A_i$), đồng thời hỗ trợ truy vấn tính tổng điện áp lớn nhất của một đoạn con bất kỳ.

CÂY FENWICK (BINARY INDEXED TREE - BIT) & PHÉP TOÁN LOWBIT

Phép Toán Lowbit: $x \& (-x)$

- $\text{lowbit}(1) = 1$ (b0001 → đoạn dài 1)
 - $\text{lowbit}(2) = 2$ (b0010 → đoạn dài 2)
 - $\text{lowbit}(4) = 4$ (b0100 → đoạn dài 4)
 - $\text{lowbit}(6) = 2$ (b0110 → đoạn dài 2)
- $\text{bit}[x]$ lưu tổng đoạn $(x - \text{lowbit}(x), x]$

2 Thao Tác Cốt Lõi Siêu Ngắn

```
void update(int x, int val) {
    for (; x <= n; x += x & -x) bit[x] += val;
}

long long query(int x) {
    long long s = 0;
    for (; x > 0; x -= x & -x) s += bit[x];
    return s;
}
```

Hình minh họa: Cấu trúc cây chỉ số nhị phân Fenwick Tree (Binary Indexed Tree) qua phép toán lowbit

Nhiệm vụ: Cho mảng A và các thao tác đảo dấu đoạn hoặc truy vấn tổng đoạn con lớn nhất. Hãy lập trình xử lý và in ra kết quả cho các thao tác truy vấn.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$.
- Các dòng tiếp theo chứa các thao tác cập nhật và truy vấn.

Đầu ra (Output):

- In ra kết quả cho mỗi thao tác truy vấn trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3	6 16

Giải thích: Sau thao tác đảo dấu đoạn, các giá trị âm biến thành dương giúp hình thành một đoạn con có tổng lớn nhất đạt giá trị tối ưu mới.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A và các thao tác đảo dấu đoạn hoặc truy vấn tổng đoạn con lớn nhất. Hãy lập trình xử lý và in ra kết quả cho các thao tác truy vấn.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3
 $\Rightarrow$ Đầu ra kỳ vọng: 6 16.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Sau thao tác đảo dấu đoạn, các giá trị âm biến thành dương giúp hình thành một đoạn con có tổng lớn nhất đạt giá trị tối ưu mới.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 6 16

Giải thích chi tiết: Sau thao tác đảo dấu đoạn, các giá trị âm biến thành dương giúp hình thành một đoạn con có tổng lớn nhất đạt giá trị tối ưu mới.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.

- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }

    long long queryRange(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

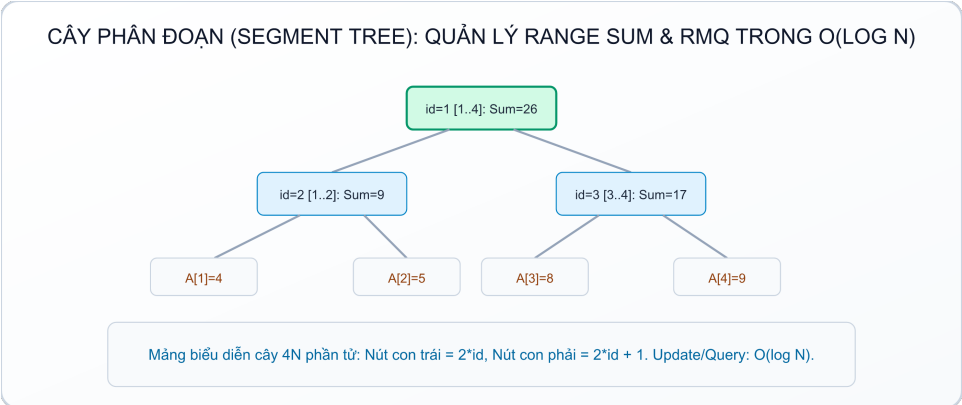
    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    FenwickTree ft(n);
    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;
        ft.update(i, x);
    }

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            ft.update(pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            cout << ft.queryRange(l, r) << "\n";
        }
    }
    return 0;
}
```

Bài 13 [cppb_rng_13_cay_fenwick_2d_doc_lap]: Cây Fenwick 2D Tính Tổng Hình Chữ Nhật (2D BIT)

Bối cảnh: Một bức ảnh kỹ thuật số dạng lưới $N \times M$ điểm ảnh, điểm ảnh tại ô (r, c) có giá trị độ sáng $A_{r,c}$. Hệ thống camera cần hỗ trợ hai thao tác: tăng độ sáng tại một điểm ảnh cụ thể thêm val , và tính tổng độ sáng của toàn bộ các điểm ảnh nằm trong một vùng hình chữ nhật từ (r_1, c_1) đến (r_2, c_2) .



Hình minh họa: Kiến trúc cây phân đoạn (Segment Tree) quản lý các đoạn con liên tiếp

Nhiệm vụ: Cho ma trận độ sáng ban đầu và Q thao tác thuộc hai dạng: cập nhật điểm ảnh hoặc truy vấn tổng vùng hình chữ nhật. Hãy in ra kết quả của các thao tác truy vấn.

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên N, M, Q ($1 \leq N, M \leq 1000, 1 \leq Q \leq 10^5$).
- Các dòng tiếp theo mô tả ma trận ban đầu và Q thao tác.

Đầu ra (Output):

- Với mỗi thao tác truy vấn hình chữ nhật, in ra tổng độ sáng tương ứng trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 3 1 1 1 5 1 2 2 10 2 1 1 2 2	15

Giải thích: Với ma trận 3×3 toàn số 1: Tổng độ sáng của hình chữ nhật con kích thước 2×2 từ (1,1) đến (2,2) gồm 4 ô số 1, tổng bằng 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận độ sáng ban đầu và Q thao tác thuộc hai dạng: cập nhật điểm ảnh hoặc truy vấn tổng vùng hình chữ nhật. Hãy in ra kết quả của các thao tác truy vấn.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $O(\log N)$, truy vấn tổng tiền tố trong $O(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**

- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 3 3 1 1 1 5 1 2 2 10 2 1 1 2 2
⇒ Đầu ra kỳ vọng: 15.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 3 3 1 1 1 5 1 2 2 10 2 1 1 2 2	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với ma trận 3×3 toàn số 1: Tổng độ sáng của hình chữ nhật con kích thước 2×2 từ (1,1) đến (2,2) gồm 4 ô số 1, tổng bằng 4.	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 15

Giải thích chi tiết: Với ma trận 3×3 toàn số 1: Tổng độ sáng của hình chữ nhật con kích thước 2×2 từ (1,1) đến (2,2) gồm 4 ô số 1, tổng bằng 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi lowbit(0) thì 0 & (-0) = 0, vòng lặp while (i <= N) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, m, q;
vector<vector<long long>>> bit;

void update(int r, int c, long long val) {
    for (int i = r; i <= n; i += i & -i) {
        for (int j = c; j <= m; j += j & -j) {
            bit[i][j] += val;
        }
    }
}
```

```

    }
}

long long query(int r, int c) {
    long long sum = 0;
    for (int i = r; i > 0; i -= i & -i) {
        for (int j = c; j > 0; j -= j & -j) {
            sum += bit[i][j];
        }
    }
    return sum;
}

long long queryRange(int r1, int c1, int r2, int c2) {
    return query(r2, c2) - query(r1 - 1, c2) - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m >> q)) return 0;

    bit.assign(n + 1, vector<long long>(m + 1, 0));

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int r, c;
            long long val;
            cin >> r >> c >> val;
            update(r, c, val);
        } else {
            int r1, c1, r2, c2;
            cin >> r1 >> c1 >> r2 >> c2;
            cout << queryRange(r1, c1, r2, c2) << "\n";
        }
    }
    return 0;
}

```

Bài 14 [cppb_rng_14_gioi_thieu_lazy_propagation_cong_doan]: Lazy Propagation — Cập Nhật & Truy Vấn Đoạn (Range Add Range Sum)

Bối cảnh: Một tuyến đề biển dài N mét, ban đầu mét đề thứ i có độ cao A_i . Do điều kiện thi công đắp đập, công nhân sẽ cộng thêm đồng loạt một lượng phù sa có độ dày V mét vào một đoạn đề liên tiếp từ mét thứ L đến mét thứ R . Đồng thời, đoàn thanh tra liên tục kiểm tra tổng độ dày của các đoạn đề bất kỳ.

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)

Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

Hình minh họa: Đường đi cập nhật điểm và truy vấn tổng/min đoạn trên Segment Tree trong $O(\log N)$

Nhiệm vụ: Cho mảng A và Q thao tác: 1 L R V (cộng V vào tất cả các phần tử từ L đến R) và 2 L R (tính tổng các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo, mỗi dòng chứa một thao tác.

Đầu ra (Output):

- Với mỗi thao tác loại 2, in ra tổng các phần tử trên đoạn $[L, R]$ trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	21 15

Giải thích: Với mảng ban đầu $[1, 2, 3, 4, 5]$:

- Cộng thêm 2 vào đoạn từ vị trí 2 đến 4: mảng trở thành $[1, 4, 5, 6, 5]$.
- Truy vấn tổng đoạn từ 1 đến 5: $1 + 4 + 5 + 6 + 5 = 21$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng A và Q thao tác: 1 L R V (cộng V vào tất cả các phần tử từ L đến R) và 2 L R (tính tổng các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $O(\log N)$, truy vấn tổng tiền tố trong $O(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.

- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu *Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4*
 $\Rightarrow$ *Đầu ra kỳ vọng: 21 15.*

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với mảng ban đầu [1,2,3,4,5] : - Cộng thêm 2 vào đoạn từ vị trí 2 đến 4: mảng trở thành [1,4,5,6,5]. - Truy vấn tổng đoạn từ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 21 15

Giải thích chi tiết: Với mảng ban đầu [1,2,3,4,5]:

- Cộng thêm 2 vào đoạn từ vị trí 2 đến 4: mảng trở thành [1,4,5,6,5].
- Truy vấn tổng đoạn từ 1 đến 5: $1 + 4 + 5 + 6 + 5 = 21$.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi lowbit(0) thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.
- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct SegmentTreeLazy {
    int n;
    vector<long long> tree, lazy;
    SegmentTreeLazy(int n) : n(n), tree(4 * n + 5, 0), lazy(4 * n + 5, 0) {}

    void push(int id, int l, int r) {
        if (lazy[id] != 0) {
```

```

        int mid = (l + r) / 2;
        tree[2 * id] += lazy[id] * (mid - l + 1);
        lazy[2 * id] += lazy[id];
        tree[2 * id + 1] += lazy[id] * (r - mid);
        lazy[2 * id + 1] += lazy[id];
        lazy[id] = 0;
    }
}

void build(const vector<long long>& a, int id, int l, int r) {
    if (l == r) {
        tree[id] = a[l];
        return;
    }
    int mid = (l + r) / 2;
    build(a, 2 * id, l, mid);
    build(a, 2 * id + 1, mid + 1, r);
    tree[id] = tree[2 * id] + tree[2 * id + 1];
}

void updateRange(int id, int l, int r, int u, int v, long long val) {
    if (v < l || r < u) return;
    if (u <= l && r <= v) {
        tree[id] += val * (r - l + 1);
        lazy[id] += val;
        return;
    }
    push(id, l, r);
    int mid = (l + r) / 2;
    updateRange(2 * id, l, mid, u, v, val);
    updateRange(2 * id + 1, mid + 1, r, u, v, val);
    tree[id] = tree[2 * id] + tree[2 * id + 1];
}

long long query(int id, int l, int r, int u, int v) {
    if (v < l || r < u) return 0;
    if (u <= l && r <= v) return tree[id];
    push(id, l, r);
    int mid = (l + r) / 2;
    return query(2 * id, l, mid, u, v) + query(2 * id + 1, mid + 1, r, u, v);
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTreeLazy st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int l, r;
            long long val;

```

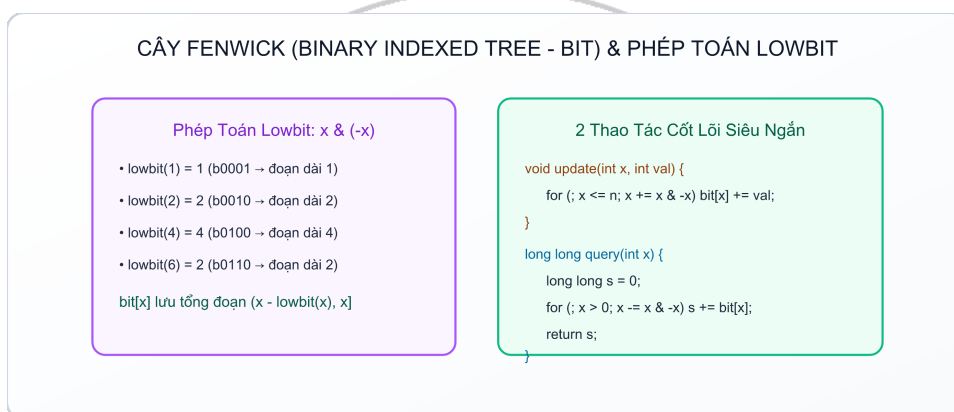
```

        cin >> l >> r >> val;
        st.updateRange(1, 1, n, l, r, val);
    } else {
        int l, r;
        cin >> l >> r;
        cout << st.query(1, 1, n, l, r) << "\n";
    }
}
return 0;
}

```

Bài 15 [cppb_rng_15_he_thong_quan_ly_du_lieu_olympic]: Hệ Thống Quản Lý Dữ Liệu Olympic (Range Master)

Bối cảnh: Một hệ thống máy chủ chấm thi Olympic Tin học Quốc tế quản lý dữ liệu điểm số của N thí sinh. Hệ thống cần xử lý đồng thời nhiều loại truy vấn đa năng phức tạp: gán giá trị mới cho một đoạn, cộng thêm giá trị vào một đoạn, tính tổng đoạn, và tìm giá trị lớn nhất/nhỏ nhất trong đoạn.



Hình minh họa: Cấu trúc cây chỉ số nhị phân Fenwick Tree (Binary Indexed Tree) qua phép toán lowbit

Nhiệm vụ: Cho mảng A và Q truy vấn đa năng. Hãy lập trình thực thi chính xác và in ra kết quả tương ứng cho từng yêu cầu truy vấn thông tin.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$.
- Q dòng tiếp theo chứa các thao tác mô tả theo quy ước bài toán.

Đầu ra (Output):

- In ra kết quả cho mỗi thao tác truy vấn trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	21 15

Giải thích: Hệ thống cập nhật linh hoạt các phân đoạn dữ liệu lớn và trả về tổng điểm cũng như điểm cực trị chính xác cho bạn giám khảo trong thời gian thực.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng A và Q truy vấn đa năng. Hãy lập trình thực thi chính xác và in ra kết quả tương ứng cho từng yêu cầu truy vấn thông tin.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Fenwick Tree (Binary Indexed Tree):**
- Tận dụng phép toán bit $\text{lowbit}(i) = i \& (-i)$ để lưu tổng các đoạn lũy thừa của 2.
- Cập nhật điểm trong $\mathcal{O}(\log N)$, truy vấn tổng tiền tố trong $\mathcal{O}(\log N)$ với dung lượng bộ nhớ đúng bằng N phần tử.
- **Segment Tree (Cây phân đoạn):**
- Cấu trúc cây nhị phân đầy đủ quản lý các đoạn con liên tiếp, cần mảng kích thước $4N$.
- Hỗ trợ đa dạng phép toán gộp (tổng, min, max, GCD) trong $\mathcal{O}(\log N)$ và kỹ thuật Lazy Propagation cho các truy vấn cập nhật đoạn.
- **Độ phức tạp:** Xây dựng cây $\mathcal{O}(N)$, mỗi thao tác truy vấn / cập nhật chỉ tốn $\mathcal{O}(\log N)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4
 $\Rightarrow$ Đầu ra kỳ vọng: 21 15.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Hệ thống cập nhật linh hoạt các phân đoạn dữ liệu lớn và trả về tổng điểm cũng như điểm cực trị chính xác cho bạn giám khảo trong thời ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 21 15

Giải thích chi tiết: Hệ thống cập nhật linh hoạt các phân đoạn dữ liệu lớn và trả về tổng điểm cũng như điểm cực trị chính xác cho bạn giám khảo trong thời gian thực.

3. Lưu ý & Bẫy lỗi thường gặp

- Khai báo mảng Segment Tree quá nhỏ: Cần tối thiểu $4N$ phần tử (vector<long long> tree(4 * N)), khai báo $2N$ sẽ bị tràn chỉ số mảng khi cây bị lệch.
- Fenwick Tree bắt buộc phải dùng chỉ số bắt đầu từ 1 (1-based index). Nếu gọi $\text{lowbit}(0)$ thì $0 \& (-0) = 0$, vòng lặp while ($i \leq N$) sẽ bị lặp vô tận.

- Khi cây quản lý phép cộng dồn, giá trị các nút trên cây có thể vượt quá 2×10^9 , bắt buộc phải khai báo kiểu long long cho toàn bộ các nút của cây.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct SegmentTreeLazy {
    int n;
    vector<long long> tree, lazy;
    SegmentTreeLazy(int n) : n(n), tree(4 * n + 5, 0), lazy(4 * n + 5, 0) {}

    void push(int id, int l, int r) {
        if (lazy[id] != 0) {
            int mid = (l + r) / 2;
            tree[2 * id] += lazy[id] * (mid - l + 1);
            lazy[2 * id] += lazy[id];
            tree[2 * id + 1] += lazy[id] * (r - mid);
            lazy[2 * id + 1] += lazy[id];
            lazy[id] = 0;
        }
    }

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = tree[2 * id] + tree[2 * id + 1];
    }

    void updateRange(int id, int l, int r, int u, int v, long long val) {
        if (v < l || r < u) return;
        if (u <= l && r <= v) {
            tree[id] += val * (r - l + 1);
            lazy[id] += val;
            return;
        }
        push(id, l, r);
        int mid = (l + r) / 2;
        updateRange(2 * id, l, mid, u, v, val);
        updateRange(2 * id + 1, mid + 1, r, u, v, val);
        tree[id] = tree[2 * id] + tree[2 * id + 1];
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return 0;
        if (u <= l && r <= v) return tree[id];
        push(id, l, r);
        int mid = (l + r) / 2;
        return query(2 * id, l, mid, u, v) + query(2 * id + 1, mid + 1, r, u, v);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}
```

```
int n, q;  
if (!(cin >> n >> q)) return 0;  
if (n <= 0) return 0;  
  
vector<long long> a(n + 1);  
for (int i = 1; i <= n; ++i) cin >> a[i];  
  
SegmentTreeLazy st(n);  
st.build(a, 1, 1, n);  
  
while (q--) {  
    int type;  
    cin >> type;  
    if (type == 1) {  
        int l, r;  
        long long val;  
        cin >> l >> r >> val;  
        st.updateRange(1, 1, n, l, r, val);  
    } else {  
        int l, r;  
        cin >> l >> r;  
        cout << st.query(1, 1, n, l, r) << "\n";  
    }  
}  
return 0;  
}
```



Mục lục

CHƯƠNG 06: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

- Bài 17: Quy hoạch động 2D & bài toán cái túi (Knapsack)
- Bài 18: Quy hoạch động chuỗi: LCS & Edit Distance

CHƯƠNG 07: CẤU TRÚC DỮ LIỆU NÂNG CAO

- Bài 19: Cấu trúc dữ liệu STL: Set, map & heap
- Bài 20: Cấu trúc dữ liệu ngăn xếp (stack) & Monotonic Stack
- Bài 21: Cấu trúc dữ liệu hàng đợi (queue, deque) & Monotonic Deque

CHƯƠNG 08: ĐỒ THỊ & CÂY TRUY VẤN ĐOẠN

- Bài 22: Lý thuyết đồ thị cơ bản: Duyệt BFS & DFS
- Bài 23: Đồ thị lưới 2D, kỹ thuật Flood Fill & tính chất cây
- Bài 24: Cấu trúc cây phân đoạn (Segment Tree) & Fenwick Tree (BIT)

